

Universidade Federal do Rio Grande do Norte

Instituto Metr pole Digital

Relat rio Final da Primeira Unidade

Computa  o de Alto Desempenho

Relat rios das atividades pr ticas e experimentais

Douglas Felipe

Natal – RN

2026

Sumário

1	Atividade 1 – Multiplicação matriz por vetor e acesso a memória	2
2	Atividade 2 – Paralelismo em nível de instrução e otimizações do compilador	35
3	Atividade 3 – Processos, threads, concorrência e compartilhamento de memória	71
4	Atividade 4 – Integral pelo método do trapézio e análise de paralelização	93
5	Atividade 5 – Contagem de números primos com OpenMP	124
6	Atividade 6 – Integral pelo método do trapézio: sincronização em OpenMP	163
7	Atividade 7 – Escopo de variáveis e estratégias de escalonamento em OpenMP	207
8	Atividade 8 – Paralelismo de tarefas, paralelismo de dados e regiões paralelas aninhadas	259

Lista de Figuras

Lista de Tabelas

Introdução

Este documento reúne os relatórios das atividades desenvolvidas na primeira unidade da disciplina de Computação de Alto Desempenho.

Capítulo 1

Atividade 1 – Multiplicação matriz por vetor e acesso a memória

1.1 Enunciado da tarefa

Implemente duas versões da multiplicação de matriz por vetor ($M \times V$) em C: uma versão com acesso a matriz por linhas, utilizando linha externa e coluna interna; outra versão com acesso a matriz por colunas, utilizando coluna externa e linha interna.

Meça o tempo de execução de cada versão utilizando uma função apropriada de temporização e execute testes com diferentes tamanhos de matriz.

Identifique a partir de que tamanho os tempos das duas implementações passam a divergir significativamente e explique por que isso ocorre, relacionando as observações com o uso da memória cache e o padrão de acesso a memória.

1.2 Objetivos e requisitos

A atividade tem como objetivo avaliar experimentalmente o impacto do padrão de acesso à memória sobre o desempenho da multiplicação matriz-vetor.

As duas implementações executam a mesma operação matemática e possuem a mesma complexidade assintótica, diferindo essencialmente na ordem em que os elementos da matriz são percorridos.

Os principais requisitos identificados para o experimento são:

- implementar a multiplicação matriz-vetor com percurso da matriz por linhas;
- implementar a mesma operação com percurso da matriz por colunas;
- utilizar a mesma representação de dados nas duas versões;

- variar sistematicamente a dimensão da matriz;
- medir exclusivamente o trecho correspondente à multiplicação;
- utilizar uma fonte de tempo monotônica;
- realizar múltiplas medições para reduzir a influência de ruído experimental;
- validar a equivalência funcional entre as duas implementações;
- apresentar os tempos de forma comparável;
- calcular uma métrica adicional que represente diretamente a diferença entre os dois padrões de acesso;
- relacionar os resultados obtidos com localidade espacial, stride de memória e hierarquia de cache.

Como métrica comparativa adicional foi definida a razão:

$$C/L = \frac{T_{\text{coluna}}}{T_{\text{linha}}}, \quad (1.1)$$

em que T_{coluna} representa o tempo da implementação com percurso por colunas e T_{linha} representa o tempo da implementação com percurso por linhas.

A interpretação da métrica é direta:

- $C/L \approx 1$: tempos semelhantes;
- $C/L > 1$: a versão por colunas apresenta maior tempo;
- $C/L < 1$: a versão por colunas apresenta menor tempo na medição considerada.

1.3 Fundamentação conceitual

1.3.1 Multiplicação matriz-vetor

Considere uma matriz quadrada A de dimensão $N \times N$, um vetor x com N elementos e um vetor resultado y .

A multiplicação matriz-vetor é definida por:

$$y_i = \sum_{j=0}^{N-1} A_{ij}x_j, \quad i = 0, 1, \dots, N-1. \quad (1.2)$$

Para cada posição do vetor de saída é calculado o produto interno entre uma linha da matriz e o vetor de entrada.

As duas implementações avaliadas realizam aproximadamente N^2 multiplicações e N^2 acumulações. Portanto, ambas apresentam complexidade assintótica:

$$O(N^2). \quad (1.3)$$

A atividade não compara algoritmos com ordens de complexidade distintas. O aspecto central é a diferença entre os padrões de acesso aos dados.

1.3.2 Organização row-major

Na representação utilizada pelo programa, a matriz é armazenada em um único bloco contínuo de memória.

O elemento lógico:

$$A[i][j] \quad (1.4)$$

é acessado por:

$$A[i \times N + j]. \quad (1.5)$$

Essa representação corresponde à organização *row-major*, em que os elementos pertencentes a uma mesma linha ocupam posições consecutivas na memória.

Assim, a sequência:

$A[i][0], A[i][1], A[i][2], \dots$

corresponde a acessos contíguos.

Por outro lado, a sequência:

$A[0][j], A[1][j], A[2][j], \dots$

realiza saltos entre posições pertencentes a linhas diferentes.

1.3.3 Localidade espacial

A hierarquia de memória transfere dados em blocos, e não necessariamente um elemento isolado por acesso.

Quando um programa utiliza uma posição de memória, elementos próximos também podem ser disponibilizados em níveis rápidos da hierarquia.

O percurso por linhas explora esse comportamento porque os elementos subsequentes da linha estão armazenados em posições adjacentes.

Esse comportamento é associado à **localidade espacial**.

1.3.4 Stride de memória

O *stride* representa a distância, em bytes, entre endereços sucessivos acessados por uma determinada sequência de execução.

Na implementação por linhas, utilizando dados do tipo `int`, o stride principal sobre a matriz é aproximadamente:

$$S_{\text{linha}} = \text{sizeof}(\text{int}). \quad (1.6)$$

No ambiente utilizado:

$$\text{sizeof}(\text{int}) = 4 \text{ bytes}. \quad (1.7)$$

Portanto:

$$S_{\text{linha}} = 4 \text{ bytes}. \quad (1.8)$$

Na implementação por colunas:

$$S_{\text{coluna}} = N \times \text{sizeof}(\text{int}). \quad (1.9)$$

Por exemplo, para $N = 2000$:

$$S_{\text{coluna}} = 2000 \times 4 = 8000 \text{ bytes}. \quad (1.10)$$

Para $N = 5000$:

$$S_{\text{coluna}} = 5000 \times 4 = 20000 \text{ bytes}. \quad (1.11)$$

O aumento do stride reduz a capacidade de aproveitar imediatamente os dados vizinhos carregados pela hierarquia de memória.

1.3.5 Hierarquia de memória

O desempenho observado depende da interação entre o padrão de acesso e a hierarquia de memória:

Registradores

|

Cache L1

|

Cache L2

|

Cache L3

|
Memória principal

Os níveis mais próximos do processador apresentam menor capacidade, mas menor latência.

Conforme o conjunto de dados aumenta, uma parcela crescente dos acessos pode depender de níveis mais distantes da hierarquia.

Entretanto, o tamanho nominal de uma cache não constitui, isoladamente, um limiar rígido de desempenho. Associatividade, políticas de substituição, prefetching, tradução de endereços e os demais dados utilizados pelo programa também influenciam o comportamento observado.

1.4 Interpretação didática da tarefa

O aspecto didático central da atividade é demonstrar que a complexidade assintótica não caracteriza sozinha o desempenho real de uma implementação.

As duas versões executam a mesma multiplicação e possuem complexidade $O(N^2)$. Entretanto, a ordem dos laços modifica a sequência de endereços acessados.

A versão por linhas apresenta o padrão:

$A[0][0] \rightarrow A[0][1] \rightarrow A[0][2] \rightarrow \dots$

enquanto a versão por colunas apresenta:

$A[0][0] \rightarrow A[1][0] \rightarrow A[2][0] \rightarrow \dots$

A primeira acompanha a organização física dos dados em memória. A segunda introduz um stride proporcional a N .

A atividade permite, portanto, estabelecer a relação:

ordem dos laços $\rightarrow$ padrão de acesso $\rightarrow$ localidade $\rightarrow$ comportamento da cache $\rightarrow$ tempo de execução.
(1.12)

1.5 Modelagem da solução

1.5.1 Estrutura geral

O experimento foi estruturado como um benchmark controlado.

Para cada dimensão N , são realizadas as seguintes etapas:

1. alocação da matriz e dos vetores;

2. inicialização determinística dos dados;
3. execução de aquecimento das duas implementações;
4. validação inicial dos resultados;
5. execução das medições válidas;
6. cálculo da mediana dos tempos;
7. cálculo da razão C/L ;
8. apresentação dos resultados;
9. liberação da memória.

As dimensões avaliadas são:

$$N = 100, 200, 300, \dots, 5000. \quad (1.13)$$

O intervalo de 100 permite acompanhar progressivamente o comportamento da hierarquia de memória sem restringir o experimento a poucos pontos.

1.5.2 Principais funções e componentes

A implementação é organizada nos seguintes componentes:

inicializar_dados() Inicializa a matriz e o vetor com valores inteiros determinísticos.

mxv_linhas() Executa a multiplicação utilizando linha externa e coluna interna.

mxv_colunas() Executa a multiplicação utilizando coluna externa e linha interna.

tempo_decorrido() Calcula o intervalo entre dois valores de `struct timespec`.

medir_linhas() Cronometra exclusivamente a execução do kernel por linhas.

medir_colunas() Cronometra exclusivamente a execução do kernel por colunas.

calcular_mediana() Determina a mediana das repetições válidas.

validar_resultados() Verifica a igualdade entre os vetores produzidos pelas duas versões.

main() Controla a variação de N , as repetições, as validações e a apresentação dos resultados.

1.6 Decisões de projeto e trade-offs

1.6.1 Uso do tipo inteiro

A versão final do experimento utiliza dados do tipo `int`.

Essa decisão simplifica a implementação e permite validar as duas versões por igualdade exata, sem a necessidade de tolerâncias numéricas próprias de operações em ponto flutuante.

A inicialização utiliza valores reduzidos:

```
x[i] = (i % 5) + 1
```

```
A[i][j] = ((i + j) % 10) + 1
```

Assim, mesmo no maior caso avaliado, o valor acumulado permanece dentro da faixa de representação de um inteiro de 32 bits.

Um limite superior conservador é:

$$5000 \times 10 \times 5 = 250000. \quad (1.14)$$

O uso de `int` também reduz pela metade o volume da matriz em relação a uma implementação equivalente com `double`. Por esse motivo, resultados obtidos anteriormente com `double` não são misturados ao conjunto experimental final.

1.6.2 Representação contínua da matriz

A matriz é alocada como um único vetor de N^2 elementos.

Essa representação foi escolhida para:

- garantir contiguidade física dos elementos;
- tornar explícita a organização row-major;
- evitar o overhead estrutural de um vetor de ponteiros;
- permitir que o stride seja derivado diretamente da expressão $iN + j$.

1.6.3 Mediana em vez da média

Cada implementação é medida dez vezes para cada dimensão.

A mediana foi utilizada como estatística representativa por ser menos sensível a medições excepcionalmente altas causadas por escalonamento do sistema operacional, interrupções e outras interferências temporárias.

1.6.4 Alternância da ordem de execução

A ordem entre as versões por linhas e por colunas é alternada a cada repetição.

Nas repetições pares:

`linhas -> colunas`

e nas repetições ímpares:

`colunas -> linhas`

Essa estratégia reduz o viés introduzido pela execução sistemática de uma das versões antes da outra.

1.6.5 Região cronometrada

A alocação, a inicialização dos dados, a validação e a impressão dos resultados permanecem fora da região cronometrada.

Na implementação por colunas, o vetor de saída precisa ser zerado antes de cada execução devido ao uso de acumulações sucessivas. Essa operação também é realizada fora da região medida.

Dessa forma, o tempo registrado corresponde diretamente ao kernel de multiplicação.

1.6.6 Otimização do compilador

O programa foi compilado com `-O2`.

Esse nível permite avaliar uma implementação otimizada em condições realistas, mantendo o mesmo conjunto de opções para ambas as versões.

Como consequência, transformações realizadas automaticamente pelo compilador podem influenciar os tempos absolutos. Entretanto, as duas implementações permanecem submetidas às mesmas condições de compilação.

1.7 Representação estrutural da implementação

A Figura 1.1 apresenta os principais componentes da implementação e suas relações.

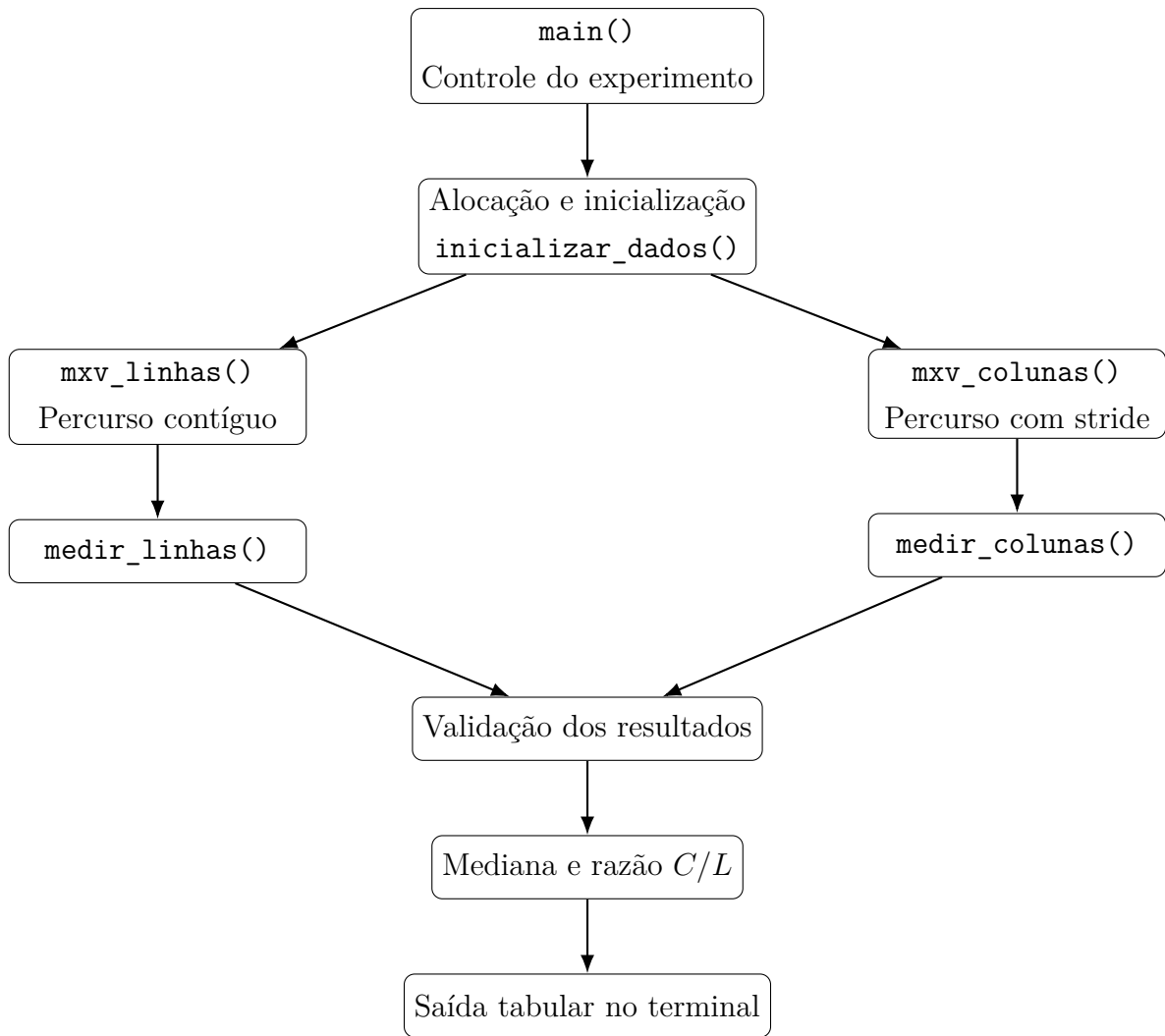


Figura 1.1: Estrutura funcional do benchmark da Atividade 1.

1.8 Fluxo de execução

A Figura 1.2 representa o fluxo executado para cada valor de N .

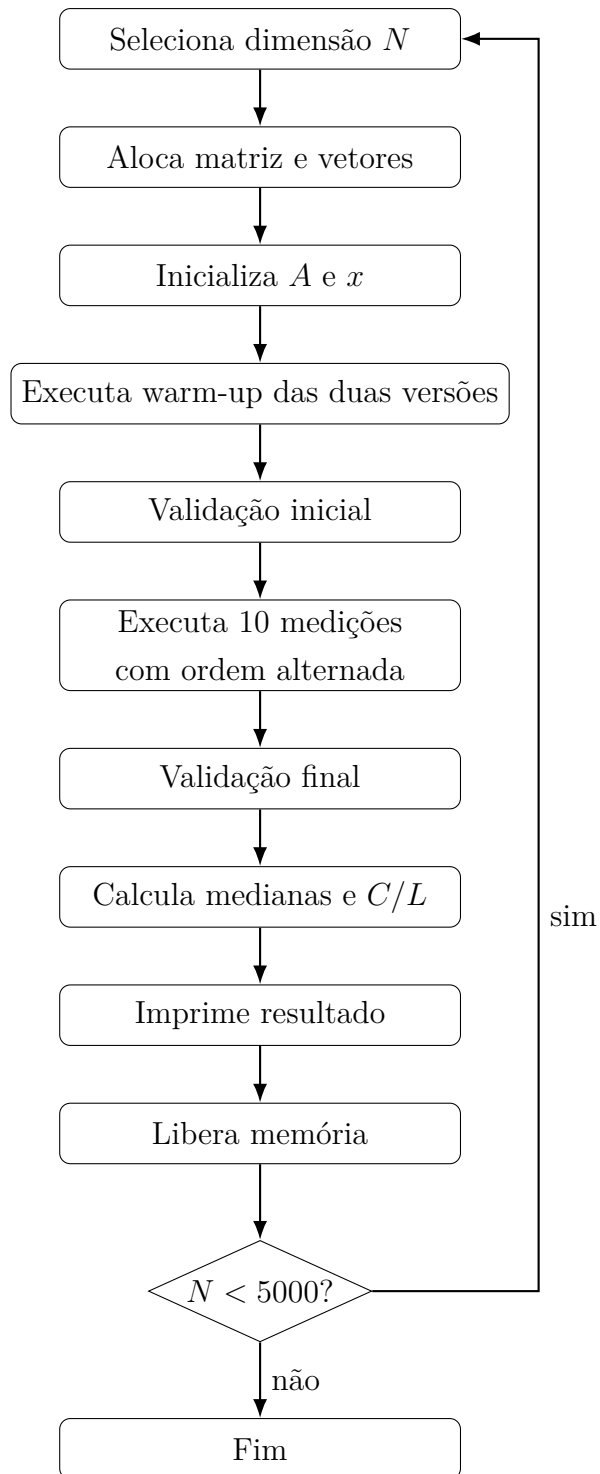


Figura 1.2: Fluxo experimental executado para cada dimensão da matriz.

1.9 Metodologia experimental

1.9.1 Ambiente de execução

O experimento foi executado em uma máquina equipada com processador Intel Core i7-14700K.

As informações de cache obtidas no sistema operacional foram:

Tabela 1.1: Configuração de hardware considerada no experimento.

Característica	Valor
Processador	Intel Core i7-14700K
Cache L2 total reportada	28672 KiB (28 MiB)
Cache L3 reportada	33792 KiB (33 MiB)

O valor de L2 apresentado corresponde ao total reportado pelo sistema e não deve ser interpretado como uma única cache L2 compartilhada de 28 MiB disponível integralmente para uma thread.

O ambiente de software é apresentado na Tabela 1.2.

Tabela 1.2: Configuração de software utilizada.

Característica	Configuração
Sistema	Windows
Ambiente	MSYS2 UCRT64
Compilador	GCC 15.1.0
Linguagem	C
Padrão	C11
Otimização	-O2
Relógio	CLOCK_MONOTONIC
Tipo principal	int

1.9.2 Configuração das medições

Os parâmetros utilizados no benchmark são apresentados na Tabela 1.3.

Tabela 1.3: Parâmetros experimentais da Atividade 1.

Parâmetro	Valor
$N_{\min}$	100
$N_{\max}$	5000
Incremento	100
Warm-up	1 execução por implementação
Medições	10 por implementação e por N
Estatística	Mediana
Ordem das versões	Alternada entre repetições
Unidade apresentada	Segundos
Métrica derivada	$C/L = T_{\text{coluna}}/T_{\text{linha}}$

O comando utilizado para compilação e execução foi:

```
gcc -O2 -std=c11 -Wall -Wextra -pedantic \
ATVD1.c -o ATVD1.exe && ./ATVD1.exe
```

1.10 Código-fonte desenvolvido

O código completo empregado no experimento é apresentado na Listagem 1.1.

Os comentários presentes no arquivo descrevem tecnicamente a finalidade das funções, as regiões de medição, os procedimentos de validação e os mecanismos utilizados para preservação do benchmark durante a otimização.

```

1  #define _POSIX_C_SOURCE 199309L
2
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <time.h>
7  #include <stdint.h>
8
9
10 /* =====
11  * PARÂMETROS DO EXPERIMENTO
12  * ===== */
13
14 #define N_MIN      100
15 #define N_MAX      5000
16 #define PASSO      100
17 #define REPETICOES  10
18
19
20 /*
21  * Impede que o compilador elimine os cálculos durante
22  * a otimização do programa.
23  */

```

```

24 static volatile long long benchmark_sink = 0;
25
26
27 /* =====
28  * TEMPORIZAÇÃO
29  * ===== */
30
31 /*
32  * Calcula o intervalo entre dois instantes de tempo.
33  * O resultado é retornado em segundos.
34  */
35 static double tempo_decorrido(
36     struct timespec inicio,
37     struct timespec fim
38 )
39 {
40     double segundos =
41         (double)(fim.tv_sec - inicio.tv_sec);
42
43     double nanossegundos =
44         (double)(fim.tv_nsec - inicio.tv_nsec) / 1e9;
45
46     return segundos + nanossegundos;
47 }
48
49
50 /* =====
51  * INICIALIZAÇÃO DOS DADOS
52  * ===== */
53
54 /*
55  * Inicializa a matriz e o vetor com valores inteiros pequenos
56  * e determinísticos.
57  *
58  * A matriz é armazenada em um bloco contínuo de memória,
59  * seguindo a organização row-major:
60  *
61  *     A[i][j] -> A[i * n + j]
62  */
63 static void inicializar_dados(
64     int *A,
65     int *x,
66     size_t n
67 )
68 {
69     for (size_t i = 0; i < n; ++i) {
70
71         x[i] = (int)(i % 5) + 1;
72
73         for (size_t j = 0; j < n; ++j) {
74
75             A[i * n + j] =
76                 (int)((i + j) % 10) + 1;
77         }
78     }
79 }
80
81
82 /* =====

```

```

83  * MULTIPLICAÇÃO MATRIZ-VETOR POR LINHAS
84  * ===== */
85
86  /*
87  * Percorre primeiro as linhas e depois as colunas.
88  *
89  * A ordem de acesso segue:
90  *
91  *     A[0][0], A[0][1], A[0][2], ...
92  *     A[1][0], A[1][1], A[1][2], ...
93  *
94  * Esse padrão acompanha a disposição row-major dos dados.
95  */
96  static void mxv_linhas(
97      const int *A,
98      const int *x,
99      int *y,
100     size_t n
101 )
102 {
103     for (size_t i = 0; i < n; ++i) {
104
105         int soma = 0;
106
107         for (size_t j = 0; j < n; ++j) {
108
109             soma += A[i * n + j] * x[j];
110         }
111
112         y[i] = soma;
113     }
114 }
115
116
117  /* =====
118  * MULTIPLICAÇÃO MATRIZ-VETOR POR COLUNAS
119  * ===== */
120
121  /*
122  * Percorre primeiro as colunas e depois as linhas.
123  *
124  * A ordem de acesso segue:
125  *
126  *     A[0][0], A[1][0], A[2][0], ...
127  *     A[0][1], A[1][1], A[2][1], ...
128  *
129  * Nesse caso, acessos consecutivos à matriz apresentam
130  * distância aproximada de n * sizeof(int) bytes.
131  */
132  static void mxv_colunas(
133      const int *A,
134      const int *x,
135      int *y,
136      size_t n
137 )
138 {
139     for (size_t j = 0; j < n; ++j) {
140
141         int xj = x[j];

```

```

142
143     for (size_t i = 0; i < n; ++i) {
144
145         y[i] += A[i * n + j] * xj;
146     }
147 }
148 }
149
150
151 /* =====
152  * CÁLCULO DA MEDIANA
153  * ===== */
154
155 /*
156  * Função auxiliar para ordenação dos tempos medidos.
157  */
158 static int comparar_double(
159     const void *a,
160     const void *b
161 )
162 {
163     double x = *(const double *)a;
164     double y = *(const double *)b;
165
166     return (x > y) - (x < y);
167 }
168
169
170 /*
171  * Calcula a mediana dos tempos obtidos nas repetições.
172  */
173 static double calcular_mediana(
174     double *valores,
175     size_t quantidade
176 )
177 {
178     qsort(
179         valores,
180         quantidade,
181         sizeof(double),
182         comparar_double
183     );
184
185     if (quantidade % 2 == 1) {
186
187         return valores[quantidade / 2];
188     }
189
190     return (
191         valores[quantidade / 2 - 1]
192         +
193         valores[quantidade / 2]
194     ) / 2.0;
195 }
196
197
198 /* =====
199  * MEDIÇÃO DO ACESSO POR LINHAS
200  * ===== */

```

```

201
202 static double medir_linhas(
203     const int *A,
204     const int *x,
205     int *y,
206     size_t n
207 )
208 {
209     struct timespec inicio;
210     struct timespec fim;
211
212     if (clock_gettime(CLOCK_MONOTONIC, &inicio) != 0) {
213
214         perror("clock_gettime");
215         exit(EXIT_FAILURE);
216     }
217
218     mxv_linhas(A, x, y, n);
219
220     if (clock_gettime(CLOCK_MONOTONIC, &fim) != 0) {
221
222         perror("clock_gettime");
223         exit(EXIT_FAILURE);
224     }
225
226     benchmark_sink += y[n / 2];
227
228     return tempo_decorrido(inicio, fim);
229 }
230
231
232 /* =====
233  * MEDIÇÃO DO ACESSO POR COLUNAS
234  * ===== */
235
236 static double medir_colunas(
237     const int *A,
238     const int *x,
239     int *y,
240     size_t n
241 )
242 {
243     struct timespec inicio;
244     struct timespec fim;
245
246     /*
247      * A implementação por colunas acumula valores em y.
248      * O vetor é zerado antes da região cronometrada.
249      */
250     memset(
251         y,
252         0,
253         n * sizeof(int)
254     );
255
256     if (clock_gettime(CLOCK_MONOTONIC, &inicio) != 0) {
257
258         perror("clock_gettime");
259         exit(EXIT_FAILURE);

```

```

260     }
261
262     mxv_colunas(A, x, y, n);
263
264     if (clock_gettime(CLOCK_MONOTONIC, &fim) != 0) {
265
266         perror("clock_gettime");
267         exit(EXIT_FAILURE);
268     }
269
270     benchmark_sink += y[n / 2];
271
272     return tempo_decorrido(inicio, fim);
273 }
274
275
276 /* =====
277  * VALIDAÇÃO DOS RESULTADOS
278  * ===== */
279
280 /*
281  * Verifica se as duas implementações produziram exatamente
282  * o mesmo vetor resultado.
283  */
284 static int validar_resultados(
285     const int *a,
286     const int *b,
287     size_t n
288 )
289 {
290     for (size_t i = 0; i < n; ++i) {
291
292         if (a[i] != b[i]) {
293
294             return 0;
295         }
296     }
297
298     return 1;
299 }
300
301
302 /* =====
303  * PROGRAMA PRINCIPAL
304  * ===== */
305
306 int main(void)
307 {
308     printf(
309         "%-8s %-15s %-15s %-10s\n",
310         "N",
311         "Linha(s)",
312         "Coluna(s)",
313         "C/L"
314     );
315
316     printf(
317         "-----\n"
318     );

```

```

319
320
321  /*
322   * Executa o experimento para matrizes quadradas entre
323   * N_MIN e N_MAX, utilizando o incremento definido em PASSO.
324   */
325  for (
326      size_t n = N_MIN;
327      n <= N_MAX;
328      n += PASSO
329  ) {
330
331      /*
332       * Verifica a possibilidade de overflow antes
333       * da alocação da matriz.
334       */
335      if (
336          n > SIZE_MAX / n
337          ||
338          n * n > SIZE_MAX / sizeof(int)
339      ) {
340
341          fprintf(
342              stderr,
343              "Erro: tamanho da matriz excede "
344              "o limite de alocação.\n"
345          );
346
347          return EXIT_FAILURE;
348      }
349
350
351      size_t elementos = n * n;
352
353
354      /* =====
355       * ALOCAÇÃO DE MEMÓRIA
356       * ===== */
357
358      int *A =
359          malloc(elementos * sizeof(int));
360
361      int *x =
362          malloc(n * sizeof(int));
363
364      int *y_linhas =
365          malloc(n * sizeof(int));
366
367      int *y_colunas =
368          malloc(n * sizeof(int));
369
370
371      if (
372          A == NULL
373          ||
374          x == NULL
375          ||
376          y_linhas == NULL
377          ||

```

```

378     y_colunas == NULL
379 ) {
380
381     fprintf(
382         stderr,
383         "Erro: falha de alocação para N = %zu.\n",
384         n
385     );
386
387     free(A);
388     free(x);
389     free(y_linhas);
390     free(y_colunas);
391
392     return EXIT_FAILURE;
393 }
394
395
396 /* =====
397  * INICIALIZAÇÃO
398  * ===== */
399
400 inicializar_dados(
401     A,
402     x,
403     n
404 );
405
406
407 /* =====
408  * AQUECIMENTO
409  * =====
410  *
411  * Executa previamente as duas implementações.
412  * Essas execuções não participam das medições.
413  */
414
415 mxv_linhas(
416     A,
417     x,
418     y_linhas,
419     n
420 );
421
422 memset(
423     y_colunas,
424     0,
425     n * sizeof(int)
426 );
427
428 mxv_colunas(
429     A,
430     x,
431     y_colunas,
432     n
433 );
434
435
436 /* =====

```

```

437     * VALIDAÇÃO INICIAL
438     * ===== */
439
440     if (
441         !validar_resultados(
442             y_linhas,
443             y_colunas,
444             n
445         )
446     ) {
447
448         fprintf(
449             stderr,
450             "Erro: resultados divergentes para N = %zu.\n",
451             n
452         );
453
454         free(A);
455         free(x);
456         free(y_linhas);
457         free(y_colunas);
458
459         return EXIT_FAILURE;
460     }
461
462
463     /* =====
464     * REPETIÇÕES DO BENCHMARK
465     * ===== */
466
467     double tempos_linhas[REPETICOES];
468     double tempos_colunas[REPETICOES];
469
470
471     for (
472         size_t repeticao = 0;
473         repeticao < REPETICOES;
474         ++repeticao
475     ) {
476
477         /*
478          * A ordem das implementações é alternada entre
479          * as repetições para reduzir viés de execução.
480          */
481         if (repeticao % 2 == 0) {
482
483             tempos_linhas[repeticao] =
484                 medir_linhas(
485                     A,
486                     x,
487                     y_linhas,
488                     n
489                 );
490
491             tempos_colunas[repeticao] =
492                 medir_colunas(
493                     A,
494                     x,
495                     y_colunas,

```

```

496         n
497     );
498
499     } else {
500
501         tempos_colunas[repeticao] =
502             medir_colunas(
503                 A,
504                 x,
505                 y_colunas,
506                 n
507             );
508
509         tempos_linhas[repeticao] =
510             medir_linhas(
511                 A,
512                 x,
513                 y_linhas,
514                 n
515             );
516     }
517 }
518
519
520 /* =====
521  * VALIDAÇÃO FINAL
522  * ===== */
523
524 if (
525     !validar_resultados(
526         y_linhas,
527         y_colunas,
528         n
529     )
530 ) {
531
532     fprintf(
533         stderr,
534         "Erro: resultados divergentes apos benchmark "
535         "para N = %zu.\n",
536         n
537     );
538
539     free(A);
540     free(x);
541     free(y_linhas);
542     free(y_colunas);
543
544     return EXIT_FAILURE;
545 }
546
547
548 /* =====
549  * RESULTADOS
550  * ===== */
551
552 double tempo_linhas =
553     calcular_mediana(
554         tempos_linhas,

```

```

555         REPETICOES
556     );
557
558     double tempo_colunas =
559         calcular_mediana(
560             tempos_colunas,
561             REPETICOES
562         );
563
564
565     /*
566     * A métrica C/L representa a razão entre o tempo
567     * por colunas e o tempo por linhas.
568     *
569     * C/L = 1 -> tempos equivalentes
570     * C/L > 1 -> acesso por colunas mais lento
571     * C/L < 1 -> acesso por colunas mais rápido
572     */
573     double razao = 0.0;
574
575     if (tempo_linhas > 0.0) {
576
577         razao =
578             tempo_colunas
579             /
580             tempo_linhas;
581     }
582
583
584     printf(
585         "%-8zu %-15.9f %-15.9f %-10.3f\n",
586         n,
587         tempo_linhas,
588         tempo_colunas,
589         razao
590     );
591
592     fflush(stdout);
593
594
595     /* =====
596     * LIBERAÇÃO DE MEMÓRIA
597     * ===== */
598
599     free(A);
600     free(x);
601     free(y_linhas);
602     free(y_colunas);
603 }
604
605
606 return EXIT_SUCCESS;
607 }

```

Listing 1.1: Código-fonte da Atividade 1.

1.11 Validação da implementação

A validação é realizada antes e depois das medições.

A versão por linhas calcula cada elemento do vetor resultado utilizando um acumulador local.

A versão por colunas acumula sucessivamente as contribuições em $y[i]$. Por esse motivo, o vetor correspondente é zerado antes de cada execução.

Após a execução das duas versões, os vetores são comparados elemento a elemento:

$$y_i^{\text{linha}} = y_i^{\text{coluna}}, \quad \forall i. \quad (1.15)$$

Como o experimento final utiliza somente operações inteiras e os valores permanecem dentro da faixa de representação, a comparação pode ser realizada por igualdade exata.

A execução apresentada completou todas as dimensões entre $N = 100$ e $N = 5000$ sem emissão de mensagens de divergência, indicando equivalência funcional entre as duas implementações para todos os casos testados.

A variável global `benchmark_sink`, declarada como `volatile`, também recebe parte do resultado após a medição. Esse mecanismo mantém o resultado observável e reduz o risco de eliminação do cálculo por otimizações do compilador.

1.12 Resultados experimentais

A Tabela 1.4 apresenta integralmente os resultados obtidos com a versão final baseada em `int`.

Tabela 1.4: Tempos obtidos para os percursos por linhas e por colunas.

N	Linha (s)	Coluna (s)	C/L
100	0.000002600	0.000002800	1.077
200	0.000010500	0.000010500	1.000
300	0.000023300	0.000022600	0.970
400	0.000040400	0.000052900	1.309
500	0.000064250	0.000082350	1.282
600	0.000092250	0.000112700	1.222
700	0.000123600	0.000162700	1.316
800	0.000158450	0.000299400	1.890
900	0.000199250	0.000313600	1.574
1000	0.000249350	0.000395500	1.586
1100	0.000299200	0.000475400	1.589
1200	0.000363000	0.000621800	1.713

Continua na próxima página

N	Linha (s)	Coluna (s)	C/L
1300	0.000416250	0.000688600	1.654
1400	0.000487650	0.000833400	1.709
1500	0.000556200	0.000988800	1.778
1600	0.000638450	0.001443650	2.261
1700	0.000709550	0.001520900	2.143
1800	0.000835900	0.002114750	2.530
1900	0.000910550	0.002374200	2.607
2000	0.001043400	0.005290200	5.070
2100	0.001178500	0.005689650	4.828
2200	0.001282700	0.007745000	6.038
2300	0.001555050	0.007845800	5.045
2400	0.001581150	0.012585500	7.960
2500	0.001675900	0.015662600	9.346
2600	0.001883250	0.017138500	9.100
2700	0.002053250	0.022817450	11.113
2800	0.002072150	0.024142200	11.651
2900	0.002399600	0.024988600	10.414
3000	0.002504650	0.028835600	11.513
3100	0.002700500	0.029189850	10.809
3200	0.002812800	0.035132500	12.490
3300	0.003690800	0.034381000	9.315
3400	0.003189100	0.036880900	11.565
3500	0.003439000	0.038748050	11.267
3600	0.004232800	0.044581250	10.532
3700	0.004204500	0.045294700	10.773
3800	0.004056450	0.048054400	11.846
3900	0.004325000	0.049052650	11.342
4000	0.004547050	0.056459450	12.417
4100	0.004807850	0.057017050	11.859
4200	0.005020850	0.061201900	12.190
4300	0.005286200	0.063863850	12.081
4400	0.005501450	0.070521800	12.819
4500	0.005782650	0.074178500	12.828
4600	0.006221750	0.079845450	12.833
4700	0.006389000	0.082503350	12.913
4800	0.006527850	0.093638850	14.345
4900	0.007172400	0.092629950	12.915
5000	0.007585550	0.099973250	13.179

1.13 Visualização dos resultados

1.13.1 Tempos de execução

A Figura 1.3 apresenta a evolução dos tempos das duas implementações.

A escala logarítmica no eixo vertical permite visualizar simultaneamente os tempos das menores e das maiores dimensões.

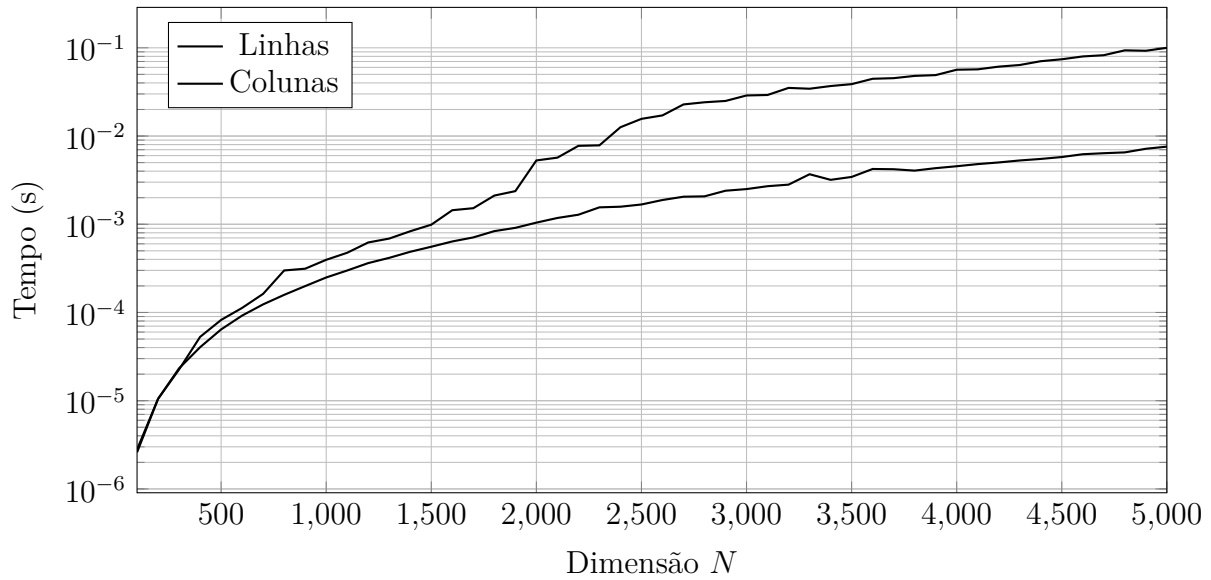


Figura 1.3: Tempo de execução das implementações por linhas e por colunas.

1.13.2 Evolução da razão C/L

A Figura 1.4 apresenta diretamente a razão entre os tempos.

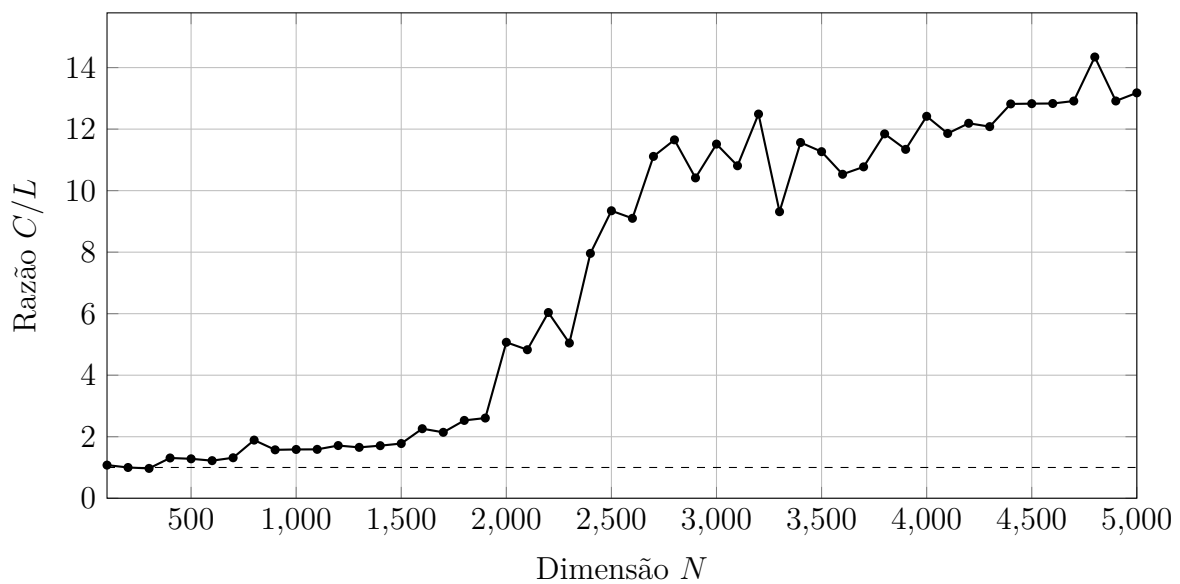


Figura 1.4: Evolução da razão C/L em função da dimensão da matriz.

1.14 Análise e discussão

1.14.1 Região de matrizes pequenas

Entre $N = 100$ e $N = 700$, os tempos permanecem relativamente próximos.

A mediana da razão C/L nessa faixa é:

$$\widetilde{C/L}_{100-700} \approx 1.222. \quad (1.16)$$

Em $N = 200$, por exemplo:

$$C/L = 1.000. \quad (1.17)$$

Em $N = 300$, foi observado:

$$C/L = 0.970. \quad (1.18)$$

Esse valor isolado abaixo de 1 não caracteriza uma vantagem estrutural do percurso por colunas. Os tempos absolutos nessa região são da ordem de microssegundos e, conseqüentemente, mais sensíveis ao ruído relativo da medição.

1.14.2 Início da divergência

Entre $N = 800$ e $N = 1500$, a diferença passa a aparecer de forma mais regular.

A mediana da razão nessa faixa é aproximadamente:

$$\widetilde{C/L}_{800-1500} \approx 1.681. \quad (1.19)$$

Em $N = 800$, por exemplo:

$$C/L = 1.890. \quad (1.20)$$

Essa região pode ser considerada o início experimentalmente observável da divergência entre os dois padrões de acesso.

1.14.3 Divergência consistente

Entre $N = 1600$ e $N = 1900$, a razão passa a permanecer acima de 2.

Os valores observados foram:

N	C/L
1600	2.261
1700	2.143
1800	2.530
1900	2.607

A mediana da faixa é aproximadamente:

$$\widetilde{C/L}_{1600-1900} \approx 2.396. \quad (1.21)$$

Nesse ponto, a penalização do percurso por colunas já é consistente e não depende de um único valor isolado.

1.14.4 Mudança expressiva em N=2000

A transição mais evidente ocorre entre $N = 1900$ e $N = 2000$.

Para $N = 1900$:

$$T_{\text{linha}} = 0.000910550 \text{ s}, \quad (1.22)$$

$$T_{\text{coluna}} = 0.002374200 \text{ s}, \quad (1.23)$$

$$C/L = 2.607. \quad (1.24)$$

Para $N = 2000$:

$$T_{\text{linha}} = 0.001043400 \text{ s}, \quad (1.25)$$

$$T_{\text{coluna}} = 0.005290200 \text{ s}, \quad (1.26)$$

$$C/L = 5.070. \quad (1.27)$$

Enquanto o tempo por linhas cresce de forma moderada, o tempo da versão por colunas mais que dobra nessa transição.

Assim, $N = 2000$ caracteriza uma mudança expressiva no comportamento experimental.

1.14.5 Regime de forte penalização

Entre $N = 2000$ e $N = 2700$, a mediana de C/L é aproximadamente:

$$\widetilde{C/L}_{2000-2700} \approx 6.999. \quad (1.28)$$

Em $N = 2700$:

$$C/L = 11.113. \quad (1.29)$$

A partir de $N = 2800$, a razão permanece predominantemente na faixa de aproximadamente 10 a 13.

A mediana obtida entre $N = 2800$ e $N = 5000$ é:

$$\widetilde{C/L}_{2800-5000} \approx 11.859. \quad (1.30)$$

1.14.6 Maior dimensão avaliada

Para $N = 5000$, foram obtidos:

$$T_{\text{linha}} = 0.007585550 \text{ s}, \quad (1.31)$$

$$T_{\text{coluna}} = 0.099973250 \text{ s}, \quad (1.32)$$

$$C/L = 13.179. \quad (1.33)$$

A versão por colunas levou, portanto, aproximadamente 13.18 vezes o tempo da versão por linhas nesse caso.

Esse resultado ocorre apesar de ambas as implementações permanecerem em $O(N^2)$.

1.14.7 Relação com o tamanho da matriz

Como cada elemento possui quatro bytes, o espaço ocupado apenas pela matriz é:

$$M(N) = 4N^2 \text{ bytes}. \quad (1.34)$$

Alguns pontos relevantes são apresentados na Tabela 1.5.

Tabela 1.5: Tamanho aproximado da matriz em pontos selecionados.

N	Matriz (MiB)	C/L
200	0.15	1.000
1000	3.81	1.586
1800	12.36	2.530
2000	15.26	5.070
3000	34.33	11.513
4000	61.04	12.417
5000	95.37	13.179

1.14.8 Relação com a cache L3

A cache L3 reportada para a máquina possui:

$$33792 \text{ KiB} = 33 \text{ MiB.} \quad (1.35)$$

Igualando aproximadamente o tamanho da matriz a essa capacidade:

$$4N^2 \approx 33 \times 1024^2, \quad (1.36)$$

obtém-se:

$$N \approx 2941. \quad (1.37)$$

De fato:

$$M(2900) \approx 32.08 \text{ MiB,} \quad (1.38)$$

$$M(3000) \approx 34.33 \text{ MiB.} \quad (1.39)$$

Portanto, a matriz atravessa numericamente a capacidade nominal reportada da L3 entre $N = 2900$ e $N = 3000$.

Entretanto, a forte divergência entre as versões já aparece em $N = 2000$, quando a matriz ocupa aproximadamente 15.26 MiB.

Esse resultado é importante porque demonstra que não é adequado explicar o comportamento simplesmente pela afirmação de que “a matriz deixou de caber na L3”.

A capacidade total da cache é apenas um dos fatores envolvidos.

O padrão de acesso por colunas já apresenta:

$$S_{\text{coluna}} = 8000 \text{ bytes} \quad (1.40)$$

em $N = 2000$, enquanto o percurso por linhas continua utilizando acessos contíguos de quatro bytes.

Assim, a diferença observada deve ser interpretada principalmente pela interação entre:

- tamanho crescente do conjunto de dados;
- baixa localidade espacial do percurso por colunas;
- stride crescente;
- aproveitamento das linhas de cache;
- mecanismos de prefetching;

- tradução de endereços;
- políticas de substituição da hierarquia de memória.

1.14.9 Cache L2 reportada

O sistema reportou 28672 KiB de cache L2, correspondentes a 28 MiB.

Numericamente, uma matriz de inteiros atinge aproximadamente esse volume em:

$$N \approx 2709. \quad (1.41)$$

Essa proximidade não deve ser interpretada como evidência de que uma matriz de aproximadamente 28 MiB esteja integralmente armazenada em uma única L2 durante o benchmark.

O valor reportado corresponde à capacidade total agregada apresentada pelo sistema para o processador. Portanto, ele é registrado como característica do hardware, mas não é utilizado como um limiar causal direto para a interpretação de uma única thread.

1.14.10 Valor máximo de C/L

O maior valor observado foi:

$$C/L = 14.345 \quad (1.42)$$

em $N = 4800$.

Nesse ponto:

$$T_{\text{linha}} = 0.006527850 \text{ s}, \quad (1.43)$$

$$T_{\text{coluna}} = 0.093638850 \text{ s}. \quad (1.44)$$

Como em $N = 4900$ e $N = 5000$ a razão retorna para 12.915 e 13.179, respectivamente, o valor em $N = 4800$ não caracteriza isoladamente uma nova mudança de regime.

Ele é tratado como uma variação dentro da região de forte penalização já estabelecida.

1.15 Limitações

Embora o experimento apresente uma tendência clara, os resultados devem ser interpretados dentro das seguintes limitações:

- o benchmark foi executado em uma única máquina;

- os tempos podem sofrer influência do escalonamento do sistema operacional e de processos concorrentes;
- variações de frequência do processador podem alterar tempos absolutos entre execuções;
- não foram utilizados contadores de hardware para medir diretamente cache misses, TLB misses ou largura de banda;
- os tamanhos de cache reportados pelo sistema não representam necessariamente a quantidade efetivamente disponível para uma única thread durante toda a execução;
- mecanismos automáticos de prefetching podem modificar o custo de determinados padrões de acesso;
- o compilador com `-O2` pode aplicar transformações que afetam os kernels, embora ambas as versões utilizem as mesmas opções;
- as menores matrizes apresentam tempos absolutos muito curtos e, consequentemente, maior sensibilidade relativa ao overhead e ao ruído;
- a inversão da ordem dos laços também modifica o comportamento de acesso ao vetor de saída: a versão por linhas acumula em uma variável local e grava `y[i]` uma vez, enquanto a versão por colunas realiza atualizações sucessivas no vetor de saída;
- o estudo utiliza `int`; outros tipos de dado alteram o tamanho do working set e podem deslocar as regiões de transição.

Essas limitações não invalidam a tendência observada, mas impedem atribuir um ponto específico exclusivamente a um único nível de cache sem medições microarquiteturais adicionais.

1.16 Conclusões da atividade

O experimento demonstrou que duas implementações com a mesma operação matemática e a mesma complexidade assintótica podem apresentar desempenhos substancialmente diferentes em função da ordem de acesso aos dados.

Para matrizes pequenas, os tempos das duas versões permaneceram próximos.

A partir de aproximadamente $N = 800$, tornou-se possível observar o início de uma divergência sistemática.

Entre $N = 1600$ e $N = 1900$, a versão por colunas passou a exigir aproximadamente duas vezes o tempo da versão por linhas.

Em $N = 2000$, a razão alcançou:

$$C/L = 5.070, \quad (1.45)$$

caracterizando a mudança mais expressiva da região intermediária.

Para matrizes maiores, a penalização aumentou substancialmente. Entre $N = 2800$ e $N = 5000$, a mediana da razão foi aproximadamente:

$$C/L = 11.859. \quad (1.46)$$

No maior tamanho avaliado:

$$C/L = 13.179. \quad (1.47)$$

Os resultados são compatíveis com a diferença entre os padrões de acesso. O percurso por linhas acompanha a organização row-major e apresenta elevada localidade espacial. O percurso por colunas utiliza um stride proporcional a N , reduzindo o aproveitamento imediato dos dados carregados pela hierarquia de memória.

A atividade também evidencia que a capacidade nominal das caches não deve ser utilizada isoladamente como explicação. A forte divergência surge antes de a matriz alcançar a capacidade nominal da L3, indicando que stride, localidade, prefetching e demais mecanismos da arquitetura também participam do comportamento observado.

Assim, o principal resultado da atividade pode ser sintetizado pela seguinte relação:

$$\boxed{\text{mesmo } O(N^2) \not\Rightarrow \text{mesmo desempenho}} \quad (1.48)$$

quando os padrões de acesso à memória são diferentes.

1.17 Síntese

Os principais pontos para apresentação oral da atividade são:

1. **Problema.** Comparar duas implementações da mesma multiplicação matriz-vetor, alterando apenas a ordem principal de percurso da matriz.
2. **Hipótese.** Como a matriz é armazenada em row-major, o percurso por linhas deve explorar melhor a localidade espacial do que o percurso por colunas.
3. **Complexidade.** As duas versões permanecem em $O(N^2)$. Portanto, a diferença não decorre de uma mudança na ordem assintótica do algoritmo.
4. **Metodologia.** Foram avaliadas matrizes de $N = 100$ até $N = 5000$, em passos de 100, com uma execução de aquecimento e dez medições por versão. A mediana foi utilizada como tempo representativo.

5. **Métrica.** Foi utilizada a razão $C/L = T_{\text{coluna}}/T_{\text{linha}}$, permitindo interpretar diretamente quantas vezes o percurso por colunas foi mais lento.
6. **Primeira divergência.** A diferença começa a se tornar visível em torno de $N = 800$.
7. **Divergência consistente.** Entre $N = 1600$ e $N = 1900$, o percurso por colunas apresenta tempo aproximadamente duas vezes maior.
8. **Mudança mais expressiva.** Em $N = 2000$, $C/L = 5.070$.
9. **Regime de matrizes grandes.** Entre $N = 2800$ e $N = 5000$, a mediana de C/L foi aproximadamente 11.859.
10. **Maior caso.** Para $N = 5000$, a implementação por colunas levou aproximadamente 13.18 vezes o tempo da implementação por linhas.
11. **Explicação arquitetural.** Por linhas, o stride principal é de quatro bytes. Por colunas, o stride é $4N$ bytes, atingindo 20.000 bytes em $N = 5000$.
12. **Relação com cache.** A matriz cruza aproximadamente os 33 MiB nominais da L3 entre $N = 2900$ e $N = 3000$, mas a forte divergência já aparece antes. Portanto, não é correto atribuir o fenômeno apenas ao instante em que a matriz deixa de caber na L3.
13. **Conclusão.** A atividade evidencia que organização dos dados e localidade de memória podem determinar diferenças de desempenho superiores a uma ordem de grandeza mesmo quando a complexidade algorítmica permanece inalterada.

Capítulo 2

Atividade 2 – Paralelismo em nível de instrução e otimizações do compilador

2.1 Enunciado da tarefa

Implemente três laços em C para investigar os efeitos do paralelismo em nível de instrução (*Instruction-Level Parallelism* – ILP) e as possíveis otimizações realizadas pelo compilador.

Os três casos devem inicializar um vetor utilizando um cálculo simples, somar seus elementos de forma acumulativa criando dependência entre as iterações, e quebrar essa dependência utilizando múltiplas variáveis de acumulação.

Compare o tempo de execução das versões compiladas com diferentes níveis de otimização: -O0, -O2 e -O3, utilizando diferentes tamanhos de vetor.

2.2 Objetivos e requisitos

A atividade teve como objetivo investigar experimentalmente a relação entre dependências de dados, paralelismo em nível de instrução e otimizações realizadas pelo compilador em laços implementados na linguagem C.

Mais especificamente, buscou-se observar como duas formas semanticamente equivalentes de realizar uma redução sobre um vetor podem apresentar comportamentos diferentes em função da quantidade de dependências existentes entre as instruções.

Foram estabelecidos os seguintes requisitos para o experimento:

- implementar o experimento integralmente em C;
- utilizar o tipo `int` para o vetor e para as variáveis de acumulação;
- implementar um laço de inicialização do vetor;

- implementar uma redução com um único acumulador;
- implementar uma redução equivalente utilizando quatro acumuladores independentes;
- variar o tamanho N do vetor;
- compilar o mesmo código com -O0, -O2 e -O3;
- medir tempo decorrido utilizando `CLOCK_MONOTONIC`, e não contadores de ciclos do processador;
- realizar aquecimentos antes das medições efetivas;
- realizar múltiplas medições e utilizar a mediana como estatística representativa;
- validar que as duas implementações de soma produzam o mesmo resultado;
- apresentar os resultados de forma simples e textual no terminal.

2.3 Fundamentação conceitual

2.3.1 Paralelismo em nível de instrução

O paralelismo em nível de instrução, ou ILP, corresponde à possibilidade de um processador executar ou manter simultaneamente em diferentes estágios múltiplas instruções pertencentes ao mesmo fluxo de execução.

Diferentemente do paralelismo baseado em *threads*, o ILP não exige que o programa possua múltiplas linhas de execução. Um único fluxo sequencial pode conter instruções independentes que sejam executadas de forma sobreposta por uma arquitetura superscalar.

Considere, por exemplo, duas operações independentes:

```
a = x + y;
b = z + w;
```

A segunda operação não necessita do resultado produzido pela primeira. Dessa forma, a arquitetura possui maior liberdade para escalonar ambas sobre diferentes recursos internos.

A situação muda quando existe uma dependência:

```
a = x + y;
b = a + z;
```

Nesse caso, a segunda instrução somente pode prosseguir após a produção de **a**. A latência da primeira operação passa a integrar o caminho crítico da execução.

2.3.2 Dependências carregadas entre iterações

O segundo laço da atividade realiza uma redução na forma

$$S_{i+1} = S_i + v_i, \quad (2.1)$$

em que v_i representa o elemento do vetor processado na iteração i .

Em C, sua forma conceitual é

```
sum += vector[i];
```

A variável `sum` produz uma dependência entre iterações consecutivas:

```
sum0 -> sum1 -> sum2 -> sum3 -> sum4 -> ...
```

Cada nova adição precisa do resultado calculado na iteração imediatamente anterior. Trata-se de uma dependência verdadeira de dados, do tipo RAW (*Read After Write*), carregada entre iterações do laço.

Essa cadeia reduz a quantidade de operações de soma independentes disponíveis simultaneamente para o processador.

2.3.3 Quebra da cadeia de dependência

Uma forma clássica de aumentar o ILP disponível consiste em utilizar múltiplos acumuladores.

Em vez de manter uma única cadeia,

```
S -> S -> S -> S -> S -> S
```

podem ser criadas várias cadeias independentes:

```
S0 -> S0 -> S0
```

```
S1 -> S1 -> S1
```

```
S2 -> S2 -> S2
```

```
S3 -> S3 -> S3
```

No experimento foram empregados quatro acumuladores:

```
sum0 += vector[i];
```

```
sum1 += vector[i + 1];
```

```
sum2 += vector[i + 2];
```

```
sum3 += vector[i + 3];
```

Somente ao final as quatro somas parciais são combinadas.

Essa reorganização mantém o resultado lógico da redução, porém expõe operações independentes que podem ser escalonadas de forma sobreposta pelo processador.

2.3.4 ILP, desenrolamento de laços e vetorização

A presença de múltiplas operações independentes também pode facilitar transformações realizadas pelo compilador.

Entre as otimizações relacionadas a laços encontram-se o desenrolamento (*loop unrolling*), o reordenamento e escalonamento de instruções e a vetorização SIMD.

É importante distinguir ILP de vetorização. ILP é um conceito mais amplo: diferentes instruções escalares podem ser executadas simultaneamente ou de maneira sobreposta. Vetorização, por outro lado, utiliza instruções capazes de operar sobre vários elementos de dados de uma única vez.

Consequentemente, uma melhoria observada após a compilação com níveis agressivos de otimização não pode ser atribuída automaticamente a uma única técnica sem analisar o código gerado ou os relatórios do compilador.

2.3.5 Efeito do nível de otimização

O mesmo código-fonte foi compilado com três níveis:

- 00 minimiza transformações de otimização e mantém maior proximidade entre o código-fonte e o código gerado;
- 02 habilita um conjunto amplo de otimizações consideradas adequadas para uso geral;
- 03 adiciona transformações mais agressivas, particularmente relevantes para laços, exposição de paralelismo e vetorização.

A comparação entre esses níveis permite investigar não apenas a estrutura manual dos laços, mas também a capacidade do compilador de transformar o código.

2.3.6 Hierarquia de memória

O tamanho do vetor também influencia o desempenho.

Os dados utilizados por um programa transitam por diferentes níveis da hierarquia de memória, que apresentam capacidades e latências distintas. Em termos simplificados:

Registadores $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ memória principal

Para vetores pequenos, uma parcela maior do conjunto de dados pode permanecer próxima ao processador. À medida que N aumenta, cresce a pressão sobre os níveis de cache e sobre o subsistema de memória.

Portanto, um ganho obtido por aumento de ILP pode tornar-se proporcionalmente menor quando o custo de movimentação de dados passa a representar uma parcela maior do tempo total.

2.4 Interpretação didática da tarefa

A tarefa foi interpretada como uma investigação da cadeia causal

estrutura do código → dependências → ILP disponível → otimizações → código de máquina → tempo
(2.2)

O primeiro laço representa um caso no qual cada posição do vetor pode ser calculada independentemente.

O segundo representa deliberadamente uma redução com dependência serial entre iterações.

O terceiro mantém o mesmo objetivo matemático do segundo, porém reorganiza a computação de forma a disponibilizar quatro cadeias de dependência independentes.

Assim, o experimento não consiste apenas em determinar qual versão é mais rápida. O interesse principal está em observar em quais condições a exposição manual de ILP produz benefício e como esse benefício se modifica quando o compilador recebe maior liberdade para transformar o código.

2.5 Modelagem da solução

2.5.1 Estrutura geral

Foi desenvolvido um único programa, `benchmark.c`, compilado três vezes a partir do mesmo código-fonte.

As três compilações diferem pelo nível de otimização:

-O0
-O2
-O3

Dessa forma, evita-se manter implementações diferentes para cada configuração e reduz-se o risco de introduzir alterações acidentais entre os experimentos.

Para cada executável, o programa percorre internamente todos os valores de N , executa aquecimentos, realiza as medições, valida os resultados e apresenta as medianas no terminal.

2.5.2 Inicialização do vetor

O vetor foi inicializado de acordo com

$$v_i = (i \ \& \ 7) + 1. \quad (2.3)$$

Consequentemente, os elementos repetem a sequência

1 2 3 4 5 6 7 8

Essa escolha possui três propriedades úteis para o experimento.

Primeiro, os elementos permanecem pequenos. Segundo, a soma esperada pode ser calculada deterministicamente. Terceiro, mesmo para o maior vetor utilizado, a soma permanece abaixo do limite representável por `int`.

Como

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 = 36, \quad (2.4)$$

e todos os valores de N utilizados são múltiplos de oito, a soma esperada é

$$S_{\text{esperado}} = \frac{N}{8} \cdot 36. \quad (2.5)$$

2.5.3 Redução com acumulador único

A implementação de referência emprega uma única variável:

```
int sum = 0;

for (size_t i = 0; i < n; ++i) {
    sum += vector[i];
}
```

Essa versão apresenta diretamente a cadeia de dependência que constitui o principal objeto de estudo da atividade.

2.5.4 Redução com quatro acumuladores

A segunda redução utiliza quatro somas parciais:

```
for (; i + 3 < n; i += 4) {
    sum0 += vector[i];
    sum1 += vector[i + 1];
    sum2 += vector[i + 2];
    sum3 += vector[i + 3];
}
```

Ao final, calcula-se

$$S = S_0 + S_1 + S_2 + S_3. \quad (2.6)$$

Foi mantido ainda um laço residual para garantir a correção da função caso um tamanho que não seja múltiplo de quatro seja utilizado futuramente.

2.5.5 Principais funções e componentes

A implementação foi decomposta nos seguintes componentes:

`initialize_vector()` executa o primeiro laço e inicializa o vetor;

`sum_single_accumulator()` executa a redução dependente;

`sum_four_accumulators()` executa a redução com quatro cadeias independentes;

`measure_initialization()` mede somente a inicialização;

`measure_single()` mede somente a redução com acumulador único;

`measure_multi4()` mede somente a redução com quatro acumuladores;

`median()` calcula a mediana das medições;

`expected_sum()` calcula o resultado esperado para validação;

`run_for_n()` coordena todos os testes associados a um tamanho de vetor.

2.6 Decisões de projeto e trade-offs

2.6.1 Uso de `int`

Foi utilizado `int` tanto para os elementos do vetor quanto para os acumuladores, conforme definido para o experimento.

Essa decisão torna necessário controlar a possibilidade de *overflow*. Por esse motivo, foram escolhidos valores pequenos para os elementos do vetor e foi implementada uma verificação do valor esperado.

Para o maior vetor,

$$N = 16\,777\,216, \tag{2.7}$$

a soma esperada é

$$75\,497\,472, \tag{2.8}$$

valor amplamente inferior ao limite de um `int` de 32 bits utilizado no ambiente.

2.6.2 Quatro acumuladores

Foram utilizados quatro acumuladores independentes.

Um número maior poderia aumentar ainda mais o paralelismo explícito, mas também introduziria maior pressão sobre registradores e transformaria a quantidade de acumuladores em uma nova variável experimental.

Quatro acumuladores representam um compromisso entre simplicidade, clareza didática e capacidade de exposição de ILP.

2.6.3 Variação de N

Foram utilizados os seguintes tamanhos:

Tabela 2.1: Tamanhos de vetor utilizados no experimento.

N	Memória ocupada pelo vetor
32 768	128 KiB
65 536	256 KiB
262 144	1 MiB
1 048 576	4 MiB
2 097 152	8 MiB
4 194 304	16 MiB
16 777 216	64 MiB

Os tamanhos foram definidos em potências de dois e abrangem desde conjuntos de dados de centenas de KiB até um vetor substancialmente maior que o cache de último nível informado para a máquina.

Essa escolha permite observar tanto situações dominadas pela execução dos laços quanto situações nas quais a movimentação de dados possui maior peso relativo.

2.6.4 Mediana em vez de uma única execução

Uma única medição está sujeita a interferências do sistema operacional, interrupções, variações de frequência e processos concorrentes.

Foram utilizados três aquecimentos e vinte medições efetivas por configuração. O valor reportado corresponde à mediana.

A mediana reduz a influência de observações isoladas anormalmente altas.

2.6.5 Alternância da ordem das reduções

Executar sempre a redução com acumulador único antes da versão com quatro acumuladores poderia favorecer sistematicamente a segunda implementação por causa do estado da hierarquia de memória.

Por esse motivo, a ordem foi alternada:

repetição par: Single -> Multi4
repetição ímpar: Multi4 -> Single

A alternância não elimina todos os efeitos de cache, mas reduz um viés sistemático simples entre as duas versões.

2.6.6 Ausência de `-march=native`

Embora a máquina tenha apresentado suporte a AVX, AVX2 e outras extensões durante a caracterização inicial, `-march=native` não foi utilizado no experimento principal.

A intenção foi manter como principal variável de compilação apenas o nível de otimização:

-O0
-O2
-O3

Adicionar `-march=native` criaria uma segunda variável, relacionada ao conjunto de instruções disponível para geração de código.

2.6.7 Uso de `noinline`

As funções correspondentes aos laços foram marcadas para evitar *inlining*.

Essa decisão facilita a separação conceitual dos experimentos e uma eventual inspeção do código de máquina de cada função.

Como contrapartida, existe um pequeno custo de chamada de função, particularmente relevante para os menores tamanhos de vetor. Esse efeito é discutido posteriormente entre as ameaças à validade.

2.7 Representação estrutural da implementação

A Figura 2.1 resume a organização do experimento.

2.8 Fluxo de execução

Para cada valor de N , a execução segue o fluxo apresentado na Figura 2.2.

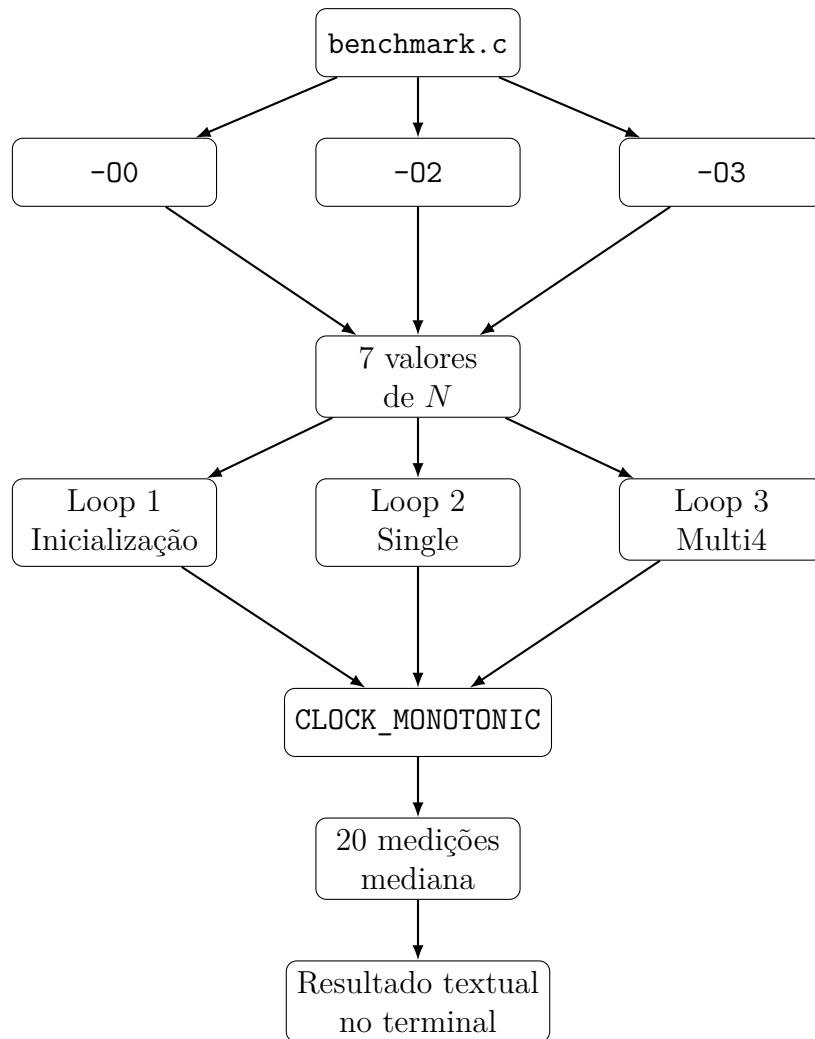


Figura 2.1: Representação estrutural do experimento de ILP.

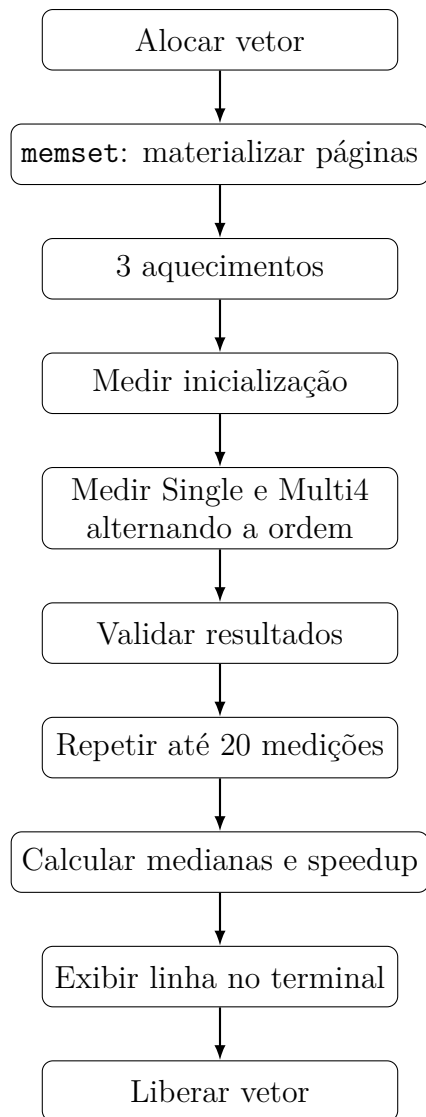


Figura 2.2: Fluxo de execução para cada tamanho de vetor.

2.9 Metodologia experimental

2.9.1 Ambiente de execução

O experimento foi executado em uma máquina com as características apresentadas na Tabela 2.2.

Tabela 2.2: Características do ambiente utilizado no experimento.

Componente	Configuração
Processador	Intel Core i5-10300H @ 2,50 GHz
Núcleos físicos	4
Processadores lógicos	8
L2 reportada pelo sistema	1024 KiB
L3 reportada pelo sistema	8192 KiB
Memória RAM	31,84 GiB
Memória configurada	2667 MHz
Sistema operacional	Windows 11 64 bits
Plano de energia	Alto desempenho
Compilador identificado	GCC 15.1.0
Tipo utilizado	int, 4 bytes

Durante a caracterização inicial, o compilador também reportou tamanho de linha de cache de 64 bytes e parâmetro de cache L1 de 32 KiB.

A execução final apresentada nesta seção foi realizada a partir de um terminal MSYS2/UCRT64.

Como o *target* específico do compilador não foi novamente consultado após a mudança para esse terminal, esse detalhe não é utilizado como base para nenhuma conclusão sobre os resultados.

2.9.2 Configuração das medições

As três versões foram compiladas com os comandos equivalentes a:

```
gcc -std=c11 -Wall -Wextra -O0 \  
-DOPT_LEVEL='"00"' benchmark.c -o benchmark_00.exe
```

```
gcc -std=c11 -Wall -Wextra -O2 \  
-DOPT_LEVEL='"02"' benchmark.c -o benchmark_02.exe
```

```
gcc -std=c11 -Wall -Wextra -O3 \  
-DOPT_LEVEL='"03"' benchmark.c -o benchmark_03.exe
```

Nenhuma flag adicional de arquitetura foi acrescentada à matriz principal.

Tabela 2.3: Parâmetros do protocolo de medição.

Parâmetro	Valor
Aquecimentos	3
Medições efetivas	20
Estatística reportada	Mediana
Relógio	CLOCK_MONOTONIC
Acumuladores da versão Multi4	4
Níveis de otimização	-O0, -O2, -O3
Quantidade de valores de N	7

Cada configuração utilizou:

O tempo medido contém somente o trecho correspondente ao laço de interesse.

Alocação de memória, liberação, impressão, ordenação das amostras, cálculo da mediana e validação dos resultados permanecem fora das regiões cronometradas.

Foi utilizado ainda um *compiler barrier* com efeito sobre memória em torno das regiões cronometradas, reduzindo a possibilidade de movimentação de operações através das fronteiras de medição pelo compilador.

Antes do início dos aquecimentos, um `memset` percorre a região alocada. O objetivo é materializar previamente as páginas utilizadas e reduzir a influência de faltas de página de primeiro acesso sobre a inicialização cronometrada.

2.9.3 Métrica de speedup

A comparação direta entre as duas formas de redução foi definida como

$$\text{Speedup}_{\text{Multi4}} = \frac{T_{\text{Single}}}{T_{\text{Multi4}}}. \quad (2.9)$$

Assim:

$$\text{Speedup} > 1 \quad (2.10)$$

indica vantagem da versão com quatro acumuladores, enquanto

$$\text{Speedup} < 1 \quad (2.11)$$

indica que a versão com acumulador único foi mais rápida.

2.10 Código-fonte desenvolvido

O código completo utilizado no experimento é apresentado na Listagem 2.1.

```
1 #define _POSIX_C_SOURCE 200809L
2
```

```

3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <stdint.h>
6  #include <stddef.h>
7  #include <time.h>
8  #include <string.h>
9
10 #ifndef OPT_LEVEL
11 #define OPT_LEVEL "unknown"
12 #endif
13
14 #define WARMUP_RUNS 3
15 #define MEASURE_RUNS 20
16 #define NUM_ACCUMULATORS 4
17
18 #if defined(__GNUC__)
19 #define NOINLINE __attribute__((noinline))
20 #define COMPILER_BARRIER() \
21     __asm__ __volatile__("" ::: "memory")
22 #else
23 #define NOINLINE
24 #define COMPILER_BARRIER() ((void)0)
25 #endif
26
27
28 /*
29  * Mantém os resultados das operações observáveis durante a
30  * execução e impede a eliminação dos cálculos pelo compilador.
31  */
32 static volatile int benchmark_sink = 0;
33
34
35 /*
36  * Define os tamanhos de vetor utilizados no experimento.
37  *
38  * Considerando sizeof(int) = 4 bytes:
39  *
40  *      32.768 elementos      = 128 KiB
41  *      65.536 elementos      = 256 KiB
42  *      262.144 elementos     = 1 MiB
43  *      1.048.576 elementos   = 4 MiB
44  *      2.097.152 elementos   = 8 MiB
45  *      4.194.304 elementos   = 16 MiB
46  *      16.777.216 elementos  = 64 MiB
47  */
48 static const size_t N_VALUES[] = {
49     32768,
50     65536,
51     262144,
52     1048576,
53     2097152,
54     4194304,
55     16777216
56 };
57
58 #define NUM_N_VALUES \
59     (sizeof(N_VALUES) / sizeof(N_VALUES[0]))
60
61

```

```

62  /* =====
63  * TEMPORIZAÇÃO
64  * ===== */
65
66  /*
67  * Calcula o intervalo entre dois instantes obtidos por
68  * clock_gettime() e retorna o tempo em milissegundos.
69  */
70  static double elapsed_ms(
71      const struct timespec *start,
72      const struct timespec *end
73  )
74  {
75      double seconds =
76          (double)(end->tv_sec - start->tv_sec);
77
78      double nanoseconds =
79          (double)(end->tv_nsec - start->tv_nsec);
80
81      return (seconds * 1000.0)
82          + (nanoseconds / 1000000.0);
83  }
84
85
86  /* =====
87  * INICIALIZAÇÃO DO VETOR
88  * ===== */
89
90  /*
91  * Inicializa os elementos do vetor com valores inteiros
92  * periódicos no intervalo de 1 a 8.
93  *
94  * Cada posição é escrita independentemente das demais.
95  */
96  static NOINLINE void initialize_vector(
97      int *vector,
98      size_t n
99  )
100  {
101      for (size_t i = 0; i < n; ++i) {
102          vector[i] = (int)((i & 7u) + 1u);
103      }
104  }
105
106
107  /* =====
108  * SOMA COM UM ACUMULADOR
109  * ===== */
110
111  /*
112  * Realiza a redução do vetor utilizando um único acumulador.
113  *
114  * A atualização de sum em cada iteração depende do resultado
115  * produzido na iteração anterior, formando uma única cadeia
116  * de dependência.
117  */
118  static NOINLINE int sum_single_accumulator(
119      const int *vector,
120      size_t n

```

```

121 )
122 {
123     int sum = 0;
124
125     for (size_t i = 0; i < n; ++i) {
126         sum += vector[i];
127     }
128
129     return sum;
130 }
131
132
133 /* =====
134  * SOMA COM QUATRO ACUMULADORES
135  * ===== */
136
137 /*
138  * Realiza a redução utilizando quatro acumuladores
139  * independentes.
140  *
141  * Os quatro acumuladores estabelecem cadeias de dependência
142  * distintas e aumentam o paralelismo em nível de instrução
143  * disponível durante a execução do laço.
144  */
145 static NOINLINE int sum_four_accumulators(
146     const int *vector,
147     size_t n
148 )
149 {
150     int sum0 = 0;
151     int sum1 = 0;
152     int sum2 = 0;
153     int sum3 = 0;
154
155     size_t i = 0;
156
157     for (; i + 3 < n; i += 4) {
158         sum0 += vector[i];
159         sum1 += vector[i + 1];
160         sum2 += vector[i + 2];
161         sum3 += vector[i + 3];
162     }
163
164     /*
165      * Processa elementos residuais quando n não é múltiplo
166      * de quatro.
167      */
168     for (; i < n; ++i) {
169         sum0 += vector[i];
170     }
171
172     return sum0 + sum1 + sum2 + sum3;
173 }
174
175
176 /* =====
177  * MEDIÇÃO DA INICIALIZAÇÃO
178  * ===== */
179

```

```

180  /*
181  * Mede exclusivamente o intervalo correspondente à
182  * inicialização do vetor.
183  */
184  static double measure_initialization(
185      int *vector,
186      size_t n
187  )
188  {
189      struct timespec start;
190      struct timespec end;
191
192      COMPILER_BARRIER();
193
194      if (clock_gettime(
195          CLOCK_MONOTONIC,
196          &start
197      ) != 0) {
198
199          perror("clock_gettime");
200          exit(EXIT_FAILURE);
201      }
202
203      COMPILER_BARRIER();
204
205      initialize_vector(vector, n);
206
207      COMPILER_BARRIER();
208
209      if (clock_gettime(
210          CLOCK_MONOTONIC,
211          &end
212      ) != 0) {
213
214          perror("clock_gettime");
215          exit(EXIT_FAILURE);
216      }
217
218      COMPILER_BARRIER();
219
220      return elapsed_ms(&start, &end);
221  }
222
223
224  /* =====
225  * MEDIÇÃO DA SOMA COM UM ACUMULADOR
226  * ===== */
227
228  /*
229  * Mede exclusivamente o tempo da redução realizada com um
230  * único acumulador.
231  */
232  static double measure_single(
233      const int *vector,
234      size_t n,
235      int *result
236  )
237  {
238      struct timespec start;

```

```

239     struct timespec end;
240
241     COMPILER_BARRIER();
242
243     if (clock_gettime(
244         CLOCK_MONOTONIC,
245         &start
246     ) != 0) {
247
248         perror("clock_gettime");
249         exit(EXIT_FAILURE);
250     }
251
252     COMPILER_BARRIER();
253
254     *result =
255         sum_single_accumulator(
256             vector,
257             n
258         );
259
260     COMPILER_BARRIER();
261
262     if (clock_gettime(
263         CLOCK_MONOTONIC,
264         &end
265     ) != 0) {
266
267         perror("clock_gettime");
268         exit(EXIT_FAILURE);
269     }
270
271     COMPILER_BARRIER();
272
273     return elapsed_ms(&start, &end);
274 }
275
276
277 /* =====
278  * MEDIÇÃO DA SOMA COM QUATRO ACUMULADORES
279  * ===== */
280
281 /*
282  * Mede exclusivamente o tempo da redução realizada com quatro
283  * acumuladores independentes.
284  */
285 static double measure_multi4(
286     const int *vector,
287     size_t n,
288     int *result
289 )
290 {
291     struct timespec start;
292     struct timespec end;
293
294     COMPILER_BARRIER();
295
296     if (clock_gettime(
297         CLOCK_MONOTONIC,

```

```

298         &start
299     ) != 0) {
300
301         perror("clock_gettime");
302         exit(EXIT_FAILURE);
303     }
304
305     COMPILER_BARRIER();
306
307     *result =
308         sum_four_accumulators(
309             vector,
310             n
311         );
312
313     COMPILER_BARRIER();
314
315     if (clock_gettime(
316         CLOCK_MONOTONIC,
317         &end
318     ) != 0) {
319
320         perror("clock_gettime");
321         exit(EXIT_FAILURE);
322     }
323
324     COMPILER_BARRIER();
325
326     return elapsed_ms(&start, &end);
327 }
328
329
330 /* =====
331  * ESTATÍSTICA
332  * ===== */
333
334 /*
335  * Implementa a comparação utilizada por qsort() para ordenar
336  * os tempos armazenados como valores do tipo double.
337  */
338 static int compare_double(
339     const void *a,
340     const void *b
341 )
342 {
343     double x = *(const double *)a;
344     double y = *(const double *)b;
345
346     if (x < y) {
347         return -1;
348     }
349
350     if (x > y) {
351         return 1;
352     }
353
354     return 0;
355 }
356

```

```

357
358 /*
359  * Calcula a mediana dos valores fornecidos.
360  */
361 static double median(
362     double values[],
363     size_t count
364 )
365 {
366     qsort(
367         values,
368         count,
369         sizeof(double),
370         compare_double
371     );
372
373     if ((count % 2) == 0) {
374
375         return (
376             values[(count / 2) - 1]
377             +
378             values[count / 2]
379         ) / 2.0;
380     }
381
382     return values[count / 2];
383 }
384
385
386 /* =====
387  * VALIDAÇÃO DO RESULTADO
388  * ===== */
389
390 /*
391  * Calcula o resultado esperado para o padrão utilizado na
392  * inicialização.
393  *
394  * A sequência periódica
395  *
396  *   1 2 3 4 5 6 7 8
397  *
398  * possui soma igual a 36 para cada bloco de oito elementos.
399  */
400 static int expected_sum(size_t n)
401 {
402     long long value =
403         (long long)(n / 8u) * 36LL;
404
405     if (value > INT32_MAX) {
406
407         fprintf(
408             stderr,
409             "Erro: soma esperada excede "
410             "o limite de int.\n"
411         );
412
413         exit(EXIT_FAILURE);
414     }
415

```

```

416     return (int)value;
417 }
418
419
420 /* =====
421  * BENCHMARK PARA UM VALOR DE N
422  * ===== */
423
424 /*
425  * Executa o procedimento experimental para um tamanho
426  * especifico de vetor.
427  */
428 static int run_for_n(size_t n)
429 {
430     size_t bytes =
431         n * sizeof(int);
432
433     int *vector =
434         malloc(bytes);
435
436     if (vector == NULL) {
437
438         fprintf(
439             stderr,
440             "Erro: nao foi possivel "
441             "alocar %zu bytes.\n",
442             bytes
443         );
444
445         return 0;
446     }
447
448
449     /*
450     * Materializa previamente as páginas de memória associadas
451     * ao bloco alocado antes do início das medições.
452     */
453     memset(
454         vector,
455         0,
456         bytes
457     );
458
459     COMPILER_BARRIER();
460
461
462     /*
463     * Executa as etapas de aquecimento.
464     *
465     * Os tempos dessas execuções não são incluídos no resultado
466     * final do benchmark.
467     */
468     for (
469         int w = 0;
470         w < WARMUP_RUNS;
471         ++w
472     ) {
473
474         initialize_vector(

```

```

475         vector,
476         n
477     );
478
479     int warm_single =
480         sum_single_accumulator(
481             vector,
482             n
483         );
484
485     int warm_multi =
486         sum_four_accumulators(
487             vector,
488             n
489         );
490
491     if (warm_single != warm_multi) {
492
493         fprintf(
494             stderr,
495             "Erro durante warm-up "
496             "para N=%zu.\n",
497             n
498         );
499
500         free(vector);
501
502         return 0;
503     }
504
505     benchmark_sink =
506         warm_single;
507
508     benchmark_sink =
509         warm_multi;
510 }
511
512
513 double initialization_times[MEASURE_RUNS];
514 double single_times[MEASURE_RUNS];
515 double multi_times[MEASURE_RUNS];
516
517 int last_single = 0;
518 int last_multi = 0;
519
520
521 /*
522  * Executa as medições válidas.
523  *
524  * A ordem entre Single e Multi4 é alternada a cada
525  * repetição para reduzir possíveis vantagens sistemáticas
526  * relacionadas ao estado da hierarquia de memória.
527  */
528 for (
529     int r = 0;
530     r < MEASURE_RUNS;
531     ++r
532 ) {
533

```

```

534     initialization_times[r] =
535         measure_initialization(
536             vector,
537             n
538         );
539
540     int single_result = 0;
541     int multi_result = 0;
542
543
544     if ((r % 2) == 0) {
545
546         single_times[r] =
547             measure_single(
548                 vector,
549                 n,
550                 &single_result
551             );
552
553         benchmark_sink =
554             single_result;
555
556
557         multi_times[r] =
558             measure_multi4(
559                 vector,
560                 n,
561                 &multi_result
562             );
563
564         benchmark_sink =
565             multi_result;
566
567     } else {
568
569         multi_times[r] =
570             measure_multi4(
571                 vector,
572                 n,
573                 &multi_result
574             );
575
576         benchmark_sink =
577             multi_result;
578
579
580         single_times[r] =
581             measure_single(
582                 vector,
583                 n,
584                 &single_result
585             );
586
587         benchmark_sink =
588             single_result;
589     }
590
591
592     /*

```

```

593     * Valida a equivalência entre os resultados produzidos
594     * pelas duas implementações.
595     */
596     if (single_result != multi_result) {
597
598         fprintf(
599             stderr,
600             "Erro: resultados diferentes "
601             "para N=%zu "
602             "(single=%d, multi=%d).\n",
603             n,
604             single_result,
605             multi_result
606         );
607
608         free(vector);
609
610         return 0;
611     }
612
613     last_single =
614         single_result;
615
616     last_multi =
617         multi_result;
618 }
619
620
621 /*
622  * Valida o resultado final em relação ao valor esperado.
623  */
624 int expected =
625     expected_sum(n);
626
627 if (
628     last_single != expected
629     ||
630     last_multi != expected
631 ) {
632
633     fprintf(
634         stderr,
635         "Erro de validacao para N=%zu. "
636         "Esperado=%d, single=%d, multi=%d\n",
637         n,
638         expected,
639         last_single,
640         last_multi
641     );
642
643     free(vector);
644
645     return 0;
646 }
647
648
649 /*
650  * Calcula as medianas das medições realizadas.
651  */

```

```

652     double init_median =
653         median(
654             initialization_times,
655             MEASURE_RUNS
656         );
657
658     double single_median =
659         median(
660             single_times,
661             MEASURE_RUNS
662         );
663
664     double multi_median =
665         median(
666             multi_times,
667             MEASURE_RUNS
668         );
669
670
671     /*
672     * Calcula o speedup da implementação Multi4 em relação
673     * à implementação Single.
674     */
675     double speedup = 0.0;
676
677     if (multi_median > 0.0) {
678
679         speedup =
680             single_median
681             /
682             multi_median;
683     }
684
685
686     /*
687     * Converte o tamanho do vetor para MiB.
688     */
689     double size_mib =
690         (double)bytes
691         /
692         (1024.0 * 1024.0);
693
694
695     printf(
696         "%-11zu %10.3f %12.6f "
697         "%14.6f %14.6f %9.3fx\n",
698         n,
699         size_mib,
700         init_median,
701         single_median,
702         multi_median,
703         speedup
704     );
705
706
707     free(vector);
708
709     return 1;
710 }

```

```

711
712
713 /* =====
714  * PROGRAMA PRINCIPAL
715  * ===== */
716
717 int main(void)
718 {
719     /*
720     * Apresenta as configurações gerais do benchmark.
721     */
722     printf("\n");
723     printf("ILP Benchmark\n");
724
725     printf(
726         "Optimization : %s\n",
727         OPT_LEVEL
728     );
729
730     printf(
731         "Data type      : int (%zu bytes)\n",
732         sizeof(int)
733     );
734
735     printf(
736         "Clock           : CLOCK_MONOTONIC\n"
737     );
738
739     printf(
740         "Warm-ups        : %d\n",
741         WARMUP_RUNS
742     );
743
744     printf(
745         "Measurements    : %d\n",
746         MEASURE_RUNS
747     );
748
749     printf(
750         "Accumulators    : %d\n",
751         NUM_ACCUMULATORS
752     );
753
754     printf("\n");
755
756
757     /*
758     * Apresenta o cabeçalho da tabela de resultados.
759     */
760     printf(
761         "%-11s %10s %12s "
762         "%14s %14s %9s\n",
763         "N",
764         "Size(MiB)",
765         "Init(ms)",
766         "Single(ms)",
767         "Multi4(ms)",
768         "Speedup"
769     );

```

```

770
771     printf(
772         "-----"
773         "-----"
774         "-----\n"
775     );
776
777
778     /*
779     * Executa o benchmark para todos os tamanhos definidos.
780     */
781     for (
782         size_t i = 0;
783         i < NUM_N_VALUES;
784         ++i
785     ) {
786
787         if (!run_for_n(N_VALUES[i])) {
788
789             printf(
790                 "\nStatus: ERROR\n"
791             );
792
793             return EXIT_FAILURE;
794         }
795     }
796
797
798     printf(
799         "-----"
800         "-----"
801         "-----\n"
802     );
803
804     printf("Status: OK\n");
805
806     printf(
807         "Sink: %d\n",
808         benchmark_sink
809     );
810
811     printf("\n");
812
813     return EXIT_SUCCESS;
814 }

```

Listing 2.1: Código-fonte da Atividade 2.

2.11 Validação da implementação

A validação foi realizada em dois níveis.

Primeiramente, após cada execução das reduções, os resultados de `sum_single_accumulator()` e `sum_four_accumulators()` foram comparados diretamente.

Em seguida, o valor foi confrontado com a soma analiticamente esperada:

$$S_{\text{esperado}} = \frac{N}{8} \cdot 36. \quad (2.12)$$

Para o maior tamanho,

$$N = 16\,777\,216, \quad (2.13)$$

$$S_{\text{esperado}} = \frac{16\,777\,216}{8} \cdot 36 \quad (2.14)$$

$$= 75\,497\,472. \quad (2.15)$$

As três execuções terminaram com

Status: OK

Sink: 75497472

confirmando que as duas implementações produziram o mesmo resultado para todos os tamanhos testados e que o maior caso produziu o valor esperado.

A variável `benchmark_sink`, declarada como `volatile`, também assegura um efeito observável para os resultados das reduções, reduzindo o risco de eliminação completa do trabalho pelo compilador.

2.12 Resultados experimentais

2.12.1 Resultados com -O0

A Tabela 2.4 apresenta os resultados obtidos sem otimização.

Tabela 2.4: Resultados obtidos com -O0.

N	Init (ms)	Single (ms)	Multi4 (ms)	Speedup
32 768	0,058600	0,063750	0,023450	2,719
65 536	0,116500	0,127150	0,047600	2,671
262 144	0,465550	0,507800	0,187250	2,712
1 048 576	1,866300	2,095650	0,831800	2,519
2 097 152	3,760450	4,262400	1,716550	2,483
4 194 304	8,255650	9,086550	4,330800	2,098
16 777 216	30,736250	34,333000	14,028250	2,447

A versão Multi4 foi mais rápida em todos os tamanhos, apresentando speedup entre $2,098\times$ e $2,719\times$.

2.12.2 Resultados com -O2

Os resultados com -O2 são apresentados na Tabela 2.5.

Tabela 2.5: Resultados obtidos com -O2.

N	Init (ms)	Single (ms)	Multi4 (ms)	Speedup
32 768	0,018800	0,016400	0,005500	2,982
65 536	0,024300	0,020100	0,006600	3,045
262 144	0,094200	0,080650	0,027350	2,949
1 048 576	0,397600	0,351900	0,135200	2,603
2 097 152	0,819450	0,810600	0,455900	1,778
4 194 304	1,655500	1,678100	0,917700	1,829
16 777 216	7,108050	7,179200	4,652300	1,543

O maior speedup de toda a matriz foi observado em

$$N = 65\,536, \quad (2.16)$$

com

$$\text{Speedup} = 3,045 \times . \quad (2.17)$$

Entretanto, a vantagem relativa diminuiu para os vetores maiores.

2.12.3 Resultados com -O3

A Tabela 2.6 apresenta os resultados obtidos com otimização -O3.

Tabela 2.6: Resultados obtidos com -O3.

N	Init (ms)	Single (ms)	Multi4 (ms)	Speedup
32 768	0,005000	0,002600	0,003300	0,788
65 536	0,009900	0,005600	0,006800	0,824
262 144	0,038500	0,025150	0,028550	0,881
1 048 576	0,168300	0,179950	0,198800	0,905
2 097 152	0,651300	0,545700	0,606050	0,900
4 194 304	1,380250	1,014250	1,043000	0,972
16 777 216	5,765100	4,054550	4,271200	0,949

O comportamento foi qualitativamente diferente dos níveis anteriores. Todos os valores de speedup ficaram abaixo de 1, indicando que a implementação Single foi mais rápida que Multi4 em todos os tamanhos testados com -O3.

2.13 Visualização dos resultados

A Figura 2.3 reúne os speedups das três compilações.

A inversão observada em -O3 torna-se particularmente evidente: enquanto -O0 e -O2 permanecem acima da linha de igualdade, todos os pontos de -O3 permanecem abaixo dela.

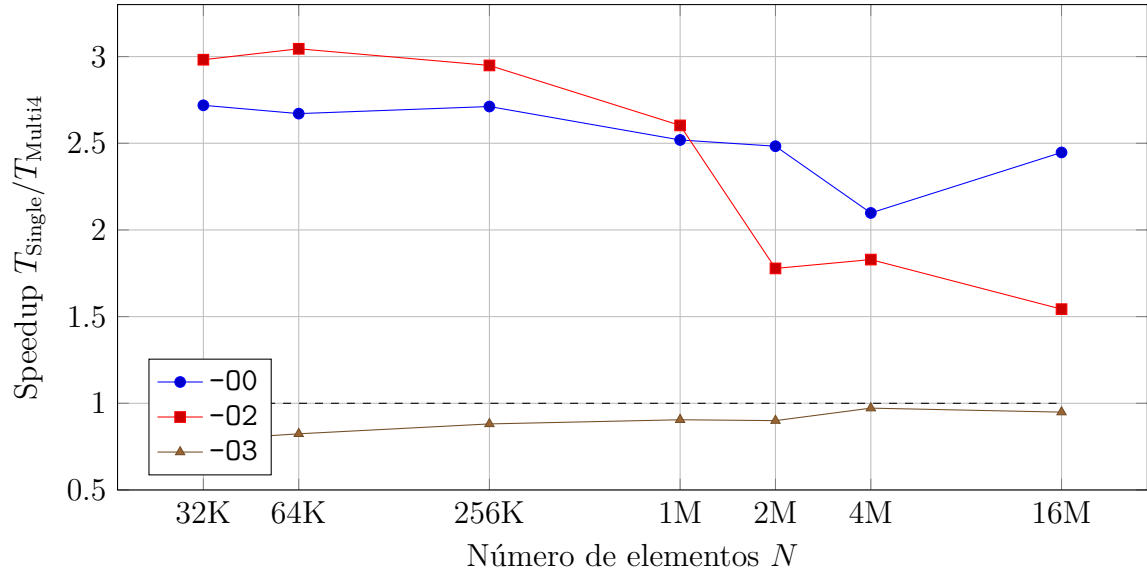


Figura 2.3: Speedup da implementação com quatro acumuladores em relação ao acumulador único. A linha tracejada em $1\times$ representa igualdade de desempenho.

A Figura 2.4 compara os tempos das duas reduções para o maior vetor.

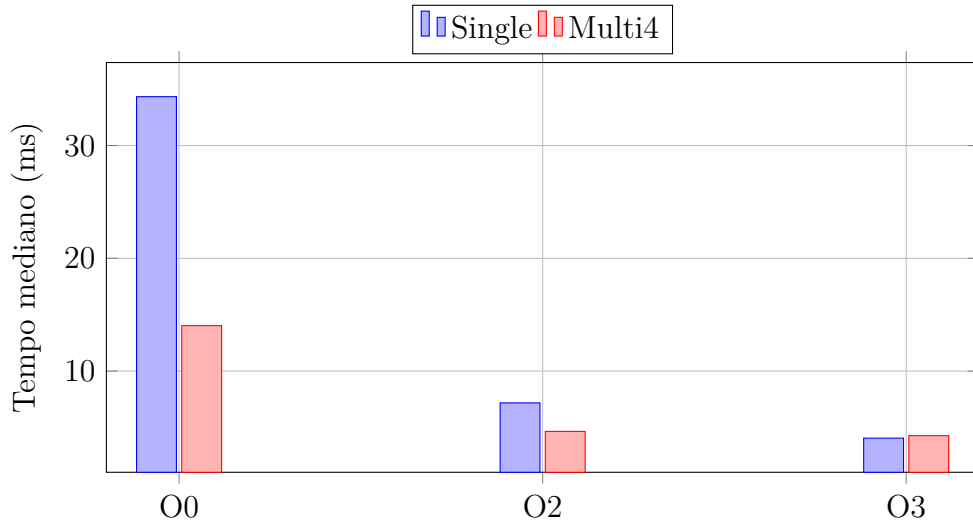


Figura 2.4: Tempos das duas reduções para $N = 16\,777\,216$ elementos (64 MiB).

2.14 Análise e discussão

2.14.1 Comportamento em -00

Os resultados de -00 apresentam de forma particularmente clara o efeito da reestruturação manual.

A implementação com um acumulador obteve, para o primeiro tamanho,

$$T_{\text{Single}} = 0,063750 \text{ ms}, \quad (2.18)$$

enquanto Multi4 obteve

$$T_{\text{Multi4}} = 0,023450 \text{ ms}, \quad (2.19)$$

produzindo speedup de $2,719\times$.

A vantagem permaneceu superior a $2\times$ em todos os tamanhos.

Esse resultado é compatível com a fundamentação de ILP: a versão Single apresenta uma cadeia de dependência contínua, enquanto Multi4 disponibiliza quatro cadeias independentes.

Em -00, com menor intervenção do compilador sobre a estrutura original, a vantagem da reorganização explícita torna-se especialmente visível.

2.14.2 Comportamento em -02

Com -02, Multi4 também foi mais rápida em todos os casos.

Para $N = 65\,536$, foram medidos:

$$T_{\text{Single}} = 0,020100 \text{ ms}, \quad (2.20)$$

$$T_{\text{Multi4}} = 0,006600 \text{ ms}, \quad (2.21)$$

correspondendo a $3,045\times$.

Nos três menores conjuntos de dados, o speedup permaneceu próximo de $3\times$.

Entretanto, esse valor caiu progressivamente nos conjuntos maiores, chegando a $1,543\times$ para 64 MiB.

A redução da vantagem relativa é coerente com uma maior influência da hierarquia de memória à medida que o *working set* cresce.

No entanto, como não foram coletados contadores de hardware de *cache miss*, largura de banda ou latência de memória, essa associação deve ser interpretada como consistente com os resultados, e não como uma demonstração causal isolada.

2.14.3 Inversão observada em -03

O resultado mais importante da atividade ocorreu com -03.

Ao contrário de -00 e -02, a versão com quatro acumuladores não apresentou vantagem em nenhum tamanho.

Para $N = 32\,768$,

$$T_{\text{Single}} = 0,002600 \text{ ms}, \quad (2.22)$$

$$T_{\text{Multi4}} = 0,003300 \text{ ms}, \quad (2.23)$$

resultando em speedup de apenas $0,788\times$.

Mesmo para o maior vetor,

$$T_{\text{Single}} = 4,054550 \text{ ms}, \quad (2.24)$$

$$T_{\text{Multi4}} = 4,271200 \text{ ms}, \quad (2.25)$$

com speedup de $0,949\times$.

Portanto, a estrutura manualmente otimizada que apresentava vantagem de aproximadamente duas a três vezes nos outros níveis deixou de ser a versão mais rápida quando foi utilizado -03.

2.14.4 Efeito do compilador sobre a versão Single

O maior vetor permite visualizar a magnitude das transformações associadas ao nível de otimização.

Para Single:

$$T_{\text{O0}} = 34,333000 \text{ ms}, \quad (2.26)$$

$$T_{\text{O2}} = 7,179200 \text{ ms}, \quad (2.27)$$

$$T_{\text{O3}} = 4,054550 \text{ ms}. \quad (2.28)$$

A razão entre -00 e -03 é aproximadamente

$$\frac{34,333000}{4,054550} \approx 8,47. \quad (2.29)$$

Assim, para esse caso, a versão Single em -03 executou em aproximadamente $1/8,47$ do tempo registrado em -00.

Para Multi4:

$$\frac{14,028250}{4,271200} \approx 3,28. \quad (2.30)$$

A versão manualmente reestruturada ainda se beneficiou de -03, porém em proporção consideravelmente menor.

2.14.5 Interpretação da inversão

Os resultados são compatíveis com a hipótese de que, em -O3, o compilador tenha conseguido transformar agressivamente o laço Single, reduzindo a limitação representada pela estrutura aparente do código-fonte.

Transformações possíveis incluem vetorização da redução, desenrolamento do laço, uso de somas parciais internas e escalonamento de instruções.

Nesse cenário, a versão Single escrita no código-fonte não corresponde necessariamente a uma única cadeia escalar de acumulação no código efetivamente executado.

Ao mesmo tempo, Multi4 já fornece explicitamente múltiplas cadeias, podendo existir menos oportunidade para ganho adicional, além de maior complexidade estrutural e eventual pressão sobre registradores.

Entretanto, os tempos medidos isoladamente não permitem afirmar qual transformação específica foi aplicada.

Como os relatórios finais de vetorização e o *assembly* das três funções não foram utilizados nesta etapa da análise, a vetorização automática deve permanecer apresentada como uma hipótese fortemente compatível com o comportamento observado, e não como fato experimentalmente confirmado.

2.14.6 Inicialização

O primeiro laço também respondeu fortemente aos níveis de otimização.

Para o vetor de 64 MiB:

$$T_{\text{Init},-\text{O0}} = 30,736250 \text{ ms}, \quad (2.31)$$

$$T_{\text{Init},-\text{O2}} = 7,108050 \text{ ms}, \quad (2.32)$$

$$T_{\text{Init},-\text{O3}} = 5,765100 \text{ ms}. \quad (2.33)$$

A razão entre os extremos é

$$\frac{30,736250}{5,765100} \approx 5,33. \quad (2.34)$$

Como as iterações da inicialização escrevem posições distintas do vetor, não existe a mesma dependência de redução presente no segundo laço. Essa estrutura oferece naturalmente mais liberdade ao compilador para otimizações.

2.14.7 Síntese dos comportamentos

Os três níveis produziram padrões claramente distintos:

Configuração	Comportamento observado
-00	Multi4 entre $2,098\times$ e $2,719\times$
-02	Multi4 entre $1,543\times$ e $3,045\times$
-03	Single vence; speedup Multi4 entre $0,788\times$ e $0,972\times$

Portanto, uma otimização manual benéfica em níveis menos agressivos não necessariamente permanece superior após transformações mais avançadas realizadas pelo compilador.

2.15 Limitações

O experimento foi projetado para reduzir fontes simples de ruído, mas permanece sujeito a limitações características de microbenchmarks executados em um sistema operacional de uso geral.

A execução ocorreu no Windows 11, que continua realizando escalonamento, interrupções e tarefas em segundo plano durante as medições.

O processador utilizado é um processador móvel. Sua frequência efetiva pode variar devido a mecanismos de gerenciamento energético, *turbo* e condições térmicas. O plano de energia utilizado foi Alto desempenho, porém isso não garante frequência absolutamente constante.

Não foi realizada afinidade explícita da execução com um núcleo específico.

Também não foram coletados contadores de desempenho de hardware. Consequentemente, as associações entre mudanças de comportamento e níveis da hierarquia de cache não podem ser tratadas como medições diretas de *cache misses* ou largura de banda.

As funções dos laços foram marcadas como `noinline`. Isso facilita o isolamento e a inspeção conceitual, mas mantém um custo de chamada que representa parcela proporcionalmente maior nos casos mais curtos.

Esse ponto merece atenção especialmente em -03, onde o menor tempo medido foi de

$$0,002600 \text{ ms} = 2,6 \mu\text{s}. \quad (2.35)$$

Por isso, as conclusões principais não dependem exclusivamente dos dois menores tamanhos de vetor.

Por fim, os resultados permitem observar o efeito global do nível de otimização, mas não identificam isoladamente quais passes do compilador produziram cada melhoria. Uma análise posterior com relatório de vetorização e inspeção de *assembly* seria necessária para estabelecer essa relação de forma direta.

2.16 Conclusões da atividade

A atividade demonstrou experimentalmente que dependências entre instruções possuem impacto direto sobre o desempenho de laços e que a reorganização do código pode expor paralelismo em nível de instrução.

Com -00, a utilização de quatro acumuladores produziu speedups entre $2,098\times$ e $2,719\times$, mostrando de forma clara o benefício da quebra da cadeia de dependência.

Com -02, a vantagem atingiu até $3,045\times$, embora tenha diminuído à medida que o tamanho do vetor cresceu.

O resultado mais relevante ocorreu em -03. Nesse nível, a versão originalmente dependente tornou-se mais rápida que a implementação manual com quatro acumuladores em todos os tamanhos testados.

Esse comportamento evidencia que a estrutura do código-fonte não deve ser analisada isoladamente do compilador. Um laço aparentemente serial no código C pode ser profundamente transformado durante a otimização, enquanto uma transformação manual anteriormente vantajosa pode tornar-se redundante ou menos eficiente.

A atividade também demonstrou a importância de uma metodologia experimental controlada. Variação de N , aquecimentos, múltiplas medições, mediana, alternância da ordem das implementações, validação dos resultados e delimitação cuidadosa da região cronometrada foram necessários para tornar a comparação mais robusta.

Assim, o principal resultado da atividade não é simplesmente que Multi4 ou Single seja a melhor implementação. A conclusão é que o desempenho emerge da interação entre dependências do algoritmo, exposição de ILP, hierarquia de memória e transformações realizadas pelo compilador.

2.17 Síntese

Para uma defesa oral, os principais pontos da atividade podem ser sintetizados da seguinte forma:

1. **O que é ILP?** É a possibilidade de executar ou sobrepor instruções independentes dentro de um único fluxo de execução. Não é paralelismo por *threads*.
2. **Qual é o problema do acumulador único?** A instrução `sum += vector[i]` cria uma dependência RAW entre iterações. A próxima soma depende do resultado da anterior.
3. **Por que quatro acumuladores ajudam?** Eles criam quatro cadeias independentes de dependência, disponibilizando mais operações simultaneamente para o processador.

4. **Como o experimento foi medido?** Foram utilizados sete tamanhos de vetor, três aquecimentos, vinte medições por configuração, mediana e `CLOCK_MONOTONIC`.
5. **Por que variar N ?** Para observar a execução em diferentes tamanhos de conjunto de dados e a mudança de importância relativa entre computação e hierarquia de memória.
6. **O que aconteceu em `-O0`?** Multi4 foi de aproximadamente $2,1\times$ a $2,7\times$ mais rápida em termos da métrica de speedup utilizada.
7. **O que aconteceu em `-O2`?** Multi4 continuou superior, chegando a $3,045\times$, mas a vantagem diminuiu nos maiores vetores.
8. **O que aconteceu em `-O3`?** O comportamento se inverteu. Single foi mais rápida em todos os tamanhos e os speedups de Multi4 ficaram abaixo de $1\times$.
9. **Por que `-O3` pode inverter o resultado?** Os dados são compatíveis com transformações agressivas realizadas pelo compilador, como vetorização, desenrolamento ou criação interna de somas parciais. Entretanto, o tempo sozinho não identifica qual transformação ocorreu.
10. **Qual é a principal conclusão?** Uma otimização manual não deve ser julgada apenas pelo código-fonte. O resultado final depende da interação entre dependências, arquitetura, hierarquia de memória e código efetivamente produzido pelo compilador.
11. **Como a correção foi validada?** Single e Multi4 foram comparadas entre si e com uma soma esperada calculada analiticamente. Todas as execuções terminaram com `Status: OK`.
12. **Qual seria o próximo passo para aprofundar a análise?** Gerar os relatórios de vetorização do GCC e comparar o *assembly* de Single e Multi4 em `-O0`, `-O2` e `-O3`, permitindo associar diretamente os tempos às transformações do compilador.

Capítulo 3

Atividade 3 – Processos, threads, concorrência e compartilhamento de memória

3.1 Enunciado da tarefa

Analise e execute os códigos disponibilizados e explique o comportamento observado nos valores de saída da variável `contador`.

Discuta o que acontece em cada código, identificando possíveis problemas, causas dos comportamentos observados e uma solução adequada para cada caso.

Em seguida, discuta os conceitos de processo e thread, destacando as principais diferenças entre eles, especialmente em relação a compartilhamento de memória e execução concorrente.

3.2 Objetivos e requisitos

A atividade teve como objetivo comparar, por meio de dois programas curtos em C, o comportamento de uma variável global quando a execução concorrente é realizada com processos e quando é realizada com threads.

O elemento observado nos dois casos foi a variável global `contador`. Em ambos os programas, são criadas quatro unidades de execução, e cada uma realiza um milhão de incrementos.

Os objetivos específicos considerados foram:

- analisar o comportamento de `contador` quando processos filhos são criados com `fork()`;

- analisar o comportamento de `contador` quando threads são criadas com `pthread_create()`;
- explicar por que os incrementos realizados pelos processos filhos não alteram o contador do processo pai;
- explicar por que as threads acessam a mesma variável global;
- identificar a condição de corrida existente no programa com threads;
- distinguir isolamento de memória, compartilhamento de memória e execução concorrente;
- propor soluções adequadas para cada caso, sem alterar a finalidade didática dos códigos fornecidos.

Os códigos analisados foram mantidos como programas demonstrativos. Assim, o objetivo principal não foi otimizar desempenho, mas compreender o modelo de memória associado a processos e threads.

3.3 Fundamentação conceitual

3.3.1 Processos

Um processo é uma instância de um programa em execução. Cada processo possui seu próprio espaço de endereçamento virtual, contendo, entre outras regiões:

- segmento de código;
- variáveis globais;
- heap;
- pilha de execução;
- descritores de arquivos e demais recursos associados pelo sistema operacional.

Quando um processo é criado por meio de `fork()`, o processo filho inicia como uma cópia lógica do processo pai.

Essa cópia inclui o valor das variáveis no instante da criação do filho. Entretanto, pai e filho não passam a compartilhar automaticamente as mesmas posições de memória. Após o `fork()`, cada processo executa em seu próprio espaço de endereçamento.

Consequentemente, se um processo filho modifica uma variável global, essa modificação ocorre em sua própria cópia da variável. O processo pai não vê essa alteração.

3.3.2 Threads

Uma thread é uma linha de execução dentro de um processo.

Diferentemente dos processos, threads pertencentes ao mesmo processo compartilham o mesmo espaço de endereçamento. Isso significa que variáveis globais, memória alocada dinamicamente e outros dados do processo podem ser acessados por todas as threads.

Cada thread, porém, possui elementos próprios, como:

- contador de programa;
- registradores;
- pilha de execução;
- estado de escalonamento.

Portanto, threads facilitam a comunicação por memória compartilhada, mas exigem cuidado quando múltiplas threads leem e escrevem os mesmos dados.

3.3.3 Concorrência

Concorrência ocorre quando duas ou mais unidades de execução possuem progresso sobreposto no tempo.

Em uma máquina com múltiplos núcleos, esse progresso pode ser efetivamente paralelo. Em uma máquina com um único núcleo, o sistema operacional pode alternar rapidamente entre unidades de execução, produzindo interleaving.

Nos dois programas desta atividade, a concorrência aparece de formas distintas:

- no programa com processos, existem quatro processos filhos executando laços de incremento independentes;
- no programa com threads, existem quatro threads executando laços de incremento sobre a mesma variável global.

3.3.4 Condição de corrida

Uma condição de corrida ocorre quando o resultado de um programa depende da ordem não controlada em que diferentes unidades de execução acessam dados compartilhados.

A operação

```
contador++;
```

parece uma única operação no código-fonte, mas conceitualmente envolve três etapas:

1. ler o valor atual de `contador`;
2. somar uma unidade;
3. gravar o novo valor em `contador`.

Se duas threads executarem essas etapas simultaneamente, ambas podem ler o mesmo valor antigo e escrever resultados que sobrescrevem atualizações feitas pela outra thread.

Esse fenômeno é chamado de atualização perdida.

3.4 Interpretação didática da tarefa

A atividade utiliza deliberadamente dois programas pequenos para isolar o conceito observado.

No primeiro programa, o foco é mostrar que a criação de processos não implica compartilhamento automático de variáveis globais. Embora os filhos executem o mesmo código e incrementem uma variável chamada `contador`, cada um manipula uma cópia privada.

No segundo programa, o foco é mostrar o caso oposto: threads compartilham a variável global, mas esse compartilhamento não garante correção. Como não há sincronização, as atualizações concorrentes sobre `contador` podem ser perdidas.

A atividade, portanto, não deve ser interpretada como um benchmark de desempenho. O ponto central é semântico:

Mecanismo	Memória	Efeito sobre <code>contador</code>
Processos com <code>fork()</code>	Espaços separados	Cada filho incrementa sua própria cópia.
Threads POSIX	Espaço compartilhado	Todas acessam a mesma variável, mas sem sincronização.

Essa diferença é uma base importante para Computação de Alto Desempenho, pois influencia diretamente a forma de decompor problemas, compartilhar dados e preservar a correção.

3.5 Modelagem da solução

3.5.1 Estrutura geral

A atividade foi organizada em dois programas independentes.

O primeiro programa, apresentado no Código 3.1, utiliza processos. A Figura 3.1 apresenta essa modelagem em forma de diagrama UML de componentes.

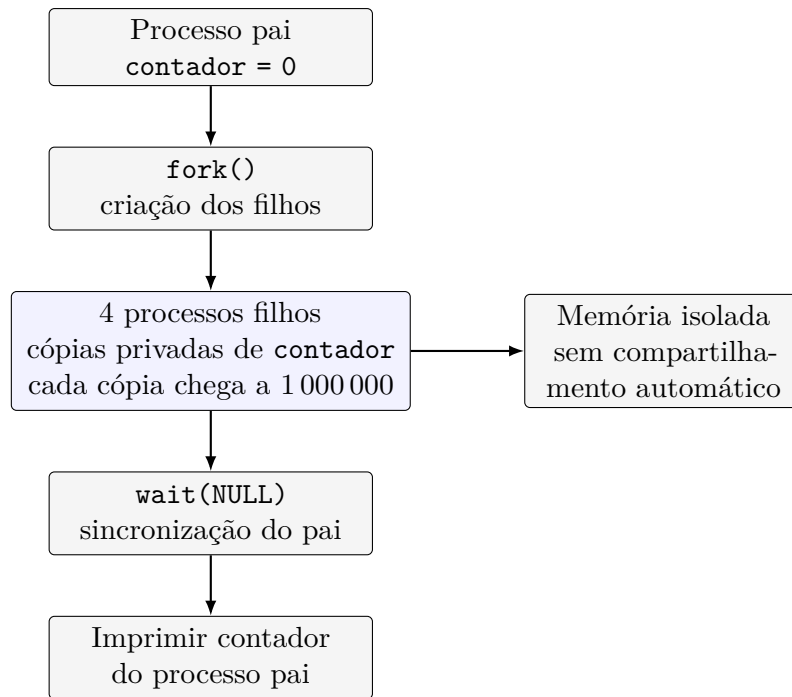


Figura 3.1: Modelagem UML do programa baseado em processos.

O segundo programa, apresentado no Código 3.2, utiliza threads. A Figura 3.2 mostra a modelagem correspondente.

Em ambos os casos, as constantes utilizadas são:

- quatro unidades de execução;
- um milhão de incrementos por unidade.

Assim, se todos os incrementos fossem acumulados em um único contador compartilhado e corretamente sincronizado, o valor esperado seria:

$$4 \times 1\,000\,000 = 4\,000\,000. \quad (3.1)$$

Esse valor, entretanto, não é o resultado esperado do programa com processos, pois nesse caso o contador não é compartilhado entre pai e filhos.

3.5.2 Principais funções e componentes

Os principais componentes dos programas são apresentados na Tabela 3.1.

3.6 Decisões de projeto e trade-offs

3.6.1 Uso de variável global

A variável `contador` foi declarada globalmente nos dois programas.

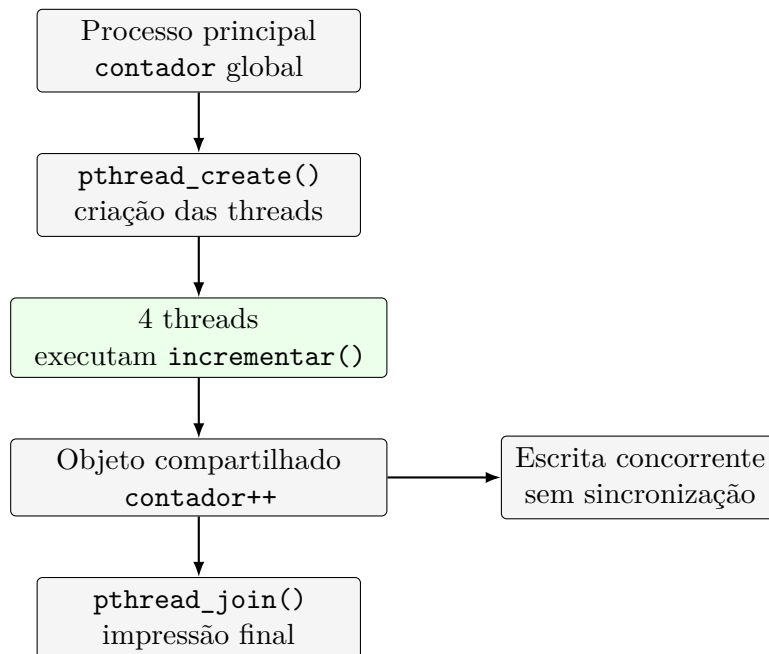


Figura 3.2: Modelagem UML do programa baseado em threads.

Tabela 3.1: Principais componentes analisados na Atividade 3.

Componente	Responsabilidade
contador	Variável global usada para evidenciar compartilhamento ou isolamento de memória.
fork()	Cria processos filhos no primeiro programa.
wait(NULL)	Faz o processo pai aguardar a finalização dos filhos.
getpid()	Identifica o processo filho que imprimiu determinado resultado.
pthread_create()	Cria threads no segundo programa.
incrementar()	Rotina executada por cada thread, responsável por incrementar contador .
pthread_join()	Faz a thread principal aguardar a finalização das threads criadas.

Essa escolha torna o experimento mais direto: a mesma forma de declaração produz comportamentos diferentes dependendo do modelo de execução adotado.

No programa com processos, a variável global pertence ao espaço de memória de cada processo.

No programa com threads, a variável global pertence ao processo e é compartilhada pelas threads.

3.6.2 Quatro unidades de execução

O uso de quatro processos e quatro threads facilita a comparação entre os programas.

Em ambos os casos, há quatro unidades concorrentes executando o mesmo trabalho lógico.

Com isso, a diferença observada não está na quantidade de trabalho solicitada, mas na forma como a memória é tratada.

3.6.3 Um milhão de incrementos

O valor de `INCREMENTOS` foi definido como um milhão.

Esse número é suficientemente grande para tornar perceptível o efeito da condição de corrida no programa com threads. Com poucas iterações, uma execução poderia ocasionalmente produzir um resultado próximo do esperado, mas isso não demonstraria de forma clara o problema.

3.6.4 Ausência de sincronização no programa com threads

O programa com threads não utiliza `mutex`, `atomic` ou outra forma de sincronização.

Essa decisão preserva a finalidade didática do código: evidenciar que o compartilhamento de memória exige controle explícito quando há escrita concorrente.

Como trade-off, o resultado final do contador deixa de ser determinístico e não pode ser tratado como correto.

3.6.5 Ausência de memória compartilhada explícita no programa com processos

O programa com processos também não utiliza memória compartilhada, *pipes*, arquivos temporários ou outro mecanismo de comunicação entre processos.

Essa decisão isola o comportamento padrão de `fork()`: cada processo filho possui sua própria cópia da variável global.

Se o objetivo fosse somar os incrementos dos filhos em um resultado comum, seria necessário acrescentar algum mecanismo de comunicação ou compartilhamento.

3.7 Representação estrutural da implementação

A Figura 3.3 resume o caso com processos.

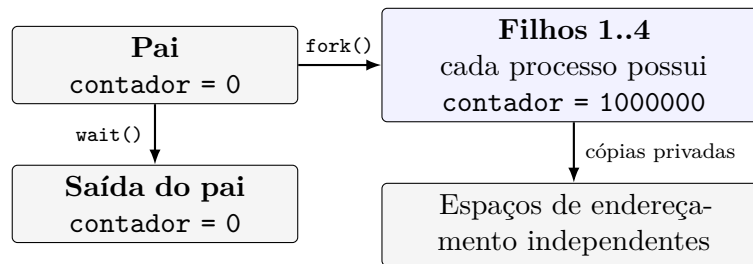


Figura 3.3: Diagrama UML estrutural do programa com processos.

A Figura 3.4 resume o caso com threads.

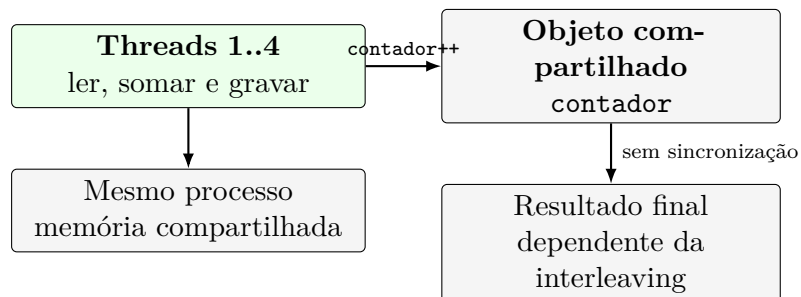


Figura 3.4: Diagrama UML estrutural do programa com threads.

3.8 Fluxo de execução

3.8.1 Fluxo do programa com processos

O fluxo do programa `cont1.c` é:

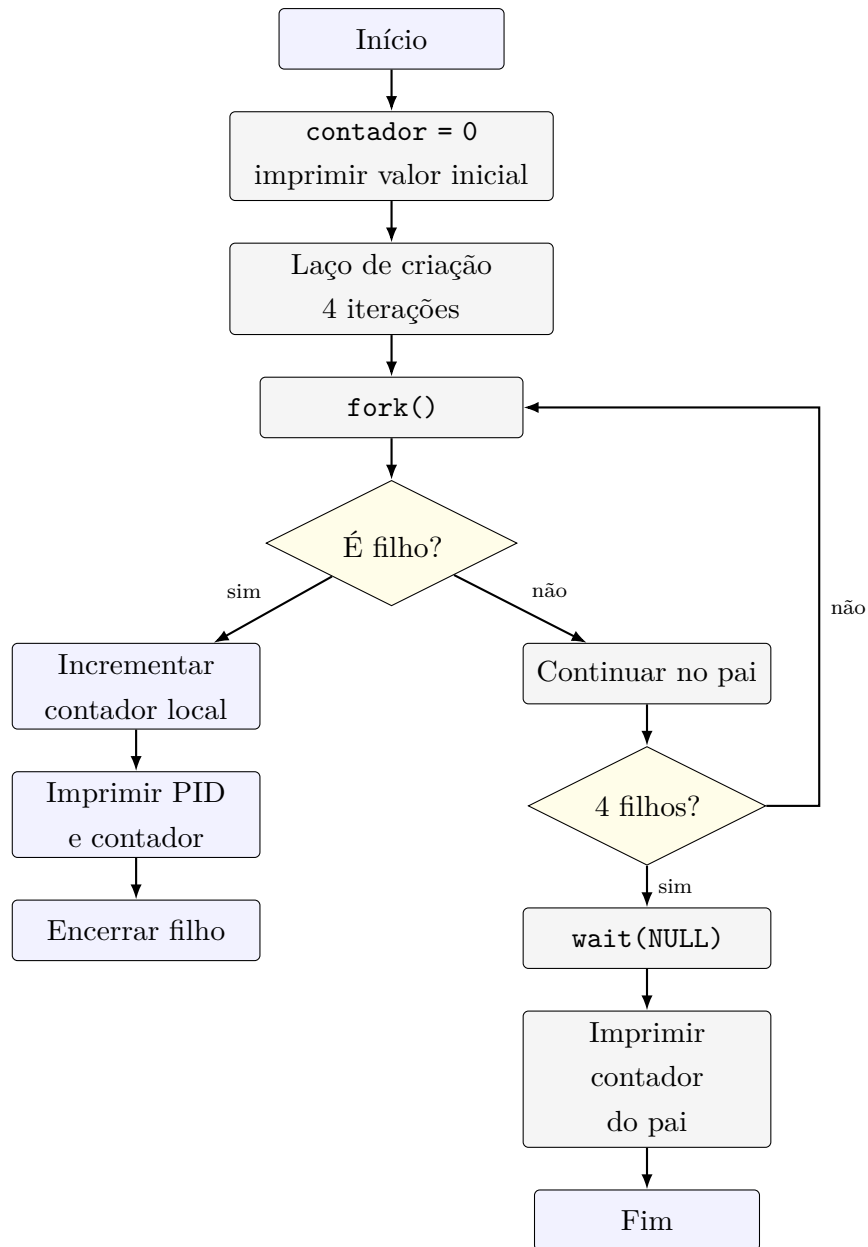


Figura 3.5: Diagrama UML de atividade do fluxo com processos.

O ponto central desse fluxo é que o pai aguarda os filhos, mas não recebe automaticamente as alterações feitas no contador de cada um.

3.8.2 Fluxo do programa com threads

O fluxo do programa `cont2.c` é:

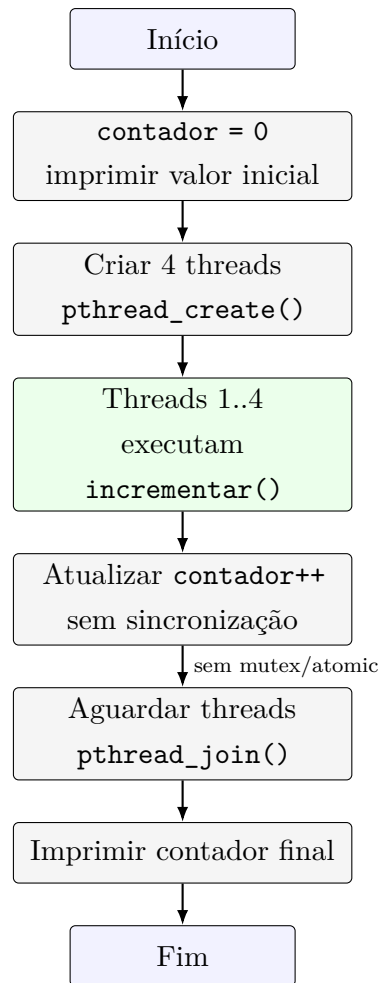


Figura 3.6: Diagrama UML de atividade do fluxo com threads.

Nesse caso, o ponto central é que todas as threads escrevem na mesma variável sem sincronização.

3.9 Metodologia experimental

3.9.1 Ambiente de execução

Os arquivos da atividade registram os comandos de compilação conceituais:

```
gcc cont1.c -o cont1
gcc cont2.c -o cont2 -pthread
```

O primeiro programa depende das interfaces POSIX `fork()` e `wait()`. O segundo depende da biblioteca POSIX Threads.

Nesta consolidação do relatório, foram utilizados os códigos-fonte e as saídas já presentes no diretório da atividade. O objetivo foi documentar o comportamento observado, não refazer um benchmark temporal.

3.9.2 Configuração das medições

Não houve medição de tempo nesta atividade.

A validação experimental consistiu em executar os programas e observar os valores impressos para `contador`.

Foram considerados:

- três execuções registradas do programa com processos;
- dez execuções registradas do programa com threads.

As execuções do programa com processos foram usadas para confirmar que o contador do processo pai permanece em zero.

As execuções do programa com threads foram usadas para evidenciar que o contador final varia e fica abaixo do valor lógico esperado de 4 000 000, devido a atualizações perdidas.

3.10 Código-fonte desenvolvido

3.10.1 Programa com processos

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 /*
7  * Exemplo de concorrencia com processos.
8  *
9  * Compilacao:
10  *     gcc cont1.c -o cont1
11  *
12  * Este programa cria quatro processos filhos com fork().
13  * Cada filho recebe sua propria copia da variavel global
14  * "contador" e realiza um milhao de incrementos localmente.
15  *
16  * Como os processos possuem espacos de memoria independentes,
17  * o valor alterado por cada filho nao e compartilhado com o
18  * processo pai. Por isso, ao final da execucao, o pai ainda
19  * imprime o valor original do contador global.
20  */
21
22 #define NUM_PROCESSOS 4
23 #define INCREMENTOS 1000000
24
25 /* Variavel global usada para ilustrar o efeito de memoria separada. */
26 int contador = 0;
27
28 int main() {
29     printf("Contador inicial: %d\n", contador);
30 }
```

```

31     for (int i = 0; i < NUM_PROCESSOS; i++) {
32         pid_t pid = fork();
33
34         /* Verifica falha na criacao do processo filho. */
35         if (pid < 0) {
36             perror("Erro ao criar processo");
37             exit(EXIT_FAILURE);
38         }
39
40         if (pid == 0) {
41             /*Codigo executado apenas no processo filho. */
42             for (int j = 0; j < INCREMENTOS; j++) {
43                 contador++;
44             }
45
46             /*
47              * Cada filho mostra o proprio PID e o valor do contador
48              * presente em sua copia privada da memoria do processo.
49              */
50             printf("Filho %d: contador = %d\n", getpid(), contador);
51
52             exit(EXIT_SUCCESS);
53         }
54     }
55
56     /* O processo pai aguarda o termino de todos os processos filhos. */
57     for (int i = 0; i < NUM_PROCESSOS; i++) {
58         wait(NULL);
59     }
60
61     printf("\nContador no processo pai: %d\n", contador);
62
63     return 0;
64 }

```

Listing 3.1: Código-fonte da Atividade 3 com processos.

3.10.2 Programa com threads

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4
5  /*
6   * Exemplo de concorrencia com threads.
7   *
8   * Compilacao:
9   *     gcc cont2.c -o cont2 -pthread
10  *
11  * Este programa cria quatro threads que executam a mesma funcao
12  * de incremento sobre a variavel global "contador".
13  *
14  * Diferentemente do caso com processos, todas as threads compartilham
15  * o mesmo espaco de memoria do processo. Assim, todas acessam a mesma
16  * variavel global durante a execucao.
17  *
18  * Observacao tecnica: o codigo nao usa mecanismos de sincronizacao

```

```

19  * como mutex. Portanto, ele foi mantido exatamente no formato da
20  * atividade, servindo para demonstrar acesso concorrente a memoria
21  * compartilhada.
22  */
23
24  #define NUM_THREADS 4
25  #define INCREMENTOS 1000000
26
27  /* Variavel global compartilhada entre todas as threads criadas. */
28  int contador = 0;
29
30  void *incrementar(void *arg) {
31      /*
32       * Cada thread executa a mesma rotina de incremento repetido.
33       * O parametro "arg" existe por compatibilidade com pthread_create,
34       * embora nao seja utilizado nesta implementacao.
35       */
36      for (int i = 0; i < INCREMENTOS; i++) {
37          contador++;
38      }
39
40      return NULL;
41  }
42
43  int main() {
44      pthread_t threads[NUM_THREADS];
45      printf("Contador inicial: %d\n", contador);
46
47      /* Cria as threads e associa cada uma a rotina "incrementar". */
48      for (int i = 0; i < NUM_THREADS; i++) {
49          int erro = pthread_create(
50              &threads[i],
51              NULL,
52              incrementar,
53              NULL
54          );
55
56          /* Interrompe o programa caso a thread nao possa ser criada. */
57          if (erro != 0) {
58              fprintf(stderr, "Erro ao criar thread\n");
59              exit(EXIT_FAILURE);
60          }
61      }
62
63      /* Aguarda a finalizacao de todas as threads antes de encerrar. */
64      for (int i = 0; i < NUM_THREADS; i++) {
65          pthread_join(threads[i], NULL);
66      }
67
68      printf("\nContador final: %d\n", contador);
69
70      return 0;
71  }

```

Listing 3.2: Código-fonte da Atividade 3 com threads.

3.11 Validação da implementação

3.11.1 Validação do programa com processos

No programa com processos, cada filho realiza exatamente:

$$1\,000\,000 \quad (3.2)$$

incrementos em sua própria cópia de `contador`.

Assim, espera-se que cada filho imprima:

$$\text{contador} = 1\,000\,000. \quad (3.3)$$

Como o pai não compartilha essa variável com os filhos, espera-se que o processo pai imprima:

$$\text{contador} = 0. \quad (3.4)$$

As três execuções registradas confirmaram esse comportamento.

3.11.2 Validação do programa com threads

No programa com threads, se os incrementos fossem executados de forma atômicamente correta sobre o contador compartilhado, o resultado esperado seria:

$$4\,000\,000. \quad (3.5)$$

Entretanto, o código utiliza:

```
contador++;
```

sem exclusão mútua ou operação atômica.

Portanto, o resultado final não é deterministicamente igual a 4 000 000.

As saídas registradas confirmam que o valor final varia entre execuções e permanece abaixo do valor esperado.

3.11.3 Observação sobre a impressão inicial

Nas saídas registradas do programa com processos, a linha

```
Contador inicial: 0
```

aparece mais de uma vez.

Isso não significa que o código tenha executado o trecho inicial várias vezes antes de criar os filhos. O `printf()` ocorre antes do `fork()`.

A repetição pode ocorrer quando a saída padrão está sendo redirecionada ou bufferizada. O conteúdo ainda presente no buffer de saída pode ser copiado para os processos filhos durante o `fork()` e depois descarregado por cada processo.

Esse detalhe reforça outro aspecto importante da criação de processos: além das variáveis, estados associados à biblioteca de E/S também podem ser duplicados no momento da criação do filho.

3.12 Resultados experimentais

3.12.1 Resultados do programa com processos

Nas três execuções registradas, cada processo filho apresentou contador igual a um milhão, e o processo pai apresentou contador igual a zero.

Tabela 3.2: Resultados observados no programa com processos.

Execução	Contador dos filhos	Contador no pai
1	1 000 000 em cada filho	0
2	1 000 000 em cada filho	0
3	1 000 000 em cada filho	0

O resultado mostra que os filhos executam corretamente seu laço local, mas essas alterações não são refletidas no processo pai.

3.12.2 Resultados do programa com threads

Nas dez execuções registradas, o contador final variou conforme apresentado na Tabela 3.3.

Tabela 3.3: Resultados observados no programa com threads.

Execução	Contador final
1	1 431 347
2	1 305 740
3	1 171 730
4	1 306 585
5	1 235 595
6	1 292 238
7	1 222 249
8	1 681 956
9	1 240 474
10	1 347 679

O menor valor registrado foi:

$$1\,171\,730, \tag{3.6}$$

e o maior foi:

$$1\,681\,956. \tag{3.7}$$

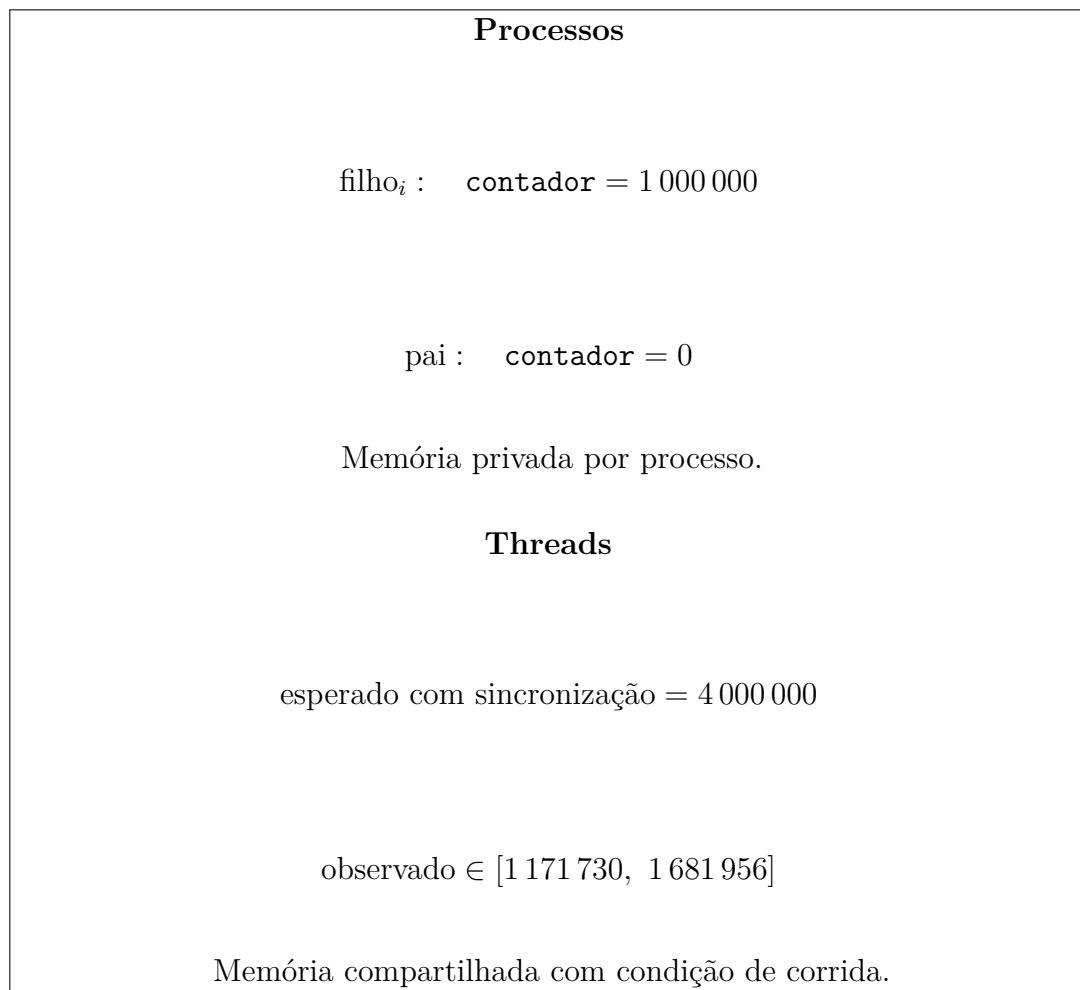
Todos os valores ficaram abaixo de:

$$4\,000\,000. \tag{3.8}$$

Isso evidencia perda de atualizações durante a execução concorrente.

3.13 Visualização dos resultados

O contraste entre os dois programas pode ser resumido por:



Uma forma textual de visualizar o problema das threads é:

Valor inicial: `contador = 10`

Thread A le contador = 10

Thread B le contador = 10

Thread A calcula 11

Thread B calcula 11

Thread A grava 11

Thread B grava 11

Resultado final: 11

Incrementos realizados: 2

Incrementos preservados: 1

Esse exemplo simplificado mostra por que o resultado final pode ficar muito abaixo de quatro milhões.

3.14 Análise e discussão

3.14.1 Comportamento do programa com processos

O programa `cont1.c` cria quatro processos filhos.

Cada filho executa um laço com um milhão de incrementos e imprime seu próprio valor de `contador`.

Como cada filho possui uma cópia independente da variável global, todos os filhos conseguem chegar ao valor:

$$1\,000\,000. \quad (3.9)$$

O pai, entretanto, nunca incrementa sua própria cópia de `contador`. Ele apenas cria filhos e aguarda sua finalização.

Portanto, o resultado:

$$\text{contador no processo pai} = 0 \quad (3.10)$$

é o comportamento correto para esse código.

Não há, nesse caso, uma condição de corrida sobre `contador`, porque os processos não estão escrevendo na mesma posição de memória.

3.14.2 Problema conceitual no programa com processos

O problema do primeiro programa não é erro de sincronização.

O problema é de expectativa: se o objetivo fosse obter no pai a soma dos incrementos realizados pelos filhos, o programa atual não fornece comunicação entre processos.

O valor esperado de quatro milhões só faria sentido se os filhos atualizassem um estado comum ou enviassem seus resultados ao pai.

Soluções adequadas seriam:

- utilizar memória compartilhada entre processos;
- utilizar *pipes* para cada filho enviar seu resultado ao pai;
- utilizar arquivos ou outro mecanismo de comunicação;
- fazer cada filho retornar uma contribuição e o pai agregá-la por IPC apropriado.

Como os filhos são processos independentes, a agregação precisa ser explícita.

3.14.3 Comportamento do programa com threads

O programa `cont2.c` cria quatro threads dentro do mesmo processo.

Todas executam a função `incrementar()` e todas acessam a mesma variável global.

Nesse caso, se cada incremento fosse preservado, o resultado deveria ser:

$$4 \times 1\,000\,000 = 4\,000\,000. \quad (3.11)$$

Entretanto, a ausência de sincronização faz com que múltiplas threads executem simultaneamente a sequência leitura-soma-escrita.

Como resultado, parte dos incrementos se perde.

Esse comportamento explica por que as execuções registradas produziram valores como:

$$1\,431\,347, \quad 1\,305\,740, \quad 1\,171\,730. \quad (3.12)$$

Os valores não são iguais entre si porque a ordem de escalonamento das threads pode mudar a cada execução.

3.14.4 Problema conceitual no programa com threads

O problema do segundo programa é uma condição de corrida.

Ao contrário do caso com processos, as threads realmente compartilham a variável. O erro surge justamente porque a escrita concorrente não é controlada.

Soluções adequadas seriam:

- proteger o incremento com `pthread_mutex_lock()` e `pthread_mutex_unlock()`;
- utilizar uma operação atômica para o incremento;
- fazer cada thread acumular em uma variável local e somar os quatro resultados ao final;
- utilizar primitivas de redução em modelos de programação que as ofereçam, como OpenMP.

Para esse caso específico, a solução com acumuladores locais tende a ser mais adequada conceitualmente do que proteger cada incremento individual, pois evita um milhão de sincronizações por thread.

O modelo seria:

```
Thread 1 calcula local_1 = 1000000
Thread 2 calcula local_2 = 1000000
Thread 3 calcula local_3 = 1000000
Thread 4 calcula local_4 = 1000000
```

Thread principal soma:

```
contador = local_1 + local_2 + local_3 + local_4
```

Esse padrão preserva a correção e reduz o custo de sincronização.

3.14.5 Comparação entre processos e threads

A Tabela 3.4 resume as diferenças observadas.

Tabela 3.4: Comparação conceitual entre processos e threads na atividade.

Aspecto	Processos	Threads
Criação	<code>fork()</code>	<code>pthread_create()</code>
Memória global	Copiada para cada processo	Compartilhada no mesmo processo
Comunicação pelo contador	Não ocorre automaticamente	Ocorre, mas sem controle
Problema principal	Falta de comunicação entre processos	Condição de corrida
Resultado observado	Pai permanece com contador 0	Contador final varia e fica abaixo de 4 milhões
Solução típica	IPC ou memória compartilhada explícita	Mutex, operação atômica ou acumulação local

3.15 Limitações

Os resultados devem ser interpretados dentro do escopo demonstrativo da atividade.

Primeiramente, os programas não medem tempo de execução. Portanto, não é possível concluir, a partir desta atividade, que processos ou threads são mais rápidos para uma aplicação geral.

Em segundo lugar, o programa com threads possui comportamento não determinístico. Os valores registrados demonstram a condição de corrida, mas uma nova execução pode produzir outros valores.

Além disso, o comportamento da impressão inicial no programa com processos pode variar conforme a saída esteja ligada ao terminal ou redirecionada para arquivo. A repetição da linha inicial está associada ao estado do buffer de saída no momento do `fork()`.

Outra limitação é que o programa com processos não utiliza mecanismos reais de comunicação entre processos. Assim, ele demonstra isolamento de memória, mas não compara diferentes estratégias de IPC.

Por fim, o programa com threads não implementa uma versão corrigida com `mutex`, operação atômica ou acumuladores privados. A discussão da solução é conceitual, conforme solicitado pelo enunciado.

3.16 Conclusões da atividade

A atividade demonstrou que processos e threads oferecem modelos de memória distintos.

No programa com processos, cada filho criado por `fork()` recebeu uma cópia própria da variável global `contador`. Por isso, cada filho imprimiu o valor local de um milhão, enquanto o processo pai manteve seu contador igual a zero.

Esse comportamento não representa erro no código. Ele é consequência do isolamento de memória entre processos.

No programa com threads, todas as threads compartilharam a mesma variável global. Nesse caso, a atualização de `contador` ocorreu sobre um estado comum.

Entretanto, como o incremento não foi protegido por sincronização, ocorreu uma condição de corrida. O resultado final variou entre execuções e ficou abaixo do valor logicamente esperado de quatro milhões.

Assim, a atividade evidencia duas lições complementares:

1. processos isolam memória e exigem comunicação explícita quando dados precisam ser combinados;
2. threads compartilham memória, mas exigem sincronização quando há escrita concorrente.

Em termos de Computação de Alto Desempenho, essa distinção é essencial. A correção de uma aplicação concorrente depende tanto da divisão do trabalho quanto da forma como os dados são compartilhados, protegidos ou combinados.

3.17 Síntese

Os principais pontos para apresentação oral da atividade são:

1. A atividade compara dois programas: um com processos e outro com threads.
2. Nos dois casos há quatro unidades de execução, e cada uma realiza um milhão de incrementos.
3. No programa com processos, os filhos são criados com `fork()`.
4. Após o `fork()`, cada processo filho possui sua própria cópia da variável global `contador`.
5. Por isso, cada filho imprime `contador = 1000000`, mas o pai imprime `contador = 0`.
6. O pai aguarda os filhos com `wait(NULL)`, mas aguardar não significa receber as alterações feitas na memória dos filhos.
7. Se fosse necessário somar os resultados no pai, seria preciso usar comunicação entre processos ou memória compartilhada explícita.
8. No programa com threads, as threads são criadas com `pthread_create()`.
9. Todas as threads pertencem ao mesmo processo e acessam a mesma variável global.
10. O valor correto, caso todos os incrementos fossem preservados, seria 4 000 000.
11. O código usa `contador++` sem sincronização.
12. O incremento não é atomicamente seguro: envolve ler, somar e escrever.
13. Quando duas threads fazem isso ao mesmo tempo, atualizações podem ser perdidas.
14. As execuções registradas produziram valores finais variáveis, entre 1 171 730 e 1 681 956.
15. Essa variação caracteriza uma condição de corrida.
16. Uma solução com threads poderia usar `mutex`, operação atômica ou acumuladores locais por thread.

17. Para este problema, acumuladores locais seguidos de soma final são uma solução eficiente, pois evitam sincronizar cada incremento.
18. A conclusão principal é que processos isolam memória, enquanto threads compartilham memória. Isolamento exige comunicação explícita; compartilhamento exige sincronização.

Capítulo 4

Atividade 4 – Integral pelo método do trapézio e análise de paralelização

4.1 Enunciado da tarefa

Implemente uma versão sequencial de um programa para o cálculo de integrais definidas utilizando o método do trapézio.

O programa deve permitir calcular numericamente a integral de uma função $f(x)$ em um intervalo $[a, b]$, utilizando n subdivisões.

Em seguida, analise o código e discuta quais partes podem ser paralelizadas. Também devem ser identificados possíveis desafios ou problemas relacionados à paralelização desse problema, como dependências entre operações, acesso concorrente a variáveis compartilhadas, necessidade de sincronização e redução dos resultados parciais.

4.2 Objetivos e requisitos

A atividade teve como objetivo implementar uma versão sequencial do método composto do trapézio e utilizar essa implementação como base para analisar oportunidades e limitações de uma futura paralelização.

A parte prática foi deliberadamente mantida sequencial. A discussão de paralelismo foi realizada a partir da estrutura do algoritmo e das dependências observadas no laço responsável pela acumulação das contribuições dos pontos internos.

Os principais objetivos considerados foram:

- implementar corretamente o método composto do trapézio em C;
- utilizar aritmética inteira de 64 bits para tornar o cálculo determinístico e evitar efeitos de arredondamento em ponto flutuante;

- variar o número de subdivisões N e observar o comportamento do erro numérico e do tempo de execução;
- utilizar uma metodologia de temporização capaz de medir somente a região computacional de interesse;
- executar aquecimentos e múltiplas medições para reduzir o efeito de interferências externas;
- validar analiticamente cada resultado antes de considerá-lo válido para análise de desempenho;
- identificar quais dados seriam privados ou compartilhados em uma implementação paralela;
- discutir a condição de corrida associada ao acumulador da soma;
- identificar a operação de redução como mecanismo natural para uma futura implementação paralela.

4.3 Fundamentação conceitual

4.3.1 Integração numérica

Uma integral definida

$$I = \int_a^b f(x) dx \quad (4.1)$$

pode ser interpretada geometricamente como a área algébrica delimitada pela função $f(x)$ no intervalo $[a, b]$.

Quando a integral não é calculada diretamente por uma expressão analítica, métodos numéricos podem aproximar seu valor utilizando amostras discretas da função.

Nesta atividade foi utilizado o método composto do trapézio.

4.3.2 Método composto do trapézio

Se o intervalo $[a, b]$ for dividido em N partes de mesma largura, define-se

$$h = \frac{b - a}{N}. \quad (4.2)$$

Os pontos da discretização são dados por

$$x_i = a + ih, \quad i = 0, 1, \dots, N. \quad (4.3)$$

Em cada subintervalo, a curva é aproximada por um segmento de reta. A área aproximada do trapézio correspondente ao intervalo $[x_i, x_{i+1}]$ é

$$A_i = \frac{h}{2} [f(x_i) + f(x_{i+1})]. \quad (4.4)$$

A forma composta pode ser escrita como

$$I_N = h \left[\frac{f(a) + f(b)}{2} + \sum_{i=1}^{N-1} f(a + ih) \right]. \quad (4.5)$$

Para a implementação inteira adotada nesta atividade, utilizou-se a forma algebricamente equivalente

$$I_N = h \sum_{i=1}^{N-1} f(x_i) + \frac{h [f(a) + f(b)]}{2}. \quad (4.6)$$

Essa reorganização evita a formação intermediária de $2 \sum f(x_i)$, reduzindo o risco de aproximação desnecessária do limite representável por `int64_t`.

4.3.3 Função utilizada

Foi adotada a função

$$f(x) = x^2 \quad (4.7)$$

no intervalo

$$[a, b] = [0, 2\,400\,000]. \quad (4.8)$$

A integral analítica é

$$\int_0^{2\,400\,000} x^2 dx = \frac{(2\,400\,000)^3}{3}, \quad (4.9)$$

resultando em

$$I_{\text{exato}} = 4\,608\,000\,000\,000\,000\,000. \quad (4.10)$$

Esse valor permanece dentro do intervalo representável por um inteiro assinado de 64 bits.

4.3.4 Erro do método

Para uma função suficientemente suave, o erro global do método composto do trapézio é de ordem

$$O(h^2). \quad (4.11)$$

Para $f(x) = x^2$, tem-se

$$f''(x) = 2, \quad (4.12)$$

e o erro pode ser determinado exatamente por

$$E_N = \frac{(b-a)h^2}{6}. \quad (4.13)$$

Como x^2 é convexa, o método do trapézio produz uma superestimação e, portanto,

$$I_N = I_{\text{exato}} + E_N. \quad (4.14)$$

Como

$$h = \frac{b-a}{N}, \quad (4.15)$$

segue que

$$E_N = O\left(\frac{1}{N^2}\right). \quad (4.16)$$

Esse comportamento fornece também um critério independente de validação dos resultados experimentais.

4.3.5 Complexidade computacional

O laço principal avalia todos os $N - 1$ pontos internos. Dessa forma, a complexidade temporal é

$$T(N) = O(N). \quad (4.17)$$

A implementação não armazena os pontos da discretização nem os valores da função em vetores. Cada ponto é produzido somente quando necessário.

Consequentemente, a memória adicional utilizada pelo kernel é constante:

$$M(N) = O(1). \quad (4.18)$$

4.4 Interpretação didática da tarefa

Embora o domínio matemático da atividade seja a integração numérica, o objetivo didático relacionado à Computação de Alto Desempenho está na estrutura da soma

$$S = \sum_{i=1}^{N-1} f(a + ih). \quad (4.19)$$

Cada ponto é obtido por

$$x_i = a + ih. \quad (4.20)$$

Portanto, o cálculo de x_i não depende de x_{i-1} . Da mesma forma, a avaliação

$$f(x_i) \quad (4.21)$$

não depende da avaliação realizada em outra iteração.

As contribuições individuais apresentam, assim, potencial para paralelismo de dados.

Entretanto, a implementação sequencial contém a operação

```
soma += funcao_integrando(x_i);
```

que estabelece uma dependência sobre o acumulador **soma**.

Em uma paralelização ingênua, múltiplas threads poderiam ler e modificar essa variável simultaneamente, produzindo uma condição de corrida.

O problema pode ser representado conceitualmente como

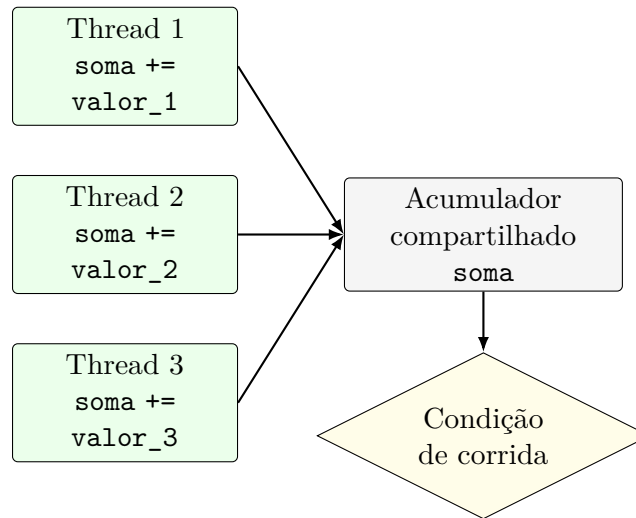


Figura 4.1: Atualização concorrente ingênua do acumulador compartilhado.

Uma estratégia mais apropriada consistiria em utilizar acumuladores privados

$$S_1, S_2, \dots, S_P \quad (4.22)$$

e combiná-los posteriormente:

$$S = \sum_{p=1}^P S_p. \quad (4.23)$$

Esse padrão corresponde a uma operação de *reduction*.

A atividade, portanto, estabelece a base conceitual para uma implementação paralela posterior: o cálculo dos pontos é independente, enquanto a acumulação exige tratamento específico.

4.5 Modelagem da solução

4.5.1 Estrutura geral

A solução foi organizada em quatro etapas principais:

1. definição do problema matemático e dos valores de N ;
2. execução do kernel sequencial do método do trapézio;
3. realização dos aquecimentos e medições temporais;
4. validação matemática e apresentação dos resultados.

A Figura 4.2 resume essa modelagem em termos de etapas e responsabilidades.

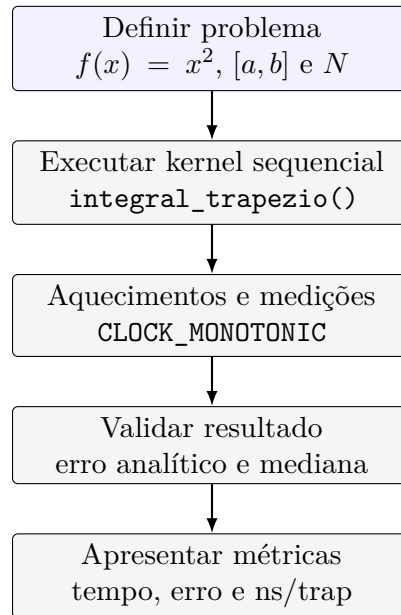


Figura 4.2: Modelagem UML da solução sequencial da Atividade 4.

Os valores de N utilizados foram

$$N \in \{24\,000, 120\,000, 240\,000, 400\,000, 800\,000, 1\,200\,000, 2\,400\,000\}. \quad (4.24)$$

Todos esses valores dividem exatamente o comprimento do intervalo

$$b - a = 2\,400\,000, \quad (4.25)$$

garantindo que

$$h = \frac{b - a}{N} \quad (4.26)$$

permaneça inteiro.

4.5.2 Principais funções e componentes

A implementação foi decomposta nas seguintes responsabilidades:

funcao_integrando() Calcula $f(x) = x^2$.

integral_trapezio() Implementa o kernel sequencial do método composto do trapézio.

obter_tempo() Obtém um instante utilizando `CLOCK_MONOTONIC`.

diferenca_ns() Calcula o intervalo entre dois instantes em nanossegundos.

comparar_uint64() Implementa a função auxiliar utilizada na ordenação dos tempos.

calcular_mediana() Calcula a mediana das vinte medições de uma configuração.

resultado_esperado() Calcula analiticamente o resultado que o método do trapézio deve produzir para determinado N .

executar_benchmark() Coordena aquecimentos, medições, validação, normalização e impressão dos resultados.

main() Percorre os diferentes valores de N e coordena o experimento.

4.6 Decisões de projeto e trade-offs

4.6.1 Uso de `int64_t`

Foi utilizada aritmética inteira em vez de `float` ou `double` no kernel.

Essa decisão teve como objetivos:

- evitar diferenças decorrentes de arredondamento em ponto flutuante;
- tornar a validação determinística;
- facilitar a futura análise de redução;

- permitir comparação exata com o resultado teórico do método.

O uso de `int` de 32 bits não seria suficiente. Por exemplo,

$$(2\,400\,000)^2 = 5\,760\,000\,000\,000, \quad (4.27)$$

valor já superior ao limite de um inteiro assinado de 32 bits.

Por esse motivo foi adotado `int64_t`.

4.6.2 Escolha do intervalo

O limite superior 2 400 000 foi escolhido de forma que:

- existissem diversos divisores adequados para N ;
- h permanecesse inteiro;
- o valor da integral ainda coubesse em `int64_t`.

Essa escolha também impôs uma limitação: para manter $h \geq 1$, o maior valor possível de N é 2 400 000.

Essa restrição contribuiu para a adoção posterior de repetições internas durante a temporização.

4.6.3 Cálculo direto de x_i

Os pontos são calculados por

$$x_i = a + ih \quad (4.28)$$

em vez de utilizar a recorrência

$$x_{i+1} = x_i + h. \quad (4.29)$$

Essa decisão evita introduzir uma dependência artificial entre iterações e torna explícito que a dependência principal do laço está apenas na acumulação da soma.

4.6.4 Ausência de vetores intermediários

Não foram armazenados vetores contendo os pontos x_i ou os valores $f(x_i)$.

Essa decisão mantém

$$M(N) = O(1) \quad (4.30)$$

e evita introduzir efeitos de alocação e acesso à memória que não fazem parte do objetivo principal da atividade.

4.6.5 Nível de otimização

A compilação experimental foi realizada com otimização

`-O2`

em vez de `-O0`.

O uso de `-O0` produziria um executável artificialmente pouco otimizado e não representativo de uma execução sequencial realista.

Entretanto, foi utilizada a opção

`-fno-tree-vectorize`

para impedir que o compilador transformasse automaticamente o laço escalar em código SIMD.

Dessa forma, a execução permaneceu sequencial e escalar para fins da atividade, ainda permitindo otimizações convencionais do compilador.

4.6.6 Proteção contra otimizações interprocedurais

Durante o desenvolvimento, uma versão inicial do benchmark apresentou tempos iguais a zero, mesmo para os maiores valores de N .

A análise indicou que impedir apenas o *inlining* não era suficiente. O compilador podia reconhecer que chamadas consecutivas do kernel, realizadas com os mesmos argumentos, produziram sempre o mesmo resultado e reutilizar a computação.

Por esse motivo, o kernel foi marcado, no GCC, com

```
__attribute__((noinline, noipa))
```

O atributo `noinline` impede a incorporação da função ao chamador, enquanto `noipa` impede otimizações interprocedurais capazes de eliminar ou reutilizar chamadas repetidas.

Adicionalmente, o resultado é armazenado em uma variável `volatile` após a região cronometrada, tornando seu uso observável.

O acumulador principal, entretanto, permanece não-`volatile`, de forma a não alterar artificialmente o desempenho do kernel.

4.6.7 Repetições internas

Após a correção das otimizações interprocedurais, as execuções individuais ainda eram curtas, variando da ordem de microssegundos até frações de milissegundo.

Para aumentar a duração efetivamente observada pelo relógio, cada medição passou a executar

$$R = 1000 \quad (4.31)$$

chamadas consecutivas do kernel.

O tempo medido para o lote é posteriormente normalizado:

$$T_{\text{execução}} = \frac{T_{\text{lote}}}{1000}. \quad (4.32)$$

Essa estratégia reduz a influência relativa do overhead de temporização sem alterar o algoritmo analisado.

4.7 Representação estrutural da implementação

A Figura 4.3 apresenta uma representação estrutural simplificada da implementação.

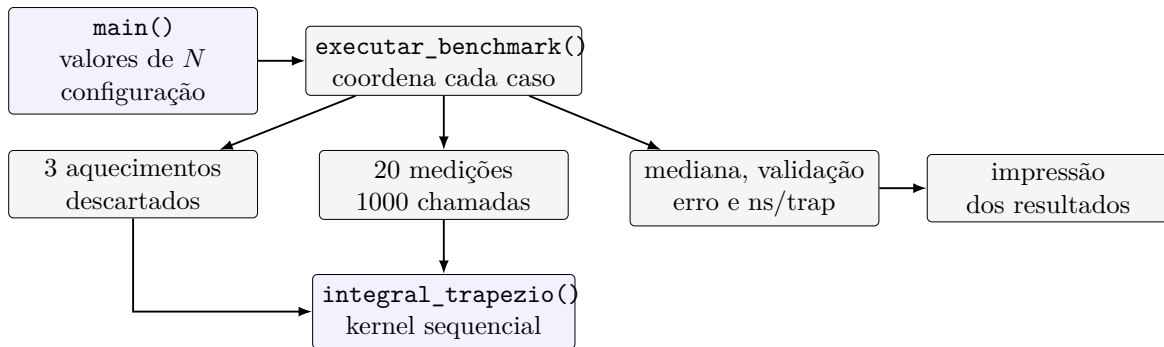


Figura 4.3: Estrutura geral da implementação da Atividade 4.

4.8 Fluxo de execução

O fluxo experimental é apresentado na Figura 4.4.

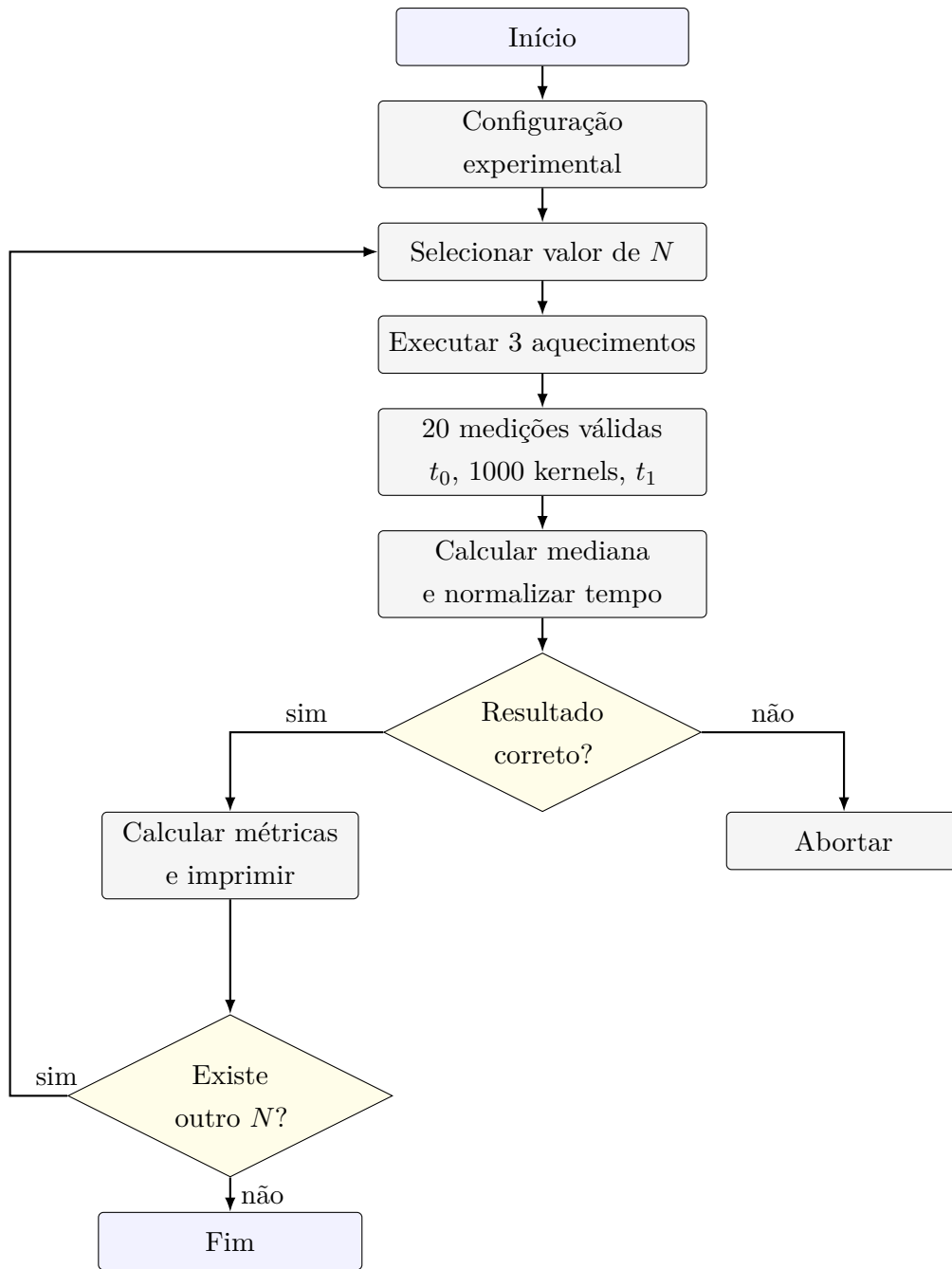


Figura 4.4: Fluxo de execução do benchmark sequencial.

4.9 Metodologia experimental

4.9.1 Ambiente de execução

Os experimentos da Atividade 4 foram executados em ambiente Windows, utilizando terminal MSYS2/UCRT64 e compilador GCC.

A compilação utilizada foi equivalente a:

```
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic \
```

```
-fno-tree-vectorize tarefa4.c -o tarefa4.exe
```

Não foi registrado, durante a execução experimental consolidada, o número de versão do compilador no conjunto de resultados utilizado neste capítulo; por isso esse dado não é atribuído posteriormente.

4.9.2 Configuração das medições

Para cada valor de N foram executados:

$$3 \tag{4.33}$$

aquecimentos, descartados das estatísticas, seguidos de

$$20 \tag{4.34}$$

medições válidas.

Cada medição executou internamente

$$1000 \tag{4.35}$$

chamadas do kernel.

O tempo do lote foi obtido por

```
clock_gettime(CLOCK_MONOTONIC, ...)
```

e normalizado para uma única execução.

A estatística representativa utilizada foi a mediana das vinte medições.

A escolha da mediana reduz a influência de observações isoladas afetadas por interrupções, escalonamento do sistema operacional ou processos em segundo plano.

4.9.3 Região cronometrada

Dentro da região medida permaneceram somente:

- cálculo da integral;
- cálculo de h ;
- cálculo dos pontos internos;
- avaliação de $f(x_i)$;
- acumulação da soma;

- combinação com os extremos.

Operações como impressão, validação, ordenação dos tempos, cálculo da mediana e formatação do resultado permaneceram fora da região cronometrada.

4.9.4 Métricas

Foram utilizadas três grandezas principais:

Tempo mediano

$$T_{\text{med}} \quad (4.36)$$

correspondente ao tempo normalizado de uma execução.

Erro absoluto

$$E_{\text{abs}} = |I_N - I_{\text{exato}}|. \quad (4.37)$$

Tempo normalizado por trapézio

$$C_N = \frac{T_{\text{med}}}{N}. \quad (4.38)$$

A métrica foi apresentada em nanossegundos por trapézio:

$$\text{ns/trap.} \quad (4.39)$$

Como o algoritmo possui complexidade $O(N)$, espera-se que essa razão permaneça aproximadamente constante para execuções suficientemente representativas.

4.10 Código-fonte desenvolvido

```

1  /*
2   * Tarefa 4 - Integração Numérica Sequencial
3   *
4   * Método: Trapézio Composto
5   * Função: f(x) = x^2
6   * Intervalo: [0, 2400000]
7   *
8   * O programa avalia diferentes números de subdivisões N e mede
9   * o tempo de execução da integração sequencial.
10  *
11  * Para tornar a medição mais estável:
12  *   - são feitos 3 aquecimentos;
13  *   - são realizadas 20 medições;
14  *   - cada medição executa o kernel 1000 vezes;
15  *   - o tempo do lote é normalizado para uma única execução;
16  *   - a mediana é utilizada como tempo representativo.
17  */

```

```

18  * A implementação utiliza int64_t para evitar erros de ponto
19  * flutuante durante o cálculo da integral.
20  */
21
22  #define _POSIX_C_SOURCE 199309L
23
24  #include <stdio.h>      /* printf, fprintf, perror */
25  #include <stdint.h>     /* int64_t, uint64_t */
26  #include <stdlib.h>     /* qsort, exit */
27  #include <inttypes.h>   /* PRId64 */
28  #include <time.h>       /* clock_gettime, timespec */
29
30
31  /* Configuração do experimento. */
32  #define NUM_AQUECIMENTOS    3
33  #define NUM_MEDICOES        20
34  #define REPETICOES_INTERNAS 1000
35
36  /* Intervalo de integração. */
37  #define LIMITE_A INT64_C(0)
38  #define LIMITE_B INT64_C(2400000)
39
40  /*
41   * Valor analítico exato:
42   *
43   *          2400000
44   *      ∫ x² dx
45   *          0
46   *
47   * = 2400000³ / 3
48   * = 4608000000000000000
49   */
50  #define INTEGRAL_EXATA INT64_C(4608000000000000000)
51
52
53  /*
54   * Recebe o resultado calculado após a região cronometrada.
55   *
56   * O uso de volatile aqui cria um efeito observável e ajuda a impedir
57   * que o compilador considere as chamadas do kernel desnecessárias.
58   *
59   * O acumulador da integral NÃO é volatile, pois isso alteraria
60   * artificialmente o desempenho do algoritmo.
61   */
62  static volatile int64_t destino_resultado = 0;
63
64
65  /*
66   * O kernel deve ser realmente executado em cada repetição.
67   *
68   * noinline:
69   *   impede que a função seja incorporada diretamente ao chamador.
70   *
71   * noipa:
72   *   impede otimizações interprocedurais que poderiam fazer o GCC
73   *   perceber que chamadas repetidas com os mesmos argumentos
74   *   produzem sempre o mesmo resultado e reutilizar o valor.
75   */
76  #if defined(__GNUC__)

```

```

77 #define KERNEL __attribute__((noinline, noipa))
78 #else
79 #define KERNEL
80 #endif
81
82
83 /*
84  * Função utilizada na integração:
85  *
86  *       $f(x) = x^2$ 
87  */
88 static inline int64_t funcao_integrando(int64_t x)
89 {
90     return x * x;
91 }
92
93
94 /*
95  * Calcula a integral utilizando o método composto do trapézio.
96  *
97  * Para N subdivisões:
98  *
99  *       $h = (b - a) / N$ 
100  *
101  * Os pontos internos são:
102  *
103  *       $x_i = a + i \cdot h$ 
104  *
105  * A formulação utilizada é:
106  *
107  *       $I = h * soma(f(x_i))$ 
108  *           $+ h * (f(a) + f(b)) / 2$ 
109  *
110  * com i variando de 1 até N-1.
111  *
112  * O ponto  $x_i$  é calculado diretamente a partir de i, evitando
113  * dependência entre  $x_i$  e  $x_{(i-1)}$ . Dessa forma, a dependência
114  * sequencial relevante do loop fica concentrada no acumulador soma.
115  */
116 static KERNEL int64_t integral_trapezio(
117     int64_t a,
118     int64_t b,
119     int64_t n)
120 {
121     /* Largura de cada subdivisão do intervalo. */
122     const int64_t h = (b - a) / n;
123
124     /* Acumula os valores da função nos pontos internos. */
125     int64_t soma = 0;
126
127     for (int64_t i = 1; i < n; ++i) {
128
129         /*
130          * O ponto atual depende apenas de a, h e i.
131          * Não depende do ponto calculado na iteração anterior.
132          */
133         const int64_t x_i = a + i * h;
134
135         soma += funcao_integrando(x_i);

```

```

136     }
137
138     /* f(a) + f(b), que recebem peso 1 no método do trapézio. */
139     const int64_t extremos =
140         funcao_integrando(a) +
141         funcao_integrando(b);
142
143     /*
144      * Os pontos internos possuem peso 2 na fórmula tradicional.
145      *
146      * A expressão abaixo é uma forma equivalente reorganizada para
147      * evitar a multiplicação intermediária 2*soma, que poderia
148      * aproximar-se desnecessariamente do limite de int64_t.
149      */
150     return h * soma + (h * extremos) / 2;
151 }
152
153
154 /*
155  * Obtém um instante usando CLOCK_MONOTONIC.
156  *
157  * CLOCK_MONOTONIC é utilizado porque representa tempo decorrido
158  * e não sofre alterações caso o relógio do sistema seja ajustado.
159  */
160 static struct timespec obter_tempo(void)
161 {
162     struct timespec tempo;
163
164     if (clock_gettime(CLOCK_MONOTONIC, &tempo) != 0) {
165         perror("clock_gettime");
166         exit(EXIT_FAILURE);
167     }
168
169     return tempo;
170 }
171
172
173 /*
174  * Calcula a diferença entre dois instantes em nanossegundos.
175  *
176  * timespec armazena separadamente:
177  *   - segundos;
178  *   - nanossegundos.
179  *
180  * O ajuste no if trata o caso em que tv_nsec do instante final
181  * é menor que tv_nsec do instante inicial devido à mudança
182  * de segundo entre as duas leituras.
183  */
184 static uint64_t diferenca_ns(
185     struct timespec inicio,
186     struct timespec fim)
187 {
188     int64_t segundos =
189         (int64_t)fim.tv_sec - (int64_t)inicio.tv_sec;
190
191     int64_t nanos =
192         (int64_t)fim.tv_nsec - (int64_t)inicio.tv_nsec;
193
194     if (nanos < 0) {

```

```

195     --segundos;
196     nanos += INT64_C(1000000000);
197 }
198
199     return
200         (uint64_t)segundos * UINT64_C(1000000000) +
201         (uint64_t)nanos;
202 }
203
204
205 /*
206  * Função auxiliar utilizada pelo qsort para ordenar
207  * os tempos de execução em ordem crescente.
208  */
209 static int comparar_uint64(const void *a, const void *b)
210 {
211     const uint64_t x = *(const uint64_t *)a;
212     const uint64_t y = *(const uint64_t *)b;
213
214     return (x > y) - (x < y);
215 }
216
217
218 /*
219  * Calcula a mediana das 20 medições.
220  *
221  * Como NUM_MEDICOES é par, a mediana corresponde à média
222  * dos dois valores centrais após a ordenação.
223  *
224  * A mediana é utilizada porque é menos sensível a medições
225  * isoladas afetadas por interrupções ou outros processos.
226  */
227 static double calcular_mediana(
228     uint64_t tempos[NUM_MEDICOES])
229 {
230     qsort(
231         tempos,
232         NUM_MEDICOES,
233         sizeof(uint64_t),
234         comparar_uint64
235     );
236
237     const size_t meio = NUM_MEDICOES / 2;
238
239     return
240         ((double)tempos[meio - 1] +
241         (double)tempos[meio]) / 2.0;
242 }
243
244
245 /*
246  * Calcula o resultado que deve ser obtido pelo método do trapézio.
247  *
248  * Para  $f(x) = x^2$ , o erro do método composto é conhecido:
249  *
250  *  $\text{erro} = (b - a) * h^2 / 6$ 
251  *
252  * Como  $x^2$  é convexa, o método do trapézio superestima a integral.
253  * Portanto:

```

```

254 *
255 *     resultado esperado = integral exata + erro
256 *
257 * Isso permite validar a implementação usando somente inteiros.
258 */
259 static int64_t resultado_esperado(
260     int64_t a,
261     int64_t b,
262     int64_t n)
263 {
264     const int64_t h = (b - a) / n;
265
266     const int64_t erro =
267         ((b - a) * h * h) / 6;
268
269     return INTEGRAL_EXATA + erro;
270 }
271
272
273 /*
274 * Executa todo o experimento correspondente a um valor de N.
275 *
276 * Etapas:
277 * 1. verificar se N é válido;
278 * 2. realizar aquecimentos;
279 * 3. executar as 20 medições;
280 * 4. validar o resultado;
281 * 5. calcular mediana e métricas;
282 * 6. imprimir uma linha da tabela.
283 */
284 static void executar_benchmark(int64_t n)
285 {
286     const int64_t a = LIMITE_A;
287     const int64_t b = LIMITE_B;
288
289     /*
290     * Como o experimento utiliza somente aritmética inteira,
291     * h = (b-a)/N deve resultar em um valor inteiro.
292     */
293     if (n <= 0 || (b - a) % n != 0) {
294         fprintf(
295             stderr,
296             "Erro: N=%" PRIu64 " não divide o intervalo.\n",
297             n
298         );
299
300         exit(EXIT_FAILURE);
301     }
302
303     int64_t resultado = 0;
304     uint64_t tempos[NUM_MEDICOES];
305
306
307     /*
308     * -----
309     * Aquecimento
310     * -----
311     *
312     * São executados 3 lotes antes das medições.

```

```

313     * Esses tempos não são armazenados.
314     *
315     * O objetivo é reduzir efeitos relacionados ao estado inicial
316     * do processador, caches e primeiras execuções do programa.
317     */
318     for (int aquecimento = 0;
319          aquecimento < NUM_AQUECIMENTOS;
320          ++aquecimento) {
321
322         for (int r = 0; r < REPETICOES_INTERNAS; ++r) {
323             resultado = integral_trapezio(a, b, n);
324         }
325
326         destino_resultado = resultado;
327     }
328
329
330     /*
331     * -----
332     * Medições
333     * -----
334     *
335     * Cada uma das 20 medições cronometra 1000 execuções.
336     *
337     * Isso aumenta a duração da região cronometrada e reduz
338     * proporcionalmente a influência do custo do temporizador.
339     *
340     * Nenhum printf ou cálculo estatístico ocorre entre inicio e fim.
341     */
342     for (int medicao = 0;
343          medicao < NUM_MEDICOES;
344          ++medicao) {
345
346         const struct timespec inicio = obter_tempo();
347
348         for (int r = 0; r < REPETICOES_INTERNAS; ++r) {
349             resultado = integral_trapezio(a, b, n);
350         }
351
352         const struct timespec fim = obter_tempo();
353
354         /*
355          * Uso observável depois da região cronometrada.
356          */
357         destino_resultado = resultado;
358
359         tempos[medicao] =
360             diferenca_ns(inicio, fim);
361     }
362
363
364     /*
365     * -----
366     * Validação
367     * -----
368     *
369     * O desempenho somente é considerado válido se o resultado
370     * obtido for exatamente o esperado.
371     */

```

```

372     const int64_t esperado =
373         resultado_esperado(a, b, n);
374
375     if (resultado != esperado) {
376         fprintf(
377             stderr,
378             "Erro de validacao para N=%" PRIu64 "\n",
379             n
380         );
381
382         exit(EXIT_FAILURE);
383     }
384
385
386     /*
387     * Diferença entre a aproximação pelo trapézio
388     * e o valor analítico da integral.
389     */
390     const int64_t erro_absoluto =
391         resultado - INTEGRAL_EXATA;
392
393
394     /*
395     * -----
396     * Tratamento das medições
397     * -----
398     */
399
400     /* Mediana do tempo gasto pelo lote de 1000 execuções. */
401     const double mediana_lote_ns =
402         calcular_mediana(tempos);
403
404     /*
405     * Normaliza o tempo do lote para obter o tempo estimado
406     * de uma única execução da integração.
407     */
408     const double mediana_execucao_ns =
409         mediana_lote_ns / REPETICOES_INTERNAS;
410
411     /* Conversão para milissegundos para apresentação. */
412     const double mediana_ms =
413         mediana_execucao_ns / 1000000.0;
414
415     /*
416     * Custo médio normalizado pelo número de trapézios.
417     * Essa métrica ajuda a verificar o comportamento O(N).
418     */
419     const double ns_por_trapezio =
420         mediana_execucao_ns / (double)n;
421
422
423     /* Uma linha de resultado para o valor atual de N. */
424     printf(
425         "%-12" PRIu64
426         " %-22" PRIu64
427         " %-16" PRIu64
428         " %-14.6f"
429         " %-12.3f\n",
430         n,

```

```

431     resultado,
432     erro_absoluto,
433     mediana_ms,
434     ns_por_trapezio
435 );
436 }
437
438
439 int main(void)
440 {
441     /*
442     * Valores de N avaliados no experimento.
443     *
444     * Todos são divisores exatos de 2400000, garantindo que:
445     *
446     *     h = (b-a)/N
447     *
448     * permaneça inteiro em todas as configurações.
449     */
450     static const int64_t valores_n[] = {
451         INT64_C(24000),
452         INT64_C(120000),
453         INT64_C(240000),
454         INT64_C(400000),
455         INT64_C(800000),
456         INT64_C(1200000),
457         INT64_C(2400000)
458     };
459
460     const size_t quantidade =
461         sizeof(valores_n) / sizeof(valores_n[0]);
462
463
464     /*
465     * Informações fixas do experimento.
466     */
467     printf("Integracao Sequencial - Metodo do Trapezio\n");
468     printf("f(x)          : x^2\n");
469     printf(
470         "Intervalo      : [%\" PRIu64 \", \"%\" PRIu64 \"]\n",
471         LIMITE_A,
472         LIMITE_B
473     );
474     printf("Tipo numerico : int64_t\n");
475     printf("Relogio       : CLOCK_MONOTONIC\n");
476     printf("Aquecimentos  : %d\n", NUM_AQUECIMENTOS);
477     printf("Medicoes      : %d\n", NUM_MEDICOES);
478     printf(
479         "Repeticoes    : %d por medicao\n\n",
480         REPETICOES_INTERNAS
481     );
482
483
484     /*
485     * Cabeçalho da tabela.
486     *
487     * Integral    = aproximação calculada;
488     * ErroAbs     = diferença para a integral exata;
489     * Mediana     = tempo normalizado de uma execução;

```

```

490     * ns/trap    = custo normalizado por trapézio.
491     */
492     printf(
493         "%-12s %-22s %-16s %-14s %-12s\n",
494         "N",
495         "Integral",
496         "ErroAbs",
497         "Mediana(ms)",
498         "ns/trap"
499     );
500
501     printf(
502         "-----\n"
503     );
504
505     /*
506     * Executa o benchmark completo para cada valor de N.
507     */
508     for (size_t i = 0; i < quantidade; ++i) {
509         executar_benchmark(valores_n[i]);
510     }
511
512     /*
513     * Mantém uma referência explícita à variável usada
514     * para tornar os resultados observáveis.
515     */
516     (void)destino_resultado;
517
518     return EXIT_SUCCESS;
519 }

```

Listing 4.1: Código-fonte da Atividade 4.

4.11 Validação da implementação

A validação foi realizada utilizando simultaneamente o valor analítico da integral e a expressão exata do erro do método para $f(x) = x^2$.

Para cada N , calculou-se

$$h = \frac{2\,400\,000}{N} \quad (4.40)$$

e

$$E_N = \frac{2\,400\,000\,h^2}{6}. \quad (4.41)$$

A Tabela 4.1 apresenta os valores esperados.

Todos os valores calculados experimentalmente coincidiram exatamente com esses valores esperados.

Tabela 4.1: Validação analítica do erro do método do trapézio.

N	h	Erro esperado
24 000	100	4 000 000 000
120 000	20	160 000 000
240 000	10	40 000 000
400 000	6	14 400 000
800 000	3	3 600 000
1 200 000	2	1 600 000
2 400 000	1	400 000

A validação, portanto, não utilizou tolerância de ponto flutuante. A comparação foi realizada diretamente com valores inteiros.

Esse resultado confirmou simultaneamente:

- a correta implementação do método;
- a ausência de overflow nos casos utilizados;
- a validade dos valores escolhidos para N ;
- a convergência esperada do método.

4.12 Resultados experimentais

A Tabela 4.2 apresenta os resultados finais obtidos após a introdução das mil repetições internas por medição.

Tabela 4.2: Resultados experimentais finais da Atividade 4.

N	Integral	Erro absoluto	Mediana (ms)	ns/trap
24 000	4608000004000000000	4000000000	0.005066	0.211
120 000	4608000000160000000	160000000	0.025484	0.212
240 000	4608000000040000000	40000000	0.050818	0.212
400 000	4608000000014400000	14400000	0.084860	0.212
800 000	4608000000003600000	3600000	0.171002	0.214
1 200 000	4608000000001600000	1600000	0.254809	0.212
2 400 000	4608000000000400000	400000	0.511678	0.213

A integral converge progressivamente para

$$4\,608\,000\,000\,000\,000\,000, \quad (4.42)$$

enquanto o erro absoluto diminui conforme o número de subdivisões aumenta.

4.13 Visualização dos resultados

Os resultados apresentam duas tendências distintas e complementares.

Comportamento numérico $N \uparrow \implies h \downarrow \implies E_{\text{abs}} \downarrow$ $E_{\text{abs}} = O(N^{-2})$ Comportamento computacional $N \uparrow \implies \text{número de iterações} \uparrow \implies \text{tempo} \uparrow$ $T(N) = O(N)$ $\frac{T(N)}{N} \approx 0.21 \text{ ns/trap}$		
--	--	--

A estabilidade da coluna **ns/trap** constitui uma forma normalizada de visualizar o comportamento linear do kernel.

Os valores obtidos foram

$$0.211, 0.212, 0.212, 0.212, 0.214, 0.212, 0.213 \text{ ns/trap.} \quad (4.43)$$

A variação é pequena ao longo de toda a faixa experimental.

4.14 Análise e discussão

4.14.1 Comportamento do erro

Os resultados confirmam a ordem de convergência esperada.

Quando N passa de

$$120\,000 \quad (4.44)$$

para

$$240\,000, \quad (4.45)$$

o número de subdivisões dobra.

O erro, entretanto, passa de

$$160\,000\,000 \quad (4.46)$$

para

$$40\,000\,000, \quad (4.47)$$

ou seja, é reduzido por um fator de quatro.

Isso é compatível com

$$E_N = O(N^{-2}). \quad (4.48)$$

A mesma propriedade pode ser observada nos demais pares de configurações compatíveis.

4.14.2 Comportamento temporal

O tempo de execução apresentou crescimento aproximadamente proporcional ao número de subdivisões.

Por exemplo, entre

$$N = 120\,000 \quad (4.49)$$

e

$$N = 240\,000, \quad (4.50)$$

o número de subdivisões dobra, enquanto o tempo passa de

$$0.025484 \text{ ms} \quad (4.51)$$

para

$$0.050818 \text{ ms}. \quad (4.52)$$

A razão entre os tempos é aproximadamente

$$\frac{0.050818}{0.025484} \approx 1.994. \quad (4.53)$$

Analogamente, entre 400 000 e 800 000,

$$\frac{0.171002}{0.084860} \approx 2.015, \quad (4.54)$$

e entre 1 200 000 e 2 400 000,

$$\frac{0.511678}{0.254809} \approx 2.008. \quad (4.55)$$

Esses resultados são compatíveis com a complexidade

$$T(N) = O(N). \quad (4.56)$$

4.14.3 Custo normalizado

O custo por trapézio permaneceu aproximadamente constante em torno de

$$0.21 \text{ ns/trap.} \quad (4.57)$$

Esse resultado reforça a interpretação de crescimento linear.

O pequeno intervalo observado,

$$0.211 \leq C_N \leq 0.214, \quad (4.58)$$

é compatível com variações normais de execução do sistema.

4.14.4 Efeito das repetições internas

As execuções individuais do kernel apresentaram tempos curtos. No menor caso, o tempo normalizado foi

$$0.005066 \text{ ms}, \quad (4.59)$$

enquanto no maior foi

$$0.511678 \text{ ms.} \quad (4.60)$$

Como cada medição executou mil chamadas, as regiões efetivamente observadas pelo relógio apresentaram duração mediana aproximada entre

$$5.066 \text{ ms} \quad (4.61)$$

e

$$511.678 \text{ ms.} \quad (4.62)$$

Isso reduziu significativamente a influência relativa do custo das chamadas ao temporizador.

Além disso, os tempos normalizados permaneceram próximos aos valores observados antes da introdução do batching, fornecendo evidência de que a alteração aumentou a robustez da medição sem modificar a tendência do kernel.

4.14.5 Análise de paralelização

O laço principal pode ser representado conceitualmente por

```
for i = 1 .. N-1
    x_i    = a + i*h
    valor  = f(x_i)
    soma   = soma + valor
```

As duas primeiras operações podem ser executadas independentemente para diferentes valores de i .

O problema aparece na terceira operação.

A atualização

$$S \leftarrow S + v_i \quad (4.63)$$

envolve leitura, operação aritmética e escrita.

Se diferentes threads realizarem essas etapas simultaneamente sobre o mesmo acumulador, atualizações podem ser perdidas.

A versão paralela ingênua teria, portanto, uma condição de corrida.

Uma possível decomposição seria:

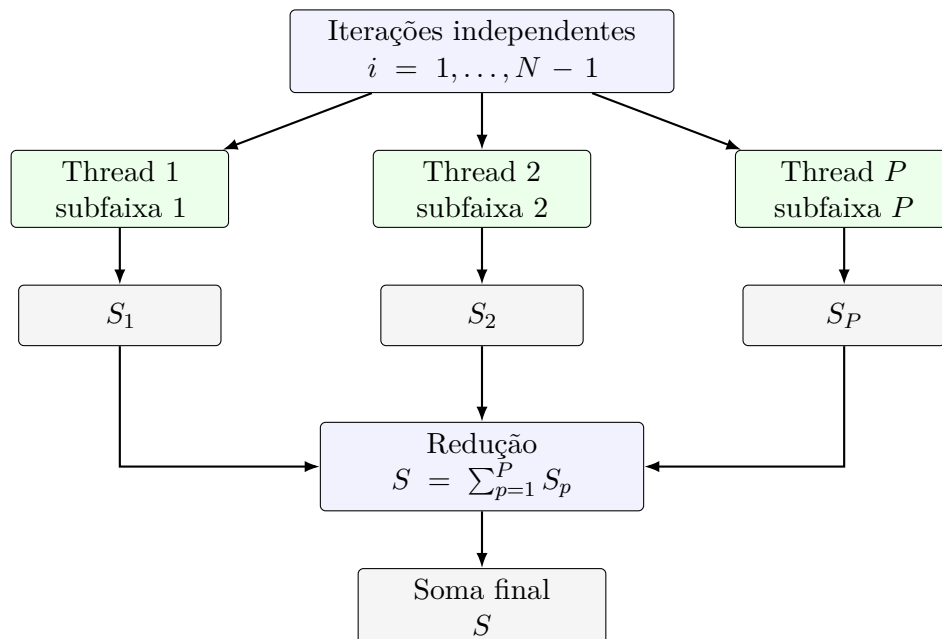


Figura 4.5: Decomposição conceitual do acumulador por redução.

Nesse modelo, cada thread mantém um acumulador privado durante sua parcela do trabalho. Somente posteriormente os resultados parciais são combinados.

Essa abordagem evita a atualização concorrente de uma única variável a cada iteração.

4.14.6 Classificação das variáveis

Em uma eventual versão paralela, as variáveis poderiam ser classificadas como apresentado na Tabela 4.3.

Tabela 4.3: Classificação conceitual das variáveis em uma paralelização.

Elemento	Classificação conceitual
a	compartilhado, somente leitura
b	compartilhado, somente leitura
N	compartilhado, somente leitura
h	compartilhado, somente leitura
i	privado por thread
x_i	privado por thread
$f(x_i)$	privado por thread
soma	variável de redução

Essa classificação mostra que a maior parte do estado do algoritmo não exige sincronização. O ponto crítico é a combinação dos resultados parciais.

4.15 Limitações

Os resultados devem ser interpretados considerando algumas limitações.

Primeiramente, o experimento utiliza uma função específica,

$$f(x) = x^2, \quad (4.64)$$

cujo custo por avaliação é baixo e aproximadamente uniforme. Funções mais complexas podem alterar significativamente a relação entre custo aritmético e overhead de paralelização.

A adoção de aritmética inteira melhora a reprodutibilidade do experimento, mas não representa todas as aplicações de integração numérica, que frequentemente utilizam ponto flutuante.

A opção `-fno-tree-vectorize` impede vetorização automática. Essa decisão é adequada para manter um baseline escalar, porém significa que os tempos não representam necessariamente o máximo desempenho sequencial que o compilador poderia produzir em uma aplicação real.

Os atributos `noinline` e `noipa` também constituem uma intervenção metodológica destinada a preservar a validade do benchmark. Eles evitam que o compilador elimine

trabalho que precisa ser medido, mas restringem algumas otimizações possíveis em um programa não experimental.

As mil repetições internas utilizam sempre os mesmos parâmetros. Essa estratégia aumenta a duração da região cronometrada e reduz o impacto do temporizador, porém não representa uma sequência de problemas independentes com dados diferentes.

Os experimentos foram executados em um sistema operacional de propósito geral. Escalonamento de processos, frequência dinâmica do processador, interrupções e outros serviços podem produzir pequenas variações entre medições.

Por fim, a atividade não implementa efetivamente uma versão paralela. Assim, as discussões sobre condição de corrida, sincronização e redução são derivadas da estrutura do algoritmo, conforme solicitado pelo enunciado, e não de medições paralelas nesta atividade.

4.16 Conclusões da atividade

A implementação sequencial do método composto do trapézio foi validada analiticamente para todos os valores de N avaliados.

O uso de $f(x) = x^2$, aritmética `int64_t` e subdivisões que mantêm h inteiro permitiu comparar exatamente o resultado calculado com o valor teoricamente esperado.

O erro absoluto diminuiu conforme

$$O(N^{-2}), \tag{4.65}$$

em concordância com a ordem de erro do método composto do trapézio.

Em relação ao custo computacional, os tempos experimentais apresentaram crescimento aproximadamente linear com N , enquanto o custo normalizado permaneceu entre aproximadamente 0.211 e 0.214 ns por trapézio.

A atividade também permitiu identificar a estrutura de paralelismo do problema. O cálculo de cada ponto

$$x_i = a + ih \tag{4.66}$$

e sua respectiva avaliação são independentes. Entretanto, a atualização do acumulador representa uma dependência de redução.

Uma paralelização direta utilizando um único acumulador compartilhado pode produzir condição de corrida. A estratégia conceitualmente mais adequada é a utilização de acumuladores privados seguida de uma etapa de redução.

Dessa forma, a atividade estabelece tanto um baseline sequencial validado quanto a fundamentação necessária para uma implementação paralela posterior.

4.17 Síntese

Para uma apresentação oral, os principais pontos da atividade podem ser sintetizados da seguinte forma:

1. A tarefa exigiu implementar somente a versão sequencial e discutir teoricamente sua paralelização.
2. Foi utilizada a função

$$f(x) = x^2$$

no intervalo

$$[0, 2\,400\,000].$$

3. O tipo `int64_t` foi adotado para evitar efeitos de arredondamento e permitir validação inteira exata.
4. O método implementado foi o trapézio composto, com

$$x_i = a + ih.$$

Essa forma evita dependência entre pontos consecutivos.

5. Foram utilizados sete valores de N , todos divisores de 2 400 000, garantindo passo h inteiro.
6. O valor exato da integral é

$$4\,608\,000\,000\,000\,000\,000.$$

7. Para $f(x) = x^2$, o erro pode ser calculado exatamente por

$$E_N = \frac{(b-a)h^2}{6}.$$

Todos os resultados experimentais coincidiram com esse valor.

8. Foram realizados 3 aquecimentos e 20 medições para cada N .
9. Cada medição executou 1000 chamadas do kernel. O tempo do lote foi dividido por 1000 para obter o tempo equivalente de uma execução.
10. Foi utilizado `clock_gettime(CLOCK_MONOTONIC)` para medir tempo decorrido.
11. Durante o desenvolvimento, uma primeira versão apresentou tempos zerados devido a otimizações interprocedurais. O problema foi corrigido utilizando `noinline`, `noipa` e consumo observável do resultado.

12. A compilação utilizou `-O2`, mas a vetorização automática foi desabilitada com `-fno-tree-vectorize` para preservar um baseline escalar.

13. O erro diminuiu com comportamento

$$O(N^{-2}).$$

14. O tempo aumentou aproximadamente de forma linear:

$$T(N) = O(N).$$

15. O custo normalizado permaneceu próximo de

$$0.21 \text{ ns/trap.}$$

16. O cálculo de cada $f(x_i)$ pode ser paralelizado, pois as iterações são independentes.

17. A variável `soma` não pode ser atualizada ingenuamente por várias threads, pois isso produziria uma condição de corrida.

18. A estratégia conceitualmente mais adequada é utilizar somas privadas por thread e combinar os resultados posteriormente por uma operação de redução.

Capítulo 5

Atividade 5 – Contagem de números primos com OpenMP

5.1 Enunciado da tarefa

Implemente um programa em C que conte quantos números primos existem entre 2 e um valor máximo n .

Depois, paralelize o laço principal utilizando a diretiva `#pragma omp parallel for`. A lógica original do algoritmo não deve ser alterada.

Compare o tempo de execução, os resultados da versão sequencial e os resultados da versão paralela utilizando diferentes valores de n . Observe possíveis diferenças no resultado e no desempenho.

Discuta também os desafios iniciais da programação paralela, considerando especialmente correção e distribuição de carga. Para a medição do tempo, utilize uma abordagem baseada em `omp_get_wtime()`.

5.2 Objetivos e requisitos

A atividade teve como objetivo investigar a transformação de um algoritmo sequencial simples em uma implementação paralela baseada em OpenMP, mantendo inalterada a lógica fundamental da computação.

O problema adotado consiste em determinar a quantidade de números primos existentes no intervalo

$$[2, n],$$

para diferentes valores do limite superior n .

Os objetivos específicos foram:

- implementar uma versão sequencial da contagem de números primos;
- paralelizar o laço externo de candidatos utilizando `#pragma omp parallel for`;
- observar o efeito de uma atualização concorrente não sincronizada sobre um contador compartilhado;
- implementar uma versão paralela correta utilizando a cláusula `reduction`;
- verificar a equivalência funcional entre a versão sequencial e a versão paralela correta;
- medir o tempo de execução utilizando `omp_get_wtime()`;
- avaliar diferentes tamanhos do problema;
- calcular o speedup obtido pela paralelização;
- discutir overhead de paralelização, condição de corrida e distribuição de carga.

O número de threads foi mantido fixo em quatro durante todo o experimento. O objetivo, portanto, não foi estudar escalabilidade em função da quantidade de threads, mas isolar o efeito da paralelização do laço principal.

5.3 Fundamentação conceitual

5.3.1 Teste de primalidade

Um número inteiro $x > 1$ é primo quando seus únicos divisores positivos são 1 e o próprio x .

Uma forma direta de verificar a primalidade consiste em testar possíveis divisores. Não é necessário testar divisores superiores a $\sqrt{x}$, pois, caso x possua um fator maior que sua raiz quadrada, necessariamente possuirá também um fator correspondente menor que ela.

Na implementação utilizada nesta atividade, após o tratamento do número 2, todos os demais números pares são descartados imediatamente. Para números ímpares, são testados apenas divisores ímpares.

A condição do laço foi expressa como

$$d \leq \frac{x}{d},$$

em vez de

$$d^2 \leq x.$$

As duas expressões são equivalentes no domínio considerado, mas a primeira evita a necessidade de calcular explicitamente a raiz quadrada e evita uma multiplicação que poderia causar overflow para valores suficientemente grandes.

5.3.2 Paralelismo de dados

A classificação de um candidato como primo ou não primo é independente da classificação dos demais candidatos.

Assim,

$$\text{primo}(i)$$

pode ser calculado independentemente de

$$\text{primo}(j), \quad i \neq j.$$

Essa característica torna o laço externo da contagem adequado ao paralelismo de dados. Conceitualmente, o espaço de candidatos pode ser distribuído entre as threads:

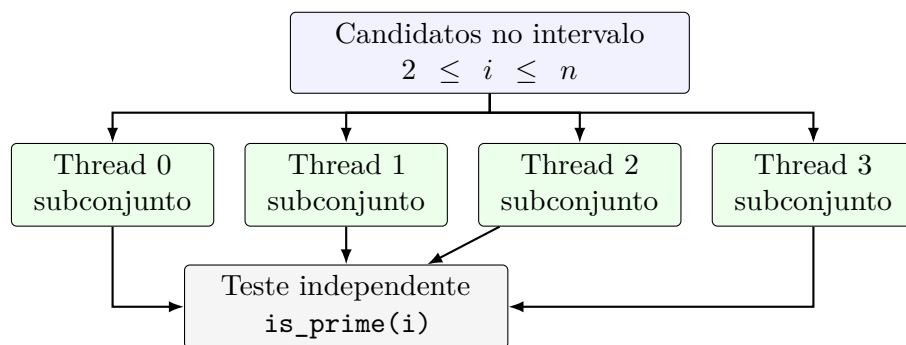


Figura 5.1: Distribuição de candidatos independentes entre threads.

Cada thread executa a mesma função de primalidade sobre diferentes elementos do intervalo.

5.3.3 Diretiva `parallel for`

OpenMP permite distribuir automaticamente as iterações de um laço por uma equipe de threads por meio da diretiva

```
#pragma omp parallel for
```

A diretiva combina a criação de uma região paralela com a distribuição do espaço de iterações do laço.

Nesta atividade, apenas o laço que percorre os candidatos entre 2 e n foi paralelizado.

5.3.4 Laços aninhados e granularidade do paralelismo

O algoritmo apresenta naturalmente dois níveis de repetição:

1. um laço externo que percorre os candidatos;
2. um laço interno que testa possíveis divisores.

A modelagem adotada paralelizou somente o primeiro nível.

O teste de divisores permaneceu sequencial dentro de cada thread.

Portanto, a estrutura adotada foi:

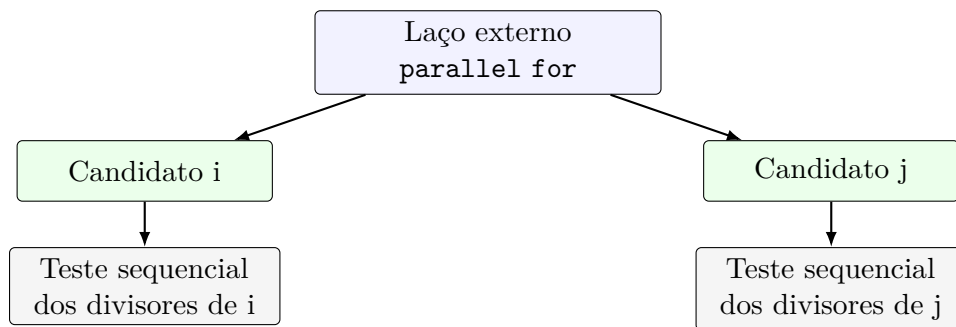


Figura 5.2: Paralelismo do laço externo com testes internos sequenciais.

Não foram utilizados paralelismo aninhado ou a cláusula `collapse`.

Essa decisão evita a criação de regiões paralelas adicionais em uma operação relativamente pequena e irregular e mantém o experimento alinhado ao objetivo do enunciado: paralelizar o laço principal.

5.3.5 Condição de corrida

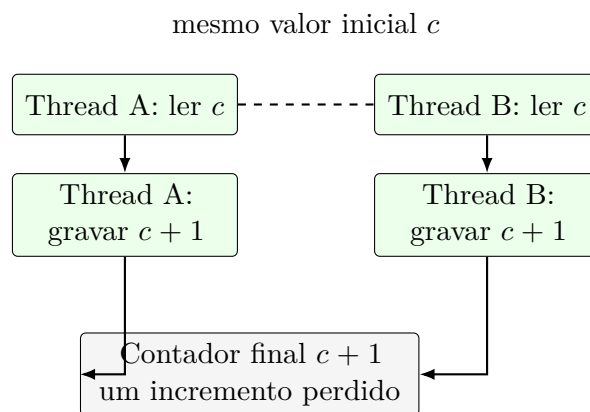


Figura 5.3: Intercalação de incrementos que causa atualização perdida.

A contagem utiliza uma variável global à computação lógica,

`count,`

incrementada sempre que um número primo é encontrado.

Na execução sequencial,

```
count++;
```

não apresenta problema.

Entretanto, em uma execução paralela, múltiplas threads podem tentar atualizar o mesmo contador simultaneamente.

A operação de incremento pode ser entendida conceitualmente como:

1. ler o valor atual;
2. adicionar 1;
3. escrever o novo valor.

Duas threads podem, por exemplo, ler simultaneamente o mesmo valor e posteriormente sobrescrever uma atualização realizada pela outra.

Essa situação caracteriza uma *race condition*, ou condição de corrida.

Dessa forma,

```
parallel for
```

por si só não garante a correção de um programa paralelo.

5.3.6 Redução

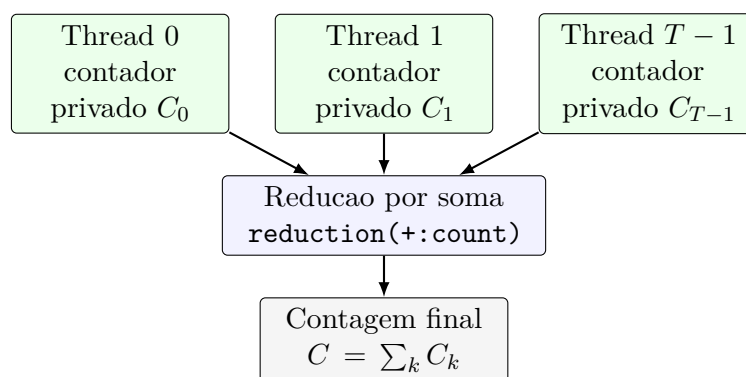


Figura 5.4: Contadores privados combinados pela redução do OpenMP.

A operação realizada sobre o contador é uma soma de contribuições independentes. Cada thread pode produzir uma contagem parcial,

$$C_0, C_1, \dots, C_{T-1},$$

e a contagem final é

$$C = \sum_{i=0}^{T-1} C_i.$$

Esse padrão corresponde diretamente à operação de redução disponível em OpenMP.

A cláusula

`reduction(+:count)`

permite que as threads trabalhem com contadores privados e que esses contadores sejam combinados ao término da região paralela.

Dessa forma, a redução fornece simultaneamente paralelismo e correção funcional.

5.3.7 Distribuição de carga

Embora cada iteração do laço externo represente o teste de um candidato, o custo das iterações não é uniforme.

Um número par pode ser eliminado quase imediatamente, enquanto um número primo grande pode exigir diversos testes de divisibilidade.

Portanto,

$$W(i) \neq W(j),$$

onde $W(x)$ representa o trabalho necessário para determinar a primalidade do candidato x .

Esse comportamento introduz a possibilidade de desequilíbrio de carga: uma thread pode concluir suas iterações antes das demais e permanecer aguardando a sincronização final.

5.3.8 Speedup

A principal métrica derivada utilizada foi o speedup,

$$S = \frac{T_{\text{seq}}}{T_{\text{omp}}},$$

onde:

- T_{seq} é o tempo da versão sequencial;
- T_{omp} é o tempo da versão paralela correta.

Assim,

$$S > 1$$

indica ganho de desempenho com a paralelização, enquanto

$$S < 1$$

indica que o overhead paralelo foi superior ao benefício obtido.

5.4 Interpretação didática da tarefa

A atividade não foi interpretada como um estudo de algoritmos avançados para geração de números primos.

O problema de primalidade funciona como uma carga computacional simples, capaz de expor conceitos fundamentais de programação paralela.

Por esse motivo, não foram utilizados métodos como o Crivo de Eratóstenes ou algoritmos distintos entre as versões.

O núcleo computacional foi mantido igual, modificando-se apenas a forma como as iterações do laço externo são executadas.

A sequência didática explorada foi:

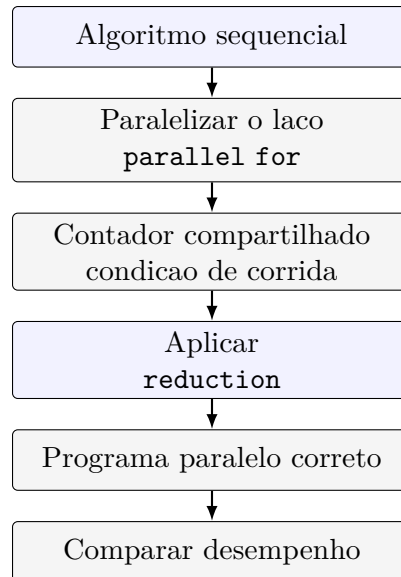


Figura 5.5: Progressão didática da paralelização até a avaliação de desempenho.

Essa sequência evidencia um princípio fundamental de HPC: paralelizar uma computação não consiste apenas em criar múltiplas threads. A preservação da semântica do programa é requisito anterior à análise de desempenho.

5.5 Modelagem da solução

5.5.1 Estrutura geral

Foram modeladas três versões do algoritmo.

Versão sequencial

A primeira implementação constitui o *baseline* do experimento.

Seu comportamento é:

```
para candidato = 2 ate N
    se candidato for primo
        count++
```

Essa versão fornece tanto a referência funcional quanto a referência de tempo.

Versão OpenMP ingênua

A segunda versão paraleliza diretamente o laço externo, mantendo o contador compartilhado sem sincronização.

Conceitualmente:

```
#pragma omp parallel for

para candidato = 2 ate N
    se candidato for primo
        count++
```

Essa implementação foi mantida propositalmente incorreta para demonstrar o efeito de uma condição de corrida.

Ela participa da validação funcional, mas não do benchmark oficial de speedup.

Versão OpenMP com redução

A terceira implementação acrescenta a redução sobre o contador:

```
#pragma omp parallel for reduction(+:count)
```

Essa é a versão paralela utilizada para a comparação válida de desempenho.

5.5.2 Principais funções e componentes

A implementação foi organizada em funções com responsabilidades distintas.

Componente	Responsabilidade
<code>is_prime()</code>	Verificar se um inteiro é primo.
<code>count_primes_sequential()</code>	Realizar a contagem sequencial entre 2 e n .
<code>count_primes_parallel_naive()</code>	Executar o laço com OpenMP mantendo a condição de corrida proposital.
<code>count_primes_parallel_reduction()</code>	Executar a versão paralela funcionalmente correta utilizando <code>reduction</code> .
<code>validate_is_prime()</code>	Validar casos conhecidos da função de primalidade.
<code>get_actual_thread_count()</code>	Confirmar a quantidade efetivamente criada de threads.
<code>run_timed_batch()</code>	Executar lotes cronometrados de chamadas da função de contagem.
<code>calibrate_internal_repetition()</code>	Determinar automaticamente o número de repetições internas necessário para produzir lotes mensuráveis.
<code>run_warmups()</code>	Realizar aquecimentos anteriores às medições oficiais.
<code>measure_per_execution()</code>	Converter o tempo total do lote no tempo equivalente a uma execução.
<code>calculate_median()</code>	Calcular a mediana das vinte medições.
<code>main()</code>	Coordenar configuração, validação, benchmark e apresentação dos resultados.

5.6 Decisões de projeto e trade-offs

5.6.1 Preservação do algoritmo

As versões sequencial e paralela utilizam exatamente a mesma função `is_prime()`.

Isso impede que eventuais diferenças de desempenho sejam atribuídas à utilização de algoritmos distintos.

A variável experimental permanece sendo a estratégia de execução.

5.6.2 Paralelização somente do laço externo

A opção por não paralelizar o teste interno de divisores reduz o overhead e evita a criação de paralelismo aninhado.

O laço externo apresenta unidades de trabalho independentes e constitui o nível natural de distribuição entre as threads.

5.6.3 Uso de **reduction** em vez de **critical** ou **atomic**

Seria possível proteger o incremento do contador por meio de mecanismos como **critical** ou **atomic**.

Entretanto, ambos introduziriam sincronização sobre atualizações individuais.

Como o problema consiste em somar contribuições independentes, **reduction** representa diretamente a operação matemática necessária e permite acumulação privada por thread.

Por esse motivo, foi adotada como solução principal.

5.6.4 Escalonamento estático

A versão paralela foi executada com

```
schedule(static)
```

durante todo o experimento.

Essa decisão mantém a política de distribuição fixa e reproduzível e evita introduzir uma segunda variável experimental.

O trade-off é que a distribuição estática de quantidades semelhantes de iterações não garante distribuição uniforme de trabalho, pois os testes de primalidade possuem custos distintos.

5.6.5 Quatro threads fixas

O número de threads foi fixado em quatro.

A atividade não foi transformada em um estudo de escalabilidade em função do número de threads.

Essa decisão permite atribuir as diferenças observadas principalmente à mudança entre execução sequencial e paralela.

5.6.6 Repetições internas

Uma primeira execução experimental mostrou que, para $n = 10\,000$, os tempos individuais eram inferiores à resolução prática observada da temporização, resultando em valores apresentados como zero e, conseqüentemente, em um speedup indefinido.

A solução adotada foi agrupar várias execuções da mesma função em um único lote cronometrado.

Para R repetições,

$$T_{\text{exec}} = \frac{T_{\text{lote}}}{R}.$$

O número de repetições é calibrado automaticamente até que as versões sequencial e paralela produzam lotes de aproximadamente pelo menos 100 ms.

Essa estratégia preserva o algoritmo original e melhora a qualidade da medição de cargas pequenas.

5.6.7 Mediana como estatística principal

Cada configuração foi medida vinte vezes.

Foi utilizada a mediana em vez de uma única execução ou do menor tempo observado. Para vinte valores ordenados,

$$t_{(1)} \leq t_{(2)} \leq \dots \leq t_{(20)},$$

a mediana utilizada é

$$\tilde{T} = \frac{t_{(10)} + t_{(11)}}{2}.$$

A mediana reduz a influência de interrupções ocasionais e outros *outliers* produzidos pelo sistema operacional.

5.7 Representação estrutural da implementação

A organização lógica do programa pode ser representada da seguinte forma:

A função de primalidade é compartilhada pelas três versões, evitando diferenças algorítmicas entre as implementações.

5.8 Fluxo de execução

O fluxo experimental implementado está modelado na Figura 5.7.

5.9 Metodologia experimental

5.9.1 Ambiente de execução

A atividade foi executada em ambiente Windows com MSYS2 utilizando o subsistema UCRT64.

O compilador validado foi:

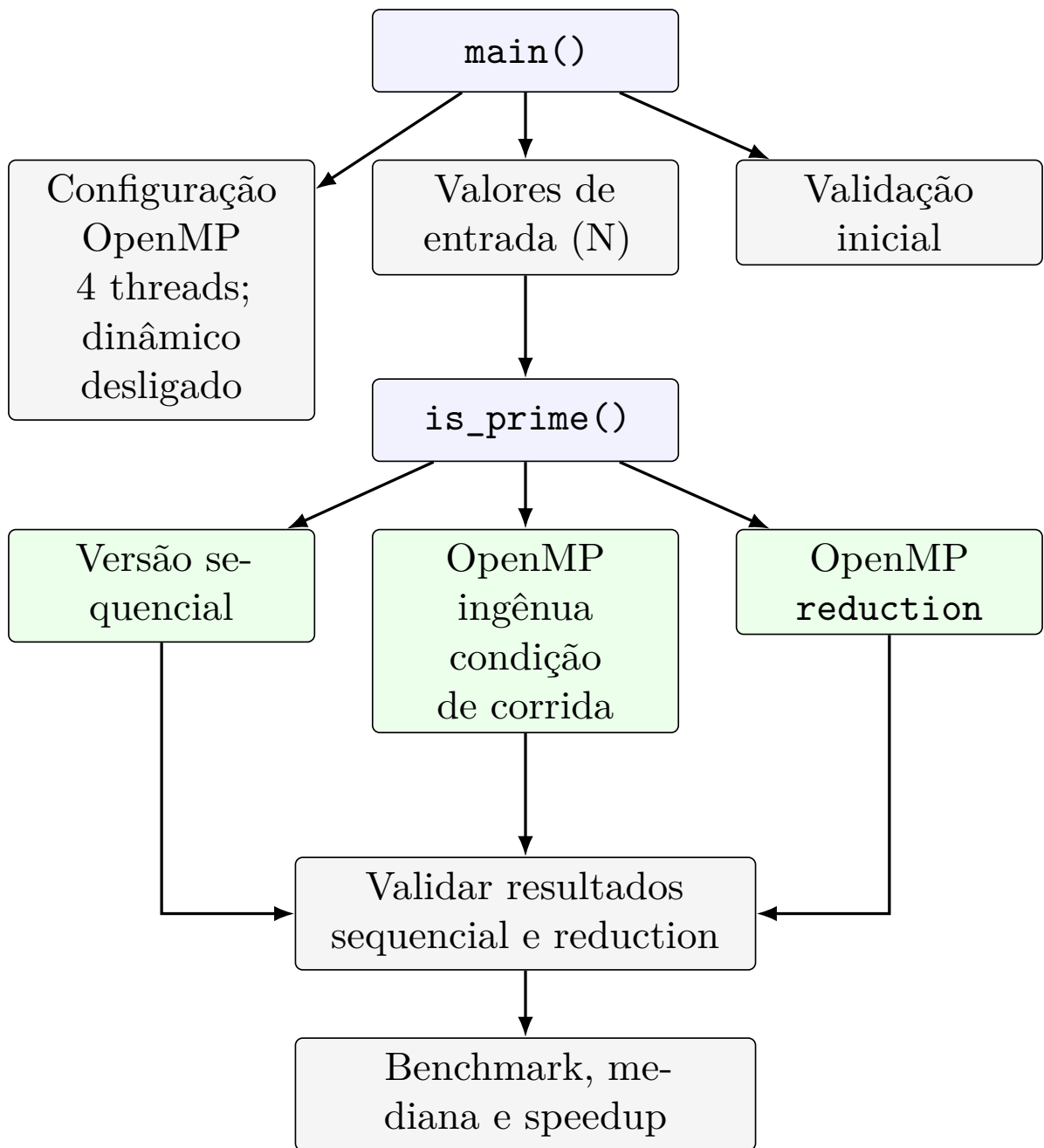


Figura 5.6: Diagrama UML de componentes e dependências do programa.

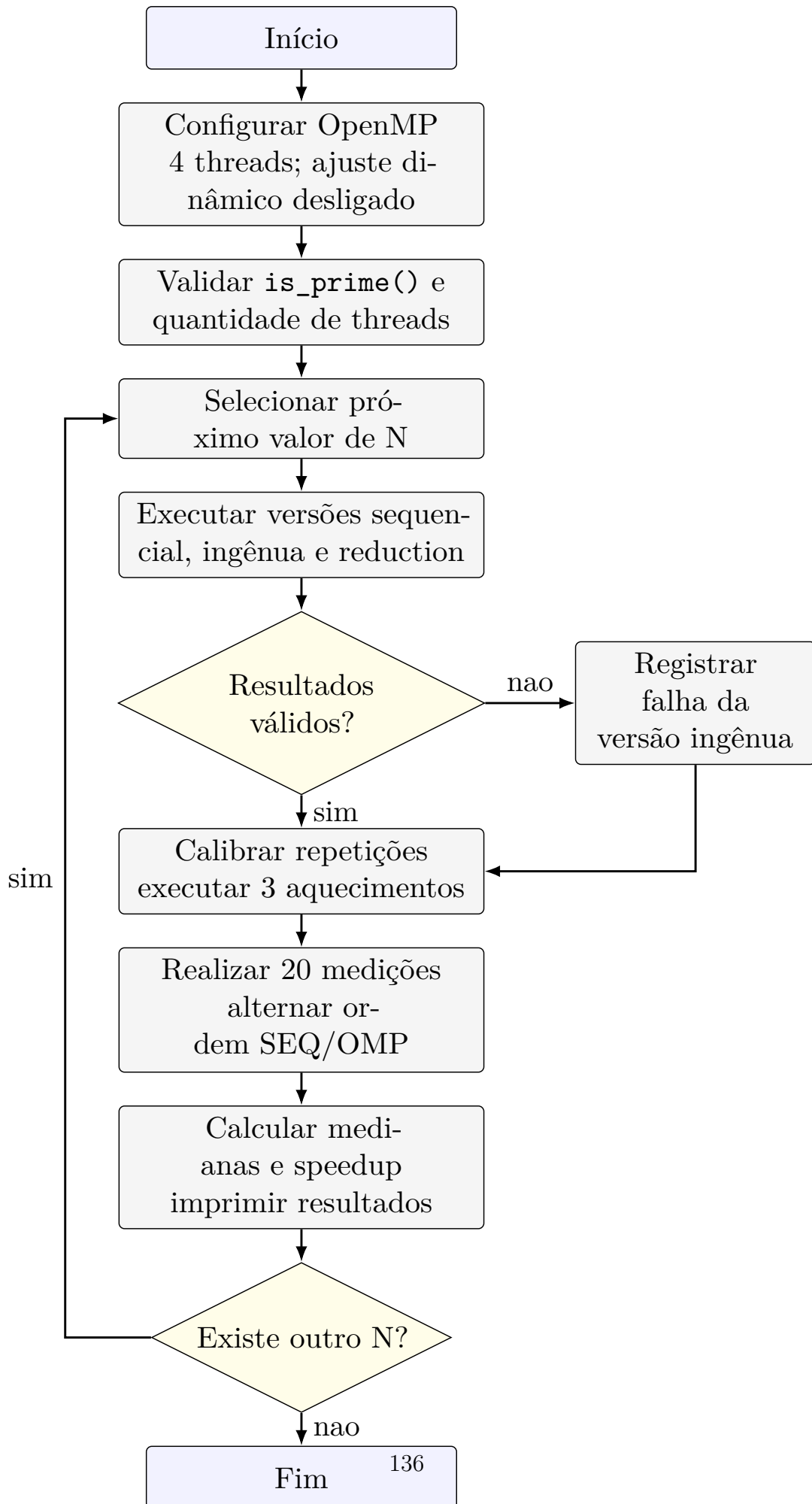


Figura 5.7: Diagrama UML de atividade do fluxo experimental.

- GCC 15.1.0;
- executável /ucrt64/bin/gcc;
- alvo x86_64-w64-mingw32;
- runtime OpenMP libgomp;
- macro `_OPENMP = 201511`, correspondente ao suporte a OpenMP 4.5.

A compilação foi realizada com:

```
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic \
    -fopenmp tarefa5.c -o tarefa5.exe
```

O mesmo executável contém as versões sequencial e paralela, garantindo o mesmo compilador e o mesmo nível de otimização para todas as comparações.

5.9.2 Valores de entrada

Foram utilizados cinco valores para o limite superior:

$$n \in \{10^4, 10^5, 10^6, 5 \times 10^6, 10^7\}.$$

As respectivas contagens conhecidas de números primos foram:

n	Quantidade de primos
10 000	1 229
100 000	9 592
1 000 000	78 498
5 000 000	348 513
10 000 000	664 579

5.9.3 Configuração das medições

A configuração experimental utilizada foi:

Parâmetro	Configuração
Threads OpenMP	4
Escalonamento	static
Aquecimentos	3
Medições	20
Estatística	mediana
Relógio	omp_get_wtime()
Lote mínimo de temporização	aproximadamente 100 ms
Otimização	-O2

5.9.4 Região cronometrada

Somente a operação de contagem de primos foi incluída na região cronometrada.

A estrutura utilizada foi equivalente a:

```
inicio = omp_get_wtime();
```

```
executar contagem;
```

```
fim = omp_get_wtime();
```

Na versão OpenMP, isso inclui o custo efetivo de:

- entrada na região paralela;
- distribuição das iterações;
- execução das threads;
- redução dos resultados;
- sincronização final.

Foram mantidos fora da região medida:

- impressão dos resultados;
- cálculo da mediana;
- cálculo do speedup;
- comparação funcional;
- configuração do OpenMP.

5.9.5 Aquecimentos

Antes das medições oficiais foram executados três aquecimentos por configuração.

Essa etapa reduz a influência da primeira ativação do runtime OpenMP e de efeitos transitórios da execução inicial.

5.9.6 Alternância da ordem

Para reduzir viés sistemático de ordem, as versões foram alternadas entre as medições.

O padrão utilizado foi:

```
medicao 1:  SEQ -> OMP
medicao 2:  OMP -> SEQ
medicao 3:  SEQ -> OMP
medicao 4:  OMP -> SEQ
...
```

Dessa forma, nenhuma implementação é sistematicamente beneficiada ou prejudicada por ser executada sempre primeiro.

5.9.7 Calibração das repetições internas

O número de repetições internas foi determinado automaticamente por duplicação:

1, 2, 4, 8, 16, ...

até que os lotes das implementações válidas alcançassem a duração mínima estabelecida.

A mesma quantidade de repetições foi utilizada para as duas versões dentro de cada valor de n .

Os tempos apresentados correspondem ao tempo normalizado de uma única contagem.

5.9.8 Prevenção de eliminação da computação

O resultado da computação possui efeito observável e é posteriormente validado.

Além disso, o *harness* de benchmark utiliza um valor protegido para o limite n durante as repetições internas.

Não foi utilizado `volatile` no algoritmo de primalidade propriamente dito, evitando alterar artificialmente seu comportamento de otimização.

5.10 Código-fonte desenvolvido

```
1 /*
2  * Tarefa 5 - Contagem de numeros primos com OpenMP
3  *
4  * Implementa tres versoes da contagem de primos:
5  * sequencial, OpenMP ingenua (com race condition) e
6  * OpenMP correta com reduction.
7  *
8  * O benchmark compara apenas as versoes corretas.
9  * O laço externo e paralelizado; o teste de primalidade
```

```

10  * permanece sequencial.
11  */
12
13  #include <stdio.h>
14  #include <stdlib.h>
15  #include <omp.h>
16
17  /* Configuracao do experimento. */
18  #define NUM_THREADS          4
19  #define NUM_WARMUPS          3
20  #define NUM_MEASUREMENTS     20
21
22  /* Duracao minima desejada para cada lote cronometrado. */
23  #define TARGET_BATCH_SECONDS  0.100
24
25  /* Limite de seguranca da calibracao automatica. */
26  #define MAX_INTERNAL_REPETITIONS 1048576
27
28  /* Valores de N avaliados no experimento. */
29  static const int N_VALUES[] = {
30      10000,
31      100000,
32      1000000,
33      5000000,
34      10000000
35  };
36
37  /* Contagens conhecidas usadas na validacao. */
38  static const long long EXPECTED_PRIMES[] = {
39      1229,
40      9592,
41      78498,
42      348513,
43      664579
44  };
45
46  #define NUM_SIZES \
47      (sizeof(N_VALUES) / sizeof(N_VALUES[0]))
48
49  /* Assinatura comum das funcoes de contagem. */
50  typedef long long (*CountFunction)(int);
51
52  /*
53   * Verifica se um valor e primo.
54   * Testa apenas divisores impares ate a raiz quadrada implicita.
55   */
56  static int is_prime(int value)
57  {
58
59      /* Casos basicos e eliminacao imediata de pares. */
60      if (value < 2) {
61          return 0;
62      }
63
64      if (value == 2) {
65          return 1;
66      }
67
68      if (value % 2 == 0) {

```

```

69     return 0;
70 }
71
72 /* Testa apenas divisores impares; value/divisor evita overflow. */
73 for (int divisor = 3;
74      divisor <= value / divisor;
75      divisor += 2) {
76
77     if (value % divisor == 0) {
78         return 0;
79     }
80 }
81
82 return 1;
83 }
84
85 /* Conta primos entre 2 e n de forma sequencial. */
86 static long long count_primes_sequential(int n)
87 {
88     long long count = 0;
89
90     for (int candidate = 2;
91          candidate <= n;
92          ++candidate) {
93
94         if (is_prime(candidate)) {
95             ++count;
96         }
97     }
98
99     return count;
100 }
101
102 /*
103  * Versao OpenMP ingenua.
104  * O contador compartilhado e atualizado sem sincronizacao,
105  * produzindo uma race condition proposital.
106  */
107 static long long count_primes_parallel_naive(int n)
108 {
109     long long count = 0;
110
111     #pragma omp parallel for schedule(static) shared(count)
112     for (int candidate = 2;
113          candidate <= n;
114          ++candidate) {
115
116         if (is_prime(candidate)) {
117             /* Atualizacao compartilhada sem protecao: race condition. */
118             ++count;
119         }
120     }
121
122     return count;
123 }
124
125 /*
126  * Versao OpenMP correta.
127  * A reduction combina com seguranca os contadores das threads.

```

```

128  */
129  static long long count_primes_parallel_reduction(int n)
130  {
131      long long count = 0;
132
133      #pragma omp parallel for schedule(static) reduction(+:count)
134      for (int candidate = 2;
135           candidate <= n;
136           ++candidate) {
137
138          if (is_prime(candidate)) {
139              ++count;
140          }
141      }
142
143      return count;
144  }
145
146  /* Valida a funcao de primalidade com casos conhecidos. */
147  static void validate_is_prime(void)
148  {
149      struct PrimeTest {
150          int value;
151          int expected;
152      };
153
154      static const struct PrimeTest tests[] = {
155          {0, 0},
156          {1, 0},
157          {2, 1},
158          {3, 1},
159          {4, 0},
160          {5, 1},
161          {9, 0},
162          {17, 1},
163          {25, 0}
164      };
165
166      const size_t num_tests =
167          sizeof(tests) / sizeof(tests[0]);
168
169      for (size_t i = 0;
170           i < num_tests;
171           ++i) {
172
173          const int result =
174              is_prime(tests[i].value);
175
176          if (result != tests[i].expected) {
177
178              fprintf(
179                  stderr,
180                  "ERROR: primality test failed for %d.\n",
181                  tests[i].value
182              );
183
184              exit(EXIT_FAILURE);
185          }
186      }

```

```

187 }
188
189 /* Retorna o numero de threads criado pelo runtime OpenMP. */
190 static int get_actual_thread_count(void)
191 {
192     int actual_threads = 1;
193
194     #pragma omp parallel
195     {
196         #pragma omp single
197         {
198             actual_threads =
199                 omp_get_num_threads();
200         }
201     }
202
203     return actual_threads;
204 }
205
206 /*
207  * Executa um lote cronometrado e valida o resultado agregado.
208  * Retorna o tempo total do lote em segundos.
209  */
210 static double run_timed_batch(
211     CountFunction function,
212     int n,
213     long long expected,
214     int repetitions
215 )
216 {
217
218     /* Evita que chamadas repetidas sejam tratadas como trabalho invariavel. */
219     volatile int guarded_n = n;
220
221     long long checksum = 0;
222
223     const double start =
224         omp_get_wtime();
225
226     for (int repetition = 0;
227         repetition < repetitions;
228         ++repetition) {
229
230         const int current_n =
231             guarded_n;
232
233         checksum +=
234             function(current_n);
235     }
236
237     const double end =
238         omp_get_wtime();
239
240     /* Validacao fora da regioa cronometrada. */
241     const long long expected_checksum =
242         expected * (long long) repetitions;
243
244     if (checksum != expected_checksum) {
245

```

```

246     fprintf(
247         stderr,
248         "ERROR: invalid benchmark result for N=%d.\n",
249         n
250     );
251
252     exit(EXIT_FAILURE);
253 }
254
255 return end - start;
256 }
257
258 /*
259  * Define repeticoes internas por duplicacao ate que os lotes
260  * sequencial e paralelo atinjam a duracao minima desejada.
261  */
262 static int calibrate_internal_repetitions(
263     int n,
264     long long expected
265 )
266 {
267     int repetitions = 1;
268
269     while (1) {
270
271         const double seq_elapsed =
272             run_timed_batch(
273                 count_primes_sequential,
274                 n,
275                 expected,
276                 repetitions
277             );
278
279         const double omp_elapsed =
280             run_timed_batch(
281                 count_primes_parallel_reduction,
282                 n,
283                 expected,
284                 repetitions
285             );
286
287         if (seq_elapsed >= TARGET_BATCH_SECONDS &&
288             omp_elapsed >= TARGET_BATCH_SECONDS) {
289
290             return repetitions;
291         }
292
293         if (repetitions >=
294             MAX_INTERNAL_REPETITIONS / 2) {
295
296             return MAX_INTERNAL_REPETITIONS;
297         }
298
299         repetitions *= 2;
300     }
301 }
302
303 /* Executa aquecimentos usando o mesmo tamanho de lote calibrado. */
304 static void run_warmups(

```

```

305     int n,
306     long long expected,
307     int repetitions
308 )
309 {
310     for (int i = 0;
311         i < NUM_WARMUPS;
312         ++i) {
313
314         /* Alterna a ordem para reduzir vies de execucao. */
315         if (i % 2 == 0) {
316
317             (void) run_timed_batch(
318                 count_primes_sequential,
319                 n,
320                 expected,
321                 repetitions
322             );
323
324             (void) run_timed_batch(
325                 count_primes_parallel_reduction,
326                 n,
327                 expected,
328                 repetitions
329             );
330         }
331         else {
332
333             (void) run_timed_batch(
334                 count_primes_parallel_reduction,
335                 n,
336                 expected,
337                 repetitions
338             );
339
340             (void) run_timed_batch(
341                 count_primes_sequential,
342                 n,
343                 expected,
344                 repetitions
345             );
346         }
347     }
348 }
349
350 /* Retorna o tempo medio de uma contagem dentro do lote. */
351 static double measure_per_execution(
352     CountFunction function,
353     int n,
354     long long expected,
355     int repetitions
356 )
357 {
358     const double elapsed =
359         run_timed_batch(
360             function,
361             n,
362             expected,
363             repetitions

```

```

364     );
365
366     return elapsed /
367         (double) repetitions;
368 }
369
370 /* Comparador usado pelo qsort na ordenacao das medicoes. */
371 static int compare_double(
372     const void *a,
373     const void *b
374 )
375 {
376     const double x =
377         *(const double *) a;
378
379     const double y =
380         *(const double *) b;
381
382     if (x < y) {
383         return -1;
384     }
385
386     if (x > y) {
387         return 1;
388     }
389
390     return 0;
391 }
392
393 /* Ordena as medicoes e calcula a mediana. */
394 static double calculate_median(
395     double values[],
396     size_t count
397 )
398 {
399     qsort(
400         values,
401         count,
402         sizeof(double),
403         compare_double
404     );
405
406     if (count % 2 != 0) {
407         return values[count / 2];
408     }
409
410     return (
411         values[count / 2 - 1]
412         +
413         values[count / 2]
414     ) / 2.0;
415 }
416
417 /* Coordena validacao, calibracao, benchmark e impressao dos resultados. */
418 int main(void)
419 {
420
421     /* Mantem fixo o numero de threads do experimento. */
422     omp_set_dynamic(0);

```

```

423
424     omp_set_num_threads(NUM_THREADS);
425
426     /* Validacao inicial da logica de primalidade. */
427     validate_is_prime();
428
429     const int actual_threads =
430         get_actual_thread_count();
431
432     if (actual_threads != NUM_THREADS) {
433
434         fprintf(
435             stderr,
436             "ERROR: expected %d OpenMP threads, "
437             "but runtime created %d.\n",
438             NUM_THREADS,
439             actual_threads
440         );
441
442         return EXIT_FAILURE;
443     }
444
445     long long sequential_results[NUM_SIZES];
446     long long naive_results[NUM_SIZES];
447     long long reduction_results[NUM_SIZES];
448
449     int internal_repetitions[NUM_SIZES];
450
451     /* Valida as tres versoes para cada N. */
452     for (size_t i = 0;
453          i < NUM_SIZES;
454          ++i) {
455
456         const int n =
457             N_VALUES[i];
458
459         sequential_results[i] =
460             count_primes_sequential(n);
461
462         naive_results[i] =
463             count_primes_parallel_naive(n);
464
465         reduction_results[i] =
466             count_primes_parallel_reduction(n);
467
468         if (sequential_results[i] !=
469             EXPECTED_PRIMES[i]) {
470
471             fprintf(
472                 stderr,
473                 "ERROR: expected %lld primes for N=%d, "
474                 "but sequential version returned %lld.\n",
475                 EXPECTED_PRIMES[i],
476                 n,
477                 sequential_results[i]
478             );
479
480             return EXIT_FAILURE;
481         }

```

```

482
483     if (reduction_results[i] !=
484         sequential_results[i]) {
485
486         fprintf(
487             stderr,
488             "ERROR: reduction result differs "
489             "from sequential for N=%d.\n",
490             n
491         );
492
493         return EXIT_FAILURE;
494     }
495 }
496
497 /* Cabecalho do experimento. */
498 printf(
499     "===== \n"
500 );
501
502 printf(
503     "PRIME COUNT - OPENMP EXPERIMENT \n"
504 );
505
506 printf(
507     "===== \n \n"
508 );
509
510 printf(
511     "Threads      : %d \n",
512     actual_threads
513 );
514
515 printf(
516     "Schedule      : static \n"
517 );
518
519 printf(
520     "Warm-ups      : %d \n",
521     NUM_WARMUPS
522 );
523
524 printf(
525     "Measurements : %d \n",
526     NUM_MEASUREMENTS
527 );
528
529 printf(
530     "Statistic     : median \n"
531 );
532
533 printf(
534     "Timing batch : >= %.0f ms \n",
535     TARGET_BATCH_SECONDS * 1000.0
536 );
537
538 /* Tabela 1: validacao dos resultados. */
539 printf(
540     "\n----- \n"

```

```

541     );
542
543     printf(
544         "RESULT VALIDATION\n"
545     );
546
547     printf(
548         "-----\n\n"
549     );
550
551     printf(
552         "%-13s %-13s %-14s %-15s %-14s\n",
553         "N",
554         "Sequential",
555         "Naive OMP",
556         "Reduction OMP",
557         "Naive Status"
558     );
559
560     printf(
561         "-----\n"
562     );
563
564     int naive_match_detected = 0;
565
566     for (size_t i = 0;
567          i < NUM_SIZES;
568          ++i) {
569
570         const int naive_matches =
571             naive_results[i] ==
572             sequential_results[i];
573
574         const char *status;
575
576         if (naive_matches) {
577
578             status = "MATCH*";
579
580             naive_match_detected = 1;
581         }
582         else {
583
584             status = "INCORRECT";
585         }
586
587         printf(
588             "%-13d %-13lld %-14lld %-15lld %-14s\n",
589             N_VALUES[i],
590             sequential_results[i],
591             naive_results[i],
592             reduction_results[i],
593             status
594         );
595     }
596
597     if (naive_match_detected) {
598
599         printf(

```

```

600         "\n"
601         "* MATCH means only that this particular execution "
602         "produced\n"
603         " the same value. The naive version still contains "
604         "a race condition.\n"
605     );
606 }
607
608 /* Tabela 2: desempenho das implementacoes validas. */
609 printf(
610     "\n\n-----\n"
611 );
612
613 printf(
614     "PERFORMANCE - VALID IMPLEMENTATIONS\n"
615 );
616
617 printf(
618     "-----\n\n"
619 );
620
621 printf(
622     "%-12s %-11s %-8s %-13s %-13s %-10s\n",
623     "N",
624     "Primes",
625     "Reps",
626     "Seq(ms)",
627     "OpenMP(ms)",
628     "Speedup"
629 );
630
631 printf(
632     "-----\n"
633 );
634
635 for (size_t i = 0;
636      i < NUM_SIZES;
637      ++i) {
638
639     const int n =
640         N_VALUES[i];
641
642     const long long expected =
643         sequential_results[i];
644
645     internal_repetitions[i] =
646         calibrate_internal_repetitions(
647             n,
648             expected
649         );
650
651     const int repetitions =
652         internal_repetitions[i];
653
654     run_warmups(
655         n,
656         expected,
657         repetitions
658     );

```

```

659
660     double sequential_times[
661         NUM_MEASUREMENTS
662     ];
663
664     double parallel_times[
665         NUM_MEASUREMENTS
666     ];
667
668     /* Alterna SEQ->OMP e OMP->SEQ entre as 20 medicoes. */
669     for (int measurement = 0;
670         measurement < NUM_MEASUREMENTS;
671         ++measurement) {
672
673         if (measurement % 2 == 0) {
674
675             sequential_times[measurement] =
676                 measure_per_execution(
677                     count_primes_sequential,
678                     n,
679                     expected,
680                     repetitions
681                 );
682
683             parallel_times[measurement] =
684                 measure_per_execution(
685                     count_primes_parallel_reduction,
686                     n,
687                     expected,
688                     repetitions
689                 );
690         }
691         else {
692
693             parallel_times[measurement] =
694                 measure_per_execution(
695                     count_primes_parallel_reduction,
696                     n,
697                     expected,
698                     repetitions
699                 );
700
701             sequential_times[measurement] =
702                 measure_per_execution(
703                     count_primes_sequential,
704                     n,
705                     expected,
706                     repetitions
707                 );
708         }
709     }
710
711     /* Medianas das 20 medicoes por implementacao. */
712     const double seq_median =
713         calculate_median(
714             sequential_times,
715             NUM_MEASUREMENTS
716         );
717

```

```

718     const double omp_median =
719         calculate_median(
720             parallel_times,
721             NUM_MEASUREMENTS
722         );
723
724     if (seq_median <= 0.0 ||
725         omp_median <= 0.0) {
726
727         fprintf(
728             stderr,
729             "ERROR: invalid timing result for N=%d.\n",
730             n
731         );
732
733         return EXIT_FAILURE;
734     }
735
736     /* Speedup = tempo sequencial / tempo OpenMP. */
737     const double speedup =
738         seq_median /
739         omp_median;
740
741     /* Conversao de segundos para milissegundos. */
742     const double seq_ms =
743         seq_median * 1000.0;
744
745     const double omp_ms =
746         omp_median * 1000.0;
747
748     printf(
749         "%-12d %-11ld %-8d %-13.3f %-13.3f %.2fx\n",
750         n,
751         expected,
752         repetitions,
753         seq_ms,
754         omp_ms,
755         speedup
756     );
757 }
758
759 return EXIT_SUCCESS;
760 }

```

Listing 5.1: Código-fonte da Atividade 5.

5.11 Validação da implementação

A validação foi realizada em três níveis.

5.11.1 Validação da função de primalidade

A função `is_prime()` foi testada utilizando casos conhecidos, incluindo números primos, compostos e valores inferiores a 2.

Entre os casos utilizados estavam:

$$0, 1, 2, 3, 4, 5, 9, 17, 25.$$

5.11.2 Validação por contagens conhecidas

A contagem sequencial foi comparada com valores conhecidos de $\pi(n)$, onde $\pi(n)$ representa a quantidade de primos menores ou iguais a n .

Todas as cinco entradas utilizadas no experimento produziram os valores esperados.

5.11.3 Comparação entre as versões

Os resultados obtidos foram:

N	Sequencial	OMP ingênua	OMP reduction	Status ingênua
10 000	1 229	279	1 229	incorreta
100 000	9 592	2 199	9 592	incorreta
1 000 000	78 498	18 260	78 498	incorreta
5 000 000	348 513	81 795	348 513	incorreta
10 000 000	664 579	156 318	664 579	incorreta

A versão sequencial e a versão com redução coincidiram em todos os casos:

$$C_{\text{seq}}(n) = C_{\text{reduction}}(n).$$

A implementação ingênua apresentou contagens inferiores em todos os tamanhos testados.

Isso confirma experimentalmente a presença da condição de corrida.

Os valores produzidos pela versão ingênua não devem ser interpretados como uma fração determinística da resposta correta. Uma condição de corrida não oferece garantia sobre o valor final, mesmo que resultados semelhantes possam ocorrer em execuções realizadas sob as mesmas condições.

5.12 Resultados experimentais

Após a calibração temporal e a execução das vinte medições por configuração, foram obtidos os seguintes resultados finais.

N	Primos	Reps	Seq. (ms)	OpenMP (ms)	Speedup
10 000	1 229	512	0,282	0,195	1,45×
100 000	9 592	64	5,836	2,984	1,96×
1 000 000	78 498	2	170,250	72,500	2,35×
5 000 000	348 513	1	1 262,000	603,000	2,09×
10 000 000	664 579	1	3 397,000	1 610,000	2,11×

Os valores de **Reps** correspondem ao número de execuções internas utilizadas em cada lote de temporização.

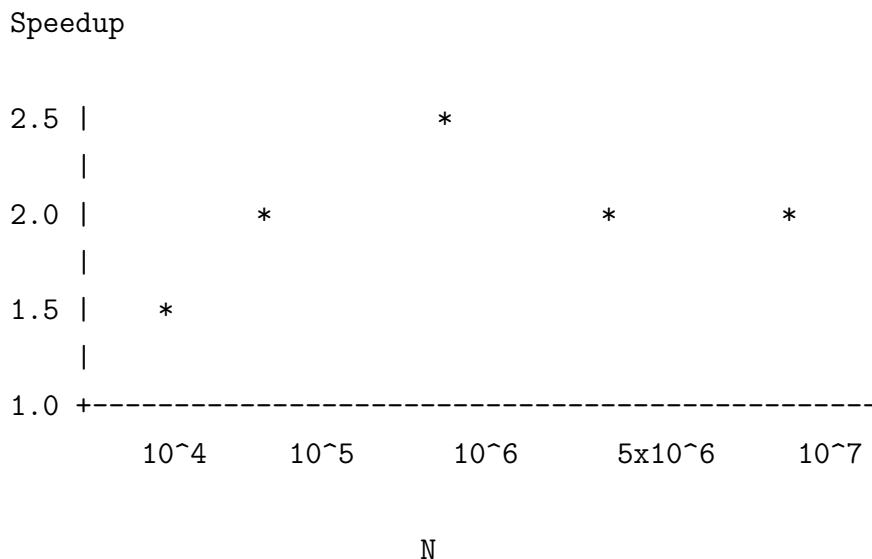
O valor reportado nas colunas de tempo continua correspondendo a uma única execução, pois o tempo total do lote foi normalizado pelo número de repetições.

5.13 Visualização dos resultados

A evolução do speedup pode ser resumida por:

N	Speedup
10 000	1,45×
100 000	1,96×
1 000 000	2,35×
5 000 000	2,09×
10 000 000	2,11×

Uma representação textual da tendência observada é:



O maior speedup da execução final foi observado em

$$N = 1\,000\,000,$$

com

$$S = 2,35 \times .$$

Nos dois maiores tamanhos avaliados, os valores observados ficaram próximos de

$$2,1 \times .$$

5.14 Análise e discussão

5.14.1 Correção da versão sequencial

A implementação sequencial produziu exatamente as contagens conhecidas para todos os valores de entrada avaliados.

Isso estabelece uma referência funcional confiável para a comparação com as implementações paralelas.

5.14.2 Falha da paralelização ingênua

A inclusão de

```
#pragma omp parallel for
```

sem tratamento adicional do contador compartilhado produziu resultados incorretos em todos os tamanhos avaliados.

Por exemplo, para

$$N = 10\,000\,000,$$

a resposta correta foi

$$664\,579,$$

enquanto a versão ingênua retornou

$$156\,318.$$

A diferença não decorre da função de primalidade, pois o mesmo `is_prime()` é utilizado nas três versões.

A causa está na atualização concorrente de `count`.

Dessa forma, o experimento demonstra concretamente que:

paralelização sintaticamente válida $\nRightarrow$ programa funcionalmente correto

5.14.3 Correção por redução

A versão com

```
reduction(+:count)
```

reproduziu exatamente a contagem sequencial para todas as entradas.

Isso ocorre porque OpenMP mantém contribuições parciais das threads e as combina por soma ao final da região paralela.

Portanto, a redução corresponde diretamente à estrutura matemática do problema.

5.14.4 Efeito do tamanho do problema

Para $N = 10\,000$, o speedup foi:

$$S = 1,45 \times .$$

Embora a versão paralela já tenha sido mais rápida, o benefício foi menor que nos tamanhos intermediários.

Isso é coerente com o efeito do overhead paralelo.

O tempo da implementação OpenMP pode ser representado conceitualmente por:

$$T_{\text{omp}} = T_{\text{computacao}} + T_{\text{overhead}},$$

onde o segundo termo inclui custos de gerenciamento da região paralela, distribuição das iterações, redução e sincronização.

Quando o problema é pequeno, esse custo representa parcela proporcionalmente maior do tempo total.

5.14.5 Speedup observado

Os speedups obtidos foram:

$$1,45\times, \quad 1,96\times, \quad 2,35\times, \quad 2,09\times, \quad 2,11\times .$$

A melhor aceleração observada ocorreu para

$$N = 10^6,$$

com

$$S = 2,35 \times .$$

Nos dois maiores tamanhos, os ganhos ficaram próximos de $2,1 \times$.

A utilização de quatro threads não implica um speedup de $4 \times$.

Para atingir o speedup ideal seria necessário, entre outros fatores:

- ausência de overhead de paralelização;
- trabalho perfeitamente balanceado;
- ausência de sincronização;
- disponibilidade integral dos recursos computacionais.

Essas condições não ocorrem no experimento real.

5.14.6 Distribuição de carga

A política utilizada foi `schedule(static)`.

Ela distribui previamente conjuntos de iterações entre as threads e possui baixo overhead.

Entretanto, igual quantidade de candidatos não corresponde necessariamente a igual quantidade de trabalho.

Um número par pode ser rejeitado rapidamente, enquanto um primo grande pode exigir testes de diversos divisores.

Além disso, candidatos maiores apresentam um limite potencial de teste também maior. Assim,

mesmo número de iterações $\nrightarrow$ mesma carga computacional.

Consequentemente, algumas threads podem terminar seus blocos antes de outras e permanecer aguardando a barreira implícita do final do `parallel for`.

Esse desequilíbrio contribui para limitar o speedup observado.

5.14.7 Crescimento do custo sequencial

Os tempos sequenciais cresceram de

0,282 ms

para

3397 ms

ao longo da faixa avaliada.

O crescimento não é linear em N , pois o custo de testar um candidato também cresce com seu valor e depende de seus fatores.

Dessa forma, aumentar N aumenta simultaneamente:

1. a quantidade de candidatos avaliados;
2. o custo potencial de algumas avaliações individuais.

5.14.8 Efeito da calibração temporal

A calibração das repetições internas mostrou-se necessária para os menores valores de N .

As quantidades finais foram:

512, 64, 2, 1, 1.

Isso produz o comportamento esperado:

$$N \uparrow \Rightarrow T_{\text{execucao}} \uparrow \Rightarrow R \downarrow.$$

Para os dois maiores tamanhos, uma única contagem já possui duração suficiente para a medição.

A técnica eliminou o problema inicialmente observado de tempos reportados como zero para cargas pequenas sem alterar o algoritmo que estava sendo comparado.

5.15 Limitações

Os resultados devem ser interpretados considerando algumas limitações.

5.15.1 Uma única máquina

As medições correspondem ao ambiente utilizado na atividade.

Arquiteturas diferentes podem apresentar outros níveis de speedup em razão de diferenças no número de núcleos, memória, frequência, sistema operacional e implementação do runtime OpenMP.

5.15.2 Número fixo de threads

Foram utilizadas quatro threads em todas as execuções.

Portanto, a atividade não permite concluir sobre escalabilidade para quantidades maiores ou menores de threads.

5.15.3 Política de escalonamento fixa

Apenas `schedule(static)` foi utilizado.

Outras políticas de OpenMP podem responder diferentemente ao custo irregular das iterações.

A comparação entre políticas de escalonamento não fazia parte do escopo experimental desta atividade.

5.15.4 Interferência do sistema operacional

Mesmo com aquecimentos e múltiplas medições, processos externos, escalonamento do sistema operacional, alterações de frequência da CPU e condições térmicas podem introduzir variação temporal.

A utilização da mediana reduz, mas não elimina completamente, esse efeito.

5.15.5 Ausência de afinidade explícita

Não foi implementado controle explícito de afinidade de threads aos processadores.

A localização efetiva das threads foi, portanto, determinada pelo runtime e pelo sistema operacional.

5.15.6 Versão ingênua

A versão com condição de corrida foi mantida exclusivamente para demonstração.

Seus resultados não podem ser utilizados para avaliação de desempenho válida, pois ela não preserva a semântica do programa sequencial.

Uma implementação incorreta não pode ser considerada uma otimização mesmo quando executa em menor tempo.

5.15.7 Resolução temporal

A primeira modelagem baseada em uma única contagem por medição mostrou-se insuficiente para os menores valores de N .

A utilização posterior de lotes calibrados mitigou essa limitação, mas também tornou necessário normalizar o tempo pela quantidade de repetições.

5.16 Conclusões da atividade

A atividade demonstrou que o laço externo da contagem de números primos é adequado ao paralelismo de dados, pois os diferentes candidatos podem ser avaliados independentemente.

Entretanto, a simples aplicação de

```
#pragma omp parallel for
```

não foi suficiente para preservar a correção.

A atualização concorrente do contador introduziu uma condição de corrida, produzindo resultados incorretos para todos os tamanhos avaliados.

A utilização de

```
reduction(+:count)
```

eliminou esse problema e fez com que a implementação OpenMP reproduzisse exatamente os resultados da versão sequencial.

Após a validação funcional, a comparação de desempenho mostrou benefício consistente da paralelização.

Os speedups observados variaram entre

$$1,45 \times$$

e

$$2,35 \times .$$

O melhor resultado foi obtido para $N = 10^6$, enquanto os dois maiores problemas apresentaram ganhos próximos de $2,1 \times$.

A atividade também evidenciou que o desempenho paralelo depende não apenas da quantidade de threads, mas de fatores como overhead, granularidade, sincronização e balanceamento de carga.

Finalmente, a necessidade de calibração temporal para cargas pequenas demonstrou que a metodologia de benchmark também constitui parte essencial de uma avaliação de desempenho: medições rápidas demais podem ser dominadas pela resolução do relógio e produzir conclusões inadequadas.

5.17 Síntese

Os principais pontos para apresentação oral da atividade são:

1. **Problema:** contar quantos números primos existem entre 2 e N .

2. **Algoritmo-base:** o mesmo teste de primalidade foi utilizado em todas as versões para garantir uma comparação justa.
3. **Paralelização:** somente o laço externo dos candidatos foi paralelizado. O laço interno dos divisores permaneceu sequencial.
4. **Versões:** foram implementadas uma versão sequencial, uma versão OpenMP ingênua e uma versão OpenMP com `reduction`.
5. **Problema da versão ingênua:** múltiplas threads atualizam `count++` simultaneamente, produzindo uma condição de corrida.
6. **Evidência da condição de corrida:** para $N = 10^7$, a contagem correta foi 664 579, enquanto a versão ingênua retornou 156 318.
7. **Correção:** a versão com `reduction` coincidiu com a versão sequencial para todos os cinco valores de N .
8. **Threads:** foram utilizadas quatro threads fixas, com ajuste dinâmico desabilitado.
9. **Scheduling:** foi utilizado `schedule(static)` em toda a atividade para manter essa variável controlada.
10. **Temporização:** foi utilizado `omp_get_wtime()`.
11. **Metodologia:** foram utilizados 3 warm-ups, 20 medições e a mediana como estatística principal.
12. **Problema de resolução temporal:** execuções pequenas inicialmente produziram tempos iguais a zero. O problema foi corrigido por repetição interna calibrada até lotes de aproximadamente 100 ms.
13. **Speedup:** foi calculado por

$$S = \frac{T_{\text{seq}}}{T_{\text{omp}}}.$$
14. **Resultado de desempenho:** os speedups finais foram

$$1,45\times, 1,96\times, 2,35\times, 2,09\times, 2,11\times.$$
15. **Melhor speedup observado:**

$$2,35\times$$
 para $N = 1\,000\,000$.
16. **Por que quatro threads não geraram $4\times$:** existem overhead de OpenMP, sincronização, competição por recursos e desequilíbrio de carga.

17. **Distribuição de carga:** candidatos diferentes possuem custos diferentes; números compostos simples podem terminar rapidamente, enquanto primos grandes demandam mais testes.
18. **Conclusão principal:** primeiro é necessário garantir correção; somente depois faz sentido avaliar desempenho. A versão com **reduction** foi simultaneamente correta e mais rápida que a implementação sequencial nos tamanhos avaliados.

Capítulo 6

Atividade 6 – Integral pelo método do trapézio: sincronização em OpenMP

6.1 Enunciado da tarefa

Implemente diferentes versões de um programa para o cálculo de integrais definidas utilizando o método do trapézio.

O programa deve permitir calcular numericamente a integral de uma função $f(x)$ em um intervalo $[a, b]$, utilizando n subdivisões.

Devem ser implementadas as versões sequencial, paralela sem tratamento para região crítica, paralela utilizando `critical`, paralela utilizando `atomic` e paralela utilizando `reduction`.

Após a implementação, compare os resultados obtidos e os tempos médios de execução de cada versão à medida que o número de subdivisões n aumenta. Utilize gráficos para apresentar e analisar os resultados.

6.2 Objetivos e requisitos

A Atividade 6 constitui uma extensão direta da Atividade 4. Na atividade anterior, o método composto do trapézio foi implementado sequencialmente e a estrutura do algoritmo foi analisada quanto às possibilidades de paralelização. Nesta atividade, essas possibilidades foram efetivamente implementadas utilizando OpenMP.

O principal objetivo foi avaliar experimentalmente diferentes formas de tratar a variável acumuladora existente no método do trapézio quando o laço principal é executado por múltiplas threads.

Foram implementadas cinco versões:

1. implementação sequencial;

2. implementação paralela sem sincronização;
3. implementação paralela utilizando `critical`;
4. implementação paralela utilizando `atomic`;
5. implementação paralela utilizando `reduction`.

Os principais requisitos experimentais foram:

- manter o mesmo problema matemático utilizado na Atividade 4;
- utilizar `int64_t` no cálculo numérico;
- variar apenas o número de subdivisões N ;
- manter fixo o número de threads;
- utilizar o mesmo escalonamento nas versões paralelas;
- medir o tempo médio de execução;
- validar a correção das versões sincronizadas;
- registrar, sem interromper o experimento, os erros da versão sem sincronização;
- calcular o speedup em relação à implementação sequencial;
- produzir dados adequados à construção de gráficos;
- analisar o impacto de sincronização, contenção e redução sobre o desempenho.

6.3 Fundamentação conceitual

6.3.1 Método composto do trapézio

O problema numérico permaneceu o mesmo da Atividade 4. Uma integral definida

$$I = \int_a^b f(x) dx \quad (6.1)$$

é aproximada dividindo-se o intervalo $[a, b]$ em N subdivisões de largura

$$h = \frac{b - a}{N}. \quad (6.2)$$

Os pontos da discretização são

$$x_i = a + ih, \quad i = 0, 1, \dots, N. \quad (6.3)$$

A formulação utilizada na implementação foi

$$I_N = h \sum_{i=1}^{N-1} f(x_i) + \frac{h[f(a) + f(b)]}{2}. \quad (6.4)$$

A função utilizada foi

$$f(x) = x^2 \quad (6.5)$$

no intervalo

$$[a, b] = [0, 2\,400\,000]. \quad (6.6)$$

A integral exata é

$$I_{\text{exato}} = \int_0^{2\,400\,000} x^2 dx = \frac{(2\,400\,000)^3}{3}, \quad (6.7)$$

resultando em

$$I_{\text{exato}} = 4\,608\,000\,000\,000\,000\,000. \quad (6.8)$$

6.3.2 Erro numérico

Para $f(x) = x^2$, o erro do método composto do trapézio pode ser determinado exatamente por

$$E_N = \frac{(b-a)h^2}{6}. \quad (6.9)$$

Como x^2 é convexa, a aproximação produzida pelo método do trapézio é superior ao valor analítico:

$$I_N = I_{\text{exato}} + E_N. \quad (6.10)$$

Como

$$h = \frac{b-a}{N}, \quad (6.11)$$

segue que

$$E_N = O\left(\frac{1}{N^2}\right). \quad (6.12)$$

6.3.3 Paralelismo de dados

O núcleo da integração contém o laço

```
for i = 1 .. N-1
    x_i = a + i*h
    soma += f(x_i)
```

O cálculo de

$$x_i = a + ih \quad (6.13)$$

é independente entre iterações. Da mesma forma, a avaliação de

$$f(x_i) = x_i^2 \quad (6.14)$$

não depende da avaliação realizada para outro valor de i .

Existe, portanto, paralelismo de dados no conjunto de iterações.

A dependência relevante está na variável acumuladora

$$\text{soma.} \quad (6.15)$$

6.3.4 Condição de corrida

A instrução

```
soma += valor;
```

pode ser interpretada conceitualmente como:

1. leitura do valor atual de `soma`;
2. realização da adição;
3. escrita do novo valor.

Se múltiplas threads executarem essas operações simultaneamente sobre o mesmo acumulador, uma atualização pode sobrescrever outra.

Esse fenômeno caracteriza uma *race condition*.

A versão sem sincronização foi implementada deliberadamente dessa forma para demonstrar experimentalmente as consequências da concorrência não controlada.

6.3.5 Região critical

A diretiva

```
#pragma omp critical
```

determina que apenas uma thread por vez execute determinada região crítica.

Na implementação desta atividade, somente a atualização

```
soma += valor;
```

foi incluída na região crítica.

O cálculo de x_i e de $f(x_i)$ permaneceu fora da região serializada.

Essa estratégia garante correção, porém todas as iterações continuam competindo pelo mesmo recurso de exclusão mútua.

6.3.6 Operação `atomic`

A diretiva

```
#pragma omp atomic update
```

protege uma operação simples de atualização compartilhada.

No experimento, foi utilizada em

```
soma += valor;
```

A operação `atomic` apresenta granularidade menor que uma região `critical` genérica. Entretanto, todas as threads continuam atualizando o mesmo endereço de memória, de forma que permanece a contenção sobre o acumulador.

6.3.7 Redução

A operação de redução transforma o acumulador compartilhado em acumuladores privados durante a execução paralela.

No OpenMP, foi utilizada a cláusula

```
reduction(+:soma)
```

Conceitualmente, para P threads:

$$S_1, S_2, \dots, S_P \tag{6.16}$$

são calculados independentemente e posteriormente combinados:

$$S = S_1 + S_2 + \dots + S_P. \tag{6.17}$$

Dessa forma, não é necessária uma operação de sincronização sobre o mesmo acumulador a cada iteração do laço.

6.3.8 Speedup e eficiência

O speedup de uma implementação paralela é definido por

$$S_P = \frac{T_{\text{seq}}}{T_P}, \quad (6.18)$$

onde T_{seq} é o tempo da implementação sequencial e T_P é o tempo da implementação paralela.

Valores

$$S_P > 1 \quad (6.19)$$

indicam ganho de desempenho, enquanto

$$S_P < 1 \quad (6.20)$$

indicam que a implementação paralela foi mais lenta que o baseline.

A eficiência pode ser calculada por

$$E_P = \frac{S_P}{P}. \quad (6.21)$$

Nesta atividade foram utilizadas quatro threads.

6.4 Interpretação didática da tarefa

A Atividade 4 havia identificado teoricamente que o cálculo de x_i e $f(x_i)$ poderia ser distribuído, enquanto a variável acumuladora exigiria tratamento especial.

A Atividade 6 transforma essa discussão em um experimento.

As quatro estratégias paralelas representam abordagens progressivamente diferentes para a mesma dependência:

Estratégia	Tratamento do acumulador
SemSync	nenhum
Critical	exclusão mútua
Atomic	atualização atômica
Reduction	acumuladores privados + redução

A atividade permite observar que correção e desempenho não são problemas independentes.

Uma implementação pode apresentar baixo tempo de execução e ainda assim ser inútil caso produza resultados incorretos.

Da mesma forma, uma estratégia de sincronização pode garantir correção mas introduzir um custo superior ao benefício obtido pelo paralelismo.

6.5 Modelagem da solução

6.5.1 Estrutura geral

A implementação foi construída preservando o problema matemático e a infraestrutura desenvolvida na Atividade 4.

Para cada valor de N , foram executadas cinco versões:

Sequential

SemSync

Critical

Atomic

Reduction

Os valores utilizados foram

$$N \in \{24\,000, 120\,000, 240\,000, 400\,000, 800\,000, 1\,200\,000, 2\,400\,000\}. \quad (6.22)$$

Todos dividem exatamente

$$2\,400\,000, \quad (6.23)$$

mantendo h inteiro.

6.5.2 Principais funções e componentes

A implementação foi organizada nos seguintes componentes:

funcao_integrando() Calcula $f(x) = x^2$.

finalizar_integral() Combina a soma dos pontos internos com as contribuições dos extremos.

integral_sequencial() Implementa o baseline sequencial.

integral_sem_sincronizacao() Executa o laço com OpenMP sem proteção da variável acumuladora.

integral_critical() Protege a atualização do acumulador por uma região `critical`.

integral_atomic() Protege a atualização utilizando `atomic update`.

integral_reduction() Utiliza `reduction(+:soma)`.

obter_tempo() Obtém um instante utilizando `CLOCK_MONOTONIC`.

diferenca_ns() Calcula o tempo decorrido em nanossegundos.

resultado_esperado() Calcula o resultado correto esperado para determinado N .

calcular_erro_abs() Calcula o erro absoluto de uma observação.

medir_lote() Cronometra um conjunto de execuções do kernel.

determinar_repeticoes() Determina automaticamente quantas execuções devem compor um lote.

medir_versao() Executa calibração, aquecimentos, vinte medições e agregação dos resultados.

6.6 Decisões de projeto e trade-offs

6.6.1 Reutilização do problema da Atividade 4

A função, o intervalo, o tipo numérico e os valores de N foram mantidos.

Isso permite comparar diretamente a nova implementação com um baseline sequencial previamente validado e isola o mecanismo de sincronização como a principal diferença experimental.

6.6.2 Uso de quatro threads

O número de threads foi mantido fixo em

$$P = 4. \tag{6.24}$$

A atividade tem como variável experimental principal o número de subdivisões N . Variar simultaneamente N e o número de threads introduziria uma segunda dimensão experimental.

A configuração foi explicitamente fixada por

```
omp_set_dynamic(0);  
omp_set_num_threads(4);
```

evitando alteração dinâmica do tamanho da equipe pelo runtime OpenMP.

6.6.3 Escalonamento estático

Todas as versões paralelas utilizaram

```
schedule(static)
```

A função

$$f(x) = x^2 \quad (6.25)$$

possui custo aproximadamente uniforme entre as iterações.

Nesse cenário, o escalonamento estático evita o overhead adicional de distribuição dinâmica e mantém a política de escalonamento constante entre as versões.

6.6.4 Região crítica mínima

Na versão `critical`, apenas

```
soma += valor;
```

foi colocada na região crítica.

Se o cálculo completo da iteração fosse protegido, operações independentes também seriam serializadas, prejudicando ainda mais a possibilidade de paralelismo.

Essa decisão representa a versão mais restrita da seção crítica necessária para garantir a correção do acumulador.

6.6.5 Batching adaptativo

Na Atividade 4 foram utilizadas mil repetições fixas por medição.

Esse modelo não foi mantido na Atividade 6, pois versões como `critical` e `atomic` podem ser dezenas de vezes mais lentas.

Utilizar mil repetições em todas as configurações produziria tempos desnecessariamente longos.

Foi implementado, portanto, um batching adaptativo com alvo de

$$20 \text{ ms.} \quad (6.26)$$

Inicialmente é realizada uma medição piloto. O número de repetições é então ajustado de acordo com a razão entre o tempo observado e o tempo-alvo.

Se uma única execução já ultrapassar aproximadamente 20 ms, o lote pode permanecer com apenas uma repetição.

O tempo final é sempre normalizado para uma única execução:

$$T_{\text{exec}} = \frac{T_{\text{lote}}}{R}. \quad (6.27)$$

6.6.6 Média em vez de mediana

Na Atividade 4 foi utilizada a mediana.

O enunciado da Atividade 6 solicita explicitamente a comparação dos *tempos médios de execução*. Por esse motivo, a métrica principal foi alterada para

$$\bar{T} = \frac{1}{20} \sum_{j=1}^{20} T_j. \quad (6.28)$$

6.6.7 Tratamento especial de SemSync

Sequencial, `critical`, `atomic` e `reduction` devem produzir o resultado correto em todas as medições.

Se uma dessas versões falhar, o benchmark é interrompido.

A versão `SemSync`, entretanto, é propositalmente incorreta.

Seus resultados inválidos são registrados sem interromper o experimento.

Para essa versão são observados:

- quantidade de resultados corretos em vinte medições;
- erro absoluto médio;
- tempo médio de execução.

6.6.8 Erro absoluto médio

Inicialmente, o erro da versão `SemSync` era calculado apenas a partir do último resultado observado.

Como a `race condition` torna o resultado não determinístico, essa medida não era representativa das vinte execuções.

Na versão final, calcula-se

$$\bar{E}_{\text{abs}} = \frac{1}{20} \sum_{j=1}^{20} |I_j - I_{\text{exato}}|. \quad (6.29)$$

O acumulador dessa estatística utiliza `long double`, evitando overflow durante a soma de erros da ordem de 10^{18} .

6.6.9 Speedup da versão incorreta

O programa calcula formalmente a razão temporal também para `SemSync`. Entretanto, esse valor não representa um ganho funcional, pois a implementação não preserva a correção.

Assim,

$$S_{\text{SemSync}} \quad (6.30)$$

é apresentado apenas como observação temporal e não deve ser interpretado como speedup de uma solução válida.

6.6.10 Otimização do compilador

A compilação utiliza

`-O2`

para produzir código otimizado, mas mantém

`-fno-tree-vectorize`

para impedir que a vetorização automática se torne uma variável adicional no experimento.

Os kernels utilizam ainda

`__attribute__((noinline, noipa))`

no GCC, impedindo eliminação ou reutilização interprocedural das chamadas que precisam ser efetivamente medidas.

6.7 Representação estrutural da implementação

A Figura 6.1 apresenta a estrutura geral.

6.8 Fluxo de execução

O fluxo de medição de cada combinação entre versão e N é apresentado na Figura 6.2.

6.9 Metodologia experimental

6.9.1 Ambiente de execução

A execução foi realizada em ambiente Windows por meio de terminal MSYS2/UCRT64.

O diretório utilizado na execução final foi

`/d/Tecnologia da Informacao (TI)/P10 2026.2/HPC/ATVD6`

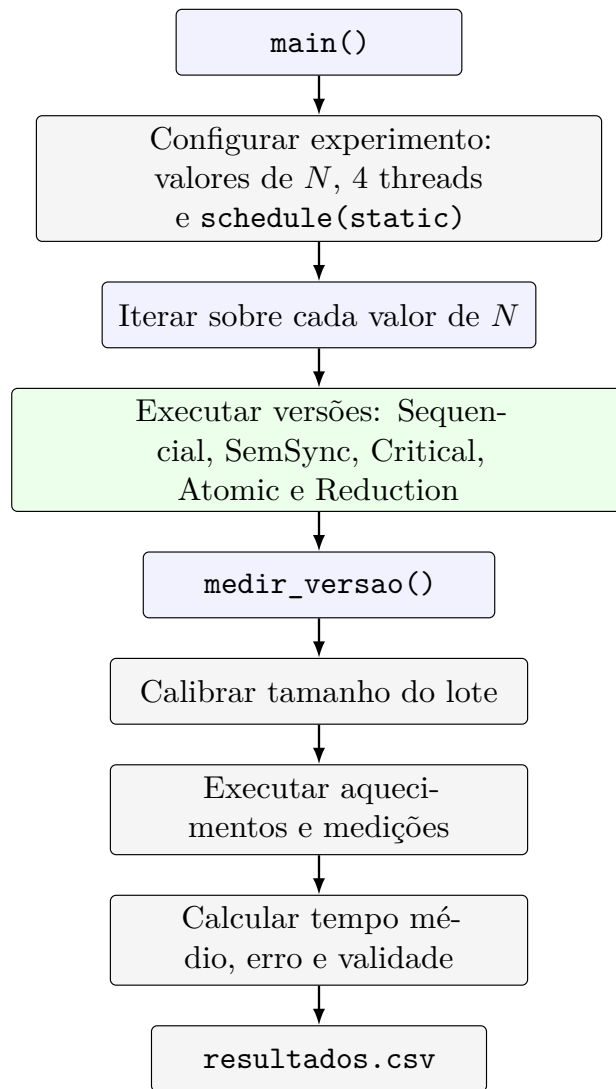


Figura 6.1: Estrutura geral da implementação da Atividade 6.

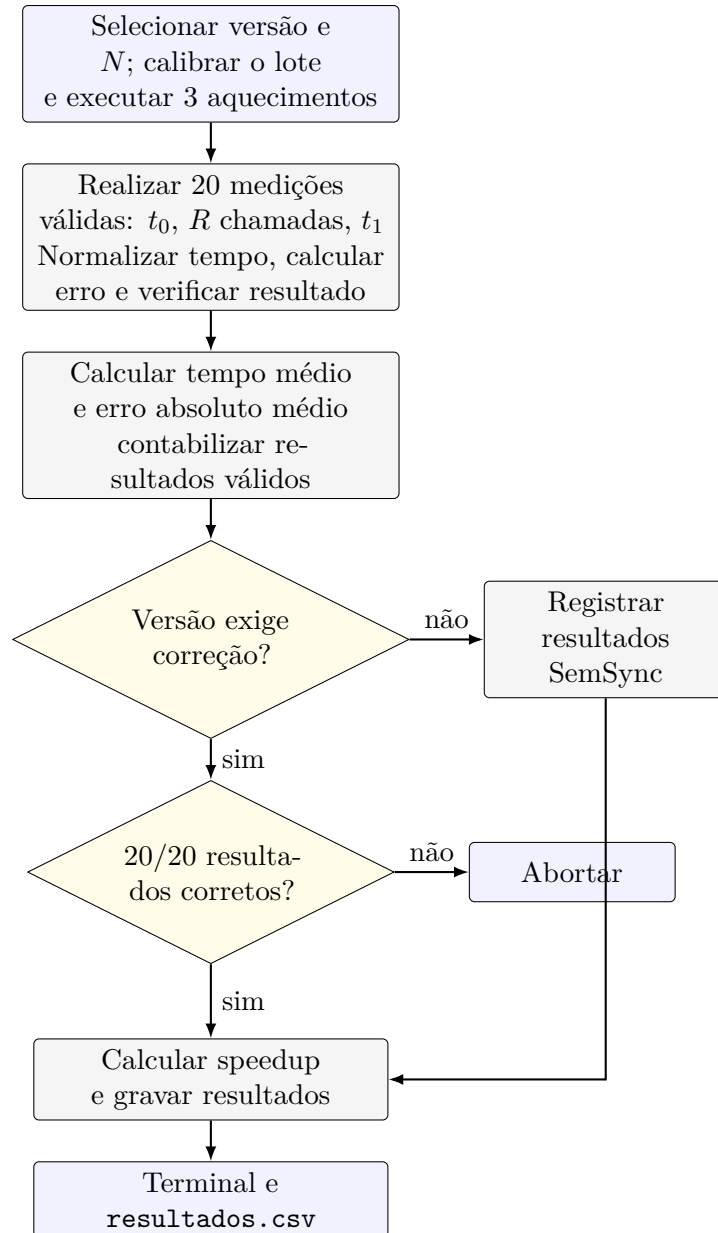


Figura 6.2: Fluxo experimental utilizado para cada implementação.

O código foi compilado com GCC e suporte a OpenMP.

O comando de compilação utilizado foi equivalente a:

```
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic \
    -fopenmp -fno-tree-vectorize \
    tarefa6.c -o tarefa6.exe
```

O modelo específico de processador e a versão exata do GCC não foram registrados no output consolidado utilizado neste capítulo e, portanto, não são atribuídos aqui.

6.9.2 Configuração das medições

A configuração experimental foi:

Tabela 6.1: Configuração experimental da Atividade 6.

Parâmetro	Configuração
Threads	4
Schedule	<code>static</code>
Aquecimentos	3
Medições	20
Temporizador	<code>CLOCK_MONOTONIC</code>
Batch	adaptativo
Tempo-alvo do batch	aproximadamente 20 ms
Estatística temporal	média aritmética
Tipo numérico	<code>int64_t</code>

6.9.3 Região cronometrada

Dentro da região cronometrada permaneceram apenas as chamadas do kernel.

A região inclui, nas versões paralelas:

- entrada na região OpenMP;
- distribuição das iterações;
- execução paralela;
- sincronizações necessárias;
- combinação dos resultados;
- encerramento da região paralela.

Essa decisão mede o custo real de cada estratégia, incluindo o overhead introduzido pelo runtime OpenMP.

6.9.4 Operações fora da região cronometrada

Permaneceram fora da região medida:

- impressão no terminal;
- escrita do arquivo CSV;
- cálculo do erro absoluto;
- contagem de resultados válidos;
- cálculo da média;
- cálculo do speedup;
- formatação dos resultados.

6.10 Código-fonte desenvolvido

```
1  /*
2  * Tarefa 6 - Integracao Numerica com OpenMP
3  *
4  * Compara cinco implementacoes do metodo composto do trapezio:
5  *   1. sequencial;
6  *   2. paralela sem sincronizacao;
7  *   3. paralela com critical;
8  *   4. paralela com atomic;
9  *   5. paralela com reduction.
10 *
11 * Problema numerico:
12 *   f(x) = x^2, intervalo [0, 2400000], aritmetica int64_t.
13 *
14 * Para cada valor de N sao executados 3 aquecimentos e 20 medicoes.
15 * O tamanho do lote e ajustado automaticamente para aproximadamente
16 * 20 ms e o tempo e normalizado para uma unica execucao antes da media.
17 *
18 * O erro absoluto apresentado corresponde a media das 20 medicoes.
19 * Essa definicao e especialmente relevante para SemSync, cuja race
20 * condition pode produzir resultados diferentes a cada execucao.
21 */
22
23 #define _POSIX_C_SOURCE 199309L
24
25 #include <stdio.h>
26 #include <stdint.h>
27 #include <stdlib.h>
28 #include <inttypes.h>
29 #include <time.h>
30 #include <omp.h>
31
32 /* Parametros fixos do experimento. */
33 #define NUM_THREADS 4
34 #define NUM_AQUECIMENTOS 3
```

```

35 #define NUM_MEDICOES 20
36 #define TEMPO_ALVO_LOTE_NS UINT64_C(20000000)
37 #define MAX_REPETICOES_LOTE 100000
38
39 /* Problema matematico. */
40 #define LIMITE_A INT64_C(0)
41 #define LIMITE_B INT64_C(2400000)
42 #define INTEGRAL_EXATA INT64_C(4608000000000000000)
43
44 /* Torna o resultado do kernel observavel fora da regio cronometrada. */
45 static volatile int64_t destino_resultado = 0;
46
47 /*
48  * Impede que o GCC elimine ou reutilize chamadas do kernel por
49  * otimizacoes interprocedurais durante o benchmark.
50  */
51 #if defined(__GNUC__)
52 #define KERNEL __attribute__((noinline, noipa))
53 #else
54 #define KERNEL
55 #endif
56
57 /* Assinatura comum a todas as implementacoes da integral. */
58 typedef int64_t (*FuncaoIntegral)(int64_t, int64_t, int64_t);
59
60 /* Metadados associados a cada implementacao avaliada. */
61 typedef struct {
62     const char *nome;
63     FuncaoIntegral funcao;
64     int exige_correcao;
65 } Versao;
66
67 /*
68  * Resultado consolidado para uma combinacao (versao, N).
69  *
70  * resultado:
71  *   ultimo resultado observado nas medicoes.
72  *
73  * erro_abs_medio:
74  *   media do erro absoluto das 20 medicoes. Para as implementacoes
75  *   corretas, o valor permanece deterministico; para SemSync, resume
76  *   a variacao provocada pela race condition.
77  */
78 typedef struct {
79     int64_t resultado;
80     long double erro_abs_medio;
81     int validos;
82     int repeticoes_lote;
83     double tempo_medio_ms;
84 } ResultadoBenchmark;
85
86
87 /* f(x) = x^2. */
88 static inline int64_t funcao_integrando(int64_t x)
89 {
90     return x * x;
91 }
92
93

```

```

94  /* Combina a soma dos pontos internos com os extremos do intervalo. */
95  static inline int64_t finalizar_integral(
96      int64_t a,
97      int64_t b,
98      int64_t h,
99      int64_t soma)
100 {
101     const int64_t extremos =
102         funcao_integrando(a) +
103         funcao_integrando(b);
104
105     return h * soma + (h * extremos) / 2;
106 }
107
108
109 /* Baseline sequencial utilizado no calculo do speedup. */
110 static KERNEL int64_t integral_sequencial(
111     int64_t a,
112     int64_t b,
113     int64_t n)
114 {
115     const int64_t h = (b - a) / n;
116     int64_t soma = 0;
117
118     for (int64_t i = 1; i < n; ++i) {
119         const int64_t x_i = a + i * h;
120         soma += funcao_integrando(x_i);
121     }
122
123     return finalizar_integral(a, b, h, soma);
124 }
125
126
127 /*
128  * Versao propositalmente incorreta.
129  *
130  * A atualizacao concorrente de soma ocorre sem sincronizacao e produz
131  * uma race condition, permitindo perda de atualizacoes entre threads.
132  */
133 static KERNEL int64_t integral_sem_sincronizacao(
134     int64_t a,
135     int64_t b,
136     int64_t n)
137 {
138     const int64_t h = (b - a) / n;
139     int64_t soma = 0;
140
141     #pragma omp parallel for schedule(static) shared(soma)
142     for (int64_t i = 1; i < n; ++i) {
143         const int64_t x_i = a + i * h;
144
145         soma += funcao_integrando(x_i);
146     }
147
148     return finalizar_integral(a, b, h, soma);
149 }
150
151
152 /* Serializa somente a atualizacao do acumulador por regioao critical. */

```

```

153 static KERNEL int64_t integral_critical(
154     int64_t a,
155     int64_t b,
156     int64_t n)
157 {
158     const int64_t h = (b - a) / n;
159     int64_t soma = 0;
160
161     #pragma omp parallel for schedule(static) shared(soma)
162     for (int64_t i = 1; i < n; ++i) {
163         const int64_t x_i = a + i * h;
164         const int64_t valor =
165             funcao_integrando(x_i);
166
167         #pragma omp critical
168         {
169             soma += valor;
170         }
171     }
172
173     return finalizar_integral(a, b, h, soma);
174 }
175
176
177 /* Protege apenas a atualizacao do acumulador com atomic. */
178 static KERNEL int64_t integral_atomic(
179     int64_t a,
180     int64_t b,
181     int64_t n)
182 {
183     const int64_t h = (b - a) / n;
184     int64_t soma = 0;
185
186     #pragma omp parallel for schedule(static) shared(soma)
187     for (int64_t i = 1; i < n; ++i) {
188         const int64_t x_i = a + i * h;
189         const int64_t valor =
190             funcao_integrando(x_i);
191
192         #pragma omp atomic update
193         soma += valor;
194     }
195
196     return finalizar_integral(a, b, h, soma);
197 }
198
199
200 /* Usa acumuladores privados e combina os resultados por reduction. */
201 static KERNEL int64_t integral_reduction(
202     int64_t a,
203     int64_t b,
204     int64_t n)
205 {
206     const int64_t h = (b - a) / n;
207     int64_t soma = 0;
208
209     #pragma omp parallel for schedule(static) reduction(+:soma)
210     for (int64_t i = 1; i < n; ++i) {
211         const int64_t x_i = a + i * h;

```

```

212     soma += funcao_integrando(x_i);
213 }
214
215
216     return finalizar_integral(a, b, h, soma);
217 }
218
219
220 /* Retorna um instante do relógio monotonicamente crescente. */
221 static struct timespec obter_tempo(void)
222 {
223     struct timespec tempo;
224
225     if (clock_gettime(CLOCK_MONOTONIC, &tempo) != 0) {
226         perror("clock_gettime");
227         exit(EXIT_FAILURE);
228     }
229
230     return tempo;
231 }
232
233
234 /* Calcula o tempo decorrido entre dois instantes em nanossegundos. */
235 static uint64_t diferenca_ns(
236     struct timespec inicio,
237     struct timespec fim)
238 {
239     int64_t segundos =
240         (int64_t)fim.tv_sec -
241         (int64_t)inicio.tv_sec;
242
243     int64_t nanos =
244         (int64_t)fim.tv_nsec -
245         (int64_t)inicio.tv_nsec;
246
247     if (nanos < 0) {
248         --segundos;
249         nanos += INT64_C(1000000000);
250     }
251
252     return
253         (uint64_t)segundos *
254         UINT64_C(1000000000) +
255         (uint64_t)nanos;
256 }
257
258
259 /* Resultado esperado para o método do trapézio aplicado a  $x^2$ . */
260 static int64_t resultado_esperado(
261     int64_t a,
262     int64_t b,
263     int64_t n)
264 {
265     const int64_t h =
266         (b - a) / n;
267
268     const int64_t erro =
269         ((b - a) * h * h) / 6;
270

```

```

271     return INTEGRAL_EXATA + erro;
272 }
273
274
275 /* Erro absoluto de uma observacao em relacao a integral analitica. */
276 static uint64_t calcular_erro_abs(int64_t resultado)
277 {
278     if (resultado >= INTEGRAL_EXATA) {
279         return
280             (uint64_t)(
281                 resultado -
282                 INTEGRAL_EXATA
283             );
284     }
285
286     return
287         (uint64_t)(
288             INTEGRAL_EXATA -
289             resultado
290         );
291 }
292
293
294 /*
295  * Cronometra um lote de chamadas do kernel.
296  *
297  * Nenhuma operacao de I/O ou tratamento estatistico ocorre dentro
298  * da regioao medida. O resultado e consumido somente apos a leitura
299  * final do tempo.
300  */
301 static uint64_t medir_lote(
302     FuncaoIntegral funcao,
303     int64_t a,
304     int64_t b,
305     int64_t n,
306     int repeticoes,
307     int64_t *resultado)
308 {
309     const struct timespec inicio =
310         obter_tempo();
311
312     for (int r = 0; r < repeticoes; ++r) {
313         *resultado =
314             funcao(a, b, n);
315     }
316
317     const struct timespec fim =
318         obter_tempo();
319
320     destino_resultado =
321         *resultado;
322
323     return
324         diferenca_ns(inicio, fim);
325 }
326
327
328 /*
329  * Ajusta o numero de repeticoes para que cada lote tenha duracao

```

```

330  * proxima a TEMPO_ALVO_LOTE_NS, reduzindo o impacto relativo do
331  * temporizador sobre a medicao.
332  */
333  static int determinar_repeticoes(
334      FuncaoIntegral funcao,
335      int64_t a,
336      int64_t b,
337      int64_t n)
338  {
339      int repeticoes = 1;
340      int64_t resultado = 0;
341
342      for (int tentativa = 0;
343           tentativa < 12;
344           ++tentativa) {
345
346          const uint64_t tempo_ns =
347              medir_lote(
348                  funcao,
349                  a,
350                  b,
351                  n,
352                  repeticoes,
353                  &resultado
354              );
355
356          if (
357              tempo_ns >= TEMPO_ALVO_LOTE_NS ||
358              repeticoes >= MAX_REPETICOES_LOTE
359          ) {
360              return repeticoes;
361          }
362
363          uint64_t proximo;
364
365          if (tempo_ns == 0) {
366
367              proximo =
368                  (uint64_t)repeticoes * 10;
369
370          } else {
371
372              proximo =
373                  (
374                      TEMPO_ALVO_LOTE_NS *
375                      (uint64_t)repeticoes +
376                      tempo_ns -
377                      1
378                  ) /
379                  tempo_ns;
380
381              if (
382                  proximo <=
383                  (uint64_t)repeticoes
384              ) {
385                  proximo =
386                      (uint64_t)repeticoes * 2;
387              }
388          }

```

```

389
390     /*
391     * Limita saltos causados por uma medicaao
392     * piloto atipica.
393     */
394     const uint64_t limite_crescimento =
395         (uint64_t)repeticoes * 10;
396
397     if (proximo > limite_crescimento) {
398         proximo =
399             limite_crescimento;
400     }
401
402     if (proximo > MAX_REPETICOES_LOTE) {
403         proximo =
404             MAX_REPETICOES_LOTE;
405     }
406
407     repeticoes =
408         (int)proximo;
409 }
410
411 return repeticoes;
412 }
413
414
415 /*
416 * Executa calibracao, aquecimentos e 20 medicoes de uma implementacao.
417 *
418 * O erro absoluto e calculado para o resultado observado ao final
419 * de cada medicaao e posteriormente promediado.
420 *
421 * Esse procedimento preserva a regioa cronometrada e fornece uma
422 * medida representativa para SemSync, cuja saida pode variar devido
423 * a race condition.
424 *
425 * As implementacoes corretas devem produzir o resultado esperado
426 * nas 20 medicoes. SemSync registra as falhas sem interromper o
427 * experimento, pois a incorrecao faz parte do comportamento avaliado.
428 */
429 static ResultadoBenchmark medir_versao(
430     const Versao *versao,
431     int64_t a,
432     int64_t b,
433     int64_t n)
434 {
435     ResultadoBenchmark benchmark = {0};
436
437     const int64_t esperado =
438         resultado_esperado(a, b, n);
439
440     const int repeticoes =
441         determinar_repeticoes(
442             versao->funcao,
443             a,
444             b,
445             n
446         );
447

```

```

448     int64_t resultado = 0;
449
450
451     /* Aquecimentos executados antes das medicoes validas. */
452     for (int aquecimento = 0;
453          aquecimento < NUM_AQUECIMENTOS;
454          ++aquecimento) {
455
456         (void)medir_lote(
457             versao->funcao,
458             a,
459             b,
460             n,
461             repeticoes,
462             &resultado
463         );
464     }
465
466
467     double soma_tempos_ns = 0.0;
468
469     /*
470      * long double evita overflow durante a soma de erros
471      * da ordem de 1018 produzidos pela versao SemSync.
472      */
473     long double soma_erros_abs = 0.0L;
474
475     int validos = 0;
476
477
478     for (int medicao = 0;
479          medicao < NUM_MEDICOES;
480          ++medicao) {
481
482         const uint64_t lote_ns =
483             medir_lote(
484                 versao->funcao,
485                 a,
486                 b,
487                 n,
488                 repeticoes,
489                 &resultado
490             );
491
492
493         /*
494          * Tempo normalizado para uma chamada do kernel.
495          */
496         soma_tempos_ns +=
497             (double)lote_ns /
498             (double)repeticoes;
499
500
501         /*
502          * Uma observacao funcional e registrada por medicao,
503          * apos o encerramento da regioa cronometrada.
504          */
505         soma_erros_abs +=
506             (long double)

```

```

507         calcular_erro_abs(resultado);
508
509
510         if (resultado == esperado) {
511             ++validos;
512         }
513     }
514
515
516     /*
517     * Sequencial, Critical, Atomic e Reduction devem
518     * produzir resultados corretos em todas as medicoes.
519     */
520     if (
521         versao->exige_correcao &&
522         validos != NUM_MEDICOES
523     ) {
524         fprintf(
525             stderr,
526             "Erro de validacao em %s para N=%" PRIu64
527             " (%d/%d resultados corretos).\n",
528             versao->nome,
529             n,
530             validos,
531             NUM_MEDICOES
532         );
533
534         exit(EXIT_FAILURE);
535     }
536
537
538     benchmark.resultado =
539         resultado;
540
541     benchmark.erro_abs_medio =
542         soma_erros_abs /
543         (long double)NUM_MEDICOES;
544
545     benchmark.validos =
546         validos;
547
548     benchmark.repeticoes_lote =
549         repeticoes;
550
551     benchmark.tempo_medio_ms =
552         (
553             soma_tempos_ns /
554             (double)NUM_MEDICOES
555         ) /
556         1000000.0;
557
558
559     return benchmark;
560 }
561
562
563 /* Imprime a configuracao fixa e o cabecalho da tabela. */
564 static void imprimir_cabecalho(void)
565 {

```

```

566     printf(
567         "Tarefa 6 - Integracao por Metodo do Trapezio com OpenMP\n"
568     );
569
570     printf(
571         "f(x)          : x^2\n"
572     );
573
574     printf(
575         "Intervalo      : [%\" PRId64 \", \"%\" PRId64 \"]\n",
576         LIMITE_A,
577         LIMITE_B
578     );
579
580     printf(
581         "Tipo numerico : int64_t\n"
582     );
583
584     printf(
585         "Threads        : %d\n",
586         NUM_THREADS
587     );
588
589     printf(
590         "Schedule        : static\n"
591     );
592
593     printf(
594         "Relogio         : CLOCK_MONOTONIC\n"
595     );
596
597     printf(
598         "Aquecimentos    : %d\n",
599         NUM_AQUECIMENTOS
600     );
601
602     printf(
603         "Medicoes        : %d\n",
604         NUM_MEDICOES
605     );
606
607     printf(
608         "Batch alvo      : %.1f ms (adaptativo)\n\n",
609         (double)TEMPO_ALVO_LOTE_NS /
610         1000000.0
611     );
612
613
614     printf(
615         "%-10s %-12s %-22s %-20s %-9s %-12s %-9s %-7s\n",
616         "N",
617         "Versao",
618         "Integral",
619         "ErroAbsMedio",
620         "Validos",
621         "Media(ms)",
622         "Speedup",
623         "Batch"
624     );

```

```

625
626
627     printf(
628         "
        ↪ -----
        ↪ n"
629     );
630 }
631
632
633 int main(void)
634 {
635     /*
636      * Todos os valores de N dividem exatamente o intervalo,
637      * mantendo h inteiro em todas as configuracoes.
638      */
639     static const int64_t valores_n[] = {
640         INT64_C(24000),
641         INT64_C(120000),
642         INT64_C(240000),
643         INT64_C(400000),
644         INT64_C(800000),
645         INT64_C(1200000),
646         INT64_C(2400000)
647     };
648
649
650     static const Versao versoes[] = {
651         {
652             "Sequencial",
653             integral_sequencial,
654             1
655         },
656         {
657             "SemSync",
658             integral_sem_sincronizacao,
659             0
660         },
661         {
662             "Critical",
663             integral_critical,
664             1
665         },
666         {
667             "Atomic",
668             integral_atomic,
669             1
670         },
671         {
672             "Reduction",
673             integral_reduction,
674             1
675         }
676     };
677
678
679     const size_t quantidade_n =
680         sizeof(valores_n) /
681         sizeof(valores_n[0]);

```

```

682
683
684     const size_t quantidade_versoes =
685         sizeof(versoes) /
686         sizeof(versoes[0]);
687
688
689     /*
690     * O numero de threads e o comportamento dinamico
691     * permanecem constantes durante todo o experimento.
692     */
693     omp_set_dynamic(0);
694     omp_set_num_threads(NUM_THREADS);
695
696
697     FILE *csv =
698         fopen(
699             "resultados.csv",
700             "w"
701         );
702
703
704     if (csv == NULL) {
705         perror("resultados.csv");
706         return EXIT_FAILURE;
707     }
708
709
710     /*
711     * O nome erro_abs e mantido no CSV para compatibilidade
712     * com o script de graficos existente.
713     *
714     * O valor corresponde ao erro absoluto medio das
715     * 20 medicoes.
716     */
717     fprintf(
718         csv,
719         "N,versao,resultado,erro_abs,validos,medicoes,"
720         "tempo_medio_ms,speedup,repeticoes_batch\n"
721     );
722
723
724     imprimir_cabecalho();
725
726
727     for (size_t i = 0;
728         i < quantidade_n;
729         ++i) {
730
731         const int64_t n =
732             valores_n[i];
733
734
735         /*
736         * Garante que h permaneça inteiro.
737         */
738         if (
739             n <= 0 ||
740             (LIMITE_B - LIMITE_A) % n != 0

```

```

741     ) {
742         fprintf(
743             stderr,
744             "Erro: N=%" PRId64
745             " nao divide o intervalo.\n",
746             n
747         );
748
749         fclose(csv);
750
751         return EXIT_FAILURE;
752     }
753
754
755     ResultadoBenchmark resultados[5];
756
757
758     /*
759     * Todas as implementacoes executam o mesmo problema
760     * para o valor corrente de N.
761     */
762     for (size_t v = 0;
763          v < quantidade_versoes;
764          ++v) {
765
766         resultados[v] =
767             medir_versao(
768                 &versoes[v],
769                 LIMITE_A,
770                 LIMITE_B,
771                 n
772             );
773     }
774
775
776     const double tempo_sequencial =
777         resultados[0].tempo_medio_ms;
778
779
780     /*
781     * O speedup de cada versao e calculado em relacao
782     * ao baseline sequencial do mesmo valor de N.
783     */
784     for (size_t v = 0;
785          v < quantidade_versoes;
786          ++v) {
787
788         const double speedup =
789             tempo_sequencial /
790             resultados[v].tempo_medio_ms;
791
792
793         printf(
794             "%-10" PRId64
795             " %-12s"
796             "%-22" PRId64
797             "%-20.0Lf"
798             "%2d/%-6d"
799             "%-12.6f"

```

```

800         " %-9.3f"
801         " %-7d\n",
802
803         n,
804         versoes[v].nome,
805         resultados[v].resultado,
806         resultados[v].erro_abs_medio,
807         resultados[v].validos,
808         NUM_MEDICOES,
809         resultados[v].tempo_medio_ms,
810         speedup,
811         resultados[v].repeticoes_lote
812     );
813
814
815     fprintf(
816         csv,
817         "%s PRId64\n",
818         versoes[v].nome,
819         resultados[v].resultado,
820         resultados[v].erro_abs_medio,
821         resultados[v].validos,
822         NUM_MEDICOES,
823         resultados[v].tempo_medio_ms,
824         speedup,
825         resultados[v].repeticoes_lote
826     );
827
828     n++;
829     versoes[v].nome,
830     resultados[v].resultado,
831     resultados[v].erro_abs_medio,
832     resultados[v].validos,
833     NUM_MEDICOES,
834     resultados[v].tempo_medio_ms,
835     speedup,
836     resultados[v].repeticoes_lote
837 }
838
839
840 printf("\n");
841 }
842
843
844 fclose(csv);
845
846 (void)destino_resultado;
847
848
849 printf(
850     "Resultados CSV: resultados.csv\n"
851 );
852
853 printf(
854     "Nota: ErroAbsMedio representa a media do erro absoluto "
855     "das %d medicoes.\n",
856     NUM_MEDICOES
857 );
858

```

```

859     printf(
860         "Nota: o speedup de SemSync nao representa "
861         "uma solucao funcionalmente valida.\n"
862     );
863
864
865     return EXIT_SUCCESS;
866 }

```

Listing 6.1: Código-fonte da Atividade 6.

6.11 Validação da implementação

A validação utilizou o resultado esperado do método do trapézio para $f(x) = x^2$.

Sequencial, Critical, Atomic e Reduction produziram resultados corretos em

$$20/20 \quad (6.31)$$

medições para todos os valores de N .

A Tabela 6.2 apresenta os resultados obtidos por essas implementações.

Tabela 6.2: Resultados numéricos das implementações corretas.

N	Integral	Erro absoluto médio
24 000	4608000004000000000	4000000000
120 000	4608000000160000000	160000000
240 000	4608000000040000000	40000000
400 000	46080000000014400000	14400000
800 000	46080000000003600000	3600000
1 200 000	46080000000001600000	1600000
2 400 000	4608000000000400000	400000

Os valores confirmam a convergência

$$E_N = O(N^{-2}). \quad (6.32)$$

6.11.1 Validação da versão sem sincronização

A versão SemSync apresentou comportamento não determinístico.

Os resultados de validação são apresentados na Tabela 6.3.

A ocorrência de

$$4/20 \quad (6.33)$$

resultados corretos em $N = 24\,000$ não elimina a condição de corrida.

Tabela 6.3: Comportamento funcional da versão sem sincronização.

N	Resultados válidos	Erro absoluto médio
24 000	4/20	2066165998700000000
120 000	0/20	4240740599952000000
240 000	0/20	3416380829987500000
400 000	0/20	3473990765996220000
800 000	0/20	3391194383999100000
1 200 000	0/20	3499195751999600000
2 400 000	0/20	3758397695999900000

Uma race condition não implica falha obrigatória em todas as execuções. Dependendo do interleaving das threads, uma execução pode ocasionalmente produzir o resultado esperado.

Para todos os demais tamanhos avaliados, nenhuma das vinte medições produziu resultado válido.

A coluna **Integral** mostrada no output representa o último resultado observado, enquanto **ErroAbsMedio** representa a média das vinte medições. Consequentemente, para **SemSync**, não é esperado que o erro da última integral impressa coincida diretamente com **ErroAbsMedio**.

6.12 Resultados experimentais

6.12.1 Tempos médios

A Tabela 6.4 apresenta os tempos médios obtidos.

Tabela 6.4: Tempos médios de execução em milissegundos.

N	Seq.	SemSync	Critical	Atomic	Reduction
24 000	0.024276	0.149859	1.510098	0.657847	0.079766
120 000	0.121317	0.106085	4.919007	2.970264	0.119469
240 000	0.245747	0.242326	9.144015	6.083695	0.162831
400 000	0.391012	0.223065	15.391255	9.872103	0.194119
800 000	0.677399	0.311087	29.877780	20.204173	0.324910
1 200 000	1.177408	0.449955	44.550875	31.542555	0.433428
2 400 000	1.874432	0.834066	92.877570	61.214450	0.875025

6.12.2 Speedup

A Tabela 6.5 apresenta os speedups calculados em relação ao baseline sequencial.

Os valores de **SemSync** são mantidos apenas como observação temporal. Como essa versão não preserva a correção, não constituem ganho de desempenho de uma solução funcionalmente válida.

Tabela 6.5: Speedup em relação à implementação sequencial.

N	SemSync	Critical	Atomic	Reduction
24 000	0.162	0.016	0.037	0.304
120 000	1.144	0.025	0.041	1.015
240 000	1.014	0.027	0.040	1.509
400 000	1.753	0.025	0.040	2.014
800 000	2.178	0.023	0.034	2.085
1 200 000	2.617	0.026	0.037	2.716
2 400 000	2.247	0.020	0.031	2.142

6.12.3 Batching adaptativo

O mecanismo adaptativo alterou automaticamente o número de chamadas por lote de acordo com o custo de cada versão.

Por exemplo, para $N = 24\,000$, foram utilizados:

- 840 repetições para Sequencial;
- 20 para SemSync;
- 10 para Critical;
- 29 para Atomic;
- 238 para Reduction.

Para $N = 2\,400\,000$, os lotes foram:

- 11 para Sequencial;
- 24 para SemSync;
- 1 para Critical;
- 1 para Atomic;
- 24 para Reduction.

No último caso, uma única execução de **Critical** ou **Atomic** já ultrapassava o tempo-alvo de aproximadamente 20 ms.

6.13 Visualização dos resultados

Foram produzidos três gráficos a partir do arquivo `resultados.csv`.

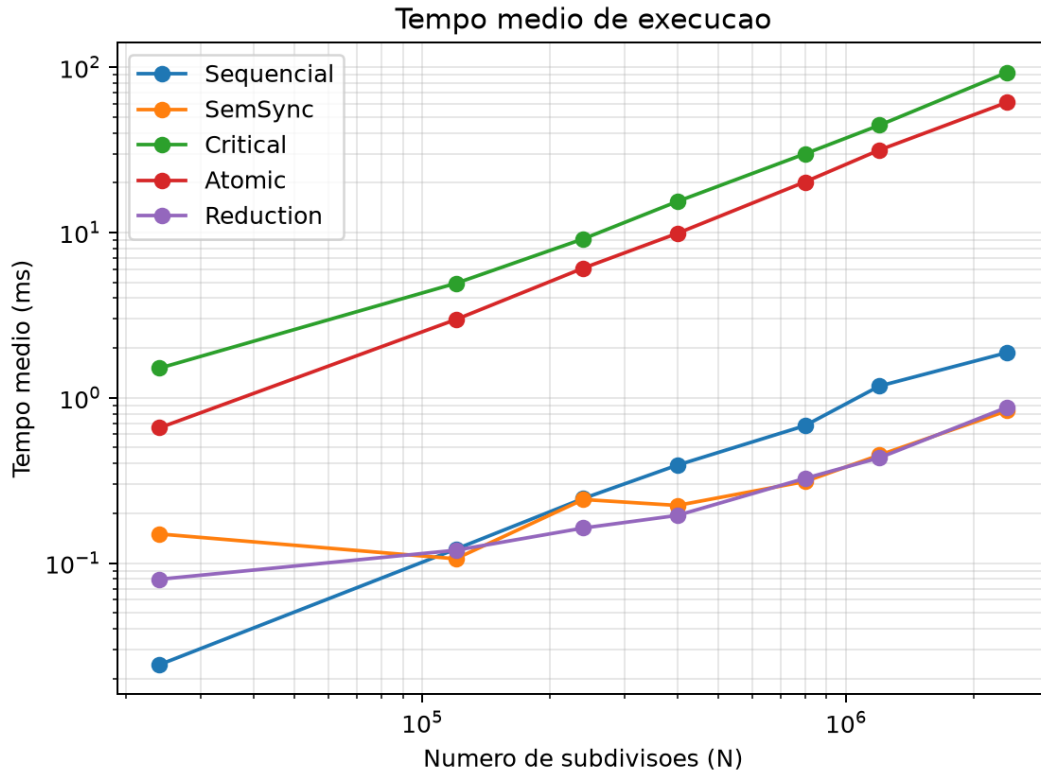


Figura 6.3: Tempo médio de execução em função do número de subdivisões.

6.13.1 Tempo médio

A escala logarítmica no eixo dos tempos permite visualizar simultaneamente implementações com diferenças superiores a uma ordem de grandeza.

Critical e **Atomic** apresentam crescimento elevado, enquanto **Sequencial** e **Reduction** permanecem em uma região consideravelmente inferior.

6.13.2 Speedup

A linha $S = 1$ representa o desempenho do baseline sequencial.

Critical e **Atomic** permanecem abaixo desse limite em todas as configurações.

Reduction ultrapassa a referência a partir da região entre $N = 24\,000$ e $N = 120\,000$, tornando o ganho mais evidente nos tamanhos posteriores.

6.13.3 Erro absoluto médio

Sequencial, **Critical**, **Atomic** e **Reduction** produzem exatamente os mesmos erros e, portanto, suas curvas se sobrepõem.

A curva de **SemSync** permanece várias ordens de grandeza acima, evidenciando o impacto da race condition sobre a correção do resultado.

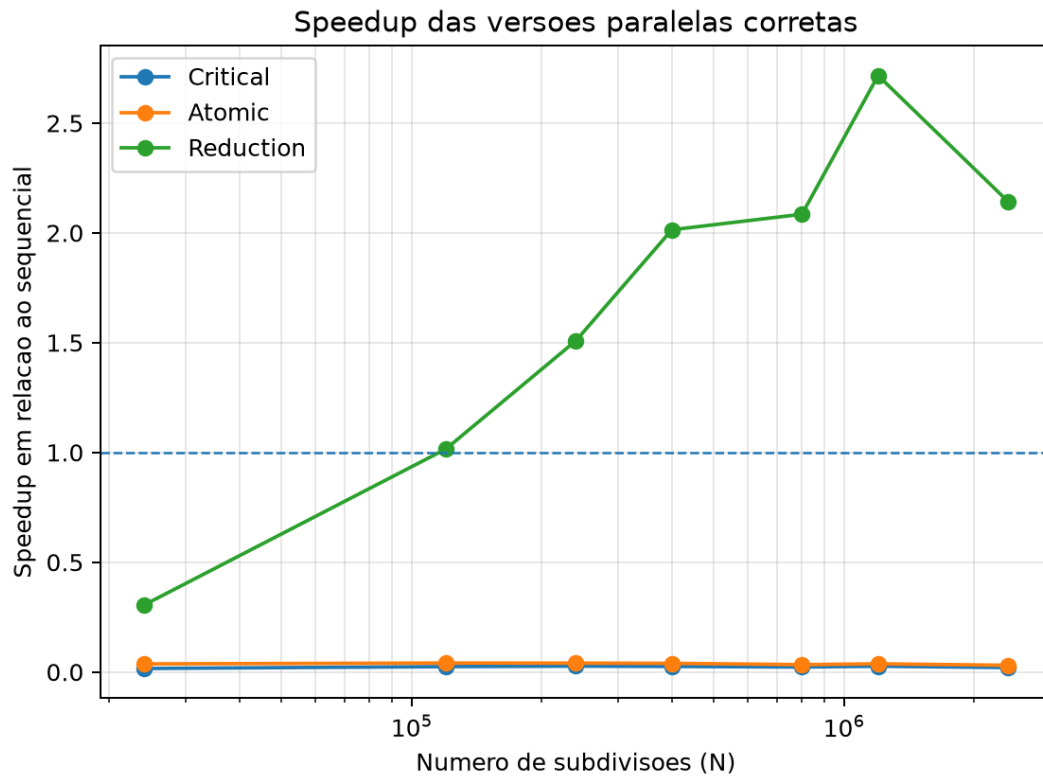


Figura 6.4: Speedup das estratégias paralelas funcionalmente válidas.

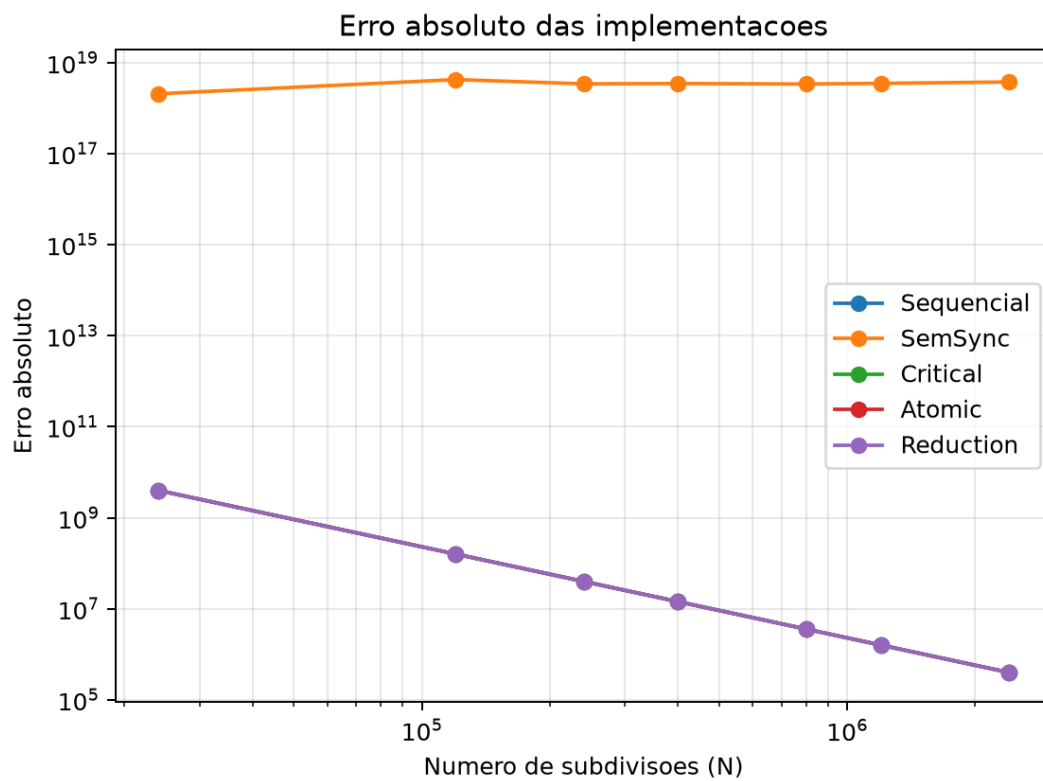


Figura 6.5: Erro absoluto médio das implementações.

6.14 Análise e discussão

6.14.1 Impacto da ausência de sincronização

A versão sem sincronização mostrou diretamente a necessidade de proteger a operação de redução.

Para $N = 24\,000$, apenas

$$4/20 \tag{6.34}$$

execuções foram corretas.

Para todos os demais valores:

$$0/20 \tag{6.35}$$

resultados foram válidos.

O erro absoluto médio permaneceu na ordem de

$$10^{18}, \tag{6.36}$$

enquanto as versões corretas apresentaram erro determinado apenas pela discretização do método.

O resultado demonstra que simplesmente distribuir as iterações não é suficiente: a semântica da atualização compartilhada precisa ser preservada.

6.14.2 Desempenho de critical

A versão **Critical** garantiu

$$20/20 \tag{6.37}$$

resultados corretos em todas as configurações.

Entretanto, apresentou o maior custo de sincronização.

Para

$$N = 2\,400\,000, \tag{6.38}$$

foram obtidos

$$T_{\text{seq}} = 1.874432 \text{ ms} \tag{6.39}$$

e

$$T_{\text{critical}} = 92.877570 \text{ ms.} \quad (6.40)$$

A razão é aproximadamente

$$\frac{92.877570}{1.874432} \approx 49.5. \quad (6.41)$$

Assim, nessa configuração, a versão **Critical** foi aproximadamente 49.5 vezes mais lenta que a sequencial.

O speedup correspondente foi

$$S_{\text{critical}} = 0.020. \quad (6.42)$$

A explicação está na serialização de cada atualização do acumulador. Embora o cálculo de x_i e $f(x_i)$ ocorra em paralelo, todas as threads precisam entrar repetidamente na mesma região de exclusão mútua.

6.14.3 Desempenho de **atomic**

A implementação **Atomic** também produziu resultados corretos em todas as medições.

Na execução final, foi mais rápida que **Critical** em todos os valores de N .

Para

$$N = 2\,400\,000, \quad (6.43)$$

o tempo foi

$$T_{\text{atomic}} = 61.214450 \text{ ms.} \quad (6.44)$$

Comparando com **Critical**:

$$\frac{92.877570}{61.214450} \approx 1.52. \quad (6.45)$$

Apesar dessa melhoria, o speedup em relação ao sequencial foi apenas

$$S_{\text{atomic}} = 0.031. \quad (6.46)$$

Portanto, **atomic** reduz o custo em relação a uma região crítica genérica, mas não elimina o gargalo principal.

Todas as threads ainda executam uma operação de leitura-modificação-escrita sobre o mesmo acumulador.

Esse padrão produz contenção e tráfego de coerência sobre o mesmo endereço de memória.

6.14.4 Desempenho de reduction

A implementação utilizando `reduction` apresentou o melhor comportamento entre as estratégias paralelas corretas.

Para $N = 24\,000$:

$$S_{\text{reduction}} = 0.304. \quad (6.47)$$

Portanto, o problema ainda era pequeno demais para compensar o overhead do OpenMP.

Para $N = 120\,000$:

$$S_{\text{reduction}} = 1.015, \quad (6.48)$$

indicando desempenho praticamente equivalente ao sequencial.

A partir de $N = 240\,000$, o ganho tornou-se mais evidente:

$$S_{\text{reduction}} = 1.509. \quad (6.49)$$

Para $N = 400\,000$:

$$S_{\text{reduction}} = 2.014. \quad (6.50)$$

O maior speedup observado ocorreu para

$$N = 1\,200\,000, \quad (6.51)$$

com

$$\boxed{S_{\text{reduction}} = 2.716}. \quad (6.52)$$

Com quatro threads, a eficiência correspondente é

$$E_4 = \frac{2.716}{4} \approx 0.679, \quad (6.53)$$

ou aproximadamente

$$\boxed{67.9\%}. \quad (6.54)$$

6.14.5 Comportamento no maior valor de N

Para

$$N = 2\,400\,000, \quad (6.55)$$

o tempo sequencial foi

$$1.874432 \text{ ms}, \quad (6.56)$$

enquanto `Reduction` apresentou

$$0.875025 \text{ ms}. \quad (6.57)$$

O speedup foi

$$S = 2.142. \quad (6.58)$$

Esse valor é inferior ao máximo de 2.716 observado para $N = 1\,200\,000$.

O speedup real não precisa crescer monotonicamente com N . Entre os fatores que podem afetar a relação estão:

- escalonamento do sistema operacional;
- frequência dinâmica do processador;
- estado térmico;
- comportamento de cache;
- overhead do runtime OpenMP.

Mesmo com essa redução no último ponto, a implementação permaneceu mais de duas vezes mais rápida que o baseline sequencial.

6.14.6 Granularidade

Os resultados demonstram a importância da granularidade.

Para cargas pequenas, o custo de:

- iniciar a região paralela;
- distribuir iterações;
- sincronizar a equipe;
- combinar os acumuladores;

é comparável ou superior ao trabalho útil.

Isso explica

$$S < 1 \quad (6.59)$$

para **Reduction** em $N = 24\,000$.

À medida que N cresce, uma quantidade maior de trabalho é distribuída entre as threads e o overhead fixo é progressivamente amortizado.

6.14.7 Correção versus desempenho

Os resultados também demonstram que desempenho não pode ser avaliado isoladamente.

Em diversas configurações, **SemSync** apresenta tempo inferior ao sequencial. Por exemplo, em $N = 1\,200\,000$, o valor temporal calculado corresponde a um fator de

$$2.617 \tag{6.60}$$

em relação ao sequencial.

Entretanto:

$$0/20 \tag{6.61}$$

resultados foram corretos.

Portanto, esse valor não pode ser interpretado como speedup de uma solução válida.

A comparação evidencia a seguinte relação:

Estratégia	Correção	Desempenho observado
Sequencial	sim	baseline
SemSync	não	não utilizável
Critical	sim	fortemente degradado
Atomic	sim	degradado por contenção
Reduction	sim	melhor estratégia paralela

6.14.8 Por que **reduction** apresenta melhor escalabilidade

Em **critical** e **atomic**, todas as threads atualizam continuamente a mesma posição de memória.

Em **reduction**, cada thread mantém sua soma privada durante a maior parte da execução.

Conceitualmente:

A sincronização deixa de ocorrer em cada iteração e passa a ser concentrada na combinação final dos resultados parciais.

Essa propriedade explica o desempenho significativamente superior observado experimentalmente.

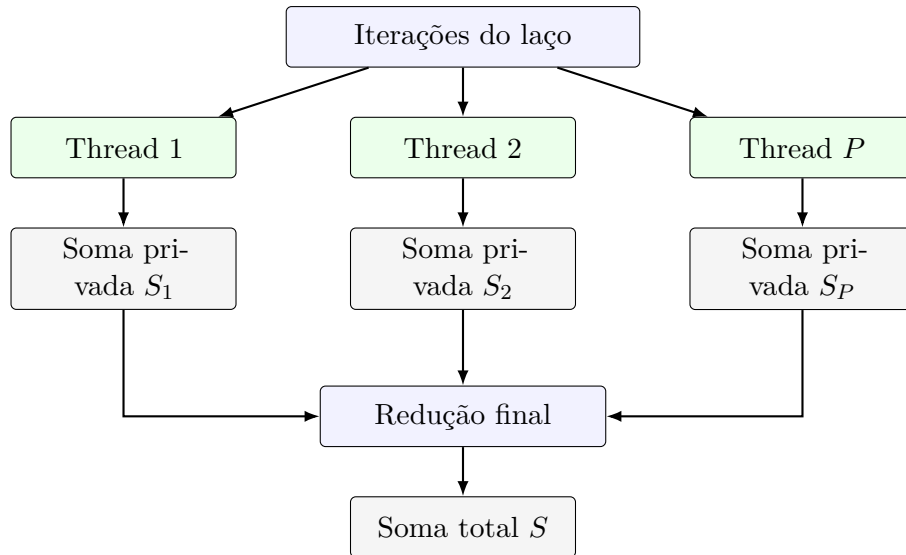


Figura 6.6: Estratégia conceitual utilizada pela redução.

6.15 Limitações

O experimento possui algumas limitações que devem ser consideradas na interpretação dos resultados.

Primeiramente, somente quatro threads foram avaliadas. Portanto, a atividade permite comparar estratégias de sincronização para uma configuração fixa, mas não caracteriza a escalabilidade em função do número de threads.

Também foi utilizada apenas a função

$$f(x) = x^2, \quad (6.62)$$

cujo custo computacional é pequeno e uniforme. Funções mais complexas podem alterar significativamente a razão entre trabalho útil e overhead de sincronização.

O uso de `schedule(static)` é adequado à uniformidade dessa carga, mas os resultados não caracterizam problemas com custo variável por iteração.

A aritmética inteira com `int64_t` evita erros de ponto flutuante e facilita a validação. Entretanto, muitas aplicações reais de integração numérica utilizam `double`, situação em que a ordem de redução também pode introduzir pequenas diferenças de arredondamento.

A compilação desabilita a vetorização automática com `-fno-tree-vectorize`. Dessa forma, a comparação prioriza os efeitos de OpenMP e sincronização, não o máximo desempenho disponível com SIMD.

Os atributos `noinline` e `noipa` são utilizados para preservar o trabalho que deve ser medido. Essa decisão é metodologicamente necessária ao benchmark, mas restringe algumas otimizações que poderiam ocorrer em um programa de produção.

O batching adaptativo utiliza uma execução piloto para estimar o tamanho do lote. Flutuações nessa medição podem resultar em lotes cuja duração real posterior se afasta

do alvo de 20 ms. Esse efeito foi especialmente visível em algumas medições da versão **SemSync**, cuja execução também é afetada pela própria condição de corrida.

A versão sem sincronização contém deliberadamente uma *race condition*. Consequentemente, seus resultados não são determinísticos e não representam uma implementação funcionalmente válida.

Por fim, os experimentos foram executados em sistema operacional de propósito geral. Escalonamento, serviços em segundo plano, frequência dinâmica e estado térmico podem provocar variações entre execuções.

6.16 Conclusões da atividade

A Atividade 6 permitiu avaliar experimentalmente quatro formas distintas de tratar a dependência sobre o acumulador do método composto do trapézio.

A versão sem sincronização demonstrou diretamente a consequência de uma *race condition*. Em $N = 24\,000$, somente quatro das vinte medições produziram o resultado esperado, enquanto todos os tamanhos posteriores apresentaram zero resultados válidos em vinte execuções.

A diretiva **critical** restaurou a correção, porém introduziu forte serialização. No maior tamanho analisado, essa versão foi aproximadamente 49.5 vezes mais lenta que a execução sequencial.

A diretiva **atomic** reduziu o custo em relação a **critical**, mas continuou apresentando forte contenção sobre o mesmo acumulador compartilhado. Para $N = 2\,400\,000$, seu speedup foi apenas 0.031.

A versão baseada em **reduction** apresentou o melhor equilíbrio entre correção e desempenho. Para problemas pequenos, seu overhead superou o benefício do paralelismo. Com o crescimento de N , entretanto, o trabalho paralelo passou a amortizar esse custo.

O maior speedup observado foi

$$\boxed{2.716} \tag{6.63}$$

com quatro threads em

$$N = 1\,200\,000, \tag{6.64}$$

correspondendo a eficiência aproximada de

$$\boxed{67.9\%}. \tag{6.65}$$

Os resultados demonstram que a mera introdução de múltiplas threads não garante ganho de desempenho. A estratégia utilizada para coordenar os dados compartilhados é

determinante.

Para o padrão de soma presente no método do trapézio, a redução mostrou-se substancialmente mais adequada que a proteção do acumulador em cada iteração.

6.17 Síntese

Os principais pontos para defesa oral podem ser sintetizados da seguinte forma:

1. A Atividade 6 implementa experimentalmente a paralelização que havia sido discutida teoricamente na Atividade 4.
2. O problema matemático foi mantido:

$$f(x) = x^2, \quad [a, b] = [0, 2\,400\,000].$$

3. Foram comparadas cinco versões: Sequencial, SemSync, Critical, Atomic e Reduction.
4. Todas as versões executam exatamente o mesmo cálculo; a principal diferença é a forma como o acumulador `soma` é tratado.
5. Foram utilizadas quatro threads e `schedule(static)`.
6. O escalonamento estático foi escolhido porque todas as iterações possuem custo aproximadamente uniforme.
7. Foram realizados três aquecimentos e vinte medições por configuração.
8. Foi utilizado batching adaptativo com alvo de aproximadamente 20 ms, porque uma quantidade fixa de repetições seria inadequada para versões com custos muito diferentes.
9. O tempo apresentado corresponde à média das vinte medições, conforme solicitado pelo enunciado.
10. A versão SemSync possui uma race condition no acumulador.
11. Em $N = 24\,000$, SemSync produziu apenas 4 resultados corretos em 20 medições; para todos os demais valores, produziu 0/20.
12. O erro de SemSync é apresentado como erro absoluto médio das vinte medições, e não como erro de uma única execução.
13. Critical resolve a race condition, mas serializa cada atualização da soma.

14. No maior caso, Critical demorou aproximadamente 92.88 ms contra 1.87 ms do sequencial, sendo cerca de 49.5 vezes mais lento.

15. Atomic também garante correção e foi mais rápido que Critical na execução final, porém todas as threads continuam disputando o mesmo acumulador.

16. No maior caso, Atomic apresentou speedup de apenas

$$0.031.$$

17. Reduction utiliza acumuladores privados e combina os resultados ao final, evitando sincronização sobre a soma em cada iteração.

18. Para $N = 24\,000$, Reduction ainda foi mais lenta que o sequencial:

$$S = 0.304.$$

19. Em $N = 120\,000$, o desempenho ficou praticamente no ponto de equilíbrio:

$$S = 1.015.$$

20. A partir dos tamanhos seguintes, Reduction passou a apresentar ganho consistente.

21. O maior speedup observado foi

$$S = 2.716$$

para

$$N = 1\,200\,000.$$

22. Com quatro threads, isso corresponde a eficiência de aproximadamente

$$67.9\%.$$

23. O speedup não cresceu monotonicamente: em $N = 2\,400\,000$, Reduction apresentou

$$S = 2.142.$$

Essa variação é compatível com fatores reais de execução como runtime OpenMP, escalonamento, frequência dinâmica e estado do sistema.

24. O aparente speedup da versão SemSync não deve ser interpretado como ganho válido, pois desempenho sem correção funcional não representa uma solução utilizável.

25. A principal conclusão é que, para uma soma paralela, proteger cada atualização com Critical ou Atomic pode introduzir contenção suficiente para tornar o programa muito mais lento.
26. Para esse padrão computacional, Reduction foi a estratégia que melhor preservou simultaneamente correção e desempenho.

Capítulo 7

Atividade 7 – Escopo de variáveis e estratégias de escalonamento em OpenMP

7.1 Enunciado da tarefa

Implemente diferentes versões de um programa para calcular quantos números primos existem entre 2 e um valor máximo n . O programa deve ser paralelizado utilizando OpenMP.

Devem ser implementadas versões com a variável do contador utilizando apenas `private`, apenas `firstprivate`, apenas `lastprivate`, `firstprivate` e `lastprivate`, uso de `default(none)` e uma versão paralela utilizando `reduction`.

A partir da versão selecionada como a de melhor comportamento, implemente versões considerando os escalonadores `static`, `dynamic`, `guided`, `auto` e `runtime`. Analise os tempos médios de execução de cada versão à medida que n aumenta e utilize gráficos para apresentar e analisar os resultados.

7.2 Objetivos e requisitos

A atividade teve como objetivo aprofundar dois aspectos fundamentais da programação paralela com OpenMP: o escopo de variáveis em regiões paralelas e a distribuição das iterações de um laço entre as threads.

O problema computacional foi mantido em continuidade com a Atividade 5: determinar a quantidade de números primos existentes no intervalo

$$[2, n].$$

Essa continuidade permitiu reutilizar uma implementação previamente validada e concentrar o experimento nos novos mecanismos do OpenMP.

Os objetivos específicos foram:

- reutilizar o algoritmo de contagem de primos desenvolvido na Atividade 5;
- preservar a mesma função de teste de primalidade em todas as versões;
- analisar o comportamento das cláusulas `private`, `firstprivate`, `lastprivate`, `firstprivate + lastprivate`, `default(none)` e `reduction`;
- verificar quais dessas construções preservam a contagem global correta;
- selecionar uma implementação paralela funcionalmente correta para o experimento de escalonamento;
- comparar as políticas `static`, `dynamic`, `guided`, `auto` e `runtime`;
- medir o tempo médio de execução das diferentes versões;
- manter fixos os demais parâmetros experimentais para isolar o efeito das cláusulas estudadas;
- representar os resultados por meio de tabelas e gráficos;
- discutir correção, sincronização, overhead e balanceamento de carga.

A atividade foi estruturada em dois experimentos principais:

1. comparação das cláusulas de escopo de dados;
2. comparação das estratégias de escalonamento utilizando `reduction`.

7.3 Fundamentação conceitual

7.3.1 Paralelismo na contagem de números primos

A contagem considera todos os candidatos pertencentes ao intervalo

$$2, 3, \dots, n.$$

Para cada candidato x , é realizada uma classificação independente:

$$P(x) = \begin{cases} 1, & \text{se } x \text{ é primo,} \\ 0, & \text{caso contrário.} \end{cases}$$

A quantidade total de números primos é, portanto,

$$C(n) = \sum_{x=2}^n P(x).$$

Como a classificação de um candidato não depende dos demais, o laço que percorre os candidatos apresenta paralelismo de dados natural.

O desafio está na agregação das contribuições individuais para o contador global.

7.3.2 Variáveis compartilhadas e privadas

Em uma região paralela OpenMP, as threads podem acessar variáveis com diferentes regras de compartilhamento.

Uma variável compartilhada possui uma única instância visível pelas threads, enquanto uma variável privada possui uma instância independente para cada thread.

Essa distinção é fundamental para a contagem de primos. Embora o trabalho de testar os candidatos seja independente, a contagem final representa uma operação global:

$$C = C_0 + C_1 + \cdots + C_{T-1},$$

onde C_i representa a quantidade encontrada pela thread i .

A simples criação de cópias privadas não realiza automaticamente essa soma.

7.3.3 `private`

A cláusula

`private(count)`

cria uma cópia independente de `count` para cada thread.

As cópias por thread são representadas na Figura 7.1.

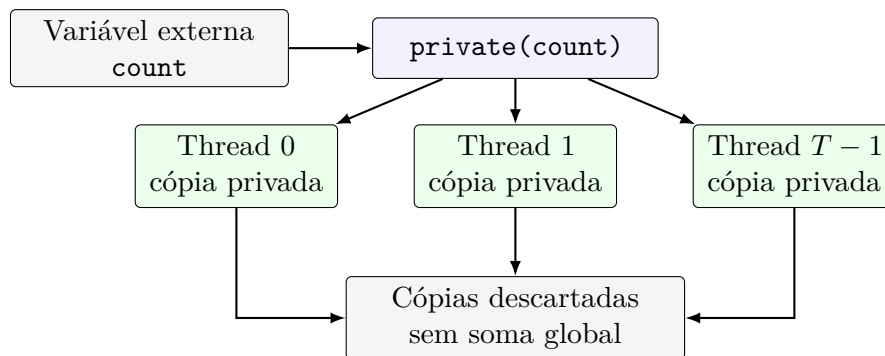


Figura 7.1: Cópias privadas independentes sem agregação ao contador externo.

As cópias privadas não recebem automaticamente o valor anteriormente armazenado na variável original e também não são automaticamente combinadas ao término da região paralela.

Assim,

private

resolve o isolamento entre as threads, mas não resolve a agregação das contagens.

7.3.4 **firstprivate**

A cláusula

firstprivate(count)

também cria uma cópia privada para cada thread, mas inicializa essa cópia com o valor existente antes da região paralela.

Se inicialmente

$count = 0$,

então todas as threads começam com

$count_i = 0$.

Ao final da região, entretanto, essas cópias continuam sendo descartadas.

Portanto,

firstprivate

resolve a inicialização das cópias, mas não implementa uma operação de redução.

7.3.5 **lastprivate**

A cláusula

lastprivate(count)

torna a variável privada durante o laço e, ao término da região paralela, transfere para a variável externa o valor correspondente à última iteração lógica do laço.

Isso não representa a soma das contribuições das threads.

Formalmente,

$$\text{lastprivate} \neq \sum_i C_i.$$

Existe ainda um cuidado importante: a cópia privada criada por `lastprivate` não deve ser lida sem inicialização.

Na implementação desta atividade, `count` é explicitamente definido antes de ser utilizado em cada iteração, evitando leitura de valor indeterminado.

7.3.6 `firstprivate` e `lastprivate`

A combinação

```
firstprivate(count) lastprivate(count)
```

faz com que cada cópia privada seja inicializada com o valor externo e que, ao final, a cópia associada à última iteração lógica seja propagada para a variável externa.

Entretanto, continua não existindo uma agregação global.

Assim,

```
firstprivate + lastprivate ≠ reduction
```

7.3.7 `default(none)`

A cláusula

```
default(none)
```

obriga que o programador declare explicitamente a classificação das variáveis utilizadas na região paralela.

Seu objetivo é aumentar a clareza da política de compartilhamento e reduzir erros decorrentes de classificações implícitas.

Entretanto,

```
default(none) não é um mecanismo de sincronização
```

Se uma variável for explicitamente declarada como compartilhada e múltiplas threads realizarem atualizações concorrentes sem proteção, a condição de corrida continua existindo.

7.3.8 Condição de corrida

A operação

```
++count;
```

não deve ser interpretada como uma única operação indivisível.

Conceitualmente, ela envolve:

1. leitura do valor;
2. incremento;
3. escrita do novo valor.

Se duas ou mais threads realizarem essas operações simultaneamente sobre o mesmo contador compartilhado, atualizações podem ser perdidas.

Essa situação constitui uma *data race*.

Os resultados produzidos por uma implementação com condição de corrida não possuem garantia de correção e uma eventual coincidência com o resultado esperado não elimina o erro de sincronização.

7.3.9 reduction

O padrão matemático da contagem de primos é uma soma de resultados parciais.

OpenMP fornece diretamente esse padrão por meio de:

`reduction(+:count)`

Cada thread realiza sua própria acumulação e, ao final da região, OpenMP combina os valores privados.

O processo de combinação é ilustrado na Figura 7.2.

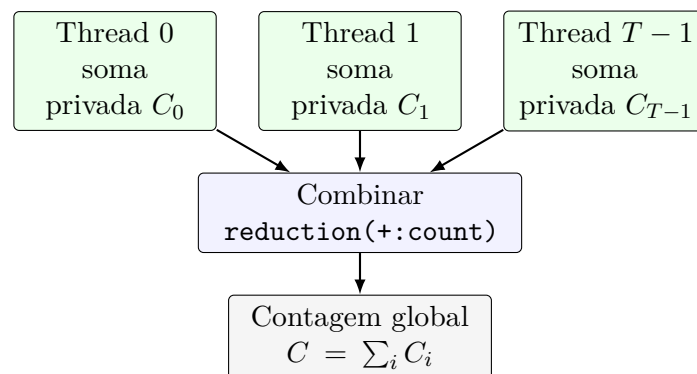


Figura 7.2: Agregação das contagens privadas pela cláusula `reduction`.

Assim,

$$C = \sum_{i=0}^{T-1} C_i.$$

A cláusula representa diretamente a operação necessária para obter a contagem global.

7.3.10 Escalonamento de iterações

Depois de definida uma implementação paralela correta, é necessário decidir como as iterações do laço serão distribuídas entre as threads.

Em OpenMP essa política pode ser especificada por meio da cláusula `schedule`.

static

Na política

`schedule(static)`

as iterações são distribuídas previamente entre as threads.

Essa abordagem possui baixo overhead e comportamento previsível.

Entretanto, uma quantidade semelhante de iterações não implica necessariamente uma quantidade semelhante de trabalho.

dynamic

Na política

`schedule(dynamic)`

novos blocos de iterações são obtidos pelas threads à medida que o trabalho anterior é concluído.

Isso pode melhorar o balanceamento de cargas irregulares, mas aumenta o custo de gerenciamento do escalonamento.

guided

Em

`schedule(guided)`

o trabalho também é distribuído dinamicamente, mas blocos maiores são utilizados inicialmente e seu tamanho é progressivamente reduzido.

A estratégia busca combinar:

baixo overhead inicial

com

melhor balanceamento próximo ao final.

auto

Com

`schedule(auto)`

a escolha da forma de distribuição é delegada à implementação OpenMP.

O programa não determina explicitamente qual estratégia interna será utilizada.

runtime

Com

`schedule(runtime)`

a configuração do escalonamento é determinada em tempo de execução.

Nesta atividade, a política foi explicitamente configurada como

`dynamic, chunk = 1.`

Dessa forma, o significado de **runtime** permaneceu reproduzível durante o experimento.

7.3.11 Irregularidade da carga

O custo do teste de primalidade não é uniforme.

Números pares são rejeitados imediatamente, enquanto candidatos ímpares, especialmente números primos maiores, podem exigir vários testes de divisibilidade.

Assim,

$$W(i) \neq W(j),$$

onde $W(x)$ representa o trabalho necessário para testar o candidato x .

Essa característica torna o problema apropriado para avaliar políticas de escalonamento.

7.3.12 Tempo médio

O enunciado solicita a análise do tempo médio de execução.

Para $M = 20$ medições,

$$T_{\text{medio}} = \frac{1}{20} \sum_{i=1}^{20} T_i.$$

Essa estatística foi utilizada como medida principal nesta atividade.

7.4 Interpretação didática da tarefa

A atividade foi interpretada como uma continuação direta do estudo iniciado na Atividade 5.

Na atividade anterior, verificou-se que uma simples paralelização do laço com um contador compartilhado produzia uma condição de corrida e que a cláusula **reduction** fornecia a agregação correta.

Nesta atividade, o objetivo foi decompor esse problema em duas questões.

A primeira foi:

Como diferentes cláusulas de escopo alteram o comportamento de uma variável utilizada como contador?

A segunda foi:

Depois de obtida uma implementação correta, como a política utilizada para distribuir as iterações influencia o desempenho?

Assim, a progressão didática adotada está representada na Figura 7.3.

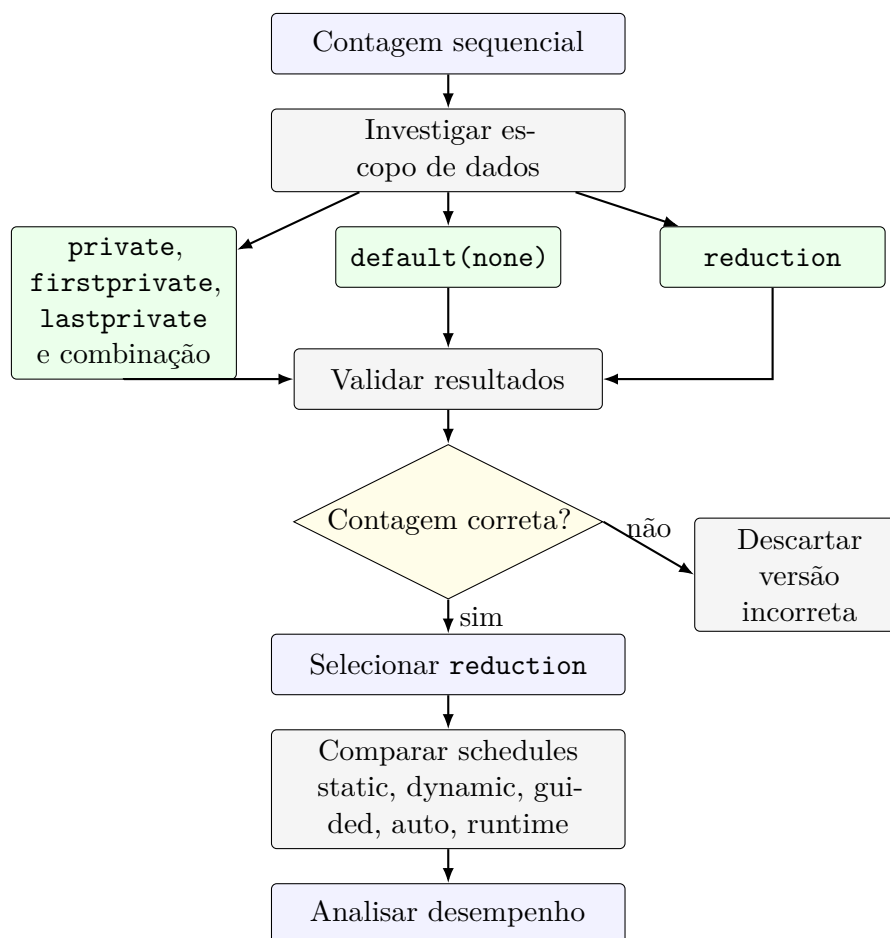


Figura 7.3: Progressão da análise de escopo até a comparação de desempenho.

A atividade, portanto, separa explicitamente dois problemas que podem ser confundidos em uma primeira abordagem à programação paralela:

compartilhamento de dados

e

distribuição do trabalho

.

7.5 Modelagem da solução

7.5.1 Estrutura geral

A solução foi implementada por adaptação direta do programa desenvolvido na Atividade 5.

Foram preservados:

- o algoritmo de primalidade;
- os valores de N ;
- as contagens conhecidas de primos;
- a temporização baseada em `omp_get_wtime()`;
- os aquecimentos;
- as repetições internas calibradas;
- a validação funcional;
- o uso de quatro threads.

A nova implementação foi dividida em dois experimentos.

Experimento A – Escopo de dados

Foram consideradas sete implementações:

1. sequencial;
2. `private`;
3. `firstprivate`;
4. `lastprivate`;

5. `firstprivate + lastprivate`;
6. `default(none)`;
7. `reduction`.

A versão sequencial foi utilizada como referência.

A seleção da implementação a ser utilizada posteriormente não foi baseada apenas no menor tempo, mas na ordem:

$\text{correção} \longrightarrow \text{desempenho}$

Experimento B – Escalonamento

A versão com `reduction`, validada no primeiro experimento, foi utilizada como base.

A única dimensão modificada passou a ser a política de escalonamento:

1. `static`;
2. `dynamic`;
3. `guided`;
4. `auto`;
5. `runtime`.

Todas as versões continuaram utilizando exatamente o mesmo teste de primalidade e a mesma redução.

7.5.2 Principais funções e componentes

A organização principal do programa é apresentada na Tabela 7.1.

7.6 Decisões de projeto e trade-offs

7.6.1 Reutilização da Atividade 5

A implementação não foi reconstruída do início.

O código anterior já fornecia:

- teste de primalidade validado;
- valores conhecidos de referência;

Tabela 7.1: Principais componentes da implementação da Atividade 7.

Componente	Responsabilidade
<code>is_prime()</code>	Teste de primalidade comum a todas as versões.
<code>count_primes_sequential()</code>	Implementação sequencial de referência.
<code>count_primes_private()</code>	Versão com cópias privadas do contador.
<code>count_primes_firstprivate()</code>	Versão cujas cópias privadas recebem o valor inicial do contador.
<code>count_primes_lastprivate()</code>	Versão que propaga o valor da última iteração lógica.
<code>count_primes_first_lastprivate()</code>	Combina inicialização privada e propagação da última iteração.
<code>count_primes_default_none()</code>	Declara explicitamente o compartilhamento, mantendo o contador compartilhado sem sincronização.
<code>count_primes_reduction()</code>	Implementação correta com redução e <code>schedule(static)</code> .
Funções de <code>schedule</code>	Implementam <code>dynamic</code> , <code>guided</code> , <code>auto</code> e <code>runtime</code> , sempre com redução.
<code>run_timed_batch()</code>	Executa várias chamadas dentro de um único intervalo cronometrado.
<code>calibrate_internal_repetitions()</code>	Determina um número comum de repetições internas por valor de N .
<code>run_warmups_rotated()</code>	Executa aquecimentos alterando a ordem inicial das versões.
<code>benchmark_function_set()</code>	Executa vinte medições por versão, com ordem rotacionada, e calcula a média.
Funções de exportação CSV	Persistem os resultados utilizados na geração e análise dos gráficos.

- controle da quantidade de threads;
- temporização;
- aquecimentos;
- calibração de lotes;
- mecanismo para impedir a eliminação da computação pelo compilador.

A reutilização reduz a quantidade de novas variáveis introduzidas no experimento e mantém continuidade entre as atividades.

7.6.2 Mesma função de primalidade

Todas as versões utilizam a mesma função `is_prime()`.

Não foram introduzidos algoritmos alternativos, como crivos ou novas otimizações específicas.

Assim, as diferenças observadas decorrem das construções OpenMP avaliadas e não de algoritmos diferentes.

7.6.3 Tratamento seguro de `private`

Uma variável criada com `private` não herda automaticamente o valor da variável externa.

Utilizar diretamente

```
++count;
```

sem inicialização produziria leitura de um valor indeterminado.

Por isso, a implementação inicializa explicitamente a cópia privada de cada thread:

```
count = 0;
```

As contagens privadas são armazenadas temporariamente apenas para manter o trabalho computacional observável durante a otimização do compilador.

O contador externo permanece zero.

7.6.4 Observabilidade das cópias privadas

Nas versões `private` e `firstprivate`, o resultado das cópias privadas seria descartado ao final da região.

Isso poderia permitir ao compilador identificar parte da computação como sem efeito observável.

Por esse motivo, as contagens parciais são utilizadas na formação de uma assinatura consumida pelo programa.

Essa estratégia preserva o significado do experimento, mas adiciona uma pequena quantidade de trabalho auxiliar às versões correspondentes.

Consequentemente, seus tempos devem ser interpretados como os tempos das implementações experimentais efetivamente utilizadas, e não como uma medida isolada do custo da cláusula OpenMP.

7.6.5 Tratamento seguro de `lastprivate`

A cópia associada a `lastprivate` não deve ser incrementada a partir de um valor não inicializado.

Para evitar comportamento indefinido, cada iteração atribui

```
count = 0;
```

antes de testar o candidato.

O valor final passa, portanto, a representar a classificação da última iteração lógica.

Como todos os valores de N utilizados são pares, a última iteração corresponde a um número não primo e o resultado observado foi zero em todos os casos.

7.6.6 Separação entre `default(none)` e `reduction`

A versão com `default(none)` foi mantida separada da versão com `reduction`.

O contador foi explicitamente declarado como compartilhado:

```
default(none) shared(n, count)
```

mas não recebeu proteção adicional.

Essa decisão permite demonstrar que declarar explicitamente o escopo das variáveis não elimina uma condição de corrida.

7.6.7 Correção antes de desempenho

Versões funcionalmente incorretas também foram cronometradas para observar seu comportamento experimental.

Entretanto, elas não foram consideradas candidatas à melhor solução.

O critério utilizado está representado na Figura 7.4.

Essa decisão evita interpretar uma implementação que realiza uma computação incorreta como uma otimização válida.

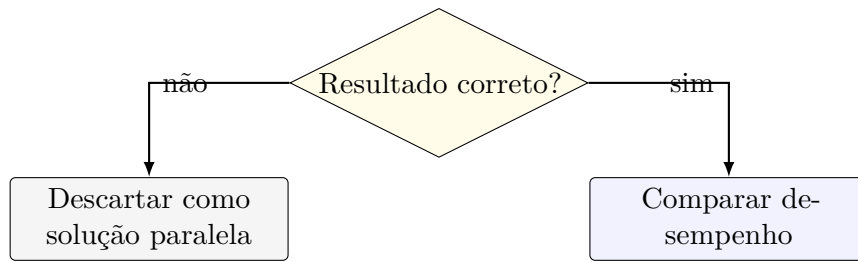


Figura 7.4: A correção funcional precede a comparação de desempenho.

7.6.8 Seleção de **reduction**

A versão com redução foi a única versão paralela do primeiro experimento que reproduziu a contagem global correta para todos os valores de N .

Ela foi, portanto, escolhida como base do experimento de escalonamento.

7.6.9 Número fixo de **threads**

Foram utilizadas quatro threads em todas as execuções.

A quantidade de threads não foi variada porque o objetivo da atividade era isolar:

1. o efeito do escopo das variáveis;
2. o efeito do escalonamento.

Adicionar diferentes quantidades de threads introduziria uma terceira dimensão experimental.

7.6.10 Não variação explícita do tamanho dos blocos

Não foi realizado um experimento separado de *chunk size*.

A comparação foi mantida concentrada nas políticas solicitadas no enunciado.

No caso de **runtime**, foi utilizada explicitamente a configuração

`dynamic, 1.`

7.6.11 Média em vez de mediana

Na Atividade 5 foi utilizada a mediana.

Nesta atividade, entretanto, o enunciado solicita tempos médios de execução.

Assim, a estatística principal foi alterada para a média aritmética das vinte medições:

$$\bar{T} = \frac{1}{20} \sum_{i=1}^{20} T_i.$$

7.6.12 Rotação da ordem de execução

Com várias versões no mesmo experimento, executar sempre as funções na mesma ordem poderia introduzir viés sistemático.

Foi implementada uma rotação determinística.

O mecanismo de rotação está representado na Figura 7.5.

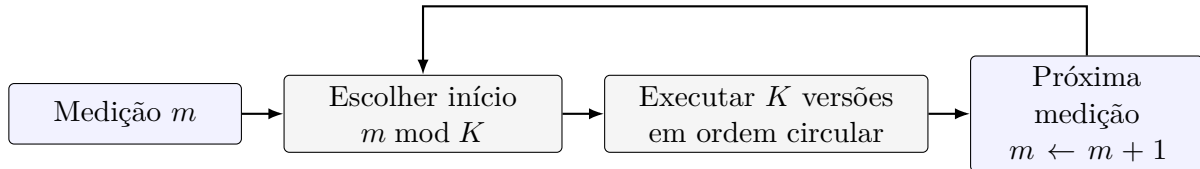


Figura 7.5: Rotação determinística da ordem das versões entre medições.

A mesma estratégia foi utilizada durante os aquecimentos.

7.6.13 Repetições internas comuns

O número de repetições internas foi calibrado para cada N utilizando a implementação sequencial e a implementação de referência com `reduction` e `static`.

O mesmo número de repetições foi então utilizado para as demais versões daquele N .

Essa abordagem preserva comparabilidade entre as configurações.

Como consequência, uma versão posteriormente mais rápida que a utilizada na calibração pode gerar um lote menor que o alvo nominal de 100 ms. O limite deve, portanto, ser entendido como um alvo da calibração da referência, e não como garantia para todas as variantes.

7.7 Representação estrutural da implementação

A estrutura geral do programa pode ser representada pelo diagrama a seguir.

7.8 Fluxo de execução

O fluxo completo da execução está modelado na Figura 7.7.

7.9 Metodologia experimental

7.9.1 Ambiente de execução

A atividade foi executada em ambiente Windows utilizando MSYS2 no subsistema UCRT64.

O ambiente previamente validado para as atividades OpenMP possuía:

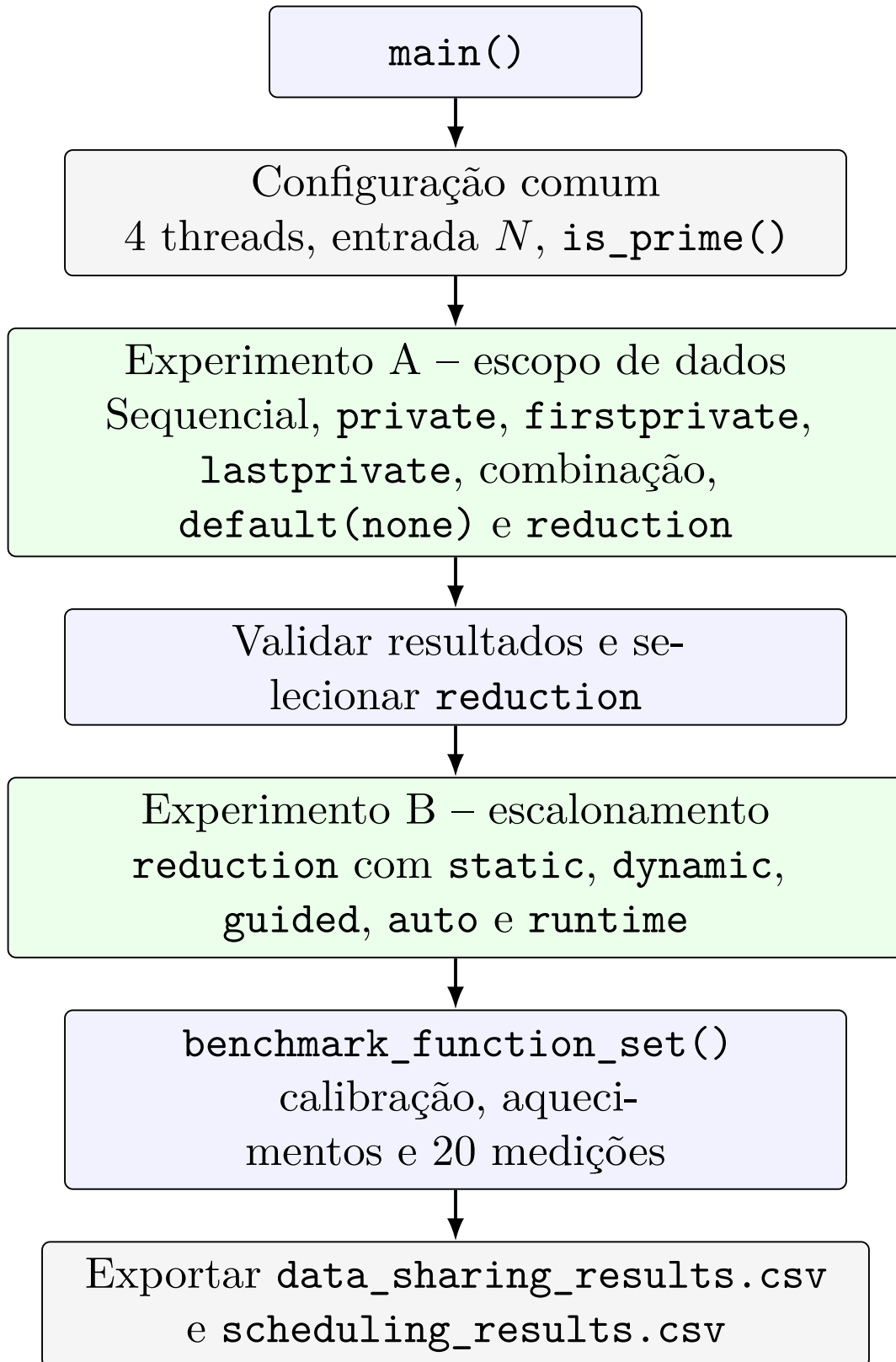


Figura 7.6: Diagrama UML de componentes dos dois experimentos da Atividade 7.

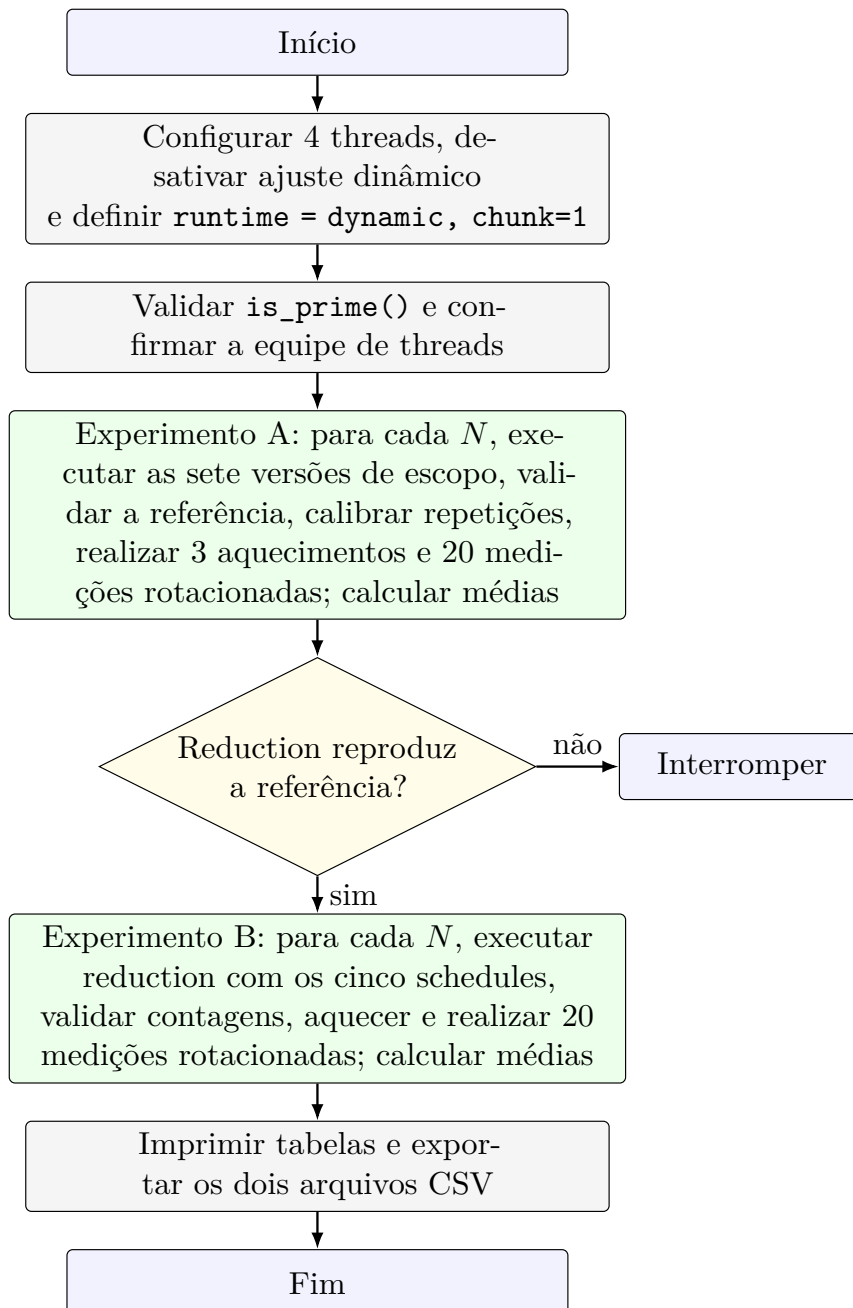


Figura 7.7: Diagrama UML de atividade do fluxo dos experimentos A e B.

- GCC 15.1.0;
- compilador /ucrt64/bin/gcc;
- alvo x86_64-w64-mingw32;
- runtime OpenMP baseado em libgomp;
- suporte identificado pela macro `_OPENMP = 201511`, correspondente ao OpenMP 4.5.

A compilação foi realizada com:

```
gcc -std=c11 -O2 -Wall -Wextra -Wpedantic \
    -fopenmp tarefa7.c -o tarefa7.exe
```

7.9.2 Valores avaliados

Foram mantidos os mesmos valores de N utilizados na atividade anterior:

$$N \in \{10\,000, 100\,000, 1\,000\,000, 5\,000\,000, 10\,000\,000\}.$$

As contagens utilizadas como referência foram:

Tabela 7.2: Contagens conhecidas utilizadas na validação.

N	Quantidade de primos
10 000	1 229
100 000	9 592
1 000 000	78 498
5 000 000	348 513
10 000 000	664 579

7.9.3 Configuração das medições

A configuração experimental foi:

7.9.4 Calibração temporal

Para cargas pequenas, uma única execução pode possuir duração insuficiente para uma medição robusta.

Por isso, várias contagens foram agrupadas dentro de um lote:

$$T_{\text{exec}} = \frac{T_{\text{lote}}}{R},$$

Tabela 7.3: Parâmetros do benchmark da Atividade 7.

Parâmetro	Valor
Threads	4
Ajuste dinâmico da equipe	desabilitado
Warm-ups	3
Medições por versão	20
Estatística	média aritmética
Temporização	<code>omp_get_wtime()</code>
Alvo de lote	aproximadamente 100 ms
Otimização	-O2
Runtime schedule	<code>dynamic,1</code>

onde R representa a quantidade de repetições internas.

A calibração inicia com

$$R = 1$$

e dobra o valor progressivamente:

$$1, 2, 4, 8, 16, \dots$$

até que as versões sequencial e `reduction + static` atinjam o alvo temporal.

As quantidades efetivamente utilizadas foram:

$$1024, \quad 64, \quad 2, \quad 1, \quad 1.$$

Portanto, a quantidade de repetições diminuiu conforme o custo de uma única contagem aumentou.

7.9.5 Aquecimentos

Antes das vinte medições oficiais foram executados três aquecimentos.

Os aquecimentos utilizaram os mesmos tamanhos de lote das medições subsequentes e também empregaram rotação da ordem das versões.

7.9.6 Ordem das medições

O conjunto de funções foi executado com deslocamento da posição inicial a cada medição.

Esse procedimento reduz a possibilidade de uma versão ser sistematicamente executada:

- sempre com a CPU mais fria;
- sempre após outra carga específica;
- sempre no início ou no final da rodada.

7.9.7 Cálculo da média

Para cada versão foram realizadas vinte medições.

O tempo reportado foi:

$$\bar{T} = \frac{1}{20} \sum_{i=1}^{20} T_i.$$

Os tempos dos lotes foram previamente divididos pela quantidade de repetições internas, de forma que os valores apresentados representam uma única execução equivalente.

7.9.8 Região cronometrada

A região cronometrada inclui a chamada efetiva da função de contagem, incluindo os custos decorrentes da execução OpenMP.

Ficam fora da região medida operações como:

- impressão das tabelas;
- escrita dos arquivos CSV;
- cálculo e apresentação posterior dos resultados.

7.9.9 Arquivos de resultados

Ao final da execução foram gerados:

```
data_sharing_results.csv  
scheduling_results.csv
```

O primeiro armazena os resultados do experimento de escopo de dados.

O segundo armazena os resultados do experimento de escalonamento.

Os arquivos preservam os tempos com maior precisão decimal do que a apresentação formatada do terminal.

7.10 Código-fonte desenvolvido

```
1 /*  
2  * Tarefa 7 - Escopo de dados e escalonamento com OpenMP  
3  *  
4  * Adaptacao da Tarefa 5. O primeiro experimento compara clausulas  
5  * de compartilhamento do contador; o segundo compara politicas de  
6  * escalonamento usando reduction.  
7  */  
8  
9 #include <stdio.h>
```

```

10 #include <stdlib.h>
11 #include <omp.h>
12
13 #define NUM_THREADS          4
14 #define NUM_WARMUPS          3
15 #define NUM_MEASUREMENTS     20
16 #define TARGET_BATCH_SECONDS  0.100
17 #define MAX_INTERNAL_REPETITIONS 1048576
18
19 /* Configuracao fixa de schedule(runtime). */
20 #define RUNTIME_SCHEDULE_KIND  omp_sched_dynamic
21 #define RUNTIME_SCHEDULE_CHUNK  1
22
23 static const int N_VALUES[] = {
24     10000, 100000, 1000000, 5000000, 10000000
25 };
26
27 static const long long EXPECTED_PRIMES[] = {
28     1229, 9592, 78498, 348513, 664579
29 };
30
31 #define NUM_SIZES (sizeof(N_VALUES) / sizeof(N_VALUES[0]))
32 #define DATA_VERSION_COUNT 7
33 #define SCHEDULE_VERSION_COUNT 5
34
35 typedef long long (*CountFunction)(int);
36
37 static volatile unsigned long long semantic_sink = 0;
38 static volatile unsigned long long benchmark_sink = 0;
39
40
41 /* Teste de primalidade comum a todas as versoes. */
42 static int is_prime(int value)
43 {
44     if (value < 2) {
45         return 0;
46     }
47
48     if (value == 2) {
49         return 1;
50     }
51
52     if (value % 2 == 0) {
53         return 0;
54     }
55
56     for (int divisor = 3; divisor <= value / divisor; divisor += 2) {
57         if (value % divisor == 0) {
58             return 0;
59         }
60     }
61
62     return 1;
63 }
64
65
66 /* Baseline sequencial. */
67 static long long count_primes_sequential(int n)
68 {

```

```

69     long long count = 0;
70
71     for (int candidate = 2; candidate <= n; ++candidate) {
72         if (is_prime(candidate)) {
73             ++count;
74         }
75     }
76
77     return count;
78 }
79
80
81 /* Mantem observavel o trabalho das copias privadas. */
82 static void observe_private_counts(const long long thread_counts[NUM_THREADS])
83 {
84     unsigned long long fingerprint = 0;
85
86     for (int i = 0; i < NUM_THREADS; ++i) {
87         fingerprint ^=
88             (unsigned long long) thread_counts[i]
89             + (unsigned long long) (i + 1);
90     }
91
92     semantic_sink ^= fingerprint;
93 }
94
95
96 /*
97  * private: cada thread possui count independente.
98  * As copias nao sao combinadas; o count externo permanece zero.
99  */
100 static long long count_primes_private(int n)
101 {
102     long long count = 0;
103     long long thread_counts[NUM_THREADS] = {0};
104
105     #pragma omp parallel private(count)
106     {
107         const int tid = omp_get_thread_num();
108         count = 0;
109
110         #pragma omp for schedule(static)
111         for (int candidate = 2; candidate <= n; ++candidate) {
112             if (is_prime(candidate)) {
113                 ++count;
114             }
115         }
116
117         thread_counts[tid] = count;
118     }
119
120     observe_private_counts(thread_counts);
121     return count;
122 }
123
124
125 /*
126  * firstprivate: cada copia inicia com o valor externo.
127  * As copias privadas sao descartadas ao final.

```

```

128  */
129  static long long count_primes_firstprivate(int n)
130  {
131      long long count = 0;
132      long long thread_counts[NUM_THREADS] = {0};
133
134      #pragma omp parallel firstprivate(count)
135      {
136          const int tid = omp_get_thread_num();
137
138          #pragma omp for schedule(static)
139          for (int candidate = 2; candidate <= n; ++candidate) {
140              if (is_prime(candidate)) {
141                  ++count;
142              }
143          }
144
145          thread_counts[tid] = count;
146      }
147
148      observe_private_counts(thread_counts);
149      return count;
150  }
151
152
153  /*
154   * lastprivate: como a copia privada nao e inicializada,
155   * cada iteracao atribui count antes do uso.
156   */
157  static long long count_primes_lastprivate(int n)
158  {
159      long long count = 0;
160
161      #pragma omp parallel for schedule(static) lastprivate(count)
162      for (int candidate = 2; candidate <= n; ++candidate) {
163          count = 0;
164
165          if (is_prime(candidate)) {
166              ++count;
167          }
168      }
169
170      return count;
171  }
172
173
174  /*
175   * firstprivate + lastprivate: as copias iniciam com o valor
176   * externo e a copia da ultima iteracao e propagada ao final.
177   */
178  static long long count_primes_first_lastprivate(int n)
179  {
180      long long count = 0;
181
182      #pragma omp parallel for schedule(static) \
183          firstprivate(count) lastprivate(count)
184      for (int candidate = 2; candidate <= n; ++candidate) {
185          if (is_prime(candidate)) {
186              ++count;

```

```

187     }
188 }
189
190     return count;
191 }
192
193
194 /*
195  * default(none): o escopo e explicito, mas count continua
196  * compartilhado e sem sincronizacao, produzindo data race.
197  */
198 static long long count_primes_default_none(int n)
199 {
200     long long count = 0;
201
202     #pragma omp parallel for default(none) shared(n, count) schedule(static)
203     for (int candidate = 2; candidate <= n; ++candidate) {
204         if (is_prime(candidate)) {
205             ++count;
206         }
207     }
208
209     return count;
210 }
211
212
213 /* reduction com schedule(static): agregacao correta. */
214 static long long count_primes_reduction(int n)
215 {
216     long long count = 0;
217
218     #pragma omp parallel for schedule(static) reduction(+:count)
219     for (int candidate = 2; candidate <= n; ++candidate) {
220         if (is_prime(candidate)) {
221             ++count;
222         }
223     }
224
225     return count;
226 }
227
228
229 /* Versoes do experimento de escalonamento. */
230 static long long count_primes_schedule_dynamic(int n)
231 {
232     long long count = 0;
233
234     #pragma omp parallel for schedule(dynamic) reduction(+:count)
235     for (int candidate = 2; candidate <= n; ++candidate) {
236         if (is_prime(candidate)) {
237             ++count;
238         }
239     }
240
241     return count;
242 }
243
244 static long long count_primes_schedule_guided(int n)
245 {

```

```

246     long long count = 0;
247
248     #pragma omp parallel for schedule(guided) reduction(+:count)
249     for (int candidate = 2; candidate <= n; ++candidate) {
250         if (is_prime(candidate)) {
251             ++count;
252         }
253     }
254
255     return count;
256 }
257
258 static long long count_primes_schedule_auto(int n)
259 {
260     long long count = 0;
261
262     #pragma omp parallel for schedule(auto) reduction(+:count)
263     for (int candidate = 2; candidate <= n; ++candidate) {
264         if (is_prime(candidate)) {
265             ++count;
266         }
267     }
268
269     return count;
270 }
271
272 static long long count_primes_schedule_runtime(int n)
273 {
274     long long count = 0;
275
276     #pragma omp parallel for schedule(runtime) reduction(+:count)
277     for (int candidate = 2; candidate <= n; ++candidate) {
278         if (is_prime(candidate)) {
279             ++count;
280         }
281     }
282
283     return count;
284 }
285
286
287 /* Validacao basica do teste de primalidade. */
288 static void validate_is_prime(void)
289 {
290     struct PrimeTest {
291         int value;
292         int expected;
293     };
294
295     static const struct PrimeTest tests[] = {
296         {0, 0}, {1, 0}, {2, 1}, {3, 1}, {4, 0},
297         {5, 1}, {9, 0}, {17, 1}, {25, 0}
298     };
299
300     const size_t count = sizeof(tests) / sizeof(tests[0]);
301
302     for (size_t i = 0; i < count; ++i) {
303         if (is_prime(tests[i].value) != tests[i].expected) {
304             fprintf(stderr, "ERROR: primality test failed for %d.\n",

```

```

305         tests[i].value);
306         exit(EXIT_FAILURE);
307     }
308 }
309 }
310
311
312 /* Confirma o numero real de threads. */
313 static int get_actual_thread_count(void)
314 {
315     int actual_threads = 1;
316
317     #pragma omp parallel
318     {
319         #pragma omp single
320         actual_threads = omp_get_num_threads();
321     }
322
323     return actual_threads;
324 }
325
326
327 /* Converte a politica do runtime para texto. */
328 static const char *schedule_name(omp_sched_t kind)
329 {
330     switch (kind) {
331         case omp_sched_static: return "static";
332         case omp_sched_dynamic: return "dynamic";
333         case omp_sched_guided: return "guided";
334         case omp_sched_auto: return "auto";
335         default: return "unknown";
336     }
337 }
338
339
340 /*
341  * Executa um lote cronometrado. O checksum e consumido depois
342  * da medicao para manter o resultado observavel.
343  */
344 static double run_timed_batch(CountFunction function, int n, int repetitions)
345 {
346     volatile int guarded_n = n;
347     unsigned long long checksum = 0;
348     const double start = omp_get_wtime();
349
350     for (int repetition = 0; repetition < repetitions; ++repetition) {
351         checksum += (unsigned long long) function(guarded_n);
352     }
353
354     const double end = omp_get_wtime();
355     benchmark_sink ^= checksum;
356
357     return end - start;
358 }
359
360
361 /* Calibra um numero comum de repeticoes para cada N. */
362 static int calibrate_internal_repetitions(int n)
363 {

```

```

364     int repetitions = 1;
365
366     while (1) {
367         const double seq_elapsed =
368             run_timed_batch(count_primes_sequential, n, repetitions);
369
370         const double omp_elapsed =
371             run_timed_batch(count_primes_reduction, n, repetitions);
372
373         if (seq_elapsed >= TARGET_BATCH_SECONDS &&
374             omp_elapsed >= TARGET_BATCH_SECONDS) {
375             return repetitions;
376         }
377
378         if (repetitions >= MAX_INTERNAL_REPETITIONS / 2) {
379             return MAX_INTERNAL_REPETITIONS;
380         }
381
382         repetitions *= 2;
383     }
384 }
385
386
387 /* Tempo medio de uma chamada dentro do lote. */
388 static double measure_per_execution(
389     CountFunction function,
390     int n,
391     int repetitions
392 )
393 {
394     return run_timed_batch(function, n, repetitions)
395         / (double) repetitions;
396 }
397
398
399 /* Aquecimentos com rotacao da ordem das versoes. */
400 static void run_warmups_rotated(
401     CountFunction functions[],
402     size_t function_count,
403     int n,
404     int repetitions
405 )
406 {
407     for (int warmup = 0; warmup < NUM_WARMUPS; ++warmup) {
408         const size_t start = (size_t) warmup % function_count;
409
410         for (size_t step = 0; step < function_count; ++step) {
411             const size_t index = (start + step) % function_count;
412
413             (void) measure_per_execution(
414                 functions[index], n, repetitions
415             );
416         }
417     }
418 }
419
420
421 /*
422  * Realiza 20 medicoes por versao com ordem rotacionada e

```

```

423  * calcula a media aritmetica dos tempos.
424  */
425  static void benchmark_function_set(
426      CountFunction functions[],
427      size_t function_count,
428      int n,
429      int repetitions,
430      double mean_seconds[]
431  )
432  {
433      for (size_t i = 0; i < function_count; ++i) {
434          mean_seconds[i] = 0.0;
435      }
436
437      for (int measurement = 0;
438           measurement < NUM_MEASUREMENTS;
439           ++measurement) {
440
441          const size_t start =
442              (size_t) measurement % function_count;
443
444          for (size_t step = 0; step < function_count; ++step) {
445              const size_t index = (start + step) % function_count;
446
447              mean_seconds[index] +=
448                  measure_per_execution(
449                      functions[index], n, repetitions
450                  );
451          }
452      }
453
454      for (size_t i = 0; i < function_count; ++i) {
455          mean_seconds[i] /= (double) NUM_MEASUREMENTS;
456      }
457  }
458
459
460  /* Valida versoes que obrigatoriamente devem ser corretas. */
461  static void require_correct_result(
462      const char *name,
463      CountFunction function,
464      int n,
465      long long expected
466  )
467  {
468      const long long result = function(n);
469
470      if (result != expected) {
471          fprintf(stderr,
472              "ERROR: %s returned %lld for N=%d; expected %lld.\n",
473              name, result, n, expected);
474          exit(EXIT_FAILURE);
475      }
476  }
477
478
479  /* Exporta resultados do experimento de data-sharing. */
480  static void write_data_sharing_csv(
481      long long results[NUM_SIZES][DATA_VERSION_COUNT],

```

```

482     double times_ms[NUM_SIZES][DATA_VERSION_COUNT],
483     int repetitions[NUM_SIZES]
484 )
485 {
486     FILE *file = fopen("data_sharing_results.csv", "w");
487
488     if (file == NULL) {
489         fprintf(stderr,
490             "ERROR: could not create data_sharing_results.csv.\n");
491         exit(EXIT_FAILURE);
492     }
493
494     fprintf(file,
495         "N,Primes,Reps,"
496         "Sequential_result,Private_result,Firstprivate_result,"
497         "Lastprivate_result,FirstLastprivate_result,"
498         "DefaultNone_result,Reduction_result,"
499         "Sequential_ms,Private_ms,Firstprivate_ms,"
500         "Lastprivate_ms,FirstLastprivate_ms,"
501         "DefaultNone_ms,Reduction_ms\n");
502
503     for (size_t i = 0; i < NUM_SIZES; ++i) {
504         fprintf(file,
505             "%d,%lld,%d,"
506             "%lld,%lld,%lld,%lld,%lld,%lld,%lld,"
507             "%.6f,%.6f,%.6f,%.6f,%.6f,%.6f,%.6f\n",
508             N_VALUES[i], EXPECTED_PRIMES[i], repetitions[i],
509             results[i][0], results[i][1], results[i][2],
510             results[i][3], results[i][4], results[i][5],
511             results[i][6], times_ms[i][0], times_ms[i][1],
512             times_ms[i][2], times_ms[i][3], times_ms[i][4],
513             times_ms[i][5], times_ms[i][6]);
514     }
515
516     fclose(file);
517 }
518
519
520 /* Exporta resultados do experimento de scheduling. */
521 static void write_scheduling_csv(
522     long long results[NUM_SIZES][SCHEDULE_VERSION_COUNT],
523     double times_ms[NUM_SIZES][SCHEDULE_VERSION_COUNT],
524     int repetitions[NUM_SIZES]
525 )
526 {
527     FILE *file = fopen("scheduling_results.csv", "w");
528
529     if (file == NULL) {
530         fprintf(stderr,
531             "ERROR: could not create scheduling_results.csv.\n");
532         exit(EXIT_FAILURE);
533     }
534
535     fprintf(file,
536         "N,Primes,Reps,"
537         "Static_result,Dynamic_result,Guided_result,"
538         "Auto_result,Runtime_result,"
539         "Static_ms,Dynamic_ms,Guided_ms,Auto_ms,Runtime_ms\n");
540

```

```

541     for (size_t i = 0; i < NUM_SIZES; ++i) {
542         fprintf(file,
543             "%d,%lld,%d,"
544             "%lld,%lld,%lld,%lld,%lld,"
545             "%.6f,%.6f,%.6f,%.6f,%.6f\n",
546             N_VALUES[i], EXPECTED_PRIMES[i], repetitions[i],
547             results[i][0], results[i][1], results[i][2],
548             results[i][3], results[i][4], times_ms[i][0],
549             times_ms[i][1], times_ms[i][2], times_ms[i][3],
550             times_ms[i][4]);
551     }
552
553     fclose(file);
554 }
555
556
557 int main(void)
558 {
559     omp_set_dynamic(0);
560     omp_set_num_threads(NUM_THREADS);
561
562     /* schedule(runtime) = dynamic,1 para reproducibilidad. */
563     omp_set_schedule(
564         RUNTIME_SCHEDULE_KIND,
565         RUNTIME_SCHEDULE_CHUNK
566     );
567
568     validate_is_prime();
569
570     const int actual_threads = get_actual_thread_count();
571
572     if (actual_threads != NUM_THREADS) {
573         fprintf(stderr,
574             "ERROR: expected %d OpenMP threads, "
575             "but runtime created %d.\n",
576             NUM_THREADS, actual_threads);
577         return EXIT_FAILURE;
578     }
579
580     omp_sched_t runtime_kind;
581     int runtime_chunk;
582     omp_get_schedule(&runtime_kind, &runtime_chunk);
583
584     CountFunction data_functions[DATA_VERSION_COUNT] = {
585         count_primes_sequential,
586         count_primes_private,
587         count_primes_firstprivate,
588         count_primes_lastprivate,
589         count_primes_first_lastprivate,
590         count_primes_default_none,
591         count_primes_reduction
592     };
593
594     CountFunction schedule_functions[SCHEDULE_VERSION_COUNT] = {
595         count_primes_reduction,
596         count_primes_schedule_dynamic,
597         count_primes_schedule_guided,
598         count_primes_schedule_auto,
599         count_primes_schedule_runtime

```

```

600     };
601
602     long long data_results[NUM_SIZES][DATA_VERSION_COUNT];
603     double data_times_ms[NUM_SIZES][DATA_VERSION_COUNT];
604
605     long long schedule_results[NUM_SIZES][SCHEDULE_VERSION_COUNT];
606     double schedule_times_ms[NUM_SIZES][SCHEDULE_VERSION_COUNT];
607
608     int internal_repetitions[NUM_SIZES];
609
610     /* Valida a referencia e coleta os resultados de data-sharing. */
611     for (size_t i = 0; i < NUM_SIZES; ++i) {
612         const int n = N_VALUES[i];
613
614         require_correct_result(
615             "Sequential",
616             count_primes_sequential,
617             n,
618             EXPECTED_PRIMES[i]
619         );
620
621         require_correct_result(
622             "Reduction",
623             count_primes_reduction,
624             n,
625             EXPECTED_PRIMES[i]
626         );
627
628         for (size_t version = 0;
629             version < DATA_VERSION_COUNT;
630             ++version) {
631
632             data_results[i][version] =
633                 data_functions[version](n);
634         }
635     }
636
637     printf(
638         "=====\n"
639         "PRIME COUNT - OPENMP TASK 7\n"
640         "=====\n\n"
641     );
642
643     printf("Threads          : %d\n", actual_threads);
644     printf("Warm-ups            : %d\n", NUM_WARMUPS);
645     printf("Measurements        : %d\n", NUM_MEASUREMENTS);
646     printf("Statistic           : mean\n");
647     printf("Timing batch        : >= %.0f ms\n",
648         TARGET_BATCH_SECONDS * 1000.0);
649     printf("Runtime schedule : %s,%d\n",
650         schedule_name(runtime_kind), runtime_chunk);
651
652     printf(
653         "\n-----\n"
654         "DATA-SHARING - RESULT VALIDATION\n"
655         "-----\n\n"
656     );
657
658     printf("%-10s %-10s %-10s %-11s %-10s %-12s %-12s %-10s\n",

```

```

659         "N", "Seq", "Private", "FirstPriv", "LastPriv",
660         "First+Last", "DefaultNone", "Reduction");
661
662     printf(
663         "-----\n"
664     );
665
666     for (size_t i = 0; i < NUM_SIZES; ++i) {
667         printf("%-10d %-10lld %-10lld %-11lld %-10lld "
668             "%-12lld %-12lld %-10lld\n",
669             N_VALUES[i],
670             data_results[i][0], data_results[i][1],
671             data_results[i][2], data_results[i][3],
672             data_results[i][4], data_results[i][5],
673             data_results[i][6]);
674     }
675
676     printf(
677         "\nOnly values equal to Seq are functionally correct for that run.\n"
678         "A coincidental match does not remove a data race.\n"
679     );
680
681     /* Benchmark das sete versoes de data-sharing. */
682     for (size_t i = 0; i < NUM_SIZES; ++i) {
683         const int n = N_VALUES[i];
684
685         internal_repetitions[i] =
686             calibrate_internal_repetitions(n);
687
688         run_warmups_rotated(
689             data_functions,
690             DATA_VERSION_COUNT,
691             n,
692             internal_repetitions[i]
693         );
694
695         double means[DATA_VERSION_COUNT];
696
697         benchmark_function_set(
698             data_functions,
699             DATA_VERSION_COUNT,
700             n,
701             internal_repetitions[i],
702             means
703         );
704
705         for (size_t version = 0;
706             version < DATA_VERSION_COUNT;
707             ++version) {
708
709             data_times_ms[i][version] =
710                 means[version] * 1000.0;
711         }
712     }
713
714     printf(
715         "\n-----\n"
716         "DATA-SHARING - MEAN EXECUTION TIME\n"
717         "-----\n\n"

```

```

718 );
719
720 printf("%-10s %-6s %-10s %-10s %-11s %-10s "
721        "%-12s %-12s %-10s\n",
722        "N", "Reps", "Seq(ms)", "Private", "FirstPriv",
723        "LastPriv", "First+Last", "DefaultNone", "Reduction");
724
725 printf(
726     "-----\n
↪ "
727 );
728
729 for (size_t i = 0; i < NUM_SIZES; ++i) {
730     printf("%-10d %-6d %-10.3f %-10.3f %-11.3f %-10.3f "
731            "%-12.3f %-12.3f %-10.3f\n",
732            N_VALUES[i], internal_repetitions[i],
733            data_times_ms[i][0], data_times_ms[i][1],
734            data_times_ms[i][2], data_times_ms[i][3],
735            data_times_ms[i][4], data_times_ms[i][5],
736            data_times_ms[i][6]);
737 }
738
739 printf(
740     "\nScheduling base: reduction (correct aggregation).\n"
741 );
742
743 /*
744  * Scheduling: todas as versoes usam reduction e devem
745  * coincidir com a referencia sequencial.
746  */
747 for (size_t i = 0; i < NUM_SIZES; ++i) {
748     const int n = N_VALUES[i];
749     const long long expected = EXPECTED_PRIMES[i];
750
751     for (size_t version = 0;
752          version < SCHEDULE_VERSION_COUNT;
753          ++version) {
754
755         schedule_results[i][version] =
756             schedule_functions[version](n);
757
758         if (schedule_results[i][version] != expected) {
759             fprintf(stderr,
760                     "ERROR: scheduling version %zu returned "
761                     "%lld for N=%d; expected %lld.\n",
762                     version, schedule_results[i][version],
763                     n, expected);
764             return EXIT_FAILURE;
765         }
766     }
767
768     run_warmups_rotated(
769         schedule_functions,
770         SCHEDULE_VERSION_COUNT,
771         n,
772         internal_repetitions[i]
773     );
774
775     double means[SCHEDULE_VERSION_COUNT];

```

```

776     benchmark_function_set(
777         schedule_functions,
778         SCHEDULE_VERSION_COUNT,
779         n,
780         internal_repetitions[i],
781         means
782     );
783
784     for (size_t version = 0;
785          version < SCHEDULE_VERSION_COUNT;
786          ++version) {
787
788         schedule_times_ms[i][version] =
789             means[version] * 1000.0;
790     }
791 }
792
793 printf(
794     "\n-----\n"
795     "SCHEDULING - RESULT VALIDATION\n"
796     "-----\n\n"
797 );
798
799 printf("%-10s %-10s %-10s %-10s %-10s %-10s %-10s\n",
800     "N", "Primes", "Static", "Dynamic",
801     "Guided", "Auto", "Runtime");
802
803 printf(
804     "-----\n"
805 );
806
807 for (size_t i = 0; i < NUM_SIZES; ++i) {
808     printf("%-10d %-10ld %-10ld %-10ld %-10ld %-10ld %-10ld\n",
809         N_VALUES[i], EXPECTED_PRIMES[i],
810         schedule_results[i][0], schedule_results[i][1],
811         schedule_results[i][2], schedule_results[i][3],
812         schedule_results[i][4]);
813 }
814
815 printf(
816     "\n-----\n"
817     "SCHEDULING - MEAN EXECUTION TIME\n"
818     "-----\n\n"
819 );
820
821 printf("%-10s %-6s %-12s %-12s %-12s %-12s %-12s\n",
822     "N", "Reps", "Static(ms)", "Dynamic(ms)",
823     "Guided(ms)", "Auto(ms)", "Runtime(ms)");
824
825 printf(
826     "-----\n"
827 );
828
829 for (size_t i = 0; i < NUM_SIZES; ++i) {
830     printf("%-10d %-6d %-12.3f %-12.3f %-12.3f %-12.3f %-12.3f\n",
831         N_VALUES[i], internal_repetitions[i],
832         schedule_times_ms[i][0], schedule_times_ms[i][1],
833         schedule_times_ms[i][2], schedule_times_ms[i][3],
834

```

```

835         schedule_times_ms[i][4]);
836     }
837
838     write_data_sharing_csv(
839         data_results,
840         data_times_ms,
841         internal_repetitions
842     );
843
844     write_scheduling_csv(
845         schedule_results,
846         schedule_times_ms,
847         internal_repetitions
848     );
849
850     printf(
851         "\nCSV files generated:\n"
852         "  data_sharing_results.csv\n"
853         "  scheduling_results.csv\n"
854     );
855
856     return EXIT_SUCCESS;
857 }

```

Listing 7.1: Código-fonte da Atividade 7.

7.11 Validação da implementação

7.11.1 Validação do teste de primalidade

A função `is_prime()` foi inicialmente testada utilizando casos conhecidos:

0, 1, 2, 3, 4, 5, 9, 17, 25.

A execução é encerrada caso qualquer desses casos produza uma classificação diferente da esperada.

7.11.2 Validação da referência sequencial

A implementação sequencial foi comparada às contagens conhecidas para todos os valores utilizados.

Os resultados coincidiram integralmente com os valores de referência.

7.11.3 Validação das cláusulas de escopo

A Tabela 7.4 apresenta os valores obtidos.

Apenas a versão paralela com `reduction` reproduziu a contagem sequencial em todos os casos:

Tabela 7.4: Resultados funcionais das versões de escopo de dados.

N	Seq.	Private	FirstPriv.	LastPriv.	First+Last	DefaultNone	Reduction
10 000	1 229	0	0	0	279	279	1 229
100 000	9 592	0	0	0	2 199	2 199	9 592
1 000 000	78 498	0	0	0	18 260	18 260	78 498
5 000 000	348 513	0	0	0	81 795	81 795	348 513
10 000 000	664 579	0	0	0	156 318	156 318	664 579

$$C_{\text{reduction}}(N) = C_{\text{seq}}(N).$$

As demais cláusulas não implementam a agregação global necessária.

7.11.4 Validação das estratégias de escalonamento

Depois da seleção de **reduction**, todas as políticas de escalonamento produziram exatamente o mesmo resultado correto.

Tabela 7.5: Validação funcional das políticas de escalonamento.

N	Primos	Static	Dynamic	Guided	Auto	Runtime
10 000	1 229	1 229	1 229	1 229	1 229	1 229
100 000	9 592	9 592	9 592	9 592	9 592	9 592
1 000 000	78 498	78 498	78 498	78 498	78 498	78 498
5 000 000	348 513	348 513	348 513	348 513	348 513	348 513
10 000 000	664 579	664 579	664 579	664 579	664 579	664 579

Portanto,

$$C_{\text{static}} = C_{\text{dynamic}} = C_{\text{guided}} = C_{\text{auto}} = C_{\text{runtime}} = C_{\text{seq}}.$$

Isso permite realizar a comparação temporal entre os escalonadores sem confundir diferenças de desempenho com erros funcionais.

7.12 Resultados experimentais

7.12.1 Tempos das versões de escopo de dados

Os tempos médios obtidos no primeiro experimento são apresentados na Tabela 7.6.

Tabela 7.6: Tempos médios das versões de escopo de dados, em milissegundos.

N	Reps	Seq.	Private	FirstPriv.	LastPriv.	First+Last	DefaultNone	Reduction
10 000	1024	0,266	0,179	0,183	0,147	0,169	0,147	0,148
100 000	64	6,023	2,491	2,480	2,414	2,416	2,452	2,403
1 000 000	2	138,900	55,325	56,550	56,075	55,550	55,625	55,975
5 000 000	1	1 291,300	506,200	506,450	516,800	512,950	521,150	526,800
10 000 000	1	3 152,100	1 183,500	1 187,300	1 176,650	1 182,150	1 181,500	1 175,150

É importante destacar que os menores tempos das versões incorretas não podem ser utilizados para classificá-las como melhores implementações.

Uma otimização somente é válida quando preserva a semântica do programa.

7.12.2 Speedup da versão correta

Considerando somente as duas implementações funcionalmente corretas relevantes para essa comparação, o speedup da versão com redução em relação à sequencial foi:

$$S = \frac{T_{\text{seq}}}{T_{\text{reduction}}}.$$

Os valores calculados são apresentados na Tabela 7.7.

Tabela 7.7: Speedup da versão `reduction + static` em relação à sequencial.

N	Seq. (ms)	Reduction (ms)	Speedup
10 000	0,266	0,148	1,79×
100 000	6,023	2,403	2,51×
1 000 000	138,900	55,975	2,48×
5 000 000	1 291,300	526,800	2,45×
10 000 000	3 152,100	1 175,150	2,68×

A versão paralela correta foi mais rápida que a sequencial em todos os tamanhos avaliados.

7.12.3 Tempos das estratégias de escalonamento

Os resultados do segundo experimento são apresentados na Tabela 7.8.

Tabela 7.8: Tempos médios das políticas de escalonamento, em milissegundos.

N	Reps	Static	Dynamic	Guided	Auto	Runtime
10 000	1024	0,124	0,301	0,100	0,118	0,431
100 000	64	1,952	3,118	1,630	2,050	4,295
1 000 000	2	45,975	47,875	36,225	45,325	57,125
5 000 000	1	453,600	412,000	364,250	455,550	441,750
10 000 000	1	1 358,400	1 169,500	1 079,550	1 333,000	1 236,650

A política `guided` apresentou o menor tempo médio em todos os tamanhos avaliados.

7.13 Visualização dos resultados

Os gráficos desta seção são produzidos diretamente a partir dos valores obtidos na execução final do experimento.

7.13.1 Comparação das cláusulas de escopo

A Figura 7.8 apresenta os tempos do primeiro experimento.

As escalas logarítmicas facilitam a visualização simultânea das cargas pequenas e grandes.

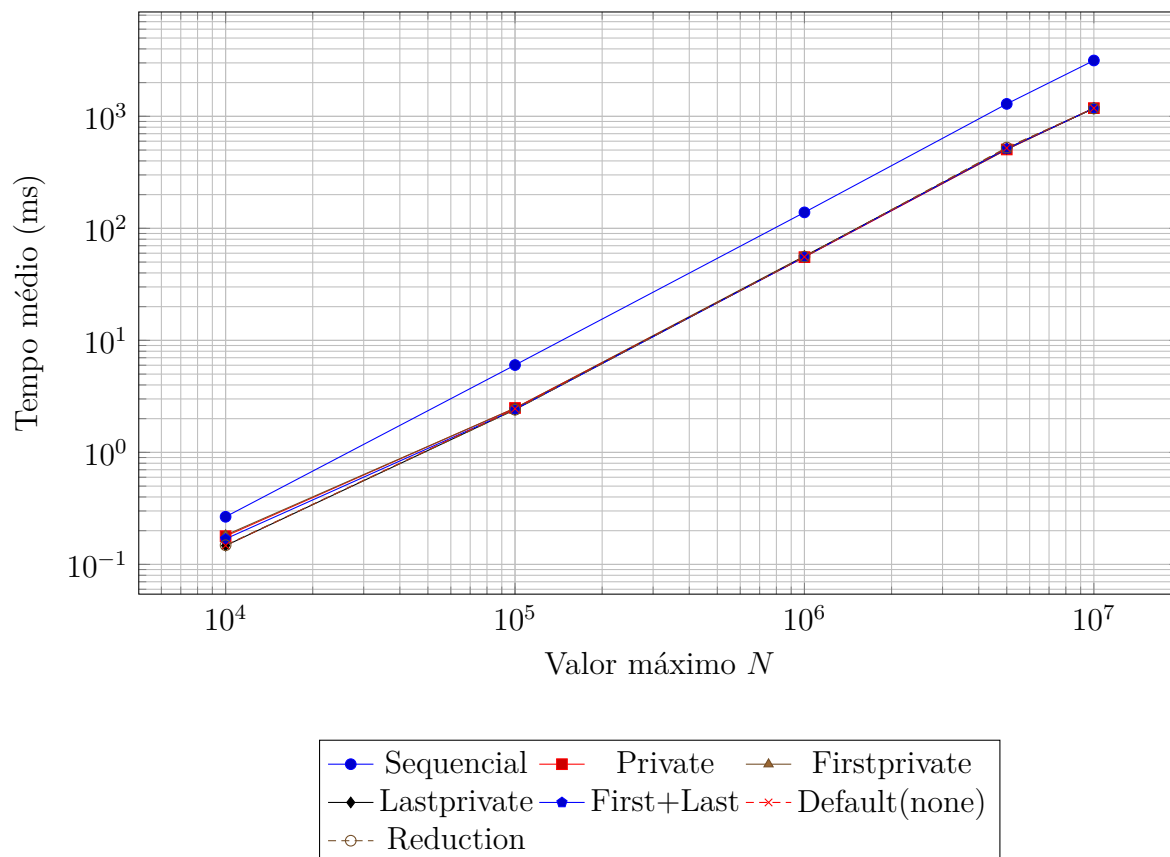


Figura 7.8: Tempo médio das versões de escopo de dados. As versões funcionalmente incorretas são exibidas apenas para caracterização experimental e não constituem soluções válidas para a contagem global.

7.13.2 Comparação das políticas de escalonamento

A Figura 7.9 apresenta a comparação das cinco políticas utilizando `reduction`.

7.13.3 Speedup da redução em relação à execução sequencial

A Figura 7.10 apresenta a aceleração obtida pela implementação correta com `reduction` + `static` no primeiro experimento.

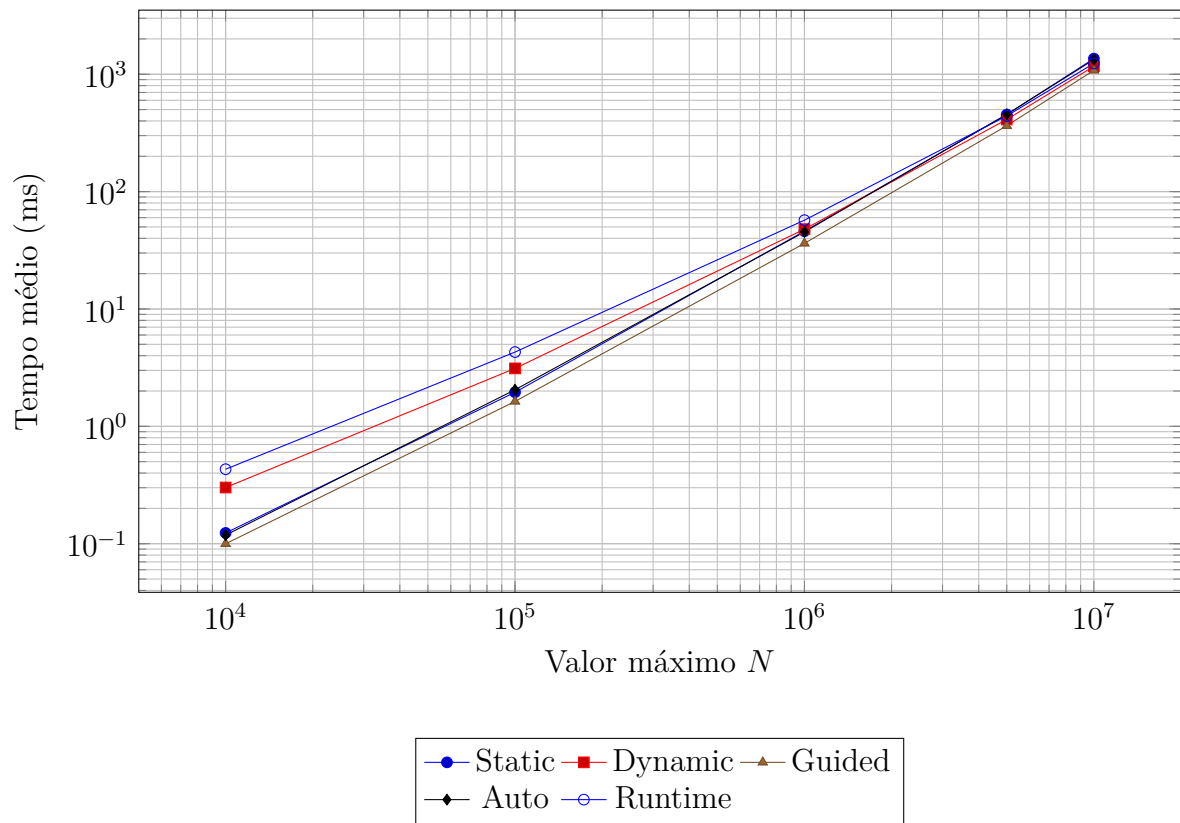


Figura 7.9: Tempo médio das políticas de escalonamento utilizando `reduction`.

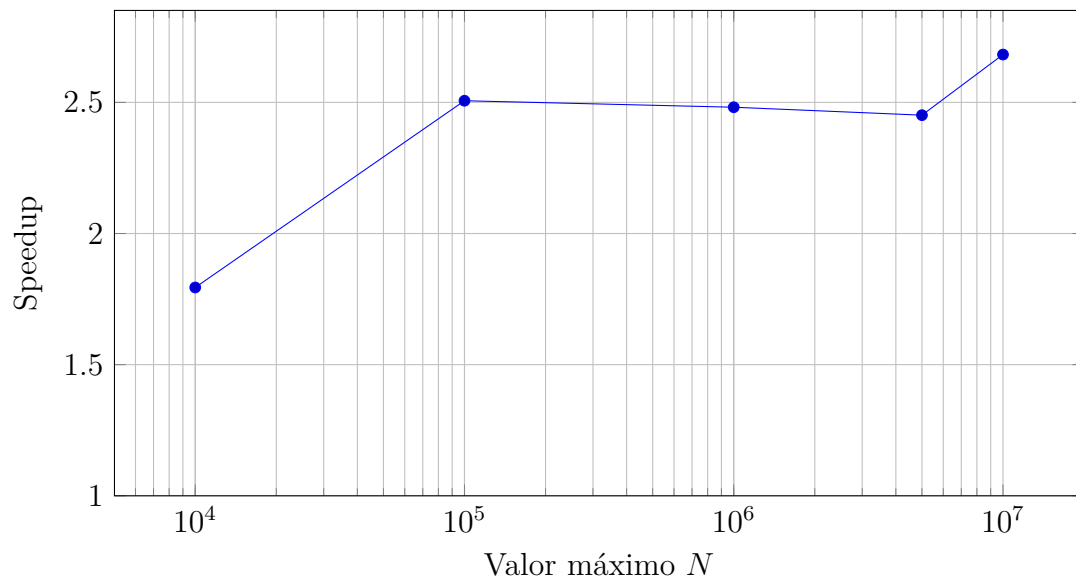


Figura 7.10: Speedup da implementação `reduction + static` em relação à versão sequencial no experimento de escopo de dados.

7.14 Análise e discussão

7.14.1 Escopo de dados não é equivalente a sincronização

O primeiro experimento demonstrou que as cláusulas de escopo possuem semânticas distintas e não podem ser utilizadas indistintamente para resolver uma operação de acumulação global.

`private`, `firstprivate` e `lastprivate` tratam da existência, inicialização ou transferência das cópias das variáveis.

Nenhuma delas, isoladamente, representa a soma das contribuições das threads.

7.14.2 Comportamento de `private`

A versão `private` retornou zero para todos os valores de N .

Isso não significa que as threads deixaram de realizar o trabalho.

Cada thread produziu sua própria contagem, mas as cópias privadas não foram combinadas.

O contador externo permaneceu com o valor inicial:

$$count = 0.$$

Portanto,

`private` fornece isolamento, não agregação

7.14.3 Comportamento de `firstprivate`

A versão `firstprivate` também retornou zero.

Cada cópia privada foi corretamente inicializada com o valor externo, também igual a zero.

Entretanto, as cópias foram descartadas ao final da região paralela.

Assim,

`firstprivate` fornece inicialização privada, não agregação

7.14.4 Comportamento de `lastprivate`

A versão `lastprivate` retornou zero para todas as entradas.

Na implementação utilizada, `count` é reinicializado em cada iteração antes da classificação do candidato.

Consequentemente, o valor propagado para fora da região corresponde à classificação da última iteração lógica.

Todos os valores escolhidos para N são pares:

$$10^4, 10^5, 10^6, 5 \times 10^6, 10^7.$$

Logo, o último candidato de cada laço não é primo e o valor transferido foi

$$0.$$

O resultado evidencia que `lastprivate` não realiza uma redução.

7.14.5 Comportamento de `firstprivate` + `lastprivate`

Os resultados observados foram:

$$279, 2199, 18260, 81795, 156318.$$

Esses valores são inferiores à contagem global.

Com `schedule(static)`, as threads recebem blocos do espaço de iterações.

A cópia privada associada à thread que executa a última iteração contém a contagem acumulada naquela porção de trabalho.

`lastprivate` propaga essa cópia para a variável externa.

O resultado é, portanto, parcial e não representa:

$$\sum_i C_i.$$

7.14.6 Comportamento de `default(none)`

A versão `default(none)` produziu nesta execução os mesmos valores numéricos observados em `firstprivate` + `lastprivate`.

Essa coincidência não indica equivalência semântica.

Na implementação com `default(none)`, `count` permanece compartilhado e é atualizado simultaneamente pelas threads sem proteção.

Existe, portanto, uma condição de corrida.

Os valores obtidos devem ser interpretados apenas como o resultado particular dessa execução.

Não existe garantia de que outra execução produza os mesmos números.

O resultado demonstra um ponto importante:

`default(none)` torna o escopo explícito, mas não garante sincronização

7.14.7 Correção de reduction

A implementação com redução reproduziu exatamente a referência sequencial nos cinco valores avaliados.

Assim,

$$C_{\text{reduction}}(N) = C_{\text{seq}}(N).$$

A redução corresponde diretamente à operação necessária:

$$C = C_0 + C_1 + C_2 + C_3.$$

Por esse motivo, ela foi selecionada como base correta do segundo experimento.

7.14.8 Desempenho da versão com redução

No primeiro experimento, a versão correta com redução apresentou speedups entre aproximadamente

$$1,79\times$$

e

$$2,68 \times .$$

Para o maior problema:

$$N = 10\,000\,000,$$

os tempos foram:

$$T_{\text{seq}} = 3152,100 \text{ ms},$$

e

$$T_{\text{reduction}} = 1175,150 \text{ ms}.$$

O speedup correspondente foi:

$$S = \frac{3152,100}{1175,150} \approx 2,68.$$

A utilização de quatro threads não implica um speedup de $4\times$, pois existem custos de criação e gerenciamento da região paralela, sincronização, redução e compartilhamento dos recursos físicos da máquina.

7.14.9 Correção das políticas de escalonamento

Todas as cinco políticas produziram resultados numericamente idênticos à referência sequencial.

Isso mostra que, uma vez resolvido corretamente o problema de agregação com **reduction**, alterar a política de escalonamento modifica a forma de distribuição do trabalho, mas não a semântica da computação.

7.14.10 **guided** como melhor política observada

A principal tendência do segundo experimento foi particularmente clara: **guided** apresentou o menor tempo médio em todos os valores de N .

Comparando com **static**, a redução aproximada do tempo foi:

N	Redução do tempo de guided vs. static
10 000	19,2%
100 000	16,5%
1 000 000	21,2%
5 000 000	19,7%
10 000 000	20,5%

A vantagem permaneceu aproximadamente entre

16,5%

e

21,2%.

Esse comportamento é compatível com a irregularidade do problema.

A estratégia **guided** começa distribuindo blocos maiores, reduzindo o custo de escalonamento, e progressivamente diminui esses blocos, melhorando o balanceamento das partes restantes.

7.14.11 Comportamento de **dynamic**

Para os menores valores de N , **dynamic** apresentou custo superior a **static**.

Para $N = 10\,000$:

$$T_{\text{static}} = 0,124 \text{ ms},$$

enquanto:

$$T_{\text{dynamic}} = 0,301 \text{ ms.}$$

Quando a carga é pequena, o gerenciamento dinâmico representa uma fração significativa do tempo.

Entretanto, para cargas maiores o comportamento se inverteu.

Para

$$N = 5\,000\,000,$$

foram observados:

$$T_{\text{static}} = 453,600 \text{ ms,}$$

e

$$T_{\text{dynamic}} = 412,000 \text{ ms.}$$

Para

$$N = 10\,000\,000,$$

foram observados:

$$T_{\text{static}} = 1358,400 \text{ ms,}$$

e

$$T_{\text{dynamic}} = 1169,500 \text{ ms.}$$

Assim, o ganho decorrente de melhor balanceamento passou a compensar o overhead adicional à medida que a carga cresceu.

7.14.12 Comportamento de **static**

A política estática apresentou bom desempenho principalmente para as cargas menores.

Seu principal benefício é o baixo custo de distribuição.

O problema, entretanto, apresenta iterações com custos diferentes.

Um bloco com maior quantidade de candidatos computacionalmente caros pode fazer com que uma thread continue trabalhando depois que outras já tenham terminado.

Esse desequilíbrio ajuda a explicar por que **guided** passou a apresentar vantagem consistente.

7.14.13 Comportamento de auto

Os tempos de **auto** permaneceram próximos aos observados para **static**:

N	T_{static}	T_{auto}
10^4	0,124	0,118
10^5	1,952	2,050
10^6	45,975	45,325
5×10^6	453,600	455,550
10^7	1358,400	1333,000

Esses resultados mostram um perfil temporal semelhante.

Entretanto, os tempos não permitem afirmar qual política interna foi selecionada pela implementação OpenMP.

A conclusão possível é apenas que **auto** apresentou, neste ambiente, desempenho próximo ao observado para **static**.

7.14.14 Comportamento de runtime

A política **runtime** foi configurada como:

dynamic,1.

Para cargas pequenas, apresentou o maior tempo entre as estratégias avaliadas.

Para

$N = 10\,000$,

o resultado foi:

0,431 ms.

Para os maiores tamanhos, a diferença relativa foi reduzida.

Em

$N = 10\,000\,000$,

o tempo foi:

1236,650 ms.

Esse valor foi inferior a **static** e **auto**, mas superior a **dynamic** e **guided**.

O resultado evidencia que a flexibilidade de selecionar a política em tempo de execução também pode envolver custos adicionais e que o desempenho observado deve ser analisado

junto à configuração efetivamente utilizada.

7.14.15 Diferença entre as duas medições de `static`

A versão `reduction` do experimento de escopo utiliza `schedule(static)`, e a política `static` aparece novamente no segundo experimento.

Os tempos não foram idênticos.

Por exemplo, para

$$N = 10\,000\,000,$$

foram obtidos:

$$1175,150\text{ ms}$$

no primeiro bloco e

$$1358,400\text{ ms}$$

no segundo.

Isso não caracteriza uma inconsistência funcional.

Os dois valores resultam de blocos de benchmark realizados em momentos diferentes durante a mesma execução.

Fatores como estado térmico, frequência da CPU, atividade do sistema operacional e histórico de execução podem afetar tempos absolutos.

Por esse motivo, as comparações principais foram realizadas **dentro de cada experimento**, e não misturando valores pertencentes a blocos diferentes.

7.15 Limitações

7.15.1 Execução em uma única máquina

Os resultados representam o ambiente utilizado na atividade.

Processadores com outras arquiteturas, quantidades de núcleos, hierarquias de cache ou runtimes OpenMP podem apresentar relações diferentes entre as estratégias.

7.15.2 Quantidade fixa de threads

Foram utilizadas quatro threads em todas as medições.

Portanto, os resultados não constituem um estudo de escalabilidade em função da quantidade de threads.

7.15.3 Ausência de afinidade explícita

Não foi implementado controle explícito de afinidade das threads.

O posicionamento nos processadores lógicos ficou sob responsabilidade do runtime e do sistema operacional.

7.15.4 Sensibilidade da média a outliers

A média aritmética é sensível a valores extremos.

Durante uma execução anterior, interrompida em condições inadequadas de energia, foi observada uma medição anômala de grande magnitude.

Essa execução foi descartada e não faz parte dos resultados apresentados neste capítulo.

A coleta final foi realizada integralmente e não apresentou aquela anomalia.

Ainda assim, a escolha da média, necessária para acompanhar o objetivo da atividade, continua mais sensível a interferências externas do que a mediana utilizada na Atividade 5.

7.15.5 Versão `default(none)` com condição de corrida

A implementação `default(none)` contém deliberadamente uma atualização concorrente não sincronizada.

Seus valores servem apenas para demonstrar a diferença entre escopo explícito e sincronização.

Eles não devem ser interpretados como resultados numéricos reproduzíveis.

7.15.6 Versões funcionalmente incorretas

Os tempos de `private`, `firstprivate`, `lastprivate`, `firstprivate + lastprivate` e `default(none)` foram medidos para caracterizar o experimento.

Entretanto, comparar esses tempos diretamente com `reduction` como se fossem soluções concorrentes seria inadequado, pois não calculam a mesma resposta global.

7.15.7 Trabalho auxiliar das versões privadas

As versões `private` e `firstprivate` utilizam uma estrutura auxiliar para manter observáveis as contagens privadas.

Esse mecanismo impede que a computação seja eliminada pelo compilador, mas introduz pequeno custo adicional que faz parte de seus tempos medidos.

7.15.8 Calibração comum

O número de repetições internas foi definido a partir da versão sequencial e da versão `reduction + static`.

A mesma quantidade foi reutilizada em todas as estratégias.

Assim, uma configuração significativamente mais rápida pode produzir um lote inferior ao alvo nominal de 100 ms.

Essa decisão foi adotada para utilizar a mesma quantidade de repetições nas versões comparadas.

7.15.9 Tamanho dos blocos

Não foi realizada uma exploração sistemática de diferentes *chunk sizes*.

Consequentemente, as conclusões dizem respeito às políticas avaliadas com as configurações utilizadas, e não à totalidade das combinações possíveis de política e tamanho de bloco.

7.15.10 `runtime` não constitui uma política independente

`schedule(runtime)` obtém sua configuração em tempo de execução.

Nesta atividade foi utilizado:

`dynamic,1.`

Portanto, os resultados da coluna `runtime` devem ser interpretados como o comportamento da implementação utilizando essa configuração por meio do mecanismo `runtime`, e não como um sexto algoritmo de escalonamento independente.

7.15.11 Ausência de medidas de dispersão

Foram realizadas vinte medições e reportadas suas médias.

O programa não exportou desvio-padrão, intervalo de confiança ou outras medidas de dispersão.

A análise está, portanto, baseada nos tempos médios observados.

7.16 Conclusões da atividade

A atividade demonstrou que mecanismos de escopo de dados e mecanismos de sincronização possuem responsabilidades distintas em OpenMP.

As cláusulas `private`, `firstprivate` e `lastprivate` alteram a forma como uma variável existe ou é transferida dentro de uma região paralela, mas não realizam automaticamente a agregação necessária à contagem global de números primos.

Da mesma forma, `default(none)` torna a classificação das variáveis explícita, mas não elimina uma condição de corrida quando uma variável compartilhada continua sendo atualizada sem sincronização.

A cláusula

`reduction(+:count)`

foi a implementação paralela do primeiro experimento que reproduziu corretamente a referência sequencial em todos os tamanhos.

Os speedups observados para essa versão em relação à sequencial variaram aproximadamente entre

$$1,79\times$$

e

$$2,68 \times .$$

Depois de estabelecida uma implementação funcionalmente correta, a comparação das políticas de escalonamento mostrou que todas preservaram a contagem correta, mas apresentaram diferenças significativas de desempenho.

A política **guided** apresentou o menor tempo médio para todos os tamanhos avaliados. Sua vantagem sobre **static** ficou aproximadamente entre

$$16,5\%$$

e

$$21,2\%.$$

O resultado é compatível com a natureza irregular do teste de primalidade: a política guiada consegue combinar blocos inicialmente maiores com redistribuição progressivamente mais fina do trabalho.

dynamic mostrou claramente o trade-off entre overhead e balanceamento: apresentou desempenho inferior a **static** nas cargas menores, mas passou a superá-lo nos dois maiores tamanhos.

A atividade evidencia, portanto, dois princípios importantes de HPC:

primeiro garantir correção, depois otimizar desempenho

e

a forma como o trabalho é distribuído pode ser tão importante quanto a criação das threads

7.17 Síntese

Os principais pontos para defesa oral da atividade são:

1. **Continuidade da atividade anterior:** a Tarefa 7 reutiliza a contagem de primos da Tarefa 5 e mantém o mesmo algoritmo de primalidade.
2. **Dois experimentos:** primeiro foram estudadas cláusulas de escopo de dados; depois foram estudadas políticas de escalonamento.
3. **Variável crítica:** o problema central é o contador global de números primos.
4. **private:** cada thread possui sua própria cópia, mas as cópias não são somadas. Por isso o contador externo permaneceu zero.
5. **firstprivate:** as cópias começam com o valor inicial correto, mas continuam sendo descartadas ao final. O resultado também foi zero.
6. **lastprivate:** apenas o valor da última iteração lógica é propagado. Como todos os valores de N eram pares, o resultado observado foi zero.
7. **firstprivate + lastprivate:** houve inicialização das cópias e propagação de uma delas, mas não soma global. Foram obtidos apenas resultados parciais.
8. **default(none):** obriga a declaração explícita do escopo, mas não sincroniza a variável. O contador compartilhado continuou sujeito a uma data race.
9. **Coincidência de resultados:** `default(none)` produziu nesta execução os mesmos valores de `firstprivate + lastprivate`, mas isso não significa equivalência entre as duas construções.
10. **reduction:** foi a implementação paralela que agregou corretamente as contagens locais e reproduziu a referência sequencial em todos os casos.
11. **Critério de escolha:** uma versão incorreta não pode ser considerada melhor apenas por ser mais rápida. A correção foi verificada antes do desempenho.
12. **Speedup da redução:** em relação à versão sequencial, foram observados aproximadamente $1,79\times$, $2,51\times$, $2,48\times$, $2,45\times$ e $2,68\times$.
13. **Segundo experimento:** todas as versões de scheduling utilizaram `reduction`, para que somente a distribuição do trabalho fosse alterada.
14. **Schedules avaliados:** `static`, `dynamic`, `guided`, `auto` e `runtime`.
15. **Correção dos schedules:** todos produziram exatamente as mesmas contagens da referência sequencial.

16. **Melhor resultado:** `guided` apresentou o menor tempo médio em todos os cinco tamanhos testados.
17. **Vantagem de `guided`:** em relação a `static`, a redução de tempo ficou aproximadamente entre 16,5% e 21,2%.
18. **Explicação para `guided`:** o teste de primalidade possui custo irregular; `guided` utiliza blocos maiores inicialmente e reduz o tamanho dos blocos progressivamente, equilibrando overhead e distribuição de carga.
19. **`dynamic`:** foi mais caro para problemas pequenos, mas superou `static` para $N = 5 \times 10^6$ e $N = 10^7$, quando o benefício do balanceamento passou a compensar seu overhead.
20. **`auto`:** apresentou tempos próximos aos de `static`, mas os resultados não permitem afirmar qual estratégia interna foi selecionada.
21. **`runtime`:** foi configurado explicitamente como `dynamic,1`; portanto seus resultados devem ser interpretados considerando essa configuração.
22. **Metodologia:** foram utilizadas quatro threads, três warm-ups, vinte medições, rotação da ordem das versões e média aritmética.
23. **Temporização:** foi utilizado `omp_get_wtime()` e repetição interna calibrada para melhorar a medição das cargas pequenas.
24. **Conclusão principal:** escopo, sincronização e escalonamento resolvem problemas diferentes. Uma implementação paralela correta exige primeiro controlar a semântica dos dados e somente depois otimizar a distribuição do trabalho.

Capítulo 8

Atividade 8 – Paralelismo de tarefas, paralelismo de dados e regiões paralelas aninhadas

8.1 Enunciado da tarefa

Implemente um código em OpenMP que realize as seguintes tarefas:

1. inicialize um vetor que representa uma função qualquer, gerando os valores necessários para o seu processamento;
2. calcule a integral desse vetor utilizando o método do trapézio;
3. calcule o vetor que representa a derivada do vetor utilizando o método de diferenças finitas.

As tarefas que não possuem dependência entre si devem ser executadas em paralelo utilizando a diretiva `sections`. Cada tarefa deve utilizar pelo menos duas threads para realizar seu processamento. Após a conclusão de cada tarefa, o resultado correspondente deve ser impresso por apenas uma thread.

Ao final, devem ser discutidas as funcionalidades do OpenMP utilizadas na implementação, considerando:

- como foi explorado o paralelismo de tarefas;
- como foi explorado o paralelismo de dados, distribuindo o processamento entre múltiplas threads;
- quais recursos do OpenMP foram utilizados para permitir o aninhamento de regiões paralelas;

- quais diretivas ou mecanismos foram utilizados para garantir que determinadas operações, como a impressão dos resultados, fossem executadas por apenas uma thread;
- quais mecanismos de sincronização estão presentes no código e qual é a função de cada um.

8.2 Objetivos e requisitos

A atividade tem como objetivo principal integrar, em uma única implementação, dois níveis distintos de paralelismo em OpenMP. No nível externo, operações independentes devem ser tratadas como tarefas distintas e executadas concorrentemente. No nível interno, o processamento de cada tarefa deve ser distribuído entre múltiplas threads.

A solução desenvolvida foi orientada pelos seguintes requisitos:

- representar numericamente uma função por meio de um vetor;
- realizar a inicialização desse vetor utilizando duas threads;
- calcular uma aproximação numérica da integral pelo método composto dos trapézios;
- calcular uma aproximação da derivada por diferenças finitas;
- executar integral e derivada como tarefas independentes utilizando **sections**;
- empregar pelo menos duas threads no processamento interno de cada tarefa;
- habilitar explicitamente o paralelismo aninhado;
- utilizar um mecanismo seguro para a acumulação da integral;
- garantir que a apresentação final dos resultados seja realizada por apenas uma thread;
- validar numericamente os resultados por comparação com soluções analíticas conhecidas;
- tornar visíveis, no terminal, tanto as threads externas responsáveis pelas **sections** quanto as equipes internas utilizadas por cada tarefa.

A atividade não foi tratada como um estudo de desempenho. Não foram definidos speedup, eficiência, escalabilidade ou comparação temporal entre versões sequenciais e paralelas. O foco foi a demonstração funcional das construções de OpenMP solicitadas no enunciado.

8.3 Fundamentação conceitual

8.3.1 Particionamento de tarefas e de dados

A paralelização de um problema pode ocorrer em níveis diferentes. No *paralelismo de tarefas*, operações funcionalmente distintas e sem dependências entre si são executadas simultaneamente. No *paralelismo de dados*, uma mesma operação é aplicada simultaneamente a diferentes partições de um conjunto de dados.

Nesta atividade, os dois modelos aparecem em conjunto. Uma vez concluída a inicialização do vetor, o cálculo da integral não depende do resultado da derivada, e o cálculo da derivada não depende do resultado da integral. Portanto, essas duas operações constituem tarefas independentes.

Pode-se representar essa dependência por

$$\text{Inicialização} \longrightarrow \begin{cases} \text{Integral,} \\ \text{Derivada.} \end{cases}$$

A inicialização precisa necessariamente ocorrer primeiro, pois ambas as tarefas subsequentes utilizam os valores armazenados no vetor da função.

No interior de cada tarefa, entretanto, existem muitas operações independentes sobre diferentes elementos do vetor. Essas operações podem ser distribuídas entre múltiplas threads, caracterizando paralelismo de dados.

8.3.2 Representação discreta da função

Foi escolhida a função

$$f(x) = x^2$$

no intervalo

$$[a, b] = [0, 10].$$

O domínio foi discretizado em

$$N = 100000$$

pontos igualmente espaçados.

O espaçamento é dado por

$$h = \frac{b - a}{N - 1},$$

de modo que

$$h = \frac{10}{99999} \approx 0,00010000100001.$$

Para cada posição i do vetor,

$$x_i = a + ih$$

e

$$y_i = f(x_i) = x_i^2.$$

A escolha de $f(x) = x^2$ foi deliberadamente simples, pois permite validar tanto a integral quanto a derivada contra expressões analíticas conhecidas, sem introduzir complexidade matemática desnecessária ao objetivo principal da atividade.

8.3.3 Método composto dos trapézios

Para uma função discretizada em pontos igualmente espaçados, o método composto dos trapézios fornece

$$I_T = h \left[\frac{y_0}{2} + \sum_{i=1}^{N-2} y_i + \frac{y_{N-1}}{2} \right].$$

O termo

$$\sum_{i=1}^{N-2} y_i$$

pode ser particionado entre diferentes threads. Entretanto, como todas as threads contribuem para o mesmo acumulador lógico, uma atualização direta de uma variável compartilhada produziria uma condição de corrida.

A construção de redução do OpenMP resolve esse problema criando acumuladores privados e combinando-os ao final da região de trabalho. Por isso, foi utilizada a cláusula

`reduction(+:sum).`

Para a função adotada, a solução analítica é

$$I = \int_0^{10} x^2 dx = \left[\frac{x^3}{3} \right]_0^{10} = \frac{1000}{3},$$

portanto

$$I \approx 333,333333333333.$$

8.3.4 Derivação por diferenças finitas

Para os pontos internos do domínio foi utilizada a diferença central

$$f'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2h}, \quad 1 \leq i \leq N - 2.$$

Como esse esquema necessita de elementos dos dois lados do ponto avaliado, ele não pode ser aplicado diretamente às extremidades.

No primeiro ponto foi utilizada a diferença progressiva

$$f'(x_0) \approx \frac{y_1 - y_0}{h},$$

enquanto no último ponto foi utilizada a diferença regressiva

$$f'(x_{N-1}) \approx \frac{y_{N-1} - y_{N-2}}{h}.$$

Para

$$f(x) = x^2,$$

a derivada analítica é

$$f'(x) = 2x.$$

Além de facilitar a validação, a função quadrática possui uma característica útil: no interior do domínio, a diferença central recupera exatamente $2x$ em aritmética exata, pois

$$\frac{(x+h)^2 - (x-h)^2}{2h} = \frac{4xh}{2h} = 2x.$$

8.3.5 Diretiva `sections`

A construção `sections` é apropriada quando diferentes blocos de código podem executar independentemente. Cada bloco delimitado por `section` é atribuído a uma thread da equipe externa.

Nesta atividade foram definidas duas seções:

`section1` $\longrightarrow$ integral

e

`section2` $\longrightarrow$ derivada.

Não foi estabelecida uma associação fixa entre uma determinada thread e uma determinada seção. Essa decisão é deixada ao runtime do OpenMP.

8.3.6 Paralelismo aninhado

Uma **section** é executada por uma única thread da região que contém a construção **sections**. Entretanto, o enunciado exige que cada tarefa utilize pelo menos duas threads em seu processamento.

Por esse motivo, cada **section** cria uma nova região paralela interna. A estrutura lógica passa a ser

região paralela externa $\rightarrow$ **sections** $\rightarrow$ região paralela interna.

Consequentemente, são explorados dois níveis distintos:

nível externo: paralelismo de tarefas

e

nível interno: paralelismo de dados.

O código habilita explicitamente o paralelismo aninhado por meio de

```
omp_set_nested(1).
```

Também é utilizado

```
omp_set_dynamic(0),
```

evitando que o runtime reduza dinamicamente o número de threads solicitado durante esta demonstração.

8.3.7 Sincronização

Não foi necessário inserir uma diretiva **barrier** explícita. A sincronização da solução é obtida principalmente por pontos de sincronização implícitos das construções OpenMP utilizadas.

Ao final da região paralela responsável pela inicialização, todas as threads foram concluídas antes que o programa avance para as tarefas que consomem o vetor.

No interior das tarefas, o término das regiões paralelas internas garante que o processamento distribuído tenha sido concluído antes da continuação da thread que criou aquela região.

A construção **sections**, sem a cláusula **nowait**, possui uma barreira implícita ao final. Assim, quando a execução alcança a etapa de impressão, tanto a integral quanto a derivada já foram finalizadas.

Finalmente, a diretiva **single** garante que apenas uma thread da equipe externa execute o bloco responsável pela apresentação consolidada dos resultados.

8.3.8 `single`, `master` e a escolha adotada

A diretiva `single` permite que uma única thread qualquer da equipe execute determinado bloco. Já a construção `master` restringe a execução especificamente à thread mestre.

Como o requisito da atividade exige apenas que a impressão seja feita por uma única thread, e não necessariamente pela thread de identificador zero, foi adotado `single`.

Além disso, essa decisão evita impor uma associação artificial entre “impressão” e “thread 0”, preservando a semântica de que qualquer thread da equipe pode assumir a responsabilidade pela operação serial.

8.4 Interpretação didática da tarefa

A atividade não consiste apenas em calcular uma integral e uma derivada numericamente. Esses cálculos funcionam como um problema simples sobre o qual podem ser demonstrados diferentes mecanismos de paralelização.

A inicialização mostra uma decomposição de dados elementar: diferentes threads escrevem em índices distintos do mesmo vetor.

Em seguida, integral e derivada representam duas funções computacionais diferentes que podem ser executadas simultaneamente. Essa etapa demonstra paralelismo de tarefas.

Cada uma dessas tarefas contém, por sua vez, quantidade suficiente de trabalho independente para ser novamente paralelizada. A integral distribui os termos do somatório entre as threads internas, enquanto a derivada distribui os índices do vetor de saída.

Assim, a atividade constrói a seguinte hierarquia conceitual:

paralelismo de tarefas + paralelismo de dados + paralelismo aninhado.

A impressão dos identificadores das equipes no terminal foi incluída para tornar essa hierarquia diretamente observável durante a execução, sem transformar a saída em um log de todas as iterações.

8.5 Modelagem da solução

8.5.1 Estrutura geral

Foram utilizados dois vetores de valores do tipo `double`:

$$y[N]$$

para armazenar a função discretizada, e

$$dy[N]$$

para armazenar a derivada numérica.

A configuração consolidada foi:

Tabela 8.1: Configuração adotada na Atividade 8.

Parâmetro	Configuração
Função	$f(x) = x^2$
Intervalo	$[0, 10]$
Número de pontos	100000
Tipo numérico	double
Threads na inicialização	2
Threads na região de sections	2
Threads internas por tarefa	2
Integral	método composto dos trapézios
Derivada interna	diferença central
Bordas da derivada	diferenças progressiva e regressiva
Acumulação da integral	reduction
Impressão consolidada	single
Paralelismo aninhado	habilitado
Medição de tempo	não realizada

8.5.2 Dependências

A dependência principal da aplicação é

inicialização do vetor $\rightarrow$ {integral, derivada}.

Integral e derivada podem começar somente depois que todos os elementos de y estiverem disponíveis.

Após essa condição ser satisfeita,

integral $\perp$ derivada,

isto é, não existe dependência de dados entre as duas operações.

Elas podem, portanto, ser processadas simultaneamente.

8.5.3 Principais funções e componentes

A implementação foi organizada em funções pequenas, cada uma responsável por uma etapa bem definida.

initialize_vector Inicializa o vetor y em uma região paralela com duas threads. Cada iteração calcula x_i e armazena x_i^2 .

compute_integral Executa o somatório interno do método dos trapézios em uma região paralela aninhada. A soma é combinada por meio de `reduction(+:sum)`.

compute_derivative Calcula os pontos internos da derivada utilizando um laço paralelo e diferença central. As duas extremidades são calculadas separadamente por diferenças unilaterais.

print_seen_threads Apresenta os identificadores das threads que participaram de determinada equipe.

print_derivative_samples Exibe cinco posições representativas da derivada em vez de imprimir os 100000 elementos do vetor.

main Configura o problema, aloca os vetores, habilita o paralelismo aninhado, inicia as etapas de processamento e realiza a apresentação consolidada dos resultados.

As estruturas `InitInfo` e `TaskInfo` foram empregadas apenas para registrar informações necessárias à demonstração didática das equipes de threads no terminal.

8.6 Decisões de projeto e trade-offs

As principais decisões adotadas são resumidas na Tabela 8.2.

A opção por `reduction`, em particular, elimina a necessidade de uma região crítica para cada termo da soma. Uma implementação baseada em `critical` preservaria a correção, mas introduziria serialização desnecessária justamente na operação que deveria explorar paralelismo.

Da mesma forma, não foi utilizado `atomic` no vetor de derivadas, porque as threads escrevem em posições distintas. Adicionar sincronização nesse caso não aumentaria a correção e apenas acrescentaria sobrecarga.

8.7 Representação estrutural da implementação

A Figura 8.1 representa os principais componentes da implementação.

A estrutura evidencia que `sections` constitui o nível de paralelismo de tarefas, enquanto as regiões criadas em `compute_integral` e `compute_derivative` constituem o nível interno de paralelismo de dados.

8.8 Fluxo de execução

O fluxo comportamental da aplicação é mostrado na Figura 8.2.

Tabela 8.2: Principais decisões de projeto da Atividade 8.

Aspecto	Escolha	Justificativa e trade-off
Função	$f(x) = x^2$	Permite validação analítica direta da integral e da derivada. A simplicidade matemática favorece o foco em OpenMP, embora não represente um problema numérico complexo.
Número de pontos	$N = 100000$	Fornece quantidade suficiente de iterações para demonstrar a divisão dos dados, mantendo baixo custo de memória.
Tipo	<code>double</code>	Adequado ao cálculo numérico da integral e da derivada e mais preciso que <code>float</code> , ao custo de maior uso de memória por elemento.
Tarefas	duas <code>sections</code>	Reflete diretamente a independência entre integral e derivada. O número de tarefas é fixo no código, característica inerente ao uso de <code>sections</code> .
Paralelismo interno	duas threads por tarefa	Satisfaz explicitamente o requisito do enunciado e mantém simples a visualização do paralelismo aninhado.
Integral	<code>reduction</code>	Evita condição de corrida sem serializar cada atualização por <code>critical</code> ou <code>atomic</code> .
Derivada	escrita por índice	Cada iteração escreve em uma posição distinta de <code>dy</code> , não sendo necessário mecanismo de exclusão mútua.
Impressão	<code>single</code>	Atende ao requisito de uma única thread sem exigir especificamente a thread mestre.
<code>nowait</code>	não utilizado	A barreira ao final de <code>sections</code> é útil porque a impressão consolidada deve ocorrer somente depois das duas tarefas.
Identificação das threads	exibida de forma resumida	Torna observável a estrutura paralela sem imprimir mensagens por iteração, o que poluiria o terminal e introduziria E/S excessiva.
Temporização	não utilizada	O enunciado não solicita benchmark. Evita interpretar uma única configuração como estudo de desempenho.

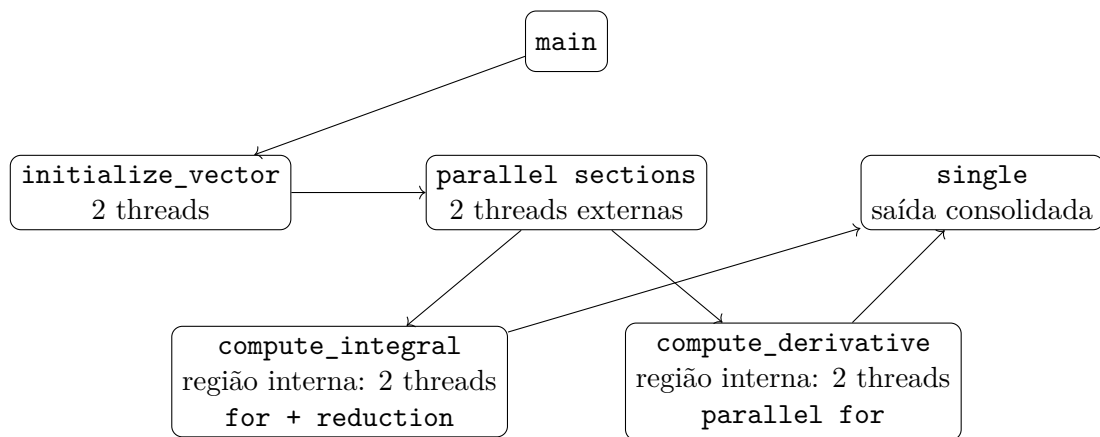


Figura 8.1: Representação estrutural da implementação da Atividade 8.

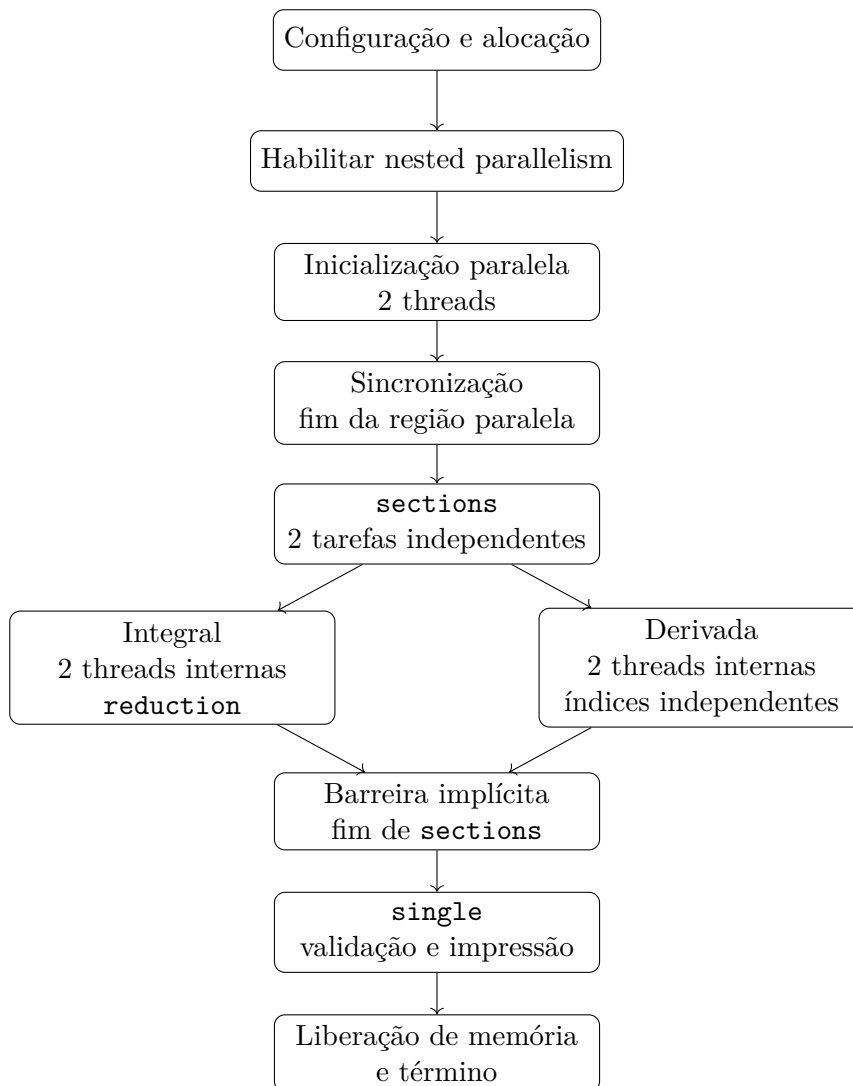


Figura 8.2: Fluxo de execução e pontos de sincronização da Atividade 8.

A sincronização existente entre a inicialização e as **sections** é fundamental: integral e derivada não podem consumir um vetor parcialmente inicializado.

A barreira implícita ao final de **sections** cumpre função diferente: ela garante que as duas tarefas independentes tenham terminado antes que o bloco **single** apresente os resultados.

8.9 Metodologia experimental

8.9.1 Ambiente de execução

A implementação foi compilada e executada no ambiente utilizado nas atividades práticas da disciplina, por meio de terminal MSYS2/UCRT64 em Windows.

A execução validada ocorreu no host

LAPTOP-AVVOLNPR

e no diretório

/d/Tecnologia da Informação (TI)/P10 2026.2/HPC/ATVD8.

O compilador utilizado foi o GCC disponível no ambiente UCRT64. A versão exata do compilador não foi registrada no output desta execução e, portanto, não é atribuída aqui.

A compilação foi realizada com suporte a OpenMP por meio de **-fopenmp**.

O comando empregado pelo script auxiliar foi:

```
1 gcc \  
2   -std=c11 \  
3   -O2 \  
4   -Wall \  
5   -Wextra \  
6   -Wpedantic \  
7   -fopenmp \  
8   tarefa8_openmp.c \  
9   -o tarefa8_openmp.exe \  
10  -lm
```

A execução foi automatizada por

./run_experiment.sh.

8.9.2 Configuração das medições

Não foram realizadas medições de tempo nesta atividade.

Consequentemente, não foram utilizados:

- aquecimentos;
- múltiplas repetições para obtenção de estatísticas;
- média ou mediana de tempos;
- speedup;
- eficiência paralela;
- comparação entre número variável de threads.

A metodologia experimental foi exclusivamente funcional e numérica. Uma execução foi utilizada para verificar:

1. se a inicialização realmente utilizava duas threads;
2. se as duas **sections** eram atribuídas às threads da equipe externa;
3. se cada tarefa criava uma equipe interna com duas threads;
4. se o paralelismo aninhado estava efetivamente habilitado;
5. se a integral correspondia ao valor analítico;
6. se a derivada correspondia à função $2x$ nos pontos avaliados.

Portanto, os resultados desta atividade não devem ser interpretados como evidência de ganho de desempenho.

8.10 Código-fonte desenvolvido

O Código 8.1 apresenta a implementação integral utilizada na execução.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <omp.h>
5
6 #define N_POINTS 100000
7 #define SECTION_THREADS 2
8 #define TASK_THREADS 2
9
10 typedef struct {
11     int outer_thread;
12     int inner_team_size;
13     int inner_seen[TASK_THREADS];
14 } TaskInfo;
15
16 typedef struct {
17     int team_size;

```

```

18     int seen[TASK_THREADS];
19 } InitInfo;
20
21 static void reset_task_info(TaskInfo *info)
22 {
23     info->outer_thread = -1;
24     info->inner_team_size = 0;
25
26     for (int i = 0; i < TASK_THREADS; ++i) {
27         info->inner_seen[i] = 0;
28     }
29 }
30
31 static void reset_init_info(InitInfo *info)
32 {
33     info->team_size = 0;
34
35     for (int i = 0; i < TASK_THREADS; ++i) {
36         info->seen[i] = 0;
37     }
38 }
39
40 /*
41  * Inicializa y[i] = f(x_i) = x_i^2 em paralelo.
42  * Cada iteracao escreve em uma posicao independente do vetor.
43  */
44 static void initialize_vector(double *y, int n, double a, double h, InitInfo *info)
45 {
46     #pragma omp parallel num_threads(TASK_THREADS) shared(y, info)
47     {
48         const int tid = omp_get_thread_num();
49
50         if (tid < TASK_THREADS) {
51             info->seen[tid] = 1;
52         }
53
54         #pragma omp single
55         {
56             info->team_size = omp_get_num_threads();
57         }
58
59         #pragma omp for
60         for (int i = 0; i < n; ++i) {
61             const double x = a + (double)i * h;
62             y[i] = x * x;
63         }
64     }
65 }
66
67 /*
68  * Calcula a integral pelo metodo composto dos trapezios.
69  * O somatorio interno e distribuido entre duas threads e combinado
70  * de forma segura por reduction.
71  */
72 static double compute_integral(
73     const double *y,
74     int n,
75     double h,
76     TaskInfo *info

```

```

77 )
78 {
79     double sum = 0.0;
80
81     #pragma omp parallel num_threads(TASK_THREADS) shared(y, info, sum)
82     {
83         const int tid = omp_get_thread_num();
84
85         if (tid < TASK_THREADS) {
86             info->inner_seen[tid] = 1;
87         }
88
89         #pragma omp single
90         {
91             info->inner_team_size = omp_get_num_threads();
92         }
93
94         #pragma omp for reduction(+:sum)
95         for (int i = 1; i < n - 1; ++i) {
96             sum += y[i];
97         }
98     }
99
100     return h * (0.5 * y[0] + sum + 0.5 * y[n - 1]);
101 }
102
103 /*
104  * Calcula a derivada numerica.
105  *
106  * Pontos internos:
107  *   diferenca central
108  *
109  * Extremidades:
110  *   diferenca progressiva no primeiro ponto
111  *   diferenca regressiva no ultimo ponto
112  */
113 static void compute_derivative(
114     const double *y,
115     double *dy,
116     int n,
117     double h,
118     TaskInfo *info
119 )
120 {
121     #pragma omp parallel num_threads(TASK_THREADS) shared(y, dy, info)
122     {
123         const int tid = omp_get_thread_num();
124
125         if (tid < TASK_THREADS) {
126             info->inner_seen[tid] = 1;
127         }
128
129         #pragma omp single
130         {
131             info->inner_team_size = omp_get_num_threads();
132         }
133
134         #pragma omp for
135         for (int i = 1; i < n - 1; ++i) {

```

```

136         dy[i] = (y[i + 1] - y[i - 1]) / (2.0 * h);
137     }
138 }
139
140 dy[0] = (y[1] - y[0]) / h;
141 dy[n - 1] = (y[n - 1] - y[n - 2]) / h;
142 }
143
144 static void print_seen_threads(const int seen[TASK_THREADS])
145 {
146     int first = 1;
147
148     for (int i = 0; i < TASK_THREADS; ++i) {
149         if (seen[i]) {
150             if (!first) {
151                 printf(", ");
152             }
153
154             printf("%d", i);
155             first = 0;
156         }
157     }
158
159     if (first) {
160         printf("nenhuma");
161     }
162
163     printf("\n");
164 }
165
166 static void print_derivative_samples(
167     const double *dy,
168     int n,
169     double a,
170     double h
171 )
172 {
173     const int indices[5] = {
174         0,
175         n / 4,
176         n / 2,
177         (3 * n) / 4,
178         n - 1
179     };
180
181     printf("%-12s %-14s %-16s %-16s\n",
182         "Indice", "x", "Numerica", "Exata");
183     printf("-----\n");
184
185     for (int k = 0; k < 5; ++k) {
186         const int i = indices[k];
187         const double x = a + (double)i * h;
188         const double exact = 2.0 * x;
189
190         printf("%-12d %-14.6f %-16.6f %-16.6f\n",
191             i, x, dy[i], exact);
192     }
193 }
194

```

```

195 int main(void)
196 {
197     const int n = N_POINTS;
198     const double a = 0.0;
199     const double b = 10.0;
200     const double h = (b - a) / (double)(n - 1);
201
202     double *y = malloc((size_t)n * sizeof(*y));
203     double *dy = malloc((size_t)n * sizeof(*dy));
204
205     if (y == NULL || dy == NULL) {
206         fprintf(stderr, "Erro: nao foi possivel alocar os vetores.\n");
207         free(y);
208         free(dy);
209         return EXIT_FAILURE;
210     }
211
212     InitInfo init_info;
213     TaskInfo integral_info;
214     TaskInfo derivative_info;
215
216     reset_init_info(&init_info);
217     reset_task_info(&integral_info);
218     reset_task_info(&derivative_info);
219
220     /*
221      * Evita que o runtime reduza dinamicamente o numero de threads e
222      * habilita regioes paralelas aninhadas, como exigido pela atividade.
223      */
224     omp_set_dynamic(0);
225     omp_set_nested(1);
226
227     printf("=====\n");
228     printf("TAREFA 8 - OPENMP\n");
229     printf("=====\n\n");
230
231     printf("CONFIGURACAO\n");
232     printf("-----\n");
233     printf("Funcao          : f(x) = x^2\n");
234     printf("Intervalo       : [%.0f, %.0f]\n", a, b);
235     printf("Pontos          : %d\n", n);
236     printf("Threads sections : %d\n", SECTION_THREADS);
237     printf("Threads por tarefa : %d\n", TASK_THREADS);
238     printf("Nested parallelism : %s\n",
239         omp_get_nested() ? "habilitado" : "desabilitado");
240
241     /*
242      * Fase 1: inicializacao do vetor.
243      * O fim da regio parallel funciona como ponto de sincronizacao:
244      * somente depois dele integral e derivada podem usar y[].
245      */
246     initialize_vector(y, n, a, h, &init_info);
247
248     printf("\n=====\n");
249     printf("INICIALIZACAO\n");
250     printf("=====\n\n");
251     printf("Inicializacao paralela:\n");
252     printf(" Equipe          : %d threads\n", init_info.team_size);
253     printf(" Threads participantes : ");

```

```

254     print_seen_threads(init_info.seen);
255     printf("\nVetor inicializado com sucesso.\n");
256
257     double integral = 0.0;
258
259     /*
260      * Fase 2: paralelismo de tarefas.
261      *
262      * Integral e derivada sao independentes depois que y[] esta pronto.
263      * Cada section e executada por uma thread externa diferente quando
264      * o runtime distribui as duas sections entre as duas threads do time.
265      *
266      * Dentro de cada section, uma nova regio parallel cria duas threads
267      * para explorar paralelismo de dados (nested parallelism).
268      */
269     #pragma omp parallel num_threads(SECTION_THREADS) \
270         shared(y, dy, integral, integral_info, derivative_info)
271     {
272         #pragma omp sections
273         {
274             #pragma omp section
275             {
276                 integral_info.outer_thread = omp_get_thread_num();
277                 integral = compute_integral(
278                     y,
279                     n,
280                     h,
281                     &integral_info
282                 );
283             }
284
285             #pragma omp section
286             {
287                 derivative_info.outer_thread = omp_get_thread_num();
288                 compute_derivative(
289                     y,
290                     dy,
291                     n,
292                     h,
293                     &derivative_info
294                 );
295             }
296         }
297
298     /*
299      * A barreira implicita ao fim de sections garante que integral
300      * e derivada terminaram antes da impressao.
301      *
302      * single garante que apenas uma thread do time externo imprima
303      * os resultados, conforme solicitado no enunciado.
304      */
305     #pragma omp single
306     {
307         const double exact_integral =
308             (b * b * b - a * a * a) / 3.0;
309         const double absolute_error =
310             fabs(integral - exact_integral);
311
312         printf("\n===== \n");

```

```

313     printf("PARALELISMO DE TAREFAS\n");
314     printf("=====\n\n");
315
316     printf("Section Integral:\n");
317     printf("  Thread externa responsavel : %d\n",
318           integral_info.outer_thread);
319     printf("  Equipe interna          : %d threads\n",
320           integral_info.inner_team_size);
321     printf("  Threads internas          : ");
322     print_seen_threads(integral_info.inner_seen);
323
324     printf("\nSection Derivada:\n");
325     printf("  Thread externa responsavel : %d\n",
326           derivative_info.outer_thread);
327     printf("  Equipe interna          : %d threads\n",
328           derivative_info.inner_team_size);
329     printf("  Threads internas          : ");
330     print_seen_threads(derivative_info.inner_seen);
331
332     printf("\n=====\n");
333     printf("INTEGRAL - METODO DOS TRAPEZIOS\n");
334     printf("=====\n\n");
335     printf("Valor numerico : %.12f\n", integral);
336     printf("Valor exato    : %.12f\n", exact_integral);
337     printf("Erro absoluto  : %.12e\n", absolute_error);
338
339     printf("\n=====\n");
340     printf("DERIVADA - DIFERENCAS FINITAS\n");
341     printf("=====\n\n");
342     print_derivative_samples(dy, n, a, h);
343
344     printf("\n=====\n");
345     printf("PROCESSAMENTO CONCLUIDO\n");
346     printf("=====\n");
347 }
348 }
349
350 free(y);
351 free(dy);
352
353 return EXIT_SUCCESS;
354 }

```

Listing 8.1: Código-fonte da Atividade 8.

A organização do código procura manter separados o cálculo numérico, o controle do paralelismo e a apresentação dos resultados.

A inicialização, a integração e a derivação foram implementadas em funções distintas, enquanto a região de `sections` permanece explícita na função principal. Dessa forma, a relação entre a modelagem conceitual e a estrutura concreta do programa permanece diretamente observável.

8.11 Validação da implementação

A validação foi realizada em dois níveis: estrutural e numérico.

8.11.1 Validação estrutural do paralelismo

A saída confirmou inicialmente:

```
1 Nested parallelism : habilitado
```

Na inicialização foram observadas duas threads:

```
1 Inicializacao paralela:
2 Equipe                  : 2 threads
3 Threads participantes   : 0, 1
```

Na região externa, a execução observada distribuiu as tarefas da seguinte forma:

```
1 Section Integral:
2 Thread externa responsavel : 1
3 Equipe interna             : 2 threads
4 Threads internas           : 0, 1
5
6 Section Derivada:
7 Thread externa responsavel : 0
8 Equipe interna             : 2 threads
9 Threads internas           : 0, 1
```

Esse resultado confirma simultaneamente:

duas tarefas externas

e

duas threads internas por tarefa.

A associação observada entre thread externa e **section** corresponde somente à execução registrada. O programa não fixa a integral na thread 1 nem a derivada na thread 0.

8.11.2 Validação da integral

O valor analítico esperado é

$$I_{\text{exato}} = \frac{1000}{3} = 333,333333333333 \dots$$

A execução produziu

$$I_{\text{num}} = 333,333333349999$$

e

$$E_{\text{abs}} = 1,666609250606 \times 10^{-8}.$$

Os valores efetivamente impressos foram:

```
1 Valor numerico : 333.333333349999
2 Valor exato    : 333.333333333333
3 Erro absoluto  : 1.666609250606e-08
```

Para uma função quadrática, o erro de discretização do método composto dos trapézios pode ser relacionado a h^2 . Como

$$f''(x) = 2,$$

a diferença teórica entre a aproximação trapezoidal e a integral exata possui magnitude

$$\frac{(b-a)h^2}{6}.$$

Com

$$h = \frac{10}{99999},$$

obtém-se aproximadamente

$$1,6667 \times 10^{-8},$$

valor consistente com o erro observado na execução.

Além disso, x^2 é convexa no intervalo utilizado. Assim, a aproximação trapezoidal é ligeiramente superior à integral analítica, exatamente como observado:

$$333,333333349999 > 333,333333333333.$$

8.11.3 Validação da derivada

A derivada analítica utilizada como referência foi

$$f'(x) = 2x.$$

Cinco pontos representativos foram impressos.

Nos pontos internos, os valores numéricos coincidiram com os valores analíticos na precisão apresentada no terminal.

Nas extremidades foi observado pequeno desvio, compatível com o uso das diferenças unilaterais de primeira ordem.

No primeiro ponto,

$$f'(0) \approx \frac{h^2}{h} = h \approx 0,000100001,$$

o que explica o valor exibido como

0,000100.

No último ponto, a aproximação regressiva para a função quadrática fornece

$$20 - h,$$

resultando aproximadamente em

19,999900.

Portanto, os desvios nas duas extremidades não representam falha do programa, mas o comportamento esperado do esquema numérico adotado.

8.12 Resultados experimentais

A execução validada produziu a seguinte configuração:

```

1 CONFIGURACAO
2 -----
3 Funcao          : f(x) = x^2
4 Intervalo       : [0, 10]
5 Pontos          : 100000
6 Threads sections : 2
7 Threads por tarefa : 2
8 Nested parallelism : habilitado

```

A distribuição das tarefas observada foi:

Tabela 8.3: Equipes de threads observadas na execução validada.

Operação	Thread externa	Equipe interna	IDs internos
Inicialização	—	2	0, 1
Integral	1	2	0, 1
Derivada	0	2	0, 1

Os resultados da integral são apresentados na Tabela 8.4.

Os cinco pontos da derivada exibidos na execução são apresentados na Tabela 8.5.

Não foram produzidos resultados de tempo de execução e, portanto, nenhuma conclusão de desempenho é associada a essas tabelas.

Tabela 8.4: Resultado numérico da integral.

Grandeza	Valor
Integral numérica	333.333333349999
Integral exata	333.333333333333
Erro absoluto	$1.666609250606 \times 10^{-8}$

Tabela 8.5: Amostras da derivada numérica obtida.

Índice	x	Numérica	Exata
0	0.000000	0.000100	0.000000
25000	2.500025	5.000050	5.000050
50000	5.000050	10.000100	10.000100
75000	7.500075	15.000150	15.000150
99999	10.000000	19.999900	20.000000

8.13 Visualização dos resultados

Nesta atividade, a visualização principal dos resultados é estrutural e tabular.

Não foi produzido gráfico de tempo, speedup ou eficiência, pois nenhuma campanha de temporização foi realizada.

A Figura 8.3 representa especificamente a distribuição de threads observada na execução validada.

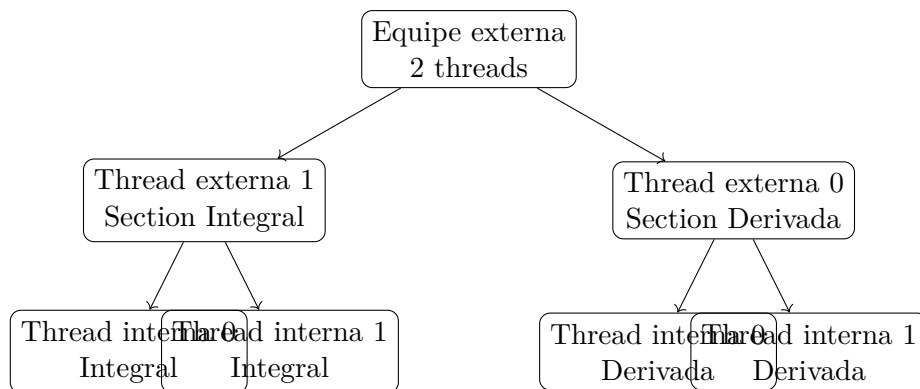


Figura 8.3: Distribuição de threads observada na execução da Atividade 8.

É importante distinguir essa figura de uma regra fixa de escalonamento. Ela documenta a execução observada; outra execução válida poderia inverter as threads externas responsáveis pelas duas **sections**.

8.14 Análise e discussão

8.14.1 Paralelismo de tarefas

O paralelismo de tarefas foi explorado após a conclusão da inicialização.

Como integral e derivada utilizam o mesmo vetor de entrada, mas não dependem dos resultados uma da outra, elas foram colocadas em duas diretivas `section` distintas.

A execução observada demonstrou:

Integral → thread externa 1

e

Derivada → thread externa 0.

Esse resultado evidencia que a atribuição é responsabilidade do runtime e não foi codificada manualmente.

8.14.2 Paralelismo de dados

O segundo nível de paralelismo aparece dentro das tarefas.

Na integral, as iterações responsáveis pelo somatório intermediário são distribuídas entre as duas threads da equipe interna.

Na derivada, diferentes índices do vetor são distribuídos entre as threads. Como cada iteração escreve em uma posição distinta de `dy`, não existe conflito de escrita entre elas.

A inicialização do vetor também utiliza paralelismo de dados, pois cada posição de `y` pode ser calculada independentemente.

8.14.3 Paralelismo aninhado

A execução confirmou que as duas regiões internas utilizaram equipes com duas threads.

Isso é importante porque uma região paralela interna poderia ser serializada caso o paralelismo aninhado não estivesse habilitado ou caso o runtime não permitisse a criação de equipes adicionais.

O output:

Equipe interna : 2 threads

para as duas tarefas fornece evidência direta de que as regiões internas foram efetivamente ativadas no ambiente testado.

8.14.4 Redução e ausência de condição de corrida

O somatório da integral representa o principal ponto em que múltiplas threads contribuem para uma mesma grandeza.

Uma implementação do tipo

`sum += y[i]`

sobre uma única variável compartilhada, sem proteção, estaria sujeita a condição de corrida.

A cláusula **reduction** evita esse problema criando estados parciais privados e combinando-os posteriormente.

Essa estratégia também é mais apropriada do que proteger cada atualização com uma seção crítica, pois uma região **critical** em cada iteração serializaria a etapa de acumulação.

8.14.5 Sincronização

A solução utiliza predominantemente sincronização implícita.

O fim da região paralela da inicialização funciona como um ponto de junção: somente depois dele o vetor pode ser consumido pelas tarefas.

As regiões paralelas internas também possuem um ponto de junção em seu término.

A construção **sections**, utilizada sem **nowait**, possui barreira implícita ao final. Esse comportamento é desejado porque o bloco **single** que apresenta os resultados só deve executar depois que integral e derivada estejam completas.

Assim, não foi necessário adicionar uma diretiva **barrier** manual.

8.14.6 Impressão por uma única thread

A impressão final ocorre no interior de uma construção **single**.

Dessa maneira, apesar de duas threads participarem da região externa, apenas uma executa o bloco de saída.

A escolha também evita que integral e derivada realizem impressão concorrente diretamente de suas respectivas **sections**, o que poderia tornar a ordem visual do terminal não determinística e dificultar a leitura.

8.14.7 Comportamento numérico da integral

O erro absoluto observado foi da ordem de

$$10^{-8}.$$

Esse resultado é coerente com a discretização adotada e com o erro esperado para o método dos trapézios aplicado à função quadrática.

A aproximação foi ligeiramente superior ao valor exato, comportamento consistente com a convexidade de x^2 .

8.14.8 Comportamento numérico da derivada

Nos pontos internos, a aproximação coincide com $2x$ na precisão exibida.

Isso não deve ser interpretado como uma propriedade universal do método de diferenças finitas. Nesse caso específico, a diferença central aplicada a uma função quadrática cancela exatamente os termos de erro de ordem superior relevantes para o cálculo.

Já nas extremidades, as diferenças progressiva e regressiva são de primeira ordem, razão pela qual foram observados:

0,000100

em $x = 0$, em vez de zero, e

19,999900

em $x = 10$, em vez de 20.

Esses valores são consistentes com o espaçamento empregado.

8.14.9 Relação entre correção e desempenho

Apesar de utilizar paralelismo, a atividade não fornece evidência experimental de que a versão paralela seja mais rápida do que uma versão sequencial.

O objetivo foi demonstrar corretamente a decomposição e a sincronização.

Qualquer afirmação sobre ganho de desempenho exigiria uma metodologia adicional, incluindo versão sequencial de referência, variação do número de threads, repetição das medições e análise estatística dos tempos.

8.15 Limitações

A primeira limitação é a ausência deliberada de temporização. A atividade valida a estrutura paralela e a correção numérica, mas não permite avaliar speedup, eficiência ou escalabilidade.

A segunda limitação é o uso de uma única função, um único intervalo e um único tamanho de vetor. A escolha de $f(x) = x^2$ é apropriada para fins didáticos, mas não representa toda a variedade de comportamentos possíveis em problemas numéricos.

A terceira limitação decorre da própria construção **sections**. O número de tarefas disponíveis é definido estaticamente no código, de modo que esse mecanismo não é destinado a problemas com grande quantidade dinâmica de tarefas.

A associação entre threads externas e **sections** também não é determinística. A execução registrada atribuiu integral à thread 1 e derivada à thread 0, mas essa associação não deve ser considerada requisito de correção.

Os identificadores 0 e 1 das equipes internas são locais às respectivas equipes OpenMP. Portanto, uma “thread interna 0” de uma tarefa não deve ser interpretada simplesmente como a mesma entidade da “thread interna 0” de outra equipe.

A soma em ponto flutuante realizada por redução também pode apresentar pequenas diferenças nos últimos dígitos caso a ordem de combinação das parcelas mude, pois a adição em ponto flutuante não é estritamente associativa.

Outra limitação é o tratamento das extremidades da derivada com fórmulas unilaterais de primeira ordem. Isso explica o erro maior nas extremidades em comparação com os pontos internos.

A validação impressa da derivada utiliza somente cinco amostras representativas. O vetor completo é calculado, mas não é integralmente apresentado para evitar uma saída de 100000 linhas.

Por fim, a versão exata do compilador GCC não foi registrada juntamente com o output. A reprodutibilidade funcional é sustentada pelo comando de compilação e pelo ambiente UCRT64 utilizado, mas uma reprodução estrita deveria também registrar a versão do compilador e do runtime OpenMP.

8.16 Conclusões da atividade

A Atividade 8 permitiu integrar, em uma única aplicação, três conceitos centrais de programação paralela com OpenMP: paralelismo de tarefas, paralelismo de dados e paralelismo aninhado.

A dependência entre as fases foi modelada explicitamente. A inicialização ocorre antes das operações numéricas, enquanto integral e derivada são executadas como tarefas independentes por meio de `sections`.

Cada tarefa cria sua própria região paralela com duas threads, atendendo ao requisito de explorar paralelismo de dados dentro do paralelismo de tarefas.

A integral utiliza `reduction` para combinar as contribuições das threads sem condição de corrida. A derivada distribui índices independentes entre as threads sem necessidade de exclusão mútua.

A sincronização necessária é obtida predominantemente pelas barreiras implícitas das construções OpenMP empregadas, e a apresentação final é executada por apenas uma thread utilizando `single`.

A execução confirmou o funcionamento do paralelismo aninhado, com duas threads internas tanto para a integral quanto para a derivada.

Numericamente, a integral apresentou erro absoluto de

$$1,666609250606 \times 10^{-8},$$

compatível com a discretização utilizada. Nos pontos internos da derivada, os resultados coincidiram com $f'(x) = 2x$ na precisão apresentada, enquanto os pequenos desvios das extremidades foram consistentes com as diferenças unilaterais adotadas.

Dessa forma, a implementação atende ao objetivo didático da atividade sem introduzir mecanismos paralelos desnecessários ou conclusões de desempenho não sustentadas experimentalmente.

8.17 Síntese

Para uma defesa oral da atividade, os pontos centrais podem ser sintetizados da seguinte forma:

1. A função escolhida foi $f(x) = x^2$, no intervalo $[0, 10]$, com 100000 pontos e valores do tipo `double`.
2. A inicialização precisa terminar antes dos demais cálculos, pois `integral` e `derivada` dependem do vetor da função.
3. Depois da inicialização, `integral` e `derivada` são independentes e, portanto, foram colocadas em duas `sections`.
4. Esse nível externo representa paralelismo de tarefas.
5. Uma `section` sozinha é executada por uma thread. Como o enunciado exige pelo menos duas threads por tarefa, cada `section` cria uma região paralela interna com duas threads.
6. As regiões internas representam paralelismo de dados e caracterizam o paralelismo aninhado.
7. Na `integral`, as threads dividem os termos do somatório. Como todas contribuem para uma única soma, foi usada `reduction(+:sum)` para evitar condição de corrida.
8. Na `derivada`, cada thread escreve em índices diferentes do vetor de saída, portanto não é necessário `critical` ou `atomic`.
9. A diferença central foi usada no interior do domínio; diferenças progressiva e regressiva foram usadas nas extremidades.
10. O paralelismo aninhado foi habilitado explicitamente com `omp_set_nested(1)` e a alteração dinâmica do número de threads foi desabilitada com `omp_set_dynamic(0)`.
11. Não foi necessário escrever uma `barrier` explícita. Foram utilizadas as sincronizações implícitas das regiões paralelas e de `sections`.

12. A impressão final é protegida por `single`, garantindo que apenas uma thread apresente os resultados.
13. Na execução observada, a integral foi atribuída à thread externa 1 e a derivada à thread externa 0. Essa associação pode mudar em outra execução e não foi fixada no programa.
14. As duas tarefas criaram equipes internas com as threads 0 e 1, confirmando que o paralelismo aninhado estava efetivamente ativo.
15. A integral numérica obtida foi 333,333333349999, contra 333,333333333333 analítico, com erro absoluto de $1,666609250606 \times 10^{-8}$.
16. Nos pontos internos, a derivada numérica coincidiu com $2x$ na precisão impressa. O pequeno erro nas extremidades decorre das diferenças unilaterais.
17. Não foi realizado benchmark. Portanto, a atividade demonstra correção e organização do paralelismo, mas não permite afirmar ganho de desempenho.

Considerações Finais